Automate Designs with Tcl !

TCL SCRIPTING

Same Script Multiple Designs Bigger Impact !

{ Tcl } Page No. 1

Tcl Basics → Variables & Expressions → Lists & Control Flow → proc { } Procedures → Files & Regex → ICC2 Collections → PD Automation → TCL HERO

VLSI PHYSICAL DESING

#!/usr/bin/tcl

Tcl Script

Tcl Shell

TCL FOR ICC2 AUTOMATION

Write Scripts Build Flows Create Impact !

Tcl scripting provides a reusable automation layer for ICC2 physical design, enabling design-object queries, filtering, analysis, decision making, reporting, checkpoints, and end-to-end RTL-to-GDSII flow automation.

ICC2 Automate with Tcl

TCL CORE BUILDING BLOCKS

- set / unset / puts / expr / info
- Variables & substitution
- Strings & lists
- if / switch / for / foreach / while
- proc / return / scope / source
- File I/O / regexp / regsub

```
# Example
set design top
puts "Hello Tcl!"
if { $design ne "" } {
    puts "Ready for ICC2"
}
```

ICC2 TCL AUTOMATION FLOW

Setup / Config	Tech files, Libraries, Design files, Run config
Initialize Design	Read design, link, check
MMMC Setup	Modes, Corners, Scenarios
Floorplan	Core, Macros, IO, Regions
Power Planning	Power nets, UPF, P/G strategy
Placement	Place std. cells, macros
CTS	Build clocks, optimize timing
Routing	Global + Detailed routing
ECO / Optimization	Fix violations, optimize QoR
Sign-off	Timing, DRC/LVS, IR, etc.
GDSII / Tapeout	Generate GDSII, final checks

Reusable Tcl Framework

run.tcl · setup.tcl · common.tcl · stage scripts · reports · checkpoints

Modular • Reusable • Configurable • Scalable

ICC2 TCL CORE

- get_cells / get_pins / get_ports / get_nets
- Collections & foreach_in_collection
- sizeof_collection / get_object_name
- filter_collection / -filter
- get_attribute / set_attribute
- Hierarchical object access

```
# Example
set cells [get_cells */U*]
set big_cells [filter_collection $cells
              -filter "area > 1000"]
foreach_in_collection c $big_cells {
    puts [get_object_name $c]
}
```

AUTOMATION GOALS

- ✓ Reduce manual effort
- ✓ Reuse common procedures
- ✓ Automate repetitive PD tasks
- ✓ Extract timing / physical / routing data
- ✓ Generate stage and QoR reports
- ✓ Implement PASS / FAIL gates

Automate → Save Time → Better QoR

TCL HERO SIGN-OFF

- ★ Automated QoR framework
- ★ Multi-scenario analysis
- ★ Configuration-driven runs
- ★ Logging & checkpoints
- ★ Report management
- ★ Multi-block automation
- ★ RTL-to-GDSII run manager
- ★ Final release package

From RTL to GDSII with Tcl !

KEY METRICS / DATA

- Area
- Utilization
- WNS
- TNS
- Hold Violations
- Congestion
- DRC
- Power
- Wirelength
- Via Count
- Clock Skew
- ... (Design QoR)

WHY TCL AUTOMATION ?

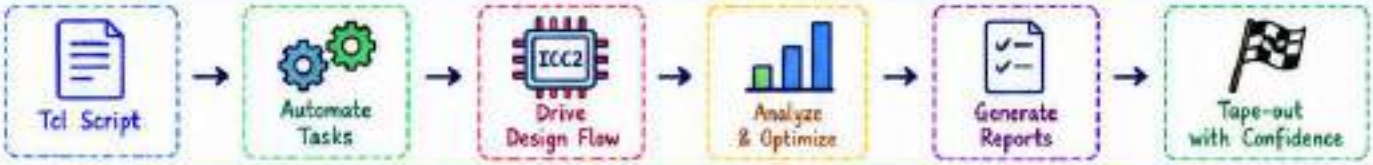
- ✓ Consistency (same flow across designs)
- ✓ Reusability (modular and maintainable)
- ✓ Speed (faster execution)
- ✓ Scalability (multi-block / multi-scenario)
- ✓ Debuggability (easy to trace and fix)
- ✓ Traceability (logs, reports, checkpoints)

Author : Konda Tharun Vivekananda

Write Once → Automate → Optimize → Sign-off

TCL & ICC2 AUTOMATION — SYLLABUS INDEX

Module No.	Module Title	Topics Covered	Starting Page No.
1	Tcl Fundamentals	• Introduction to Tcl, command structure, comments, basic commands, script execution, evaluation, special characters, basic practice	1
2	Variables, Substitution & Expressions	• Variables, naming, variable substitution, command substitution, braces, quotes, escape characters, mathematical expressions, operators, EDA-oriented calculations	9
3	Strings & Lists	• String fundamentals, string commands, manipulation, searching, concatenation, list fundamentals, list creation/access/modification, searching/sorting, string-list conversion, nested lists, EDA applications	19
4	Control Flow & Loops	• Conditional statements, logical conditions, switch, for, foreach, while, loop control, nested loops, EDA loop applications, control-flow practice	31
5	Procedures, Scope & Modular Tcl	• Procedures, arguments, return handling, variable scope, reusable procedures, debugging, source, modular design, project structure, modular Tcl practice	41
6	File Handling, Regex & Report Parsing	• File handling, reading/writing files, file-system commands, path manipulation, regular expressions, regsub, report parsing, timing/physical/routing data extraction, automated report generation	51
7	Error Handling, Debugging & Advanced Tcl	• Error handling, Tcl errors, debugging commands/techniques, arrays, dictionaries, namespaces, introspection, advanced evaluation	65
8	ICC2 Tcl Fundamentals	• ICC2 Tcl environment, command structure, design objects, get_* commands, wildcards, hierarchical access, collections, collection commands, iteration, filters, attributes	76
9	Tcl for Complete ICC2 Physical Design	• Design initialization, technology/library setup, MMMC, floorplan, macro planning, I/O planning, UPF, power planning, physical-only cells, placement, CTS, routing, post-route optimization, ECO, physical verification, signoff, GDSII/tapeout	88
10	Tcl HERO: Advanced ICC2 Automation	• QoR framework, automated stage gates, decision making, multi-scenario automation, configuration-driven Tcl, logging, checkpoints, report management, multi-block automation, performance optimization, coding standards, run manager, RTL-to-GDSII automation, advanced projects, complete Tcl HERO project	101



Note :

- Page numbers indicate the starting page of each module in this material.
- Topics are arranged in a progressive learning order from Tcl basics to advanced ICC2 automation.
- Use this index to quickly navigate to the desired module.

Author : Konda Tharun Vivekananda

Write Tcl
Build Solutions
Shape Silicon !

TCL Scripting

{Tcl}
$variable
[command]
.tcl

1. TCL Fundamentals

1.1 Introduction to Tcl

1. What is Tcl ?

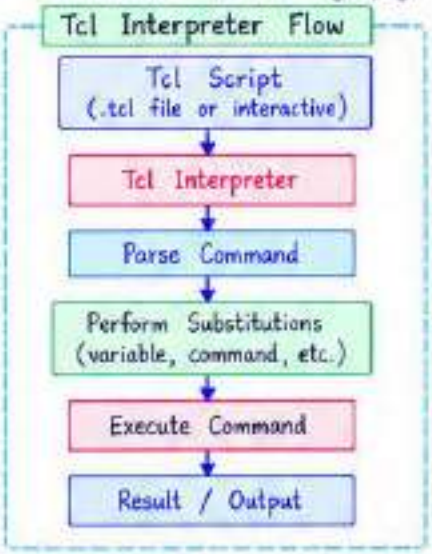
- Tcl (Tool Command Language) is a simple yet powerful scripting language.
- It provides a flexible command language with variables, control structures and rich built-in commands.
- Tcl is designed to be easy to use, easy to extend and highly portable.
- Originally created for tool control and automation, and widely adopted in EDA industry.

2. Tcl history and purpose

- Developed in late 1980s by John Ousterhout at UC Berkeley.
- First released in 1988.
- Designed for controlling and integrating tools in a simple scripting language.
- Purpose :
  - → Provide a simple and consistent command language
  - → Easy embedding in applications
  - → Enable rapid development of automation scripts

3. Tcl interpreter

- Tcl is an interpreted language (not compiled).
- Tcl interpreter reads a command, parses it, performs substitutions and then executes it.
- Interpreter can run interactively (command line) or from a script file (.tcl).
- Same interpreter is used in standalone Tcl and in EDA tools like Synopsys.



Key Point

- ☐ Tcl is a simple, powerful and widely used scripting language.
- ☐ It is the command language for Synopsys EDA tools.
- ☐ Tcl enables efficient automation and tool customization.
- ☐ Same Tcl language can be used across different EDA tools.

4. Tcl as a scripting language

- Tcl is a lightweight scripting language.
- Used to automate repetitive tasks.
- Provides variables, expressions, control statements, procedures, file handling and more.
- Suitable for writing small utility scripts to large automation frameworks.
- Easy to learn and write – simple and clean syntax.
- Highly portable across platforms (Linux, Windows, etc.).

Example : Simple Tcl Script

```
# My first Tcl script
set msg "Hello Tcl!"
puts $msg          # print message
set a 10
set b 20
puts [expr {$a + $b}] # command substitution
```

Output :

```
Hello Tcl!
30
```

5. Tcl in EDA

- Tcl is widely used in Electronic Design Automation (EDA) tools for automation, customization and flow control.
- Provides a common scripting interface across tools.
- Used to :
  - → Automate design flows
  - → Query and modify design data
  - → Parse reports
  - → Create custom utilities
  - → Integrate multiple tools in a flow

6. Tcl in Synopsys tools

- Synopsys tools like Design Compiler, ICC2, PrimeTime, etc. use Tcl as the command language.
- All tool commands (e.g. get_cells, get_ports, create_clock) are Tcl commands.
- Users can write Tcl scripts to automate synthesis, place & route, sign-off and analysis tasks.
- Tcl enables customization, batch runs and large design automation in Synopsys flow.

7. Tcl vs Shell scripting

Feature	Tcl	Shell (sh / bash)
Primary purpose	Scripting & tool control	OS / system automation
Syntax	Simple & consistent	More complex
Data handling	Rich data types (lists, strings)	Primarily text based
Portability	Highly portable	OS dependent
Usage in EDA	Widely used	Limited (mainly for OS tasks)
Control structures	Rich and easy to use	Available but less powerful

8. Tcl vs Python

Feature	Tcl	Python
Learning curve	Simple	Moderate
Syntax	Very simple	More structured
Primary use in EDA	Native in EDA tools	Used via APIs / wrappers
Data types	Basic (string, list, dict)	Rich and advanced
Tool integration	Built-in (e.g. Synopsys tools)	Needs additional interface
Usage	Command language for tools	General purpose + EDA

9. Tcl execution model

- Tcl is executed by the interpreter line by line.
- Each command is parsed, substitutions are performed (variable, command, etc.), then the command is evaluated.
- Execution can be interactive (tcl shell) or from a script file.
- The interpreter maintains state (variables, procedures, etc.) during the session.

Author : Konda Tharun Vivekananda

1. TCL Fundamentals

1.2 Tcl Command Structure

1. Command

- A Tcl command is a keyword or procedure name that tells the interpreter what operation to perform.
- Commands can be built-in (e.g. set, puts) or user defined (procedure).
- Tcl is case sensitive for command names.
- Each command is followed by its arguments, options and values.

Examples :

```
set a 10          # built-in command
puts "Hello"      # built-in command
my_proc arg1      # user defined command
```

2. Arguments

- Arguments provide input to the command.
- A command can have zero or more arguments.
- Arguments can be values, variables or other commands.

Examples :

```
puts "Hello Tcl"    # one argument
set a 10            # two arguments (a and 10)
add 5 3             # two arguments to proc 'add'
```

3. Options

- Many Tcl commands support options to modify their behaviour.
- Options usually start with a dash (-).
- Options can appear before or after arguments (depends on the command).

Examples :

```
open -r input.txt       # -r is an option
puts -nonewline "Hi"    # -nonewline is an option
file copy -force src dst
```

4. Values

- Values are the actual data passed to a command or option.
- Values can be numbers, strings, variables, lists, etc.

Examples :

```
set a 100          # numeric value
puts "Hello"       # string value
set list {1 2 3}   # list value
puts $a            # variable value
```

Key Point

- ☐ A Tcl command consists of a command name followed by arguments, options and values.
- ☐ Whitespace separates the command words.
- ☐ Commands can be terminated by newline or semicolon.
- ☐ Options (starting with -) modify command behaviour.

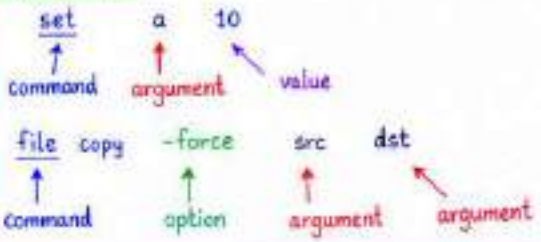
5. Command Structure

- General form of a Tcl command :



- A command can have multiple options and arguments.
- Not all commands use options.

Example :



6. Command termination

- A Tcl command is terminated by either a newline or a semicolon (;).
- After termination, the interpreter executes the command.

Example :

```
set a 10                # terminated by newline
set b 20 ; puts $b      # terminated by semicolon
puts "Hello" ; set c 30 ; puts $c
```

7. Whitespace

- One or more spaces or tabs are treated as a separator between command words (command, arguments, options, values).
- Multiple spaces are equivalent to a single space.
- Tabs can also be used.

Example :

```
set   a   10
set a    10        # multiple spaces
set a    10        # tab between words (shown as space)
puts  "Hello Tcl"
```

8. Newline

- A newline (pressing Enter) terminates the current command.
- The next line is treated as a new command.

Example :

```
set  a 10     # command 1
puts $a       # command 2 (next line)
```

9. Semicolon

- A semicolon (;) can be used to terminate a command and start another command on the same line.
- Useful to write multiple commands in one line.

Example :

```
set a 10 ; set b 20 ; puts [expr [$a + $b]]
command 1  command 2  command 3
```

- Semicolon is optional, but improves readability when writing multiple commands in one line.

1. TCL Fundamentals

1.3 Comments

1. Single-line comments

- In Tcl, a comment starts with the hash symbol (#).
- Everything from # to the end of the line is treated as a comment and ignored by the interpreter.
- Used to explain the purpose of a command or a block of code.

Example :

```
# This is a single-line comment
set a 10
set b 20        # initialize variable b
puts "Sum = [expr {$a + $b}]"    # print result
```

2. Inline comments

- Comments can be written on the same line as a command after some whitespace.
- Useful for short notes or explanations.

Example :

```
set a 10                    # value of a
set b 20                    # value of b
set sum [expr {$a + $b}]    # compute sum
puts $sum                   # print the result
```

3. Multi-line commenting style

- Tcl does not have a built-in multi-line comment syntax like /* ... */.
- To comment multiple lines, use # at the beginning of each line.
- Another common method is to enclose the lines in a block using if {0} { ... } (these lines are not executed).
- Method 1 : Using # on each line

Example :

```
# This block of code is commented
# It will not be executed
# set a 10
# set b 20
# puts $a
```

- Method 2 : Using if {0} { ... }

Example :

```
if {0} {                    # all these lines are ignored
    set a 10
    set b 20
    puts [expr {$a + $b}]
}
```

4. Documentation practices

- Use meaningful comments to explain the intent of the script, important steps and assumptions.
- Write a header section at the beginning of the file with author, date, description, and revision history.
- Comment complex logic, loops, and procedures.
- Use consistent comment style for better readability and maintenance.
- Good documentation helps others (and your future self) to understand and modify the script easily.

Example : Script header

```
# ================================
# File        : floorplan.tcl
# Author      : Konda Tharun Vivekananda
# Date        : 04-Sep-2026
# Description : This script creates a simple floorplan
#               and sets basic constraints.
# Revision History :
#   v1.0   04-Sep-2026   Initial version
# ================================

# Set design variables
set core_width  1000      # core width in microns
set core_height 800       # core height in microns

# Create floorplan
create_floorplan -core_width $core_width \
                 -core_height $core_height  # run command
```

Good vs Bad comments

Type	Example	Remarks
Bad Comment	set a 10 # temp	Not meaningful
Good Comment	set a 10 # number of rows	Clear and useful
Bad Comment	# code	Does not explain intent
Good Comment	# Calculate total area set area [expr {$w * $h}]	Explains what the code does

Key Point

- ☐ Comments start with # and are ignored by the interpreter.
- ☐ They can be single-line or inline.
- ☐ For multi-line comments, use # on each line or enclose the block in if {0} { ... }.
- ☐ Well-documentad scripts are easier to understand, maintain and share.

1. TCL Fundamentals

1.4 Basic Commands

1. puts

- puts is used to print output on the console (standard output).
- It displays strings, variable values or expression results.
- By default, puts adds a newline at the end.
- Useful for messages, debugging and displaying results.

Examples :

```
puts "Hello Tcl!"      # print a string
set a 10
puts $a                # print variable value
puts "Value = $a"
puts [expr {2+3}]      # print expression result
```

2. set

- set is used to create a variable and assign a value.
- If the variable already exists, its value is updated.
- Values can be numbers, strings, lists or command results.
- Returns the assigned value.

Examples :

```
set a 10                  # numeric value
set name "Tcl"            # string value
set list_val {1 2 3}      # list value
set result [expr {5*2}]   # command result
set msg "Value = $result"
```

3. unset

- unset is used to delete a variable.
- After unset, the variable no longer exists.
- If the variable does not exist, an error is returned (unless handled).
- Can also delete multiple variables.

Examples :

```
set a 10
set b 20
unset a          # delete variable a
unset b          # delete variable b
# unset c        # error if c does not exist
```

Check if variable exists before unset :

```
if {[info exists c]} {
    unset c
}
```

4. expr

- expr is used to evaluate arithmetic and logical expressions.
- Supports basic math operations (+ - * / %), relational and logical operators.
- Expression is written inside braces { } or as an argument.
- Returns the result of the expression.

Example :

```
set a 10
set b 3
set sum [expr {$a + $b}]      # addition
set diff [expr {$a - $b}]     # subtraction
set mul [expr {$a * $b}]      # multiplication
set div [expr {$a / $b}]      # division
set rem [expr {$a % $b}]      # modulus

set result [expr {($a > $b) && ($a < 20)}]  # logical
puts $result       # 1 (true)
```

5. info

- info provides information about the Tcl interpreter, variables, commands, procedures, etc.
- Useful for debugging and introspection.

Example :

```
info vars                 # list all variables
info exists a             # check if variable a exists
info commands             # list all commands
info commands puts        # check if 'puts' command exists
info procs                # list all procedures
info tclversion           # Tcl version
info patchlevel           # Tcl patch level
```

Command Summary

Command	Purpose	Typical Usage
puts	Print output to console	puts "Hello"
set	Create / assign variable	set a 10
unset	Delete a variable	unset a
expr	Evaluate expression	set sum [expr {a+b}]
info	Get information	info vars

Key Point

- ☐ puts displays output on the console.
- ☐ set is used to create and assign variable values.
- ☐ unset deletes a variable.
- ☐ expr evaluates arithmetic and logical expressions.
- ☐ info provides useful information about the Tcl interpreter.

1. TCL Fundamentals

1.5 Tcl Script Execution

1. Interactive Tcl shell

- Tcl provides an interactive shell (tclsh) where commands can be typed and executed immediately.
- Useful for quick testing, debugging and learning.
- Each command is executed as soon as you press Enter.
- No need to create a separate file.

Example :

```
% tclsh                    # start Tcl shell
% puts "Hello Tcl!"        # execute command
Hello Tcl!
% set a 10                 # set variable
% expr {$a * 2}            # evaluate expression
20
```

2. Tcl script file

- A group of Tcl commands can be written in a text file and executed as a script.
- Useful for automating repetitive tasks.
- Script files can contain comments, procedures, variables, etc.
- Enhances reusability and maintainability.

Example : (my_script.tcl)

```
# Simple Tcl script          # comment
set a 10                     # set variable a
set b 20                     # set variable b
set sum [expr {$a + $b}]     # compute sum
puts "Sum = $sum"            # print result
```

3. .tcl extension

- Tcl script files usually use the .tcl extension (e.g. run_me.tcl).
- The extension is not mandatory, but it is a widely adopted convention.
- Helps to identify the file as a Tcl script.

Examples :

```
✓ run.tcl      # preferred
✓ flow.tcl     # preferred
✓ script.tcl   # preferred
✗ run          # works but not recommended
```

4. Running Tcl scripts

- Tcl scripts can be executed using the tclsh interpreter from the command line.
- Syntax : `tclsh script_name.tcl`
- The interpreter reads the file and executes the commands in order.
- Any output is printed to the console.

Example :

```
% tclsh my_script.tcl      # run the Tcl script
Sum = 30
```

5. source

- The source command is used to read and execute another Tcl script file from within the current Tcl session.
- Useful for modular scripting (splitting code into multiple files).
- Variables, procedures, etc. defined in the sourced file are available in the current session.

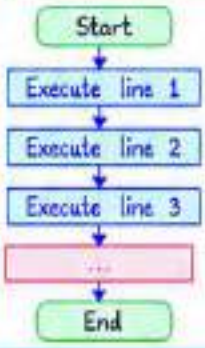
Example :

```
% source setup.tcl            # execute another script file
% source ./scripts/common.tcl # relative path
% source /home/user/util.tcl  # full path
```

6. Script execution order

- Tcl executes commands sequentially from top to bottom.
- Each command is executed only after the previous command completes.
- If a script uses source, the sourced file is executed at that point in the flow.
- Use comments and proper structure to maintain clarity.

Execution Flow



Example script :

```
# run_me.tcl
set a 10
set b 20
source util.tcl
puts "Final = [expr {$a + $b}]"
```

Executed in this order

Key Point

- ☐ Tcl can be used interactively or through script files.
- ☐ .tcl is the standard file extension for Tcl scripts.
- ☐ tclsh is used to run Tcl scripts from the command line.
- ☐ source is used to execute another Tcl script file.
- ☐ Commands in a script are executed in sequential order.

1. TCL Fundamentals

1.6 Tcl Evaluation

1. Command evaluation

- Tcl reads a complete command, performs substitutions and then evaluates (executes) the command.
- The first word is the command name, followed by its arguments.
- If the command exists, the Tcl interpreter invokes it with the evaluated arguments.

Example :

```
puts "Hello Tcl!"      # command 'puts' is evaluated
set a 10               # command 'set' is evaluated
expr {2+3}             # command 'expr' is evaluated
```

2. Argument evaluation

- Before a command is executed, each argument is evaluated (after performing substitutions).
- Arguments can be literals, variables, command results or expressions.
- Each argument is treated as a single word after substitution.

Example :

```
set a 10
set b 20
puts $a                # argument is variable (value 10)
puts [expr {$a + $b}]  # argument is command result (30)
puts "Result = $a"     # argument with substitution
```

3. Substitution

- Tcl performs substitutions before command evaluation.
- Types of substitution :
  - → Variable substitution : $var
  - → Command substitution : [command]
  - → Backslash substitution : \n \t \\
  - → (Performed inside double quotes).

Example :

```
set name "Tcl"
puts "Hello $name"     # variable substitution
puts [expr {5*2}]      # command substitution
puts "Line1\nLine2"    # backslash substitution
```

4. Parsing

- Tcl parses the input into commands and words.
- Whitespace (spaces, tabs, newlines) separate words.
- Quotes "..." and braces {...} control how words are parsed.
- Comments starting with # are ignored.

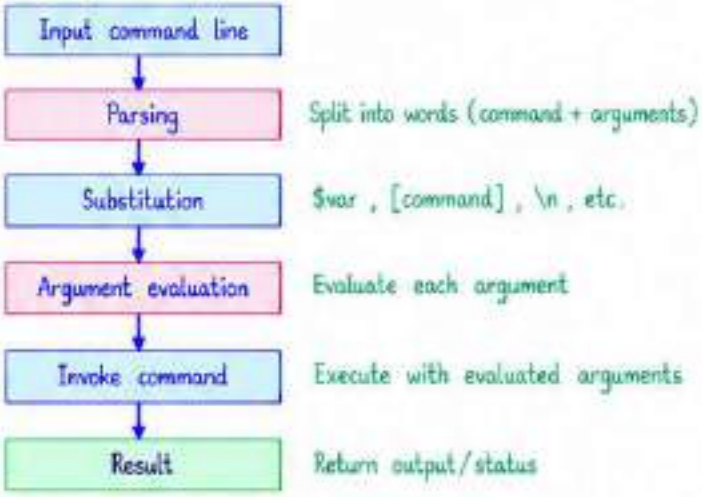
Example :

```
puts "Hello Tcl"       # two words (command + argument)
set msg {Hello Tcl}    # braces: no substitution
puts $msg              # single word 'Hello Tcl'
```

5. Evaluation order

- Tcl follows a specific order while processing a command.
- General order :
  1. Parse the command (identify words).
  2. Perform substitutions (variable, command, backslash).
  3. Evaluate each argument.
  4. Invoke the command with the evaluated arguments.
  5. Return the result.

Evaluation Flow



Example : Step by Step

```
Command : puts "Sum = [expr {$a + $b}]"
1. Parse              → words : puts "Sum = [expr {$a + $b}]"
2. Substitution       → [expr {$a + $b}] → 30
3. Argument evaluation → "Sum = 30"
4. Invoke command     → puts with argument "Sum = 30"
5. Result             → outputs : Sum = 30
```

Substitution Examples

Type	Syntax	Example	Result
Variable	$var	set a 10 $a	10
Command	[command]	[expr {2+3}]	5
Backslash	\n \t \\	"A\nB"	A B
Quotes	"..."	"Hello $name"	Substituted
Braces	{...}	{Hello $name}	No substitution

Key Point

- ☐ Tcl parses a complete command before evaluation.
- ☐ Substitutions are performed before the command is executed.
- ☐ Each argument is evaluated after substitution.
- ☐ The evaluation order is : Parse → Substitute → Evaluate → Invoke → Return result.

1. TCL Fundamentals

1.7 Tcl Special Characters

1. $ (Dollar)

- $ is used for variable substitution.
- It is replaced by the value of the variable.
- If the variable does not exist, an error is returned.

Example :

```
set name "Tcl"
puts $name          # prints Tcl
set a 10
puts "Value = $a"   # prints Value = 10
```

2. [ ] (Square Brackets)

- [ ] is used for command substitution.
- The command inside the brackets is executed first, and its result is substituted in place.
- Can be nested.

Example :

```
set a 10
set b 20
set sum [expr {$a + $b}]      # executes expr command
puts "Sum = [expr {$a + $b}]" # prints Sum = 30
puts "Today is [clock format [clock seconds]]"
```

3. { } (Braces)

- { } is used to group words and prevent substitution.
- Variables and commands inside braces are not substituted.
- Often used in procedures, conditionals and to pass literal text.

Example :

```
set a 10
puts {$a + 5}          # no substitution
puts "$a + 5"          # substitution happens
puts {Hello [expr 2+3]} # no command substitution
```

4. " " (Double Quotes)

- " " is used to group words, allowing substitutions.
- Variables and commands inside double quotes are substituted.
- Useful for strings with spaces.

Example :

```
set name "VLSI"
puts "Hello $name"          # Hello VLSI
puts "2 + 3 = [expr 2+3]"   # 2 + 3 = 5
puts "Path : C:\temp\file"  # backslash is treated
```

5. \ (Backslash)

- \ is used as an escape character.
- It is used to escape special characters or to continue a command to the next line.
- Common uses: \n (newline), \t (tab), \\ (literal backslash), \ (escape next character).

Example :

```
puts "Line1\nLine2"        # newline
puts "A\tB"                # tab
puts "C:\\temp\\file"      # literal backslash
set a 10 \                 # line continuation
      + 20
puts $a                    # 30
```

6. ; (Semicolon)

- ; is used to separate multiple commands on the same line.
- It indicates the end of a command.
- Commands can also be separated by a newline.
- Useful when writing multiple commands in one line.

Example :

```
set a 10 ; set b 20 ; set c [expr {$a + $b}]
puts $c          # 30
puts "Hello" ; puts "Tcl"     # multiple commands
```

Summary of Special Characters

Character	Name	Purpose	Example
$	Dollar	Variable substitution	puts $a
[ ]	Square brackets	Command substitution	puts [expr 2+3]
{ }	Braces	Group words (no substitution)	puts {$a}
" "	Double quotes	Group words (with substitution)	puts "Hello $a"
\	Backslash	Escape character / line continuation	puts "A\nB"
;	Semicolon	Command separator	set a 10 ; set b 20

Key Point

- ☐ $ substitutes the value of a variable.
- ☐ [ ] executes a command and substitutes the result.
- ☐ { } prevents substitution.
- ☐ " " allows substitution.
- ☐ \ is used for escaping and line continuation.
- ☐ ; separates multiple commands.

1. TCL Fundamentals

1.8 Basic Practice

1. Hello World

- The simplest Tcl program prints a message using puts command.
- Useful to verify the Tcl environment.

Example :

```
puts "Hello World!"
```

Output :

```
Hello World!
```

2. Variables

- Use set command to create and assign a variable.
- Use $ to access the variable value.
- Variable names are case sensitive.

Example :

```
set name "Tcl"          # assign string
set age 25              # assign number
puts "Name = $name"     # print string value
puts "Age = $age"       # print number value
```

Output :

```
Name = Tcl
Age = 25
```

3. Arithmetic

- Use expr to perform arithmetic operations.
- Supports + - * / % and more.
- The expression is evaluated and the result is returned.

Example :

```
set a 10                        # define variables
set b 20
set sum [expr {$a + $b}]        # addition
set mul [expr {$a * $b}]        # multiplication
puts "Sum = $sum"               # print result
puts "Product = $mul"           # print result
```

Output :

```
Sum = 30
Product = 200
```

4. Simple output

- Use puts to display text and variable values.
- You can print multiple items in a single line.

Example :

```
set a 10
set b 20
puts "Values : a = $a , b = $b"
puts "Sum = [expr {$a + $b}]"
```

Output :

```
Values : a = 10 , b = 20
Sum = 30
```

5. Example 1 : Simple Calculator Script

- This script takes two numbers, performs basic arithmetic operations and prints the results.
- Demonstrates variables, arithmetic and simple output.

File : calculator.tcl

```
# calculator.tcl
# Simple arithmetic example
set a 10                          # assign values
set b 5
set sum [expr {$a + $b}]          # addition
set diff [expr {$a - $b}]         # subtraction
set mul [expr {$a * $b}]          # multiplication
set div [expr {$a / $b}]          # division

puts "Value of a = $a"            # display values
puts "Value of b = $b"
puts "--------------------"
puts "a + b = $sum"               # display results
puts "a - b = $diff"
puts "a * b = $mul"
puts "a / b = $div"
```

Output (when run) :

```
Value of a = 10
Value of b = 5
--------------------
a + b = 15
a - b = 5
a * b = 50
a / b = 2
```

Execution flow :

set a 10 , set b 5 → Evaluate expressions → Store results in variables → Execute puts commands → Display output

6. Example 2 : User Information Script

- This script shows a simple real-world style example using variables and output.
- Demonstrates string handling and multiple puts commands.

File : user_info.tcl

```
# user_info.tcl
# Display user information
set name "Konda Tharun Vivekananda"
set branch "VLSI"
set year 2026

puts "----- User Information -----"
puts "Name   : $name"
puts "Branch : $branch"
puts "Year   : $year"
puts "---------------------------"
```

Output (when run) :

```
----- User Information -----
Name   : Konda Tharun Vivekananda
Branch : VLSI
Year   : 2026
---------------------------
```

Execution flow :

Assign variable values → Execute puts commands → Display output

Key Point

- ☐ puts is used for output.
- ☐ set creates and assigns variables.
- ☐ expr is used for arithmetic operations.
- ☐ Tcl scripts can combine multiple commands.
- ☐ These basic concepts form the foundation for larger Tcl automation scripts.

TCL Scripting

2. VARIABLES, SUBSTITUTION & EXPRESSIONS

2.1 Variables

- A variable is used to store a value (string, number, list, etc).
- Variables in Tcl are created automatically when assigned.
- Variable names are case sensitive.
- A variable is referenced using the $ symbol.

1. Creating variables

- Use set command to create a variable and assign a value.
- Tcl does not require explicit declaration.
- Variable name can contain letters, digits and _.

Example :

```
set name "Tcl"          # string variable
set age 25              # numeric variable
set pi 3.14             # floating point
set list_val {1 2 3}    # list variable
```

Output :

```
Variable 'name' = Tcl
Variable 'age' = 25
Variable 'pi' = 3.14
Variable 'list_val' = 1 2 3
```

2. Reading variables

- Use $ to read (access) the value of a variable.
- The variable name is substituted by its value.

Example :

```
set name "VLSI"
puts $name              # prints value of variable
puts "Name = $name"     # inside double quotes
puts Name: $name        # direct substitution
```

Output :

```
VLSI
Name = VLSI
Name: VLSI
```

3. Updating variables

- Use set again to update (change) the value of a variable.
- If the variable already exists, its old value is replaced.

Example :

```
set count 10
puts $count                     # original value
set count 20                    # update value
puts $count                     # new value
set count [expr {$count + 5}]   # using expression
puts $count                     # updated value
```

Output :

```
10
20
25
```

4. Removing variables

- Use unset to delete a variable.
- After unset, the variable no longer exists.
- If the variable does not exist, an error is returned (unless handled).
- You can also delete multiple variables.

Example :

```
set temp 100
puts $temp              # value before unset
unset temp              # delete variable
# puts $temp            # error (if executed)
set a 1; set b 2; set c 3
unset a b               # delete multiple variables
puts [info exists a]    # check after delete
```

Output :

```
100
0       # 'a' does not exist (returns 0)
```

5. Checking variable existence

- Use info exists to check if a variable exists.
- It returns 1 if the variable exists, otherwise 0.
- Useful to avoid errors before accessing a variable.

Example :

```
set x 50
puts [info exists x]    # 1 (exists)
puts [info exists y]    # 0 (does not exist)
if {[info exists x]} {
    puts "x = $x"
} else {
    puts "x is not defined"
}
```

Output :

```
1
0
x = 50
```

Variable Commands Summary

Command	Purpose	Example	Result
set	Create / update a variable	set a 10	a = 10
$var	Read variable value	puts $a	10
unset	Delete variable	unset a	variable removed
info exists	Check if variable exists	info exists a	1 (exists) 0 (not exists)

Key Point

- ☐ Variables are created using set.
- ☐ Use $ to read the value of a variable.
- ☐ Use unset to remove a variable.
- ☐ Use info exists to check if a variable exists.
- ☐ Variable names are case sensitive.

2. VARIABLES, SUBSTITUTION & EXPRESSIONS

2.2 Variable Naming

- Variable names in Tcl are used to identify and store values.
- Tcl variable names are case sensitive.
- Good naming conventions help in writing readable and maintainable scripts.
- Tcl supports both scalar variables and array variables.

1. Valid variable names

- Must start with a letter (a - z, A - Z) or underscore (_).
- Can contain letters, digits and underscore.
- Cannot start with a digit.
- Cannot contain spaces or special characters (except _).
- Variable names are case sensitive (name ≠ Name).

Valid examples :

```
set a 10
set name "Tcl"
set _temp 5
set data1 100
set user_name "VLSI"
```

Invalid examples :

```
set 1name 10     # starts with digit
set user-name 5  # '-' not allowed
set user name 5  # space not allowed
set $var 10      # '$' not part of name
set a+b 10       # special character
```

2. Naming conventions

- Use meaningful names that describe the purpose.
- Use lowercase letters and underscores (widely used).
- Avoid using Tcl reserved keywords as variable names.
- Keep names short but descriptive.
- Use consistent naming style in all scripts.

Recommended style :

```
set design_name "top"
set clock_period 10
set input_file "design.v"
set max_iterations 5
set is_valid 1
```

Avoid :

```
set dn "top"      # not clear
set cp 10         # unclear
set infile "design.v"
set MAX_ITER 5 # inconsistent
set Valid 1       # mixed case
```

Common Tcl reserved words (avoid using as variable names)

set	if	else	foreach	while	proc
return	break	continue	switch	expr	namespace

3. Scalar variables

- A scalar variable stores a single value (string, number, etc.).
- Most commonly used variable type in Tcl.
- Value can be accessed using the variable name preceded by $.

Example :

```
set a 100
set name "Tcl"
set pi 3.14
puts "a = $a"
puts "name = $name"
puts "pi = $pi"
```

Output :

```
a = 100
name = Tcl
pi = 3.14
```

4. Array variables

- An array variable can store multiple values indexed by a key.
- Each element is accessed using the syntax : $array_name(index)
- Arrays are useful for grouped data (e.g. configuration, timing values, design parameters).
- Array indices can be strings or numbers.

Example :

```
# create and assign array elements
set freq(0) 100
set freq(1) 200
set freq(2) 300
puts "freq(0) = $freq(0)"
puts "freq(1) = $freq(1)"
puts "freq(2) = $freq(2)"

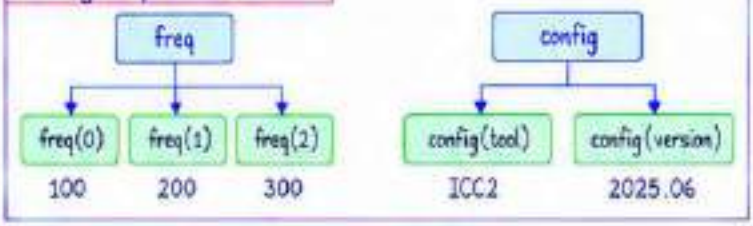
# array with string index
set config(tool) "ICC2"
set config(version) "2025.06"
puts "Tool = $config(tool)"
puts "Version = $config(version)"
```

Output :

```
freq(0) = 100
freq(1) = 200
freq(2) = 300

Tool = ICC2
Version = 2025.06
```

Array Representation :



Accessing and updating array elements :

Example :

```
set freq(0) 100
puts $freq(0)              # read element
set freq(0) 150            # update element
puts $freq(0)
unset freq(0)              # delete element
puts [info exists freq(0)] # check existence
```

Output :

```
100

150

0    (deleted)
```

Array related commands :

Command	Purpose	Example
set	Create/update array element	set arr(1) 10
unset	Delete array element	unset arr(1)
info exists	Check if element exists	info exists arr(1)
array names	List all array indices	array names arr
array size	Number of elements	array size arr
array get	Get all elements (key value pairs)	array get arr
array unset	Delete all or matching elements	array unset arr

Key Point

- ☐ Variable names are case sensitive.
- ☐ Use meaningful names and follow consistent conventions.
- ☐ Scalar variables store a single value.
- ☐ Array variables store multiple values using an index.
- ☐ Use array commands to manage array elements.

Author : Konda Tharun Vivekananda

2. VARIABLES, SUBSTITUTION & EXPRESSIONS

2.3 Variable Substitution

- Variable substitution replaces a variable reference with its value before the command is executed.
- In Tcl, variables are referenced using the $ symbol.
- Substitution happens automatically in most contexts (e.g., inside double quotes, in expressions, etc.).

☑ 1. $variable

- Use $ followed by the variable name.
- Simplest form of variable substitution.
- Works well when the variable name is a simple word (letters, digits or underscore).

Example :

```
set name "VLSI"
puts $name
set count 10
puts "Count = $count"
```

Output :

```
VLSI
Count = 10
```

☑ 2. ${variable}

- Use ${ } to explicitly specify the variable name.
- Useful when the variable name is adjacent to other characters (to avoid ambiguity).
- Works for longer variable names.

Example :

```
set tool "ICC2"
set version "2025.06"
puts "Tool = ${tool}"
puts "Version = ${version}"
puts "File = ${tool}_${version}.log"
```

Output :

```
Tool = ICC2
Version = 2025.06
File = ICC2_2025.06.log
```

Why use ${ } ?

- Without braces, Tcl may not know where the variable name ends.

Example :

```
set a 1
set b 2
puts "Value = $a b"   # wrong (tries $ab)
puts "Value = ${a}b"  # correct
```

Output :

```
Value =
Value = 1b
```

Note : Use ${ } when the variable name is followed by letters, digits or underscore.

☑ 3. Nested substitution

- A variable can hold the name of another variable.
- Tcl performs substitution from inside to out.
- Useful in dynamic scripting (e.g., accessing variable names stored in other variables).

Example :

```
set var1 "name"        # var1 holds the name of another variable
set name "Tcl"         # actual variable
puts "Value = [set $var1]"   # nested substitution

set x 5
set y "x"
puts "y = $y, value = [set $y]"
```

Output :

```
Value = Tcl
y = x, value = 5
```

Explanation :

- $var1 → name
  then [set name] → Tcl
- $y → x
  then [set x] → 5

☑ 4. Variable substitution inside strings

- Variables are substituted inside double quotes (" ").
- No substitution happens inside braces ({ }).
- Useful for creating messages and combining text with variable values.

Example :

```
set user "Vivekananda"
set tool "Tcl"
puts "Hello $user, welcome to $tool!"   # inside string
puts "User = ${user}"                   # using braces
puts "This is a test: $tool scripting"  # mixed text
puts {This is a test: $tool scripting}  # no substitution
```

Output :

```
Hello Vivekananda, welcome to Tcl!
User = Vivekananda
This is a test: Tcl scripting
This is a test: $tool scripting
```

Explanation :

- Inside double quotes, $variables are replaced with their values.
- Inside braces, the string is taken literally (no substitution).

Key Point

- ☐ Use $variable for simple substitution.
- ☐ Use ${variable} when the variable name is adjacent to other characters.
- ☐ Nested substitution allows dynamic variable references.
- ☐ Variables inside double quotes are substituted.
- ☐ Variables inside braces are not substituted.
- ☐ Substitution happens before the command is executed.

2. VARIABLES, SUBSTITUTION & EXPRESSIONS

2.4 Command Substitution

- Command substitution allows the result of a command to be used in place of the command itself.
- In Tcl, a command enclosed in square brackets [ ] is executed and replaced by its result.
- The substituted result is then used as an argument to the outer command.

1. [command]

- The command inside [ ] is executed first.
- Its output (result) replaces the bracketed command.
- Useful to use the output of one command as input to another command.

Example :

```
set date [clock format [clock seconds]]
puts "Current date : $date"

puts "Current directory : [pwd]"
puts "Tcl version : [info patchlevel]"
```

Output :

```
Current date : Thu Sep  4 22:30:15 IST 2025
Current directory : /home/user
Tcl version : 8.6.13
```

2. Nested command substitution

- A command inside [ ] can itself contain another [command].
- The inner command is executed first, then its result is used in the outer command.
- Useful for performing multi-level operations.

Example :

```
set dir_count [llength [glob -directory . *]]
puts "Number of items in current directory : $dir_count"

set today [clock format [clock seconds] -format "%Y-%m-%d"]
puts "Today is : $today"
```

Output :

```
Number of items in current directory : 5
Today is : 2025-09-04
```

3. Command result handling

- The result of a command substitution can be assigned to a variable, used inside expressions, or combined with text.
- The substituted result is treated as a string.
- Trailing newlines in the command output are removed automatically.
- If the command returns multiple lines, the entire result is inserted (can be used with further processing)

Example 1 : Assigning command result to a variable

```
set user [whoami]
puts "Logged in user : $user"

set files [glob *.tcl]
puts "Tcl files : $files"
```

Output :

```
Logged in user : user
Tcl files : script.tcl test.tcl util.tcl
```

Example 2 : Using command result in expression

```
set count [llength [glob *.tcl]]
set next [expr {$count + 1}]
puts "Number of .tcl files : $count"
puts "Next value : $next"
```

Output :

```
Number of .tcl files : 3
Next value : 4
```

Example 3 : Combining with text

```
puts "There are [llength [glob *.tcl]] Tcl files
    in [pwd]"
puts "Today: [clock format [clock seconds] -format {%A}]"
```

Output :

```
There are 3 Tcl files in /home/user
Today: Thursday
```

Key Point

- ☐ [command] executes the command and substitutes its result.
- ☐ Nested [command] is allowed (inner first, then outer).
- ☐ Command results can be stored in variables or used directly.
- ☐ Substitution result is a string (trailing newline removed).
- ☐ Useful for dynamic scripting and automation in Tcl/ICC2.

TCL Scripting

2. VARIABLES, SUBSTITUTION & EXPRESSIONS

2.5 Braces { }

- Braces { } are used to group text as a single argument.
- Content inside braces is treated literally (no substitution).
- Braces are widely used in Tcl for procedure definitions, control structures and lists.
- Useful to prevent unwanted variable or command substitution.

1. { }

- Text inside { } is taken literally.
- No variable substitution, no command substitution, and no backslash substitution.
- Helps in passing multiple words as a single argument.

Example :

```
set msg {Hello World}
puts $msg
puts {Today is $date}      # no substitution
puts {[expr 1+2]}          # no command substitution
```

Output :

```
Hello World
Today is $date
[expr 1+2]
```

2. Preventing substitution

- Use braces when you want to disable variable, command and backslash substitution.
- Useful in print messages, comments, and procedure bodies.

Example :

```
set name "Tcl"
puts "Name = $name"        # substitution happens
puts {Name = $name}        # substitution prevented
puts {Path = C:\\new\\test} # backslashes are literal
```

Output :

```
Name = Tcl
Name = $name
Path = C:\\new\\test
```

3. Grouping arguments

- Braces can be used to group multiple words into a single argument for a command.
- Useful when the argument contains spaces.

Example :

```
puts {This is a single argument}
set list {a b c d}
puts $list
```

Output :

```
This is a single argument
a b c d
```

4. Braces vs Quotes

- Braces { } → no substitution (literal text).
- Quotes " " → allow substitution (variables, commands, backslashes).
- Use braces when you want exact text. Use quotes when you want substitution.

Feature	Braces { }	Quotes " "
Substitution	No substitution	Allows variable, command and backslash substitution
Grouping	Groups multiple words	Also groups multiple words
Use case	Exact / literal text	Text with variable values
Performance	Slightly faster	Slightly slower (due to substitution)
Example	puts {Hello $name}	puts "Hello $name"
Output	Hello $name	Hello Tcl (if name = Tcl)

More Examples :

Example 1 : Using braces to store a script

```
set script {
    set a 10
    set b 20
    puts "Sum = [expr {$a + $b}]"
}
eval $script  # execute the script stored in variable
```

Output :

```
Sum = 30
```

Example 2 : Comparing braces and quotes

```
set name "VLSI"
puts {Project: $name}  # no substitution
puts "Project: $name"  # substitution
puts {Dir: [pwd]}  # no command substitution
puts "Dir: [pwd]"  # command substitution
```

Output :

```
Project: $name
Project: VLSI
Dir: [pwd]
Dir: /home/user
```

Example 3 : Passing multiple arguments

```
proc show_msg {msg} {
    puts "Message: $msg"
}

show_msg {Hello from Tcl scripting!}
```

Output :

```
Message: Hello from Tcl scripting!
```

Key Point

- ☐ Braces { } prevent variable, command and backslash substitution.
- ☐ Braces are used to group multiple words as a single argument.
- ☐ Quotes " " allow substitution.
- ☐ Use braces for exact text and quotes when you need variable values.
- ☐ Braces are commonly used in procedures, control statements, and lists.
- ☐ Choose braces or quotes based on whether substitution is needed.

Author : Konda Tharun Vivekananda

2. VARIABLES, SUBSTITUTION & EXPRESSIONS

2.6 Quotes " "

- Quotes " " are used to enclose a string in Tcl.
- Unlike braces { }, content inside quotes is subject to substitution (variables and commands are evaluated).
- Quotes allow easier creation of text with variables and command results.
- Useful for messages, formatted strings and user output.

1. " " (Double Quotes)

- Text inside " " is a string with substitution enabled.
- Variables ($) and commands ([ ]) inside quotes are replaced with their values before execution.
- Backslashes (\\ ) can be used for special characters.
- Quotes are commonly used with puts, set, and other commands that expect a string.

Example : Basic usage of quotes

```
set name "Tcl"
set msg "Hello $name"
puts $msg                          # variable substitution
puts "Welcome to Tcl scripting" # simple string
```

Output :

```
Hello Tcl
Welcome to Tcl scripting
```

2. Variable substitution inside quotes

- Variables referenced with $ are substituted with their values inside quotes.
- You can use multiple variables in the same quoted string.
- Useful for creating messages and formatted output.

Example : Using variables inside quotes

```
set tool "ICC2"
set version "2025.06"
puts "Tool = $tool"
puts "Version = $version"
puts "Tool $tool Version $version"
```

Output :

```
Tool = ICC2
Version = 2025.06
Tool ICC2 Version 2025.06
```

3. Command substitution inside quotes

- Commands enclosed in [ ] are executed inside quotes and replaced with their results.
- Useful for including dynamic information such as date, file names, or command outputs.
- You can combine variables and commands inside the same quoted string.

Example 1 : Using command substitution inside quotes

```
puts "Current date is: [date]"
puts "Current directory is: [pwd]"
puts "Number of files: [llength [glob *.tcl]]"
puts "Tool: $tool  Date: [clock format [clock seconds]]"
```

Output :

```
Current date is: Thu Sep  4 22:30:15 IST 2025
Current directory is: /home/user
Number of files: 3
Tool: ICC2  Date: 2025-09-04 22:30:15
```

4. Combining variable and command substitution

- Both variable substitution and command substitution can be used together inside quotes.
- The command is executed first, then the result is combined with variable values.

Example 2 : Combining variables and commands

```
set user "Vivekananda"
set file_count [llength [glob *.tcl]]
puts "User: $user"
puts "Total .tcl files: $file_count"
puts "Report generated at: [clock format [clock seconds]]"
puts "User $user has $file_count Tcl files in [pwd]"
```

Output :

```
User: Vivekananda
Total .tcl files: 3
Report generated at: 2025-09-04 22:30:15
User Vivekananda has 3 Tcl files in /home/user
```

Key Point

- ☐ Use " " to create strings with substitution.
- ☐ Variables inside quotes are substituted with their values.
- ☐ Commands inside [ ] are executed inside quotes.
- ☐ You can combine variables and commands in the same string.
- ☐ Use quotes for readable messages and formatted output.
- ☐ If no substitution is needed, prefer braces { }.

2. VARIABLES, SUBSTITUTION & EXPRESSIONS

2.7 Escape Characters

- Escape characters in Tcl are used with backslash (\\ ) to represent special characters.
- They are mainly used inside double quotes " ".
- Common escape sequences include \n (newline), \t (tab), etc.
- Useful for formatting output, file paths, and including special characters literally.

1. Backslash (\ )

- Backslash is the escape character in Tcl.
- Used to escape special characters or to include them literally.
- \\ can also be used to continue a command to the next line.

Example :

```
puts "This is a backslash: \\"
puts "This is a double quote: \" "
puts "This is a dollar sign: \$ "
```

Output :

```
This is a backslash: \
This is a double quote: "
This is a dollar sign: $
```

2. Newline (\n)

- \n inserts a new line in the output.
- Useful for printing multi-line messages.

Example :

```
puts "Line1\nLine2\nLine3"
puts "Hello\nTcl\nScripting"
```

Output :

```
Line1
Line2
Line3

Hello
Tcl
Scripting
```

3. Tab (\t)

- \t inserts a horizontal tab space.
- Useful for column formatting in output.

Example :

```
puts "Name\tAge\tCity"
puts "Giri\t24\tBangalore"
```

Output :

```
Name    Age    City
Giri    24     Bangalore
```

4. Carriage return (\r)

- \r moves the cursor to the beginning of the line.
- Often used to overwrite the same line (e.g., progress messages).

Example :

```
puts -nonewline "Loading..."
after 1000
puts -nonewline "\rDone!    "
```

Output :

```
Done!    # (overwrites "Loading...")
```

5. Escaping special characters

- Special characters like $, [ ], { }, " can be escaped using \ when you want to use them literally without triggering substitution.
- Useful when printing strings that contain Tcl syntax.

Example :

```
puts "Literal dollar: \$var"
puts "Literal brackets: \[date\]"
puts "Literal braces: \{not a block\}"
puts "Literal quote: \"Hello\""
```

Output :

```
Literal dollar: $var
Literal brackets: [date]
Literal braces: {not a block}
Literal quote: "Hello"
```

6. Using backslash for line continuation

- A backslash at the end of a line continues the command on the next line.
- Useful for long commands.

Example :

```
set msg "This is a very long message \
that continues on the next line"
puts $msg
```

Output :

```
This is a very long message that continues on the next line
```

7. Common Escape Sequences in Tcl

Escape Sequence	Meaning	Example	Output
\\	Backslash	puts "\\"	\
\n	New line	puts "A\nB"	A (newline) B
\t	Tab space	puts "A\tB"	A B
\r	Carriage return	puts "A\rB"	B (overwrites A)
\"	Double quote	puts "\""	"
\$	Dollar sign	puts "\$var"	$var
\[	Left bracket	puts "\[date\]"	[date]
\]	Right bracket	puts "\]"	]
\{	Left brace	puts "\{"	{
\}	Right brace	puts "\}"	}

Key Points

- ☐ Backslash (\ ) is the escape character in Tcl.
- ☐ \n → new line, \t → tab, \r → carriage return.
- ☐ Use \ to print special characters like $, [ ], { }, " literally.
- ☐ Escape sequences are mainly used inside double quotes.
- ☐ Useful for formatted output, file paths, and long commands.

Author : Konda Tharun Vivekananda

2. VARIABLES, SUBSTITUTION & EXPRESSIONS

2.8 Mathematical Expressions

- Tcl uses the expr command to evaluate mathematical expressions
- Supports integer and floating-point arithmetic
- Follows standard operator precedence rules
- Use parentheses ( ) to change the order of evaluation
- Useful for calculations, comparisons and in scripts for automation

1. expr

- expr is used to evaluate a mathematical expression
- The expression can contain numbers, variables, operators and parentheses.
- The result of expr is a number (integer or floating point)

Example :

```
set a 10
set b 5
set sum [expr $a + $b]
puts "Sum = $sum"
```

Output :

```
Sum = 15
```

2. Integer arithmetic

- By default, if all operands are integers, the result is an integer.
- Supports + , - , * , / , % (modulo).

Example :

```
set a 15
set b 4
puts "Addition       : [expr $a + $b]"
puts "Subtraction    : [expr $a - $b]"
puts "Multiplication : [expr $a * $b]"
puts "Division       : [expr $a / $b]"
puts "Modulo         : [expr $a % $b]"
```

Output :

```
Addition       : 19
Subtraction    : 11
Multiplication : 60
Division       : 3
Modulo         : 3
```

3. Floating-point arithmetic

- If any operand is a floating-point number, the result is a floating-point number.
- Useful for accurate calculations

Example :

```
set x 5.0
set y 2
puts "Result = [expr $x / $y]"
puts "Pi value = [expr 3.1415 * $y]"
puts "Average = [expr ($x + $y) / 2.0]"
```

Output :

```
Result = 2.5
Pi value = 6.283
Average = 3.5
```

Note : Use a floating-point number (e.g., 2.0) to get floating-point result.

4. Operator precedence

- Operators in expr are evaluated based on precedence.
- Multiply / divide / modulo have higher precedence than add / subtract.
- Use parentheses to make the order explicit.

Precedence	Operators	Description
1	( )	Parentheses (highest)
2	* / %	Multiply, Divide, Modulo
3	+ -	Add, Subtract (lowest)

Example :

```
set a 10
set b 5
puts "2 + 3 * 4 = [expr 2 + 3 * 4]"
puts "(2 + 3) * 4 = [expr (2 + 3) * 4]"
puts "a + b * 2 = [expr $a + $b * 2]"
puts "(a + b) * 2 = [expr ($a + $b) * 2]"
```

Output :

```
2 + 3 * 4 = 14
(2 + 3) * 4 = 20
a + b * 2 = 20
(a + b) * 2 = 30
```

5. Parentheses ( )

- Use parentheses to control the order of evaluation.
- Expressions inside ( ) are evaluated first.
- Helps avoid ambiguity and makes the expression clear.

Example :

```
set x 8
set y 4
puts "x / y + 2 = [expr $x / $y + 2]"
puts "x / (y + 2) = [expr $x / ($y + 2)]"
puts "(x + y) / 2 = [expr ($x + $y) / 2]"
```

Output :

```
x / y + 2 = 4
x / (y + 2) = 1
(x + y) / 2 = 6
```

Key Points

- ☐ Use expr to evaluate mathematical expressions.
- ☐ Supports integer and floating-point arithmetic.
- ☐ Operators: + , - , * , / , % (follow standard precedence).
- ☐ Use parentheses ( ) to change the order of evaluation.
- ☐ If any operand is floating-point, result is floating-point.
- ☐ Useful for calculations in scripts and tool automation (e.g., ICC2)

Extra Example :

```
set length 12.5
set width 8.0
set area [expr $length * $width]
puts "Area = $area"
puts "Perimeter = [expr 2 * ($length + $width)]"
```

Output :

```
Area = 100.0
Perimeter = 41.0
```

Author : Konda Tharun Vivekananda

2. VARIABLES, SUBSTITUTION & EXPRESSIONS

2.9 Operators

- Tcl supports various operators in expr for calculations and comparisons.
- Operators are used to perform arithmetic, comparison, logical and boolean operations.
- Assignment in Tcl is done using the set command (or incr for increment).
- Useful for controlling flow in scripts (if, while, etc).

1. Arithmetic operators

- Used to perform mathematical operations.
- Work with both integer and floating-point numbers.

Operator	Meaning	Example	Result
+	Addition	[expr 5 + 3]	8
-	Subtraction	[expr 5 - 3]	2
*	Multiplication	[expr 5 * 3]	15
/	Division	[expr 10 / 4]	2.5
%	Modulo (remainder)	[expr 10 % 3]	1

Example :

```
set a 10
set b 4
puts "Sum      : [expr $a + $b]"
puts "Diff     : [expr $a - $b]"
puts "Product  : [expr $a * $b]"
puts "Division : [expr $a / $b]"
puts "Modulo   : [expr $a % $b]"
```

Output :

```
Sum      : 14
Diff     : 6
Product  : 40
Division : 2.5
Modulo   : 2
```

2. Comparison operators

- Used to compare two values.
- Return 1 (true) or 0 (false).

Operator	Meaning	Example	Result
==	Equal to	[expr 5 == 5]	1
!=	Not equal to	[expr 5 != 3]	1
>	Greater than	[expr 5 > 3]	1
<	Less than	[expr 5 < 3]	0
>=	Greater or equal	[expr 5 >= 5]	1
<=	Less or equal	[expr 5 <= 4]	0

Example :

```
set a 10
set b 20
puts "a == b : [expr $a == $b]"
puts "a < b  : [expr $a < $b]"
puts "a >= 10 : [expr $a >= 10]"
```

Output :

```
a == b  : 0
a < b   : 1
a >= 10 : 1
```

3. Logical operators

- Used to combine multiple conditions.
- Return 1 (true) or 0 (false).

Operator	Meaning	Example	Result
&&	Logical AND	[expr {1 && 0}]	0
||	Logical OR	[expr {1 || 0}]	1
!	Logical NOT	[expr {!0}]	1

Example :

```
set a 1
set b 0
puts "a && b : [expr $a && $b]"
puts "a || b : [expr $a || $b]"
puts "!b     : [expr !$b]"
```

Output :

```
a && b : 0
a || b : 1
!b     : 1
```

4. Boolean operators

- In Tcl, boolean values are integers (0 = false, 1 = true).
- Any non-zero value is considered true.
- Boolean operators are the same as logical operators and are mainly used in conditional expressions.

Example :

```
set flag 1
if {$flag} {
    puts "Flag is true"
} else {
    puts "Flag is false"
}
```

Output :

```
Flag is true
```

5. Assignment concepts

- In Tcl, variables are assigned using the set command.
- You can update values and also use incr / lappend for increment and list assignment.

Example :

```
set count 10
puts "Count = $count"
set count 20        # update value
puts "Updated count = $count"
incr count          # increment by 1
puts "After incr = $count"
```

Output :

```
Count = 10
Updated count = 20
After incr = 21
```

Key Points

- ☐ Use expr to perform arithmetic, comparison and logical operations.
- ☐ Comparison and logical operators return 1 (true) or 0 (false).
- ☐ Boolean values in Tcl are integers (0 or 1).
- ☐ Variable assignment is done using set (update with set or incr).
- ☐ Operators are widely used in if, while, and other control statements.

2. VARIABLES, SUBSTITUTION & EXPRESSIONS

2.10 EDA-Oriented Calculations

- Tcl expressions are widely used in EDA flow for calculations such as area, utilization, delay, slack, percentage, density and coordinate operations.
- Helps in report parsing, constraint generation and automation.
- Use expr with variables from tool reports (ICC2, PrimeTime, etc.).
- Units should be handled carefully (e.g., um, ns, %).

1. Area Calculation

- Area can be calculated using length × width.
- Commonly used for core area, cell area, blockage area etc.

Example :

```
set length 100
set width 50
set area [expr $length * $width]
puts "Area = $area um^2"
```

Output :

```
Area = 5000 um^2
```

2. Utilization Calculation

- Utilization = (Cell Area / Core Area) × 100
- Used to check how much of the core is occupied by cells.

Example :

```
set cell_area 75000
set core_area 200000
set util [expr ($cell_area / $core_area) * 100]
puts "Utilization = $util %"
```

Output :

```
Utilization = 37.5 %
```

3. Delay Calculation

- Delay can be calculated as the difference between arrival time and required time.
- Useful for custom checks and report analysis.

Example :

```
set arrival  12.5
set required 10.0
set delay [expr $arrival - $required]
puts "Delay = $delay ns"
```

Output :

```
Delay = 2.5 ns
```

4. Slack Calculation

- Slack = Required Time − Arrival Time.
- Positive slack means timing is met, negative means violation.

Example :

```
set arrival  9.8
set required 10.0
set slack [expr $required - $arrival]
puts "Slack = $slack ns"
```

Output :

```
Slack = 0.2 ns
```

5. Percentage Calculation

- Useful for finding percentage change, improvement, etc.
- Formula: (Value / Total) × 100.

Example :

```
set value 25
set total 80
set percent [expr ($value / $total) * 100]
puts "Percentage = $percent %"
```

Output :

```
Percentage = 31.25 %
```

6. Density Calculation

- Density can be calculated as (Number of objects / Area).
- Used for cell density, pin density, via density etc.

Example :

```
set num_cells 5000
set core_area 200000
set density [expr $num_cells / $core_area]
puts "Cell density = $density cells/um^2"
```

Output :

```
Cell density =
0.025 cells/um^2
```

7. Coordinate Calculations

- Used to compute distances, mid-points, offsets etc.
- Useful in floorplanning, placement and routing automation.

a) Distance between two points

Example :

```
set x1 10
set y1 20
set x2 40
set y2 60
set dist [expr sqrt(($x2 - $x1)**2 + ($y2 - $y1)**2)]
puts "Distance = $dist um"
```

Output :

```
Distance =
56.57 um
```

b) Mid-point of two points

Example :

```
set x1 10
set y1 20
set x2 40
set y2 60
set x_mid [expr ($x1 + $x2) / 2.0]
set y_mid [expr ($y1 + $y2) / 2.0]
puts "Mid-point = ($x_mid, $y_mid)."
```

Output :

```
Mid-point =
(25.0, 40.0)
```

Key Points

- ☐ Use expr for all mathematical calculations in Tcl.
- ☐ Common EDA calculations: area, utilization, delay, slack, %, density and coordinates.
- ☐ Handle units properly (um, ns, %).
- ☐ Useful for report parsing, constraint generation and automation.
- ☐ Can be extended for more complex calculations (e.g., angles, scaling).

3. STRINGS & LISTS

3.1 String Fundamentals

- In Tcl, a string is a sequence of characters.
- Strings can be created using quotes " ", braces { }, or by assigning the result of a command.
- Strings are the basic data type in Tcl.
- Useful for messages, file names, designs, reports, etc.

1. Creating strings

- Strings can be created using double quotes, braces or direct assignment.
- Braces prevent substitution, while quotes allow substitution.

Example : Creating strings in different ways

```
set s1 "Hello World"          # using quotes
set s2 {Tcl Scripting}        # using braces
set s3 Tcl                    # direct assignment
set s4 [format "Design: %s" "CPU"] # using command
```

Output :

```
s1 = Hello World
s2 = Tcl Scripting
s3 = Tcl
s4 = Design: CPU
```

2. String representation

- Strings can contain letters, numbers, special characters and spaces.
- Tcl does not have a separate character type (single characters are also strings).

Example : String representation

```
set name "VLSI Design"
set path "/home/user/my block"
set msg "Count = 10, Status = OK!"
set ch "A"
```

Output :

```
name = VLSI Design
path = /home/user/my block
msg  = Count = 10, Status = OK!
ch   = A
```

3. String length

- The string length command returns the number of characters in a string.
- Useful to check the size of a string.

Example : Using string length

```
set s "Tcl Scripting"
puts "Length = [string length $s]"
```

Output :

```
Length = 14
```

4. String comparison

- Strings can be compared using string compare or the shortcut operators eq, ne, etc.
- Comparison is case sensitive.
- Useful for checking names, options, tool statuses, etc.

Example : Comparing strings

```
set a "hello"
set b "hello"
set c "Hello"

puts "a eq b : [string compare $a $b]"
puts "a eq c : [string compare $a $c]"

if { $a eq $b } {
    puts "Strings are equal"
}
```

Output :

```
a eq b : 0
a eq c : 1
Strings are equal
```

Note : string compare returns
- 0 → strings are equal
- < 0 → first string is less (comes before)
- > 0 → first string is greater (comes after)

5. String indexing

- Use string index to get a character at a specific position.
- Indexing starts from 0.
- You can also extract a range of characters using string range.

Example : String indexing and range

```
set s "TclScripting"
puts "First char : [string index $s 0]"
puts "Char at 3 : [string index $s 3]"
puts "Last char : [string index $s end]"
puts "Range 0-2 : [string range $s 0 2]"
puts "Range 3-end: [string range $s 3 end]"
```

Output :

```
First char : T
Char at 3 : S
Last char : g
Range 0-2 : Tcl
Range 3-end: Scripting
```

Key Points

- ☐ In Tcl, everything is a string.
- ☐ Create strings using quotes, braces or direct assignment.
- ☐ Use string length to find the number of characters.
- ☐ Compare strings using string compare or eq, ne, etc.
- ☐ Use string index and string range for character access.
- ☐ String comparison is case sensitive.
- ☐ Useful in EDA scripts for handling names, paths, reports, etc.

3. STRINGS & LISTS

3.2 String Commands

- Tcl provides a rich set of string commands to manipulate strings.
- These commands are widely used in EDA flows for parsing reports, handling file names, tool outputs, and option processing.
- String commands are case sensitive.
- General format : string <subcommand> <args>

1. string length

- Returns the number of characters in a string.
- Spaces and special characters are also counted.

Syntax : string length $string

Example :

```
set s "Tcl Scripting"
puts [string length $s]
```

Output :

```
13
```

Note :
- Counts all characters including spaces.
- Useful to check file name length, report field length etc.

2. string index

- Returns the character at a specific index (0 based).
- If index is out of range, returns an empty string.

Syntax : string index $string $index

Example :

```
set s "VLSI"
puts [string index $s 0]
puts [string index $s 2]
puts [string index $s 3]
```

Output :

```
V
S
I
```

Note :
- Index starts from 0.
- Useful for extracting specific characters (e.g., file extensions).

3. string range

- Returns a substring from start index to end index (both inclusive).
- If end index is omitted, returns till end of string.

Syntax : string range $string $start $end

Example :

```
set s "Physical Design"
puts [string range $s 0 7]
puts [string range $s 9 end]
puts [string range $s 3]
```

Output :

```
Physical
Design
Design
```

Note :
- Useful to extract substrings like tool name, path, extension etc.

4. string first

- Returns the index of the first occurrence of a substring.
- If not found, returns -1.

Syntax : string first $pattern $string

Example :

```
set s "VLSI Design Flow"
puts [string first "Design" $s]
puts [string first "Chip" $s]
```

Output :

```
5
-1
```

Note :
- Useful to check if a particular word exists in a string.

5. string last

- Returns the index of the last occurrence of a substring.
- If not found, returns -1.

Syntax : string last $pattern $string

Example :

```
set s "path/to/design.v"
puts [string last "/" $s]
puts [string last "o" $s]
```

Output :

```
8
4
```

Note :
- Useful to find last occurrence (e.g., last "/" in a path).

6. string compare

- Compares two strings lexicographically.
- Returns 0 if equal, < 0 if first string is smaller, > 0 if first string is greater.

Syntax : string compare $str1 $str2

Example :

```
puts [string compare "abc" "abc"]
puts [string compare "abc" "abd"]
puts [string compare "xyz" "abc"]
```

Output :

```
0
-1
1
```

Note :
- Useful for sorting, conditional checks and text comparisons.

7. string equal

- Checks if two strings are exactly equal.
- Returns 1 if equal, 0 otherwise.
- Case sensitive.

Syntax : string equal $str1 $str2

Example :

```
puts [string equal "Tcl" "Tcl"]
puts [string equal "Tcl" "tcl"]
```

Output :

```
1
0
```

Note :
- Use when you just want to check equality (alternative to [expr {$a eq $b}])

8. string match

- Performs pattern matching using wildcards.
- Returns 1 if the string matches the pattern, 0 otherwise.
- Wildcards: * (any string), ? (single character), [ ] (character set).

Syntax : string match $pattern $string

Example :

```
puts [string match "*.v" "design.v"]
puts [string match "*.v" "design.sv"]
puts [string match "d?sign.*" "design.v"]
puts [string match "[Dd]esign*" "Design_top"]
```

Output :

```
1
0
1
1
```

Note :
- Very useful for filtering file names, tool outputs and log messages.

Key Points

- ☐ Tcl provides powerful string commands for text manipulation.
- ☐ Indexing starts from 0 and ranges are inclusive.
- ☐ string compare and string equal are case sensitive.
- ☐ string match supports wildcards (*, ?, [ ]).
- ☐ Widely used in EDA scripts for parsing reports, file handling, and automation.

3. STRINGS & LISTS

3.3 String Manipulation

- Tcl provides several string manipulation commands to modify and process strings.
- These commands are very useful in EDA scripts for handling file names, paths, tool outputs, and text processing.
- String manipulation helps in cleaning, formatting and transforming data.
- General format : string <subcommand> <args>

1. string map

- Performs multiple substring replacements in a string.
- Takes a list of key-value pairs and replaces all occurrences of keys with corresponding values.
- Useful for quickly replacing multiple patterns.

Syntax : string map {key1 value1 key2 value2 ...} $string

Example :

```
set s "VLSI Design Flow"
set result [string map {VLSI ASIC Design Flow EDA} $s]
puts $result
```

Output :

ASIC EDA

Note :
- All replacements are done simultaneously.
- Useful when multiple substitutions are required.

2. string replace

- Replaces a portion of the string specified by start and end index with a new string.
- If end index is omitted, replaces till the end of the string.
- Useful for modifying specific parts of a string.

Syntax : string replace $string $start $end $newString

Example :

```
set s "Physical Design"
set r1 [string replace $s 0 7 "Logical"]
puts $r1
set r2 [string replace $s 9 end "Verification"]
puts $r2
```

Output :

Logical Design

Physical Verification

Note :
- Indexing starts from 0.
- End index is inclusive.

3. string trim

- Removes leading and trailing whitespace characters (spaces, tabs, newlines).
- Useful when reading inputs from files or user input.

Syntax : string trim $string

Example :

```
set s "  VLSI Design  "
puts "[$s]"
puts "[string trim $s]"
```

Output :

```
[  VLSI Design  ]
[VLSI Design]
```

Note :
- Removes only leading and trailing whitespace.
- Does not affect spaces in the middle of the string.

4. string trimleft

- Removes only the leading (left) whitespace characters.
- Useful when you want to keep trailing spaces intact.

Syntax : string trimleft $string

Example :

```
set s "  Hello World  "
puts "[$s]"
puts "[string trimleft $s]"
```

Output :

```
[  Hello World  ]
[Hello World  ]
```

Note :
- Only leading spaces are removed.
- Trailing spaces remain.

5. string trimright

- Removes only the trailing (right) whitespace characters.
- Useful when you want to keep leading spaces intact.

Syntax : string trimright $string

Example :

```
set s "  Hello World  "
puts "[$s]"
puts "[string trimright $s]"
```

Output :

```
[  Hello World  ]
[  Hello World]
```

Note :
- Only trailing spaces are removed.
- Leading spaces remain.

6. string tolower

- Converts all characters in the string to lowercase.
- Useful for case-insensitive comparisons and normalization.

Syntax : string tolower $string

Example :

```
set s "VLSI DeSign FLOW"
puts [string tolower $s]
```

Output :

vlsi design flow

Note :
- All alphabetic characters are converted to lowercase.
- Non-alphabetic characters remain unchanged.

7. string toupper

- Converts all characters in the string to uppercase.
- Useful for standardizing text and comparisons.

Syntax : string toupper $string

Example :

```
set s "VLSI DeSign Flow"
puts [string toupper $s]
```

Output :

VLSI DESIGN FLOW

Note :
- All alphabetic characters are converted to uppercase.
- Non-alphabetic characters remain unchanged.

Key Points

- ☐ string map → multiple replacements
- ☐ string replace → replace specific portion using index
- ☐ string trim/trimleft/trimright → remove whitespace
- ☐ string tolower/toupper → convert case
- ☐ Very useful for parsing tool outputs, handling file names and text cleaning in EDA scripts.

3. STRINGS & LISTS

3.4 String Searching

- Searching is one of the most common string operations in Tcl scripts.
- Tcl provides several commands to find substrings and match patterns.
- Useful in EDA flows for parsing file names, logs, reports and tool outputs.
- We can search for a specific substring, a pattern using wildcards, or check prefixes and suffixes.

☑ 1. Finding substrings

- Use string first to find the index of a substring.
- Returns the index of the first occurrence (0 based).
- If not found, returns -1.

Syntax : `string first $substr $string`

Example :

```
set s "VLSI Design Flow"
puts [string first "Design" $s]
puts [string first "Chip" $s]
```

Output :

```
5
-1
```

Note :
- Index starts from 0.
- Useful to check if a word exists in a string

☑ 2. Pattern matching

- Use string match to check if a string matches a pattern.
- Supports wildcards: * (any number of characters), ? (single character), [ ] (character set).
- Returns 1 if match is found, 0 otherwise.

Syntax : `string match $pattern $string`

Example :

```
puts [string match "*Design*" "VLSI Design"]
puts [string match "VLSI*" "VLSI Design"]
puts [string match "*chip?" "mychip1"]
puts [string match "*chip?" "mychip12"]
```

Output :

```
1
1
1
0
```

Note :
- Matching is case sensitive.
- Useful for filtering lists, files and reports.

☑ 3. Regular expression search

- Use string first with -regexp or regexp command for advanced pattern searching.
- More powerful than simple wildcard matching.
- Useful when the pattern is complex.

Syntax : `string first -regexp $pattern $string`

Example :

```
set s "cell_123, cell_456"
puts [string first -regexp {cell_[0-9]+} $s]
puts [regexp {cell_[0-9]+} $s match]
```

Output :

```
0
cell_123
```

Note :
- Regular expressions help extract patterns like numbers,
- file names, etc.

☑ 4. Prefix matching

- A prefix is a substring at the beginning of a string.
- Use string match with pattern at the start or check with string range.
- Commonly used to identify file types, command options, etc.

Method 1: : Using string match

Syntax : `string match "$prefix*" $string`

Example :

```
puts [string match "VLSI*" "VLSI_Design"]
puts [string match "AMD*" "AMD_chip"]
```

Output :

```
1
1
```

Method 2: : Using string range

Syntax : `string range $string 0 [expr {[string length $prefix]-1}]`

Example :

```
set s "VLSI_Design"
puts [string range $s 0 3]
```

Output :

```
VLSI
```

Note :
- Prefix matching is useful to filter commands, file names,
- or check tool output categories.

☑ 5. Suffix matching

- A suffix is a substring at the end of a string.
- Use string match with pattern ending in * or check with string range.
- Useful to identify file extensions (.v, .tcl, .log, etc.).

Method 1: : Using string match

Syntax : `string match "*$suffix" $string`

Example :

```
puts [string match "*.v" "design.v"]
puts [string match "*.log" "run.log"]
puts [string match "*.tcl" "script.txt"]
```

Output :

```
1
1
0
```

Method 2: : Using string range

Syntax : `string range $string [expr {[string length $string]-[string length $suffix]}] end`

Example :

```
set s "design.v"
puts [string range $s [expr {[string length $s]-2}] end]
```

Output :

```
.v
```

Note :
- Suffix matching is widely used to process files based on
- their extensions.

Key Points

- ☐ string first → find the index of a substring.
- ☐ string match → pattern matching using wildcards.
- ☐ string first -regexp / regexp → advanced searching with regular expressions.
- ☐ Prefix matching → use pattern at the beginning (e.g., "abc*").
- ☐ Suffix matching → use pattern at the end (e.g., "*.v").
- ☐ String searching is essential for parsing tool outputs, file names and reports in EDA automation scripts.

3. STRINGS & LISTS

3.5 String Concatenation

- String concatenation means joining two or more strings to create a new string.
- Tcl provides multiple ways to combine strings.
- Useful for building dynamic messages, command arguments, file paths and tool scripts.
- Tcl strings are flexible and can be created at run time using variables, commands and user input.

1. Combining strings

- The easiest way to concatenate strings is to use spaces between them or use the concat command.
- Useful for simple message construction.

Syntax 1 : Using spaces

```
set str "Hello" "World"
puts $str
```

Output :
```
Hello World
```

Syntax 2 : Using concat command

```
set str [concat "Hello" " " "Tcl"]
puts $str
```

Output :
```
Hello Tcl
```

Note :
- Spaces between quoted strings automatically concatenate.
- concat is useful when combining many strings or variables.

2. Dynamic strings

- Strings can be built dynamically using variables, commands and user input.
- Useful in scripts where values change based on conditions or runtime data.

Syntax : Using variables and commands

```
set design "mychip"
set block "top"
set msg "Design: $design Block: $block"
puts $smg
```

Output :
```
Design: mychip
Block: top
```

Example : Using command substitution

```
set date_str [clock format [clock seconds]
        -format "%Y-%m-%d"]
set msg "Run started on $date_str"
puts $msg
```

Output :
```
Run started on
2024-08-21
```

Note :
- Use $variable to insert variable values.
- Use [command] to insert command output.
- Allows scripts to adapt to different scenarios.

3. Building command arguments

- Strings are often concatenated to build tool commands with options and arguments.
- Helps in creating flexible and reusable scripts.

Syntax : Constructing a command string

```
set cmd "report_area -design $design -block $block"
puts $cmd
```

Output :
```
report_area -design
mychip -block top
```

Note :
- Useful when command options are stored in variables.
- Can be extended with conditional arguments.

Example : Adding optional arguments

```
set cmd "report_timing -delay_type max"
set cmd "$cmd -nworst 10"
puts $cmd
```

Output :
```
report_timing
-delay_type max
-nworst 10
```

Note :
- Build the base command first and append options.
- Helps in creating configurable scripts.

4. Building file paths

- Strings are widely used to construct file and directory paths.
- Useful for reading/writing reports, logs and constraint files.
- Use forward slashes (/) in Tcl (works on both Windows and Unix).

Syntax 1 : Simple concatenation

```
set dir "/home/user/project"
set file "report.tcl"
set path "$dir/$file"
puts $path
```

Output :
```
/home/user/project
/report.tcl
```

Syntax 2 : Using file join (recommended)

```
set path [file join "/home" "user" "project"
        "report.tcl"]
puts $path
```

Output :
```
/home/user/project
/report.tcl
```

Note :
- file join is safer and handles different platforms.
- Avoid hardcoding paths; use variables for flexibility.

5. Practical examples

a) Creating log file name with timestamp

```
set log_dir "logs"
set timestamp [clock format [clock seconds] -format "%Y%m%d_%H%M%S"]
set log_file [file join $log_dir "run_$timestamp.log"]
puts $log_file
```

Output :
```
logs/run_20240821_153045.log
```

b) Building EDA tool command

```
set design "mychip"
set corner "ss"
set cmd "run_sta -design $design -corner $corner"
puts $cmd
```

Output :
```
run_sta -design mychip
-corner ss
```

c) Combining multiple strings

```
set part1 "VLSI"
set part2 "Design"
set part3 "Flow"
set full_str "$part1 $part2 $part3"
puts $full_str
```

Output :
```
VLSI Design Flow
```

Key Points

- ☐ Strings can be concatenated using spaces or the concat command.
- ☐ Variables and command outputs can be embedded inside strings.
- ☐ Useful for building dynamic messages, command arguments and file paths.
- ☐ file join is recommended for constructing file paths.
- ☐ String concatenation makes scripts flexible, reusable and easier to maintain.

3. STRINGS & LISTS

3.6 List Fundamentals

- A list in Tcl is an ordered collection of elements (strings).
- Lists are one of the most powerful features in Tcl.
- Lists can hold numbers, strings or even other lists.
- Lists are widely used in EDA scripts for file lists, options, command arguments and data processing.
- Tcl automatically handles spaces and special characters in list elements.

1. What is a Tcl list ?

- A list is a collection of elements separated by spaces.
- Each element can be a string, number or another list.
- Lists preserve the boundaries of elements, even if they contain spaces, when created using braces {} or quotes "".
- You can think of a list as an array of strings.

Example :

```
set lst {apple banana "VLSI Design" 123}
puts $lst
```

Output :

```
apple banana
VLSI Design 123
```

Note :
- Elements inside {} are treated as separate list items.
- Use quotes or braces to include spaces in an element.

2. Creating lists

- Lists can be created using braces {}, the list command or by appending elements.
- The list command is preferred as it handles spaces correctly.

Syntax 1 : Using braces

```
set lst1 {a b c d}
puts $lst1
```

Output :

```
a b c d
```

Syntax 2 : Using the list command

```
set lst2 [list a "VLSI Design" 123]
puts $lst2
```

Output :

```
a VLSI Design 123
```

Syntax 3 : Appending elements

```
set lst3 [list]
lappend lst3 "file1.v"
lappend lst3 "file2.v"
puts $lst3
```

Output :

```
file1.v file2.v
```

Note :
- Use list command when elements contain spaces.
- Use lappend to add elements incrementally.

3. Reading list elements

- List elements can be accessed using index.
- Use lindex to get a specific element.
- Use llength to get the number of elements in a list.

Syntax : `lindex $list $index ;# index starts from 0`

Example :

```
set lst {cpu gpu memory io}
puts [lindex $lst 0]
puts [lindex $lst 2]
puts [llength $lst]
```

Output :

```
cpu
memory
4
```

Note :
- Indexing starts from 0.
- If index is out of range, an empty string is returned.

4. Modifying lists

- Lists are immutable as a whole, but you can create a new list with modifications using Tcl list commands.
- Commands like lappend, linsert, lreplace and lset are used to modify lists.

4.1 Adding elements - lappend

Example :

```
set lst {a b}
lappend lst c
lappend lst d
puts $lst
```

Output :

```
a b c d
```

4.2 Inserting elements - linsert

Example :

```
set lst {a b d}
set lst [linsert $lst 2 c]
puts $lst
```

Output :

```
a b c d
```

4.3 Replacing elements - lreplace

Example :

```
set lst {a b c d}
set lst [lreplace $lst 1 2 x y]
puts $lst
```

Output :

```
a x y d
```

4.4 Setting a specific element - lset

Example :

```
set lst {a b c}
lset lst 1 "VLSI"
puts $lst
```

Output :

```
a VLSI c
```

Note :
- These commands return a new list (original list is not modified unless reassigned).
- Lists are flexible and can grow dynamically.

5. Nested lists

- A list can contain other lists as elements (nested lists).
- Nested lists are useful for representing structured data like groups, configurations or multi-dimensional information.
- Use proper braces to maintain the list structure.

Example :

```
set design_list {{alu regfile} {pc controller} {mem io}}
puts $design_list
```

Output :

```
alu regfile pc controller mem io
```

```
puts [lindex $design_list 0]
```

Output :

```
alu regfile
```

```
puts [lindex $design_list 1 0]
```

Output :

```
pc
```

```
puts [llength $design_list]
```

Output :

```
3
```

Note :
- Each sublist is treated as a single element.
- Use multiple indices to access elements inside nested lists.
- Nested lists are very useful in EDA scripts for hierarchical data, pin groups, constraint sets, etc.

Key Points

- [ ] A Tcl list is an ordered collection of elements (strings).
- [ ] Lists can be created using braces, list command or lappend.
- [ ] Use lindex and llength to read list elements.
- [ ] Use lappend, linsert, lreplace and lset to modify lists.
- [ ] Lists can contain other lists (nested lists).
- [ ] Lists are widely used in EDA scripts for file lists, options and structured data.

3. STRINGS & LISTS

3.7 List Creation & Access

- Tcl lists are ordered collections of elements and are widely used in EDA scripts for handling files, options, object names, and data sets.
- This section covers the most commonly used commands for creating lists and accessing or modifying their elements.
- Commands covered : list , lindex , llength , lappend , lset

1. list - Create a list

- The list command creates a new list from the given elements.
- Elements are separated by spaces, and Tcl automatically handles proper quoting.
- Useful for storing multiple values in a single variable.

Syntax : list element1 element2 element3 ...

Example :

```
set cells [list U1 U2 U3 U4]
set nums [list 10 20 30 40]
set mixed [list "VLSI Design" 123 "top"]
```

Output :

```
U1 U2 U3 U4
10 20 30 40
VLSI Design 123 top
```

Note :
- Use list instead of braces {} when elements may contain spaces.
- list preserves each element as a single item.

2. lindex - Access list elements

- The lindex command returns the element at the specified index.
- Indexing starts from 0.
- If the index is out of range, an empty string is returned.

Syntax : lindex listName index

Example :

```
set cells [list U1 U2 U3 U4]
puts [lindex $cells 0]
puts [lindex $cells 2]
puts [lindex $cells 3]
```

Output :

```
U1
U3
U4
```

Note :
- Useful for accessing a specific element like a file name, cell name etc.
- Can also be used with nested lists (e.g., lindex $list 1 0)

3. llength - Get the number of elements

- The llength command returns the number of elements in a list.
- Very useful for looping through list elements.

Syntax : llength listName

Example :

```
set cells [list U1 U2 U3 U4 U5]
puts [llength $cells]
set empty_list [list]
puts [llength $empty_list]
```

Output :

```
5
0
```

Note :
- Returns 0 for an empty list.
- Useful to control for-loops and iterations.

4. lappend - Append elements to a list

- The lappend command adds one or more elements to the end of a list.
- If the list does not exist, it is created automatically.
- Very commonly used to build lists dynamically in loops or conditionally.

Syntax : lappend listName element1 ?element2 ...?

Example :

```
set files [list]
lappend files "netlist.v"
lappend files "constraints.sdc" "floorplan.tcl"
puts $files
```

Output :

```
netlist.v
constraints.sdc
floorplan.tcl
```

Note :
- lappend modifies the original list variable.
- Useful for building file lists, option lists, or collecting data.
- Can append multiple elements in a single command.

5. lset - Modify existing elements

- The lset command replaces the value at a specified index.
- If the index is beyond the current list length, intermediate elements are created as empty strings.
- Useful for updating list values.

Syntax : lset listName index newValue

Example :

```
set cells [list U1 U2 U3 U4]
lset cells 1 U2_new
puts $cells
```

Output :

```
U1 U2_new U3 U4
```

Note :
- Modifies the list at the given index.
- If index is out of range, new elements are created.
- Useful for updating configurations or replacing values.

Summary Table

Command	Purpose
list	Create a new list from given elements
lindex	Access an element at a specific index
llength	Return the number of elements in a list
lappend	Append one or more elements to the end of a list
lset	Modify (replace) an element at a given index

Key Points

- [ ] Tcl lists are ordered collections of elements.
- [ ] Use list to create a list with proper element separation.
- [ ] Use lindex to read specific elements (indexing starts from 0).
- [ ] Use llength to get the number of elements.
- [ ] Use lappend to build lists dynamically.
- [ ] Use lset to modify existing elements.
- [ ] Lists are widely used in EDA scripts for files, options, object names and data handling.

3. STRINGS & LISTS

3.8 List Modification

- Tcl provides several commands to modify existing lists.
- These commands help to extract portions, replace elements, insert new elements, reverse the order or concatenate lists.
- List modification is very useful in EDA scripts for handling design data, file lists, options and tool outputs.
- Commands covered : lrange , lreplace , linsert , lreverse , concat

1. lrange - Extract a sublist

- The lrange command returns a portion of a list from a specified start index to end index.
- If the end index is omitted, elements from start index to the end are returned.
- Useful for processing part of a list.

Syntax : `lrange listName startIndex endIndex`

Example :

```
set cells [list U1 U2 U3 U4 U5]
puts [lrange $cells 1 3]
puts [lrange $cells 2 end]
```

Output :

```
U2 U3 U4
U3 U4 U5
```

Note :
- Indexing starts from 0.
- Use end to get elements till the last index.

2. lreplace - Replace elements

- The lreplace command replaces elements in a list with new elements.
- It removes the elements in the specified index range and inserts the new elements.
- Useful for modifying list contents.

Syntax : `lreplace listName startIndex endIndex newElement ?...?`

Example :

```
set cells [list U1 U2 U3 U4 U5]
set new_list [lreplace $cells 1 3 A B]
puts $new_list
```

Output :

```
U1 A B U5
```

Note :
- Replaces elements from startIndex to endIndex (inclusive).
- If no new elements are given, the specified range is removed.

3. linsert - Insert elements

- The linsert command inserts new elements at a specified index in a list.
- The new elements are inserted before the element at the index.
- Useful for adding options, items or data at specific positions.

Syntax : `linsert listName index newElement ?...?`

Example :

```
set cells [list U1 U2 U3 U4]
set new_list [linsert $cells 2 X Y]
puts $new_list
```

Output :

```
U1 U2 X Y U3 U4
```

Note :
- If index is 0, elements are inserted at the beginning.
- If index is end, elements are appended at the end.

4. lreverse - Reverse a list

- The lreverse command returns a new list with the elements in reverse order.
- The original list remains unchanged.
- Useful for processing data in reverse order.

Syntax : `lreverse listName`

Example :

```
set cells [list U1 U2 U3 U4]
set rev_list [lreverse $cells]
puts $rev_list
```

Output :

```
U4 U3 U2 U1
```

Note :
- Returns a new list with elements in reverse order.
- Can be used with nested lists as well.

5. concat - Combine multiple lists

- The concat command combines two or more lists into a single list.
- It does not modify the original lists.
- Useful for joining lists from different sources.

Syntax : `concat list1 list2 ?list3 ...?`

Example :

```
set list1 [list U1 U2]
set list2 [list U3 U4]
set list3 [list U5]
set all [concat $list1 $list2 $list3]
puts $all
```

Output :

```
U1 U2 U3 U4 U5
```

Note :
- All input arguments are treated as lists.
- Very useful for merging file lists, cell lists, option lists, etc.

Additional Examples

a) Using multiple modification commands

Example :

```
set cells [list U1 U2 U3 U4 U5]
set tmp [lreplace $cells 2 2 X]
set tmp [linsert $tmp 4 Y]
set tmp [lreverse $tmp]
puts $tmp
```

Output :

```
U5 Y U4 X U2 U1
```

b) Concatenating and reversing

Example :

```
set list1 [list A B C]
set list2 [list D E F]
set merged [concat $list1 $list2]
puts [lreverse $merged]
```

Output :

```
F E D C B A
```

Key Points

- [ ] Use lrange to extract a portion of a list.
- [ ] Use lreplace to replace elements.
- [ ] Use linsert to insert new elements.
- [ ] Use lreverse to reverse the order of elements.
- [ ] Use concat to combine multiple lists.
- [ ] These commands are widely used in EDA scripts for flexible data handling.

3.9 List Searching & Sorting

- Tcl provides powerful commands to search for elements in a list, sort list elements and remove duplicates.
- These commands are widely used in EDA scripts for handling design data, file lists, and tool outputs.
- Main commands covered : lsearch , lsort
- Topics : Searching elements , Sorting elements , Removing duplicates.

1. lsearch - Search for an element

- The lsearch command searches for an element in a list and returns its index (0 based).
- If the element is not found, it returns -1.
- Supports exact match and pattern matching using -glob or -regexp options.

Syntax : lsearch ?options? listName pattern

Example 1 : Exact match

```
set nets {clk rst data valid}
set idx [lsearch $nets rst]
puts $idx
```

Output :

```
1
```

Example 2 : Using pattern matching (-glob)

```
set cells {U1 U2 U3 U10 U11}
set idx [lsearch -glob $cells "U1*"]
puts $idx
```

Output :

```
0
```

Example 3 : Using -all to get all matches

```
set pins {A B A C A}
set idx_list [lsearch -all $pins A]
puts $idx_list
```

Output :

```
0 2 4
```

Note :

- Returns the index of the first match by default.
- Use -all to get all matching indices.
- Use -glob or -regexp for pattern based search.

2. Searching elements (Other methods)

- Apart from lsearch, you can also check if an element exists in a list using expr or string match within a loop.
- Useful for conditional checks in scripts.

Example : Check if an element exists

```
set ports {clk rst data valid}
if {[lsearch $ports rst] != -1} {
    puts "rst exists in the list"
} else {
    puts "rst not found"
}
```

Output :

```
rst exists
in the list
```

Note :

- lsearch returns -1 if not found.
- Useful in if conditions to check existence of elements.
- Can also use -exact for case sensitive match.

3. lsort - Sort list elements

- The lsort command sorts the elements of a list.
- By default, sorting is in dictionary (lexicographic) order.
- Supports numeric, increasing/decreasing and unique options.

Syntax : lsort ?options? listName

Example 1 : Dictionary (alphabetical) sort

```
set names {ram alu ctrl adder}
puts [lsort $names]
```

Output :

```
adder alu ctrl ram
```

Example 2 : Numeric sort

```
set nums {10 3 25 1 7}
puts [lsort -integer $nums]
```

Output :

```
1 3 7 10 25
```

Example 3 : Decreasing order

```
set nums {10 3 25 1 7}
puts [lsort -integer -decreasing $nums]
```

Output :

```
25 10 7 3 1
```

Example 4 : Unique sort (remove duplicates)

```
set vals {A B A C B D}
puts [lsort -unique $vals]
```

Output :

```
A B C D
```

Note :

- Use -integer for numeric sorting.
- Use -decreasing for descending order.
- Use -unique to remove duplicate elements while sorting.

4. Removing duplicates

- Duplicates can be removed using lsort -unique.
- Useful for getting unique cell names, net names, file lists, etc.
- Maintains sorted order in the output.

Example : Remove duplicates from a list

```
set files {a.v b.v a.v c.v b.v}
set unique_files [lsort -unique $files]
puts $unique_files
```

Output :

```
a.v b.v c.v
```

Key Points

- [ ] Use lsearch to find the index of an element in a list.
- [ ] Use -glob or -regexp with lsearch for pattern matching.
- [ ] Use lsort to sort list elements (dictionary or numeric).
- [ ] Use -decreasing for descending order.
- [ ] Use -unique with lsort to remove duplicates.
- [ ] List searching and sorting are widely used in EDA scripts for processing tool outputs, design data and file lists.

3.10 List/String Conversion

- Tcl makes it easy to convert between strings and lists using join and split commands.
- Lists and strings are closely related in Tcl.
- Conversion is useful when reading/writing files, processing tool outputs, or building command arguments.
- Main commands covered : join , split.
- Topics : String-to-list conversion , List-to-string conversion.

☑ 1. join - List to string conversion

- The join command takes a list and combines its elements into a single string using a specified separator.
- If no separator is given, a space is used by default.
- Useful for building command arguments or generating formatted strings.

Syntax : join listName ?separator?

Example 1 : Using default separator (space)

```
set lst [list U1 U2 U3 U4]
set str [join $lst]
puts $str
```

Output :
U1 U2 U3 U4

Example 2 : Using custom separator

```
set lst [list U1 U2 U3 U4]
set str [join $lst ","]
puts $str
```

Output :
U1,U2,U3,U4

Example 3 : Building a command argument

```
set files [list file1.v file2.v file3.v]
set cmd [join $files " "]
puts $cmd
```

Output :
file1.v file2.v file3.v

Note :
- join is used for list-to-string conversion.
- Default separator is space if not specified.
- Useful for creating command options, file paths, etc.

☑ 2. split - String to list conversion

- The split command splits a string into a list using specified separator characters.
- If no separator is given, it splits on whitespace (spaces, tabs, newlines).
- Useful for parsing file data, command outputs, or user input.

Syntax : split string ?splitChars?

Example 1 : Using default separator (whitespace)

```
set str "U1 U2 U3 U4"
set lst [split $str]
puts $lst
```

Output :
U1 U2 U3 U4

Example 2 : Using custom separator

```
set str "U1,U2,U3,U4"
set lst [split $str ","]
puts $lst
```

Output :
U1 U2 U3 U4

Example 3 : Splitting a file path

```
set path "/home/user/project/file.v"
set lst [split $path "/"]
puts $lst
```

Output :
home user project file.v

Note :
- split is used for string-to-list conversion.
- You can specify multiple separator characters.
- Very useful for parsing paths, reports, or CSV-like data.

☑ 3. String-to-list conversion (Other methods)

- In Tcl, a string can often be treated as a list directly if it is properly formatted (elements separated by spaces).
- You can also use list command to explicitly create a list from strings.

Example 1 : Using list command

```
set lst [list "VLSI" "Design" "Flow"]
puts $lst
```

Output :
VLSI Design Flow

Example 2 : Using split for multiple delimiters

```
set str "U1;U2;U3;U4"
set lst [split $str ";"]
puts $lst
```

Output :
U1 U2 U3 U4

Example 3 : Reading from file (each line to list)

```
set fp [open "cells.txt" r]
set data [read $fp]
close $fp
set lst [split $data "\n"]
puts $lst
```

Output :
U1
U2
U3
U4

☑ 4. List-to-string conversion (Other methods)

- Besides join, you can also use format or string manipulation to create strings from lists.

Example 1 : Using format

```
set lst [list U1 U2 U3]
set str [format "%s,%s,%s" {*}$lst]
puts $str
```

Output :
U1,U2,U3

Example 2 : Using string cat

```
set lst [list "Path:" "/home" "/user"]
set str [string cat {*}$lst]
puts $str
```

Output :
Path:/home/user

Key Points

- ☐ Use join to convert a list to a string.
- ☐ Use split to convert a string to a list.
- ☐ You can specify custom separators.
- ☐ Useful for file I/O, parsing tool outputs, and building commands.
- ☐ Lists and strings are interchangeable in many Tcl use cases.

3.11 Nested Lists

- A nested list is a list whose elements are themselves lists.
- Useful for representing structured or hierarchical data.
- Commonly used in EDA scripts for handling multi-corner data, pin groups, constraint sets, tables, etc.
- We can access nested elements using multiple indices.
- Topics : List of lists , Accessing nested elements , Processing nested data , Two-dimensional data.

1. List of lists

- A list can contain other lists as its elements.
- Each sublist is treated as a single element.
- Useful to organize related data.

Example :

```
set corners [list \
    [list SS 0.72 -40] \
    [list TT 0.80 25] \
    [list FF 0.88 125]
]
puts $corners
```

Output :

```
{SS 0.72 -40}
{TT 0.80 25}
{FF 0.88 125}
```

Note :
- Each sublist is enclosed in {}.
- The main list contains 3 sublists.

2. Accessing nested elements

- Use multiple index levels with lindex.
- First index selects the sublist, second index selects the element within that sublist.
- Indexing starts from 0.

Syntax : `lindex $listName outerIndex innerIndex`

Example :

```
set corners [list \
    [list SS 0.72 -40] \
    [list TT 0.80 25] \
    [list FF 0.88 125]
]
puts [lindex $corners 0 0]   ;# SS
puts [lindex $corners 1 1]   ;# 0.80
puts [lindex $corners 2 2]   ;# 125
```

Output :

```
SS
0.80
125
```

Note :
- lindex $list i j → i^{th} sublist, j^{th} element
- Be careful with out-of-range indices (returns empty string)

3. Processing nested data

- Use foreach loops to iterate over nested lists.
- Useful for processing tables, reports or multi-corner data.

Example :

```
set corners [list \
    [list SS 0.72 -40] \
    [list TT 0.80 25] \
    [list FF 0.88 125]
]
foreach corner $corners {
    set name [lindex $corner 0]
    set volt [lindex $corner 1]
    set temp [lindex $corner 2]
    puts "Corner: $name V=$volt T=$temp"
}
```

Output :

```
Corner: SS V=0.72 T=-40
Corner: TT V=0.80 T=25
Corner: FF V=0.88 T=125
```

Note :
- Each $corner is a sublist.
- Use lindex to extract elements inside the loop.

4. Two-dimensional data

- Nested lists can represent a 2D matrix (rows and columns).
- Useful for storing tabular data like timing, power, or configuration values.
- Access elements using row and column indices.

Example : 2D matrix (rows x columns)

```
set matrix [list \
    [list 1 2 3] \
    [list 4 5 6] \
    [list 7 8 9]
]
puts $matrix
```

Output :

```
{1 2 3} {4 5 6} {7 8 9}
```

Accessing elements :

```
puts [lindex $matrix 0 0]   ;# row 0, col 0
puts [lindex $matrix 1 2]   ;# row 1, col 2
puts [lindex $matrix 2 1]   ;# row 2, col 1
```

Output :

```
1
6
8
```

Iterating through 2D data :

```
set row_num 0
foreach row $matrix {
    set col_num 0
    foreach val $row {
        puts "[$row_num,$col_num] = $val"
        incr col_num
    }
    incr row_num
}
```

Output :

```
[0,0] = 1
[0,1] = 2
[0,2] = 3
[1,0] = 4
[1,1] = 5
[1,2] = 6
[2,0] = 7
[2,1] = 8
[2,2] = 9
```

Note :
- Nested lists are flexible and can have different sizes.
- Useful for multi-corner multi-mode (MCMM) data, pin groups, constraint sets, etc.
- Use nested foreach to process hierarchical data.

Additional Example : Real-world use case

Storing timing data (design, corner, slack)

```
set timing_data [list \
    [list "U1" "TT" 0.12] \
    [list "U2" "SS" -0.05] \
    [list "U3" "FF" 0.20]
]
foreach entry $timing_data {
    set inst   [lindex $entry 0]
    set corner [lindex $entry 1]
    set slack  [lindex $entry 2]
    puts "Instance: $inst  Corner: $corner  Slack: $slack"
}
```

Output :

```
Instance: U1  Corner: TT  Slack: 0.12
Instance: U2  Corner: SS  Slack: -0.05
Instance: U3  Corner: FF  Slack: 0.20
```

Key Points

- [ ] A nested list is a list of lists.
- [ ] Use lindex with multiple indices to access nested elements.
- [ ] Use foreach to process nested data.
- [ ] Nested lists are useful for representing two-dimensional data.
- [ ] Commonly used in EDA scripts for multi-corner data, pin groups, tables, etc.
- [ ] Handle out-of-range indices carefully (returns empty string).

3.12 EDA Applications of Lists

- Lists are widely used in EDA tools to store and manipulate design data such as cells, nets, pins, files, layers, scenarios, paths, etc.
- Using Tcl list operations makes scripts flexible, compact and easy to maintain.
- This section shows common real-world examples.
- Topics : Cell lists , Net lists , Pin lists , File lists , Layer lists, Scenario lists , Path lists.

1. Cell lists

- Used to store a group of cell names for reporting, constraint or modification.
- Cell lists can be obtained using tool commands like get_cells and stored in a Tcl list.

Example : Create and use a cell list

```
set cell_list [get_cells -hierarchical]
puts "Total cells : [llength $cell_list]"
puts "First 5 cells : [lrange $cell_list 0 4]"
```

Output :

```
Total cells : 12560
First 5 cells :
U1 U2 U3 U4 U5
```

2. Net lists

- Used to store net names for analysis, reporting or constraint application.
- Useful for tasks like finding critical nets, modifying attributes, etc.

Example : Create and use a net list

```
set net_list [get_nets *]
puts "Total nets : [llength $net_list]"
puts "Sample nets : [lrange $net_list 0 4]"
```

Output :

```
Total nets : 8420
Sample nets :
net1 net2 net3
net4 net5
```

3. Pin lists

- Used to store pin names of a cell or a net.
- Helpful for applying constraints, checking connectivity, or reporting.

Example : Get pin list of a cell

```
set pin_list [get_pins U1/*]
puts "Pins of U1 : $pin_list"
puts "Number of pins : [llength $pin_list]"
```

Output :

```
Pins of U1 :
U1/A U1/B U1/Z
Number of pins : 3
```

4. File lists

- Used to store multiple file names (RTL, SDC, libraries, reports, etc.) for iteration and batch processing.
- Useful for reading, writing or processing multiple files.

Example : Create a file list and open each file

```
set file_list [list \
    "rtl.v" "top.sdc" \
    "lib.lib" "script.tcl" ]
foreach file $file_list {
    puts "Processing file : $file"
}
```

Output :

```
Processing file : rtl.v
Processing file : top.sdc
Processing file : lib.lib
Processing file : script.tcl
```

5. Layer lists

- Used to store technology layer names for routing, placement blockages, or analysis.
- Common in physical design scripts.

Example : Define and use a layer list

```
set layer_list [list M1 M2 M3 M4 M5]
puts "Layers : $layer_list"
puts "Top layer : [lindex $layer_list end]"
```

Output :

```
Layers : M1 M2 M3 M4 M5
Top layer : M5
```

6. Scenario lists

- Used to store different modes, corners or scenarios in a multi-corner multi-mode (MCMM) flow.
- Useful for timing analysis, signoff, and reports.

Example : Scenario (mode-corner) list

```
set scenario_list [list \
    "FUNC_SS" "FUNC_TT" \
    "FUNC_FF" "TEST_TT" ]
foreach sc $scenario_list {
    puts "Running STA for scenario : $sc"
}
```

Output :

```
Running STA for scenario : FUNC_SS
Running STA for scenario : FUNC_TT
Running STA for scenario : FUNC_FF
Running STA for scenario : TEST_TT
```

7. Path lists

- Used to store timing paths (e.g., critical paths) for analysis or debugging.
- Helpful for iterating over paths and reporting details like slack, startpoint, endpoint, etc.

Example : Get and process a path list

```
set path_list [get_timing_paths -max_paths 3]
foreach p $path_list {
    set sp [get_attribute $p startpoint]
    set ep [get_attribute $p endpoint]
    set sl [get_attribute $p slack]
    puts "Path: $sp -> $ep  Slack: $sl"
}
```

Output :

```
Path: U1/CLK -> U5/D
  Slack: -0.012
Path: U2/CLK -> U8/D
  Slack: -0.008
Path: U3/CLK -> U7/D
  Slack: 0.005
```

Summary Table

List Type	Purpose	Typical Commands / Example
Cell list	Store cell names	set cells [get_cells *]
Net list	Store net names	set nets [get_nets *]
Pin list	Store pin names	set pins [get_pins U1/*]
File list	Store file names	set files [list rtl.v top.sdc ...]
Layer list	Store technology layers	set layers [list M1 M2 M3 ...]
Scenario list	Store modes/corners	set scenarios [list FUNC_SS ...]
Path list	Store timing paths	set paths [get_timing_paths ...]

Key Points

- ☐ Lists are widely used in EDA for design data handling.
- ☐ Common list types : cells, nets, pins, files, layers, scenarios, paths.
- ☐ Use list operations (lindex, lappend, foreach, etc.) to process data.
- ☐ Helps automate repetitive tasks and improve script efficiency.
- ☐ Essential for real-world RTL, STA and PnR automation flows.

4. CONTROL FLOW & LOOPS

4.1 Conditional Statements

- Conditional statements allow the script to take different actions based on whether a condition is true or false.
- The if command is used to evaluate a condition.
- Tcl supports if, if/else, if/elseif/else and nested conditions.
- Conditions are written inside braces { } and can use operators like ==, !=, >, <, &&, || etc.
- Very useful in EDA scripts for checks, filtering, and flow control.

1. if - Basic if condition

- The if command executes a block of code only if the condition is true.
- If the condition is false, nothing is executed.

Syntax :
```
if {condition} { statements }
```

Example :
```
set value 10
if { $value > 5 } {
    puts "Value is greater than 5"
}
```

Output :
```
Value is greater than 5
```

Note :
- Condition is written inside { } (no $ for variables).
- Use comparison operators like >, <, ==, != etc.

2. Basic conditions

- Tcl supports common comparison and logical operators to build conditions.
- These can be used with numbers, strings or variables.

Common Operators :

Operator	Meaning	Example
==	Equal to	$a == $b
!=	Not equal to	$a != $b
>	Greater than	$a > $b
<	Less than	$a < $b
>=	Greater than or equal	$a >= $b
<=	Less than or equal	$a <= $b
&&	Logical AND	{$a > 0 && $b < 10}
||	Logical OR	{$a < 0 || $b > 10}
!	Logical NOT	{!$flag}

Example :
```
set a 20
set b 10
if { $a >= 20 && $b < 15 } {
    puts "Condition is true"
}
```

Output :
```
Condition is true
```

Note :
- Use && for AND, || for OR, ! for NOT.
- You can combine multiple conditions.

3. if/else - True or false case

- The if/else structure executes one block if the condition is true and another block if it is false.

Syntax :
```
if {condition} {
    statements for true case
} else {
    statements for false case
}
```

Example :
```
set value 5
if { $value > 10 } {
    puts "Value is greater than 10"
} else {
    puts "Value is 10 or less"
}
```

Output :
```
Value is 10 or less
```

4. if/elseif/else - Multiple conditions

- The if/elseif/else structure allows checking multiple conditions in sequence.
- The first true condition block will be executed.

Syntax :
```
if {cond1} {
    statements1
} elseif {cond2} {
    statements2
} elseif {cond3} {
    statements3
} else {
    statements for default case
}
```

Example :
```
set score 75
if { $score >= 90 } {
    puts "Grade : A"
} elseif { $score >= 75 } {
    puts "Grade : B"
} elseif { $score >= 50 } {
    puts "Grade : C"
} else {
    puts "Grade : F"
}
```

Output :
```
Grade : B
```

5. Nested conditions

- An if can be written inside another if to handle more complex decision making.
- Useful when multiple dependent conditions need to be checked.

Example :
```
set a 10
set b 20
if { $a > 5 } {
    if { $b > 15 } {
        puts "Both conditions are true"
    } else {
        puts "a > 5 but b <= 15"
    }
} else {
    puts "a <= 5"
}
```

Output :
```
Both conditions are true
```

4. CONTROL FLOW & LOOPS

4.2 Comparison & Logical Conditions

- Tcl provides operators to compare values and build logical conditions.
- Conditions can be numeric, string or boolean.
- Multiple conditions can be combined using logical operators AND (&&), OR (||) and NOT (!).
- These are widely used with if, while and other control statements in EDA scripts.
- Topics : Numeric comparison , String comparison , Boolean conditions , Compound conditions , AND , OR , NOT

1. Numeric comparison

- Used to compare integer or floating point numbers.
- Operators : > , < , >= , <= , == , !=

Operator	Meaning	Example (a=10, b=5)	Result
>	Greater than	$a > $b	1 (true)
<	Less than	$a < $b	0 (false)
>=	Greater or equal	$a >= $b	1 (true)
<=	Less or equal	$a <= $b	0 (false)
==	Equal to	$a == $b	0 (false)
!=	Not equal to	$a != $b	1 (true)

Example :

```
set a 20
set b 15
if { $a >= $b } {
    puts "a is greater or equal to b"
}
```

Output :

```
a is greater or
equal to b
```

2. String comparison

- Used to compare strings (lexicographical order).
- Operators : eq , ne , lt , le , gt , ge
- String comparison is case sensitive.

Operator	Meaning	Example	Result
eq	Equal to	"abc" eq "abc"	1 (true)
ne	Not equal to	"abc" ne "xyz"	1 (true)
lt	Less than	"apple" lt "banana"	1 (true)
le	Less or equal	"apple" le "apple"	1 (true)
gt	Greater than	"dog" gt "cat"	1 (true)
ge	Greater or equal	"dog" ge "cat"	1 (true)

Example :

```
set s1 "VLSI"
set s2 "vlsi"
if { $s1 ne $s2 } {
    puts "Strings are different"
}
```

Output :

```
Strings are
different
```

3. Boolean conditions

- In Tcl, a value is considered false if it is 0, an empty string "", or the literal word "false" (case insensitive).
- Everything else is considered true.

Value	Boolean result	Example
0	false	if {0} { ... } → not executed
"" (empty)	false	if {""} { ... } → not executed
false	false	if {false} { ... } → not executed
1	true	if {1} { ... } → executed
non-empty string	true	if {"abc"} { ... } → executed

Example :

```
if { [llength ""] } {
    puts "This is true"
} else {
    puts "Empty string is false"
}
```

Output :

```
Empty string
is false
```

4. Compound conditions

- Multiple conditions can be combined using logical operators.
- Use parentheses ( ) for clarity when needed.

Example :

```
set a 10
set b 20
if { $a > 5 && $b < 30 } {
    puts "Both conditions are true"
} else {
    puts "Condition is false"
}
```

Output :

```
Both conditions
are true
```

5. Logical operators

- Tcl provides && (AND) , || (OR) and ! (NOT) for building complex conditions.
- These operators work with numeric, string or boolean values.

Operator	Meaning	Example (a=1, b=0)	Result
&&	Logical AND	$a && $b	0 (false)
||	Logical OR	$a || $b	1 (true)
!	Logical NOT	!$b	1 (true)

Example 1 : Using AND and OR

```
set a 1
set b 0
if { $a && $b } {
    puts "Both true"
} elseif { $a || $b } {
    puts "At least one true"
}
```

Output :

```
At least one
true
```

Example 2 : Using NOT

```
set flag 0
if { !$flag } {
    puts "Flag is false"
}
```

Output :

```
Flag is false
```

Key Points

- ☐ Use numeric operators for numbers and string operators for text.
- ☐ 0 , empty string and "false" are considered false in Tcl.
- ☐ Combine conditions with && , || and ! for complex checks.
- ☐ Use parentheses ( ) to make conditions clear.
- ☐ These operators are widely used in EDA scripts for filtering, validation and flow control.

4.3 Switch Statements

- The switch command provides a clean way to execute different blocks of code based on the value of an expression.
- Useful when there are multiple possible values.
- Supports exact matching, pattern matching (wildcards), and a default case.
- Helps in command selection and tool automation in EDA scripts.
- Syntax : switch, Multiple cases, Default case, Pattern matching, Command-selection logic.

1. Basic switch syntax

- The switch command compares a value with a list of patterns and executes the matching block.
- If no pattern matches, the default block is executed.

Syntax :

```
switch $expression {
    pattern1 { statements }
    pattern2 { statements }
    ...
    default  { statements }
}
```

Example 1 : Simple exact matching

```
set mode "setup"
switch $mode {
    "setup"   { puts "Running setup analysis" }
    "hold"    { puts "Running hold analysis" }
    "signoff" { puts "Running signoff analysis" }
    default   { puts "Unknown mode : $mode" }
}
```

Output :

```
Running setup analysis
```

2. Multiple cases

- Multiple patterns can be specified for the same block.
- Useful when different values should execute the same command.

Example 2 : Multiple cases for same action

```
set stage "pnr"
switch $stage {
    "syn" "synthesis" { puts "Synthesis stage" }
    "pnr" "place" "route" { puts "PNR stage" }
    "signoff" "sta" { puts "Signoff stage" }
    default { puts "Unknown stage : $stage" }
}
```

Output :

```
PNR stage
```

3. Default case

- The default block is executed when no pattern matches.
- It is good practice to always include a default case.

Example 3 : Using default case

```
set tool "icc2"
switch $tool {
    "dc"   { puts "Design Compiler" }
    "icc2" { puts "Implementation Compiler II" }
    "pt"   { puts "PrimeTime" }
    default { puts "Unknown tool : $tool" }
}
```

Output :

```
Implementation Compiler II
```

4. Pattern matching

- switch supports pattern matching using wildcards (*, ?).
- Useful when matching a group of values (e.g., file extensions, commands, or names).

Example 4 : Using wildcards for pattern matching

```
set file "design.v"
switch $file {
    "*.v"   { puts "Verilog file" }
    "*.sv"  { puts "SystemVerilog file" }
    "*.tcl" { puts "Tcl script file" }
    "*"     { puts "Unknown file type" }
}
```

Output :

```
Verilog file
```

Note :
- * matches zero or more characters.
- ? matches a single character.
- Pattern matching is case sensitive.

5. Command-selection logic (EDA example)

- switch is widely used in EDA scripts to select and execute different tool commands, modes or operations.
- Makes the script compact and easy to maintain.

Example 5 : Select command based on operation

```
set op "report"
switch $op {
    "run"    { puts "Running implementation..." }
    "report" { puts "Generating reports..." }
    "clean"  { puts "Cleaning output files..." }
    "exit"   { puts "Exiting script..." }
    default  { puts "Invalid operation : $op" }
}
```

Output :

```
Generating reports...
```

6. Practical EDA example

- Determine analysis type and run appropriate command.
- Shows how switch simplifies command selection in real flows.

Example 6 : Analysis type selection

```
set analysis "hold"
switch $analysis {
    "setup"   { puts "Running setup STA";  #run_setup_sta }
    "hold"    { puts "Running hold STA";   #run_hold_sta }
    "leakage" { puts "Running leakage analysis" }
    "timing"  { puts "Running full timing analysis" }
    default   { puts "Unknown analysis type : $analysis" }
}
```

Output :

```
Running hold STA
```

Key Points

- [ ] switch provides a clean way to handle multiple conditions.
- [ ] Supports exact matching and pattern matching (wildcards).
- [ ] Always include a default case.
- [ ] Useful for command selection, tool control, and flow automation in EDA.
- [ ] Improves script readability compared to long if/elseif chains.
- [ ] Patterns are case sensitive.

4.4 For Loop

- The for loop is used to execute a block of code repeatedly for a given number of iterations.
- It has three parts : Initialization, Condition, Increment.
- Useful when the number of iterations is known in advance.
- Supports nested for loops for multi-level iterations.
- Common in EDA scripts for processing lists, files, cells, nets, layers, etc.

1. Basic for loop syntax

- The for command includes initialization, condition and increment inside the same line.
- The loop body is executed as long as the condition is true.

Syntax :

```
for {initialization} {condition} {increment} {
    statements
}
```

Example 1 : Print numbers from 0 to 4

```
for {set i 0} {$i < 5} {incr i} {
    puts "i = $i"
}
```

Output :

```
i = 0
i = 1
i = 2
i = 3
i = 4
```

Note :

- Initialization : executed once at the beginning.
- Condition : checked before each iteration.
- Increment : executed at the end of each iteration.
- Loop body : executed when condition is true.

2. Initialization

- Initialization is used to set the loop variable(s) before starting the loop.
- Multiple variables can be initialized if needed.

Example 2 : Initialize multiple variables

```
for {set i 0; set j 10} {$i < 5} {incr i; incr j} {
    puts "i = $i , j = $j"
}
```

Output :

```
i = 0 , j = 10
i = 1 , j = 11
i = 2 , j = 12
i = 3 , j = 13
i = 4 , j = 14
```

3. Condition

- The condition is a Boolean expression.
- If it evaluates to true, the loop body is executed.
- When it becomes false, the loop terminates.

Example 3 : Using different condition

```
for {set i 1} {$i <= 3} {incr i} {
    puts "Value = $i"
}
```

Output :

```
Value = 1
Value = 2
Value = 3
```

Note :

- You can use any valid Tcl expression in the condition.
- Common operators: <, <=, >, >=, ==, !=

4. Increment

- The increment part updates the loop variable after each iteration.
- Typically uses incr, but you can use any Tcl command.
- Multiple variables can be incremented.

Example 4 : Using different increment

```
for {set i 0} {$i < 5} {incr i 2} {
    puts "i = $i"
}
```

Output :

```
i = 0
i = 2
i = 4
```

5. Nested for loops

- A for loop can be placed inside another for loop.
- Useful for two-dimensional iterations (e.g., rows and columns, layers and cells, etc.)
- Common in EDA for processing hierarchical data, matrices, or multi-corner setups.

Example 5 : Nested for loop (rows and columns)

```
for {set i 0} {$i < 3} {incr i} {
    for {set j 0} {$j < 4} {incr j} {
        puts "i = $i , j = $j"
    }
}
```

Output :

```
i = 0 , j = 0
i = 0 , j = 1
i = 0 , j = 2
i = 0 , j = 3
i = 1 , j = 0
i = 1 , j = 1
i = 1 , j = 2
i = 1 , j = 3
i = 2 , j = 0
i = 2 , j = 1
i = 2 , j = 2
i = 2 , j = 3
```

Note :

- Outer loop controls rows (i).
- Inner loop controls columns (j).
- Useful for traversing 2D data structures.

6. Practical EDA examples

- Loops are widely used in EDA scripts for automating repetitive tasks like processing cells, nets, files, layers, scenarios, etc.

Example 6 : Iterate over a list using for loop

```
set cell_list [list U1 U2 U3 U4]
for {set i 0} {$i < [llength $cell_list]} {incr i} {
    set cell [lindex $cell_list $i]
    puts "Processing cell : $cell"
}
```

Output :

```
Processing cell : U1
Processing cell : U2
Processing cell : U3
Processing cell : U4
```

Example 7 : Iterate over technology layers

```
set layers [list M1 M2 M3 M4]
for {set i 0} {$i < [llength $layers]} {incr i} {
    puts "Analyzing layer : [lindex $layers $i]"
}
```

Output :

```
Analyzing layer : M1
Analyzing layer : M2
Analyzing layer : M3
Analyzing layer : M4
```

Key Points

- [ ] for loop has three parts : initialization, condition, increment.
- [ ] Use incr to update loop variables.
- [ ] Nested for loops are useful for multi-dimensional data.
- [ ] Widely used in EDA for automating repetitive tasks.
- [ ] Makes scripts compact, efficient and easy to maintain.

4.5 Foreach Loop

- The foreach command is used to iterate over the elements of a list or collection
- It automatically takes each element and assigns it to a variable during each iteration.
- Supports single or multiple variables
- Useful for processing lists of cells, nets, files, layers, scenarios, etc.
- Cleaner and easier to use compared to index based for loop

1. Basic foreach syntax

- Iterates over each element in the list and executes the statements for each element.
- The loop variable takes one value at a time from the list.

```
Syntax :  foreach variable list {
              statements
          }
```

Example 1 : Iterate over a simple list

```
set nums {1 2 3 4 5}
foreach n $nums {
    puts "Number : $n"
}
```

Output :

```
Number : 1
Number : 2
Number : 3
Number : 4
Number : 5
```

2. Iterating lists (strings)

- foreach works with string lists as well.
- Each element is taken in order.

Example 2 : Iterate over a list of strings

```
set fruits {apple banana orange}
foreach f $fruits {
    puts "Fruit : $f"
}
```

Output :

```
Fruit : apple
Fruit : banana
Fruit : orange
```

3. Multiple variables

- You can use multiple variables to iterate over grouped data.
- Each iteration takes multiple elements from the list.

Example 3 : Using multiple variables

```
set data {U1 NAND 2 U2 NOR 3 U3 INV 1}
foreach {name type fanin} $data {
    puts "Cell: $name  Type: $type  Fanin: $fanin"
}
```

Output :

```
Cell: U1  Type: NAND  Fanin: 2
Cell: U2  Type: NOR   Fanin: 3
Cell: U3  Type: INV   Fanin: 1
```

Note :

- The list is split into groups based on the number of variables.
- Elements are assigned in order.
- Useful for processing structured data (e.g., name, type, value).

4. Nested foreach

- A foreach loop can be placed inside another foreach loop.
- Useful for processing two-dimensional data like matrices, rows and columns, or hierarchical structures.

Example 4 : Nested foreach loop

```
set matrix {{1 2 3} {4 5 6} {7 8 9}}
foreach row $matrix {
    foreach val $row {
        puts -nonewline "$val "
    }
    puts ""
}
```

Output :

```
1 2 3
4 5 6
7 8 9
```

5. Processing collections (EDA examples)

- foreach is widely used in EDA scripts to process lists of cells, nets, files, layers, scenarios, etc.
- Makes scripts concise and easy to read.

Example 5 : Process a list of cells

```
set cell_list [get_cells *]
foreach c $cell_list {
    puts "Processing cell : $c"
}
```

Output :

```
Processing cell : U1
Processing cell : U2
Processing cell : U3
...
```

Example 6 : Process files in a directory

```
set files {top.v rtl.v tb.v}
foreach file $files {
    puts "Reading file : $file"
}
```

Output :

```
Reading file : top.v
Reading file : rtl.v
Reading file : tb.v
```

Example 7 : Process layers (technology example)

```
set layers {M1 M2 M3 M4 M5}
foreach layer $layers {
    puts "Checking layer : $layer"
}
```

Output :

```
Checking layer : M1
Checking layer : M2
Checking layer : M3
Checking layer : M4
Checking layer : M5
```

Key Points

- ☐ foreach iterates over each element in a list.
- ☐ Supports single or multiple variables.
- ☐ Useful for processing collections (cells, nets, files, layers, etc.).
- ☐ Nested foreach loops are used for multi-dimensional data.
- ☐ Makes scripts cleaner, shorter and easier to maintain.
- ☐ Widely used in EDA for automation and data processing.

4.6 While Loop

- The while loop repeatedly executes a block of code as long as the given condition is true.
- The condition is checked before each iteration.
- Useful when the number of iterations is not known in advance.
- Commonly used in EDA scripts for file processing, waiting for conditions, monitoring status, etc.

1. Basic while syntax

- The while command takes a condition in braces and a block of statements to execute.
- The block is executed as long as the condition is true.

```
Syntax :  while {condition} {
              statements
          }
```

Example 1 : Print numbers from 0 to 4

```
set i 0
while { $i < 5 } {
    puts "i = $i"
    incr i
}
```

Output :

```
i = 0
i = 1
i = 2
i = 3
i = 4
```

2. Loop condition

- The condition is a Boolean expression.
- If it evaluates to true, the loop body is executed.
- When it becomes false, the loop terminates.

Example 2 : Using a condition with a variable

```
set val 10
while { $val > 0 } {
    puts "Value = $val"
    incr val -1
}
```

Output :

```
Value = 10
Value = 9
Value = 8
...
Value = 1
```

3. Counter-based loop

- A counter variable is commonly used to control the loop.
- Use incr to update the counter in each iteration.

Example 3 : Find sum of first N numbers

```
set n 5
set i 1
set sum 0
while { $i <= $n } {
    set sum [expr $sum + $i]
    incr i
}
puts "Sum = $sum"
```

Output :

```
Sum = 15
```

4. Infinite-loop prevention

- If the condition never becomes false, the loop will run forever (infinite loop).
- Always ensure that the loop variable is updated and the condition will eventually become false.
- Use a counter, flag or timeout mechanism in real scripts.

Example 4 : Example of an infinite loop (avoid this!)

```
set i 0
while { 1 } {
    puts "This will run forever : i = $i"
    incr i
}
```

Note :
- Condition is always true (1)
- No exit condition.
- Press Ctrl+C to stop execution.

5. Using a flag variable

- A flag variable can be used to control the loop execution based on some condition inside the loop.

Example 5 : Using a flag to exit

```
set done 0
set count 0
while { !$done } {
    puts "Iteration : $count"
    incr count
    if { $count >= 5 } {
        set done 1
    }
}
```

Output :

```
Iteration : 0
Iteration : 1
Iteration : 2
Iteration : 3
Iteration : 4
```

6. Real-world EDA example

- while loops are useful in EDA scripts for polling tool status, waiting for a file to be generated, or checking a condition.

Example 6 : Wait for a file to exist

```
set file "report.rpt"
set timeout 10
set count 0
while { ![file exists $file] && $count < $timeout } {
    puts "Waiting for $file ... ($count)"
    after 1000          ;# wait for 1 second
    incr count
}
if { [file exists $file] } {
    puts "File $file is ready!"
} else {
    puts "Timeout! File not found."
}
```

Output :

```
Waiting for report.rpt ... (0)
Waiting for report.rpt ... (1)
...
File report.rpt is ready!
(or)
Timeout! File not found
```

Key Points

- ☐ while executes the loop body as long as the condition is true.
- ☐ Use a counter or flag to control loop execution.
- ☐ Always update the loop variable inside the loop.
- ☐ Ensure the condition will eventually become false (avoid infinite loops).
- ☐ Useful in EDA for file monitoring, waiting for tool completion, and polling.

4.7 Loop Control

- Tcl provides break and continue to control the flow of loops.
- break terminates the loop immediately.
- continue skips the remaining statements in the current iteration and moves to the next iteration.
- These are useful for early exit, skipping unwanted cases, and improving script efficiency.
- Commonly used in EDA scripts for processing large lists, files, or design objects.

1. break statement

- break terminates the nearest enclosing loop.
- Control moves to the statement just after the loop.
- Useful when a certain condition is met (e.g., item found).
- Only the innermost loop is terminated.

Syntax : break

Example 1 : Exit loop when a value is found

```
set nums {1 2 3 4 5}
foreach n $nums {
    if { $n == 3 } {
        puts "Found 3, exiting loop"
        break
    }
    puts "Value : $n"
}
```

Output :

```
Value : 1
Value : 2
Found 3, exiting loop
```

Note :
- break exits only the current (innermost) loop.
- Useful to stop processing once a condition is satisfied.
- Common in search and filtering operations.

2. continue statement

- continue skips the remaining statements in the current iteration.
- Control moves to the next iteration of the loop.
- Useful to ignore unwanted items or conditions.

Syntax : continue

Example 2 : Skip even numbers

```
foreach n {1 2 3 4 5 6} {
    if { $n % 2 == 0 } {
        continue
    }
    puts "Odd number : $n"
}
```

Output :

```
Odd number : 1
Odd number : 3
Odd number : 5
```

Note :
- When continue is executed, the loop immediately goes to the next iteration.
- Useful for filtering or skipping invalid cases.
- Remaining statements in that iteration are not executed.

3. Loop termination using conditions

- Loops can be terminated early using break with a condition.
- Helps avoid unnecessary iterations and save runtime.

Example 3 : Stop after processing 5 items

```
set count 0
foreach cell [get_cells *] {
    puts "Processing cell : $cell"
    incr count
    if { $count >= 5 } {
        puts "Reached 5 cells, stopping..."
        break
    }
}
```

Output :

```
Processing cell : U1
Processing cell : U2
Processing cell : U3
Processing cell : U4
Processing cell : U5
Reached 5 cells,
  stopping...
```

4. Skipping iterations (EDA example)

- Use continue to skip unwanted objects based on a condition.
- Commonly used to skip specified cells, nets, or file types.

Example 4 : Skip buffer cells

```
foreach cell [get_cells *] {
    set type [get_attribute $cell ref_name]
    if { [string match "*BUF*" $type] } {
        continue    ;# Skip buffer cells
    }
    puts "Processing non-buffer cell : $cell"
}
```

Output :

```
Processing non-buffer
 cell : U1
Processing non-buffer
 cell : U2
Processing non-buffer
 cell : U3
...
```

5. Nested loops with break and continue

- break and continue affect only the innermost loop.
- Useful when working with nested structures like matrices or hierarchical data.

Example 5 : Nested loop with continue and break

```
for {set i 0} {$i < 3} {incr i} {
    for {set j 0} {$j < 3} {incr j} {
        if { $j == 1} {
            continue    ;# Skip j = 1
        }
        if { $i == 2 && $j == 2 } {
            break       ;# Exit inner loop
        }
        puts "i = $i , j = $j"
    }
    puts "End of i = $i"
}
```

Output :

```
i = 0 , j = 0
i = 0 , j = 2
End of i = 0
i = 1 , j = 0
i = 1 , j = 2
End of i = 1
i = 2 , j = 0
End of i = 2
```

Key Points

- ☐ break terminates the nearest loop.
- ☐ continue skips the remaining statements in the current iteration.
- ☐ Use loop control to save runtime and avoid unnecessary processing.
- ☐ Commonly used in EDA scripts for filtering, search, and early exit.
- ☐ In nested loops, break and continue affect only the innermost loop.

4.8 Nested Loops

- Nested loops mean a loop inside another loop.
- Can be implemented using for, foreach or while loops.
- Useful for multi-level data processing like matrices, hierarchical design data, or combinations.
- Widely used in EDA for processing cells, nets, layers, scenarios, etc.

1. Nested for loops

- A for loop can be placed inside another for loop.
- Useful for processing 2D data like matrices or grid structures.
- Each iteration of the outer loop executes the inner loop completely.

Example 1 : Print a 3x3 matrix using nested for loop

```
for {set i 0} {$i < 3} {incr i} {
    for {set j 0} {$j < 3} {incr j} {
        puts -nonewline "$i $j "
    }
    puts ""
}
```

Output :

```
0 0 1 2
1 0 1 2
2 0 2 2
```

2. Nested foreach loops

- A foreach loop can be placed inside another foreach loop.
- Useful for iterating over multi-dimensional lists.
- Commonly used in EDA for processing lists of cells, nets, or layers group-wise.

Example 2 : Iterate over a list of lists using nested foreach

```
set designs { {U1 U2 U3} {U4 U5 U6} }
set block_id 0
foreach block $designs {
    incr block_id
    puts "Block $block_id :"
    foreach cell $block {
        puts "  Cell : $cell"
    }
}
```

Output :

```
Block 1 :
    Cell : U1
    Cell : U2
    Cell : U3
Block 2 :
    Cell : U4
    Cell : U5
    Cell : U6
```

3. Nested while loops

- A while loop can be placed inside another while loop.
- Useful when loop conditions are not based on a fixed count.
- Commonly used for iterative algorithms or condition-based processing.

Example 3 : Nested while loop example

```
set i 0
while {$i < 2} {
    set j 0
    while {$j < 3} {
        puts "i = $i , j = $j"
        incr j
    }
    incr i
}
```

Output :

```
i = 0 , j = 0
i = 0 , j = 1
i = 0 , j = 2
i = 1 , j = 0
i = 1 , j = 1
i = 1 , j = 2
```

4. Multi-level data processing (EDA example)

- Nested loops are widely used in EDA to process hierarchical data like designs → blocks → cells → pins.
- Can be implemented using for, foreach or while loops depending on the data structure.

Example 4 : Process hierarchical design data

```
set designs {
    {top {U1 U2}}
    {sub1 {U3 U4 U5}}
}
foreach design $designs {
    set name [lindex $design 0]
    set cells [lindex $design 1]
    puts "Design : $name"
    foreach cell $cells {
        puts "  Processing cell : $cell"
    }
}
```

Output :

```
Design : top
  Processing cell : U1
  Processing cell : U2

Design : sub1
  Processing cell : U3
  Processing cell : U4
  Processing cell : U5
```

5. Mixed nested loops (Combination)

- Different loop types can be nested together (e.g., for inside foreach).
- Useful when processing complex data structures.

Example 5 : foreach with inner for loop

```
set layers {M1 M2 M3}
foreach layer $layers {
    puts "Layer : $layer"
    for {set i 0} {$i < 2} {incr i} {
        puts "  Iteration $i for $layer"
    }
}
```

Output :

```
Layer : M1
  Iteration 0 for M1
  Iteration 1 for M1
Layer : M2
  Iteration 0 for M2
  Iteration 1 for M2
Layer : M3
  Iteration 0 for M3
  Iteration 1 for M3
```

6. Real-world EDA use case

- Nested loops are commonly used to process scenarios, corners, and mode combinations in timing or signoff flows.

Example 6 : Process modes and corners

```
set modes {func scan}
set corners {ss tt ff}
foreach mode $modes {
    foreach corner $corners {
        puts "Running STA for mode=$mode corner=$corner"
    }
}
```

Output :

```
Running STA for mode=func corner=ss
Running STA for mode=func corner=tt
Running STA for mode=func corner=ff
Running STA for mode=scan corner=ss
Running STA for mode=scan corner=tt
Running STA for mode=scan corner=ff
```

Key Points

- ☐ Nested loops are powerful for multi-level data processing.
- ☐ Can be implemented using for, foreach or while.
- ☐ Useful for matrices, hierarchical design data, and scenario processing.
- ☐ Each loop type has its own use case depending on the data structure.
- ☐ Be careful with loop conditions to avoid infinite loops.
- ☐ Widely used in EDA for automation, reporting, and signoff analysis.

4.9 EDA Loop Applications

- Loops are widely used in EDA to iterate over design objects such as cells, macros, pins, nets, reports, timing paths and scenarios
- Helps in automation, analysis, reporting and design modification
- Makes scripts flexible, scalable and tool independent
- Same concept can be used in ICC2, PrimeTime, etc.

1. Loop through cells

- Iterate over all cells in the design and perform some operation (e.g., print names, check attributes, change properties)
- Useful for reports, analysis and ECOs

Example 1 : Loop through all cells and print name

```
set cells [get_cells *]
foreach c $cells {
    puts "Cell : $c"
}
```

Output :

```
Cell : U1
Cell : U2
Cell : U3
Cell : U4
...
```

2. Loop through macros

- Iterate over all macros (blocks) in the design
- Useful for placement, floorplan checks and macro analysis

Example 2 : Loop through macros

```
set macros [get_cells -hier -filter "is_hard_macro == true"]
foreach m $macros {
    puts "Macro : $m"
}
```

Output :

```
Macro : MEM1
Macro : MEM2
Macro : CPU
Macro : DSP
...
```

3. Loop through pins

- Iterate over all pins of a cell or the design
- Useful for checking connectivity, direction, or attributes

Example 3 : Loop through pins of a cell

```
set pins [get_pins U1/*]
foreach p $pins {
    puts "Pin : $p"
}
```

Output :

```
Pin : U1/A
Pin : U1/B
Pin : U1/Z
Pin : U1/CLK
```

Note :
- You can also use [get_pins *] to get all pins in the design
- Useful for finding unconnected pins or checking pin directions

4. Loop through nets

- Iterate over all nets in the design
- Useful for connectivity checks, net property reports, etc.

Example 4 : Loop through nets

```
set nets [get_nets *]
foreach n $nets {
    puts "Net : $n"
}
```

Output :

```
Net : n1
Net : n2
Net : n3
Net : n4
...
```

5. Loop through reports

- Iterate over report results (e.g., violate cells, DRC errors, etc.)
- Useful for post-processing and custom reporting

Example 5 : Loop through lines of a report file

```
set fp [open "timing_report.rpt" r]
while { [gets $fp line] >= 0 } {
    puts $line
}
close $fp
```

Output :

```
Startpoint: U1/CLK
Endpoint  : U2/D
Slack     : -0.12
...
Startpoint: U3/CLK
Endpoint  : U5/D
Slack     : -0.08
...
```

6. Loop through timing paths

- Iterate over timing paths from STA results
- Useful for analyzing critical paths and custom path reports

Example 6 : Loop through timing paths (PrimeTime style)

```
set paths [get_timing_paths -max_paths 5]
set i 1
foreach p $paths {
    puts "Path $i :"
    puts "  Slack : [get_attribute $p slack]"
    puts "  Start : [get_attribute $p startpoint]"
    puts "  End   : [get_attribute $p endpoint]"
    incr i
}
```

Output :

```
Path 1 :
  Slack : -0.12
  Start : U1/CLK
  End   : U2/D
Path 2 :
  Slack : -0.10
  Start : U3/CLK
  End   : U5/D
...
```

7. Loop through scenarios

- Iterate over all analysis scenarios (MCMM)
- Useful for running reports or checks across multiple PVT corners

Example 7 : Loop through scenarios

```
set scenarios [all_scenarios]
foreach s $scenarios {
    puts "Scenario : $s"
    current_scenario $s
    report_summary
}
```

Output :

```
Scenario : FUNC_SS_0.72V_125C
Scenario : FUNC_TT_0.80V_25C
Scenario : FUNC_FF_0.88V_-40C
Scenario : SCAN_TT_0.80V_25C
...
```

Key Points

- ☐ Loops are essential for automating repetitive EDA tasks
- ☐ Can loop through cells, macros, pins, nets, reports, timing paths and scenarios
- ☐ Helps in design analysis, reporting, verification and ECOs
- ☐ Same loop constructs (for, foreach, while) can be used with different EDA commands
- ☐ Makes scripts more efficient, consistent and scalable

4.10 Control-Flow Practice (EDA Examples)

- Control flow constructs (if, loops, switch, etc.) are widely used in EDA scripts to automate checks and decision making.
- Helps in validating design metrics like timing, area, congestion and DRC, and reporting PASS/FAIL.
- Makes the flow more robust, repeatable and time efficient.
- Below are common examples used in real projects.

1. Timing decision logic

- Check worst slack and decide PASS/FAIL.
- Useful in signoff STA (PrimeTime/ICC2)

Example 1 : Check timing and report status

```
set worst_slack [get_attribute [get_timing_paths -max_paths 1] slack]
if { $worst_slack >= 0 } {
   puts "[format "TIMING CHECK : PASS (Worst Slack = %.3f)" $worst_slack]"
} else {
   puts "[format "TIMING CHECK : FAIL (Worst Slack = %.3f)" $worst_slack]"
}
```

Output :

```
TIMING CHECK : PASS (Worst Slack = 0.012)
or
TIMING CHECK : FAIL (Worst Slack = -0.045)
```

2. Area checks

- Compare design area with a target value.
- Useful after placement or during optimization.

Example 2 : Check cell area against target

```
set area [get_attribute [current_design] cell_area]
set target_area 1000000   ;# in um^2
if { $area <= $target_area } {
   puts "AREA CHECK : PASS (Area = $area)"
} else {
   puts "AREA CHECK : FAIL (Area = $area > $target_area)"
}
```

Output :

```
AREA CHECK : PASS (Area = 850320)
or
AREA CHECK : FAIL (Area = 1254300 > 1000000)
```

3. Congestion checks

- Check global routing congestion metrics.
- Useful to detect hotspots early.

Example 3 : Check congestion value

```
set cong [get_attribute [current_design] max_congestion]
set threshold 0.8
if { $cong <= $threshold } {
   puts "CONGESTION CHECK : PASS (Congestion = $cong)"
} else {
   puts "CONGESTION CHECK : FAIL (Congestion = $cong > $threshold)"
}
```

Output :

```
CONGESTION CHECK : PASS (Congestion = 0.62)
or
CONGESTION CHECK : FAIL (Congestion = 0.93 > 0.8)
```

4. DRC checks

- Check for any design rule violations.
- Useful in signoff flows (Calibre/IC Validator).

Example 4 : Check DRC violations

```
set drc_count [get_drc_violations -count]
if { $drc_count == 0 } {
   puts "DRC CHECK : PASS (No violations)"
} else {
   puts "DRC CHECK : FAIL (Violations = $drc_count)"
}
```

Output :

```
DRC CHECK : PASS (No violations)
or
DRC CHECK : FAIL (Violations = 25)
```

5. Combined PASS/FAIL flow

- Run multiple checks and give overall status.
- Useful in nightly regressions or signoff scripts.

Example 5 : Overall design checks

```
set pass 1
# Timing check
set wns [get_attribute [get_timing_paths -max_paths 1] slack]
if { $wns < 0 } {
   puts "Timing FAIL (WNS = $wns)"
   set pass 0
} else {
   puts "Timing PASS (WNS = $wns)"
}
# Area check
set area [get_attribute [current_design] cell_area]
if { $area > 1000000 } {
   puts "Area FAIL (Area = $area)"
   set pass 0
} else {
   puts "Area PASS (Area = $area)"
}
# DRC check
set drc_count [get_drc_violations -count]
if { $drc_count > 0 } {
   puts "DRC FAIL (Violations = $drc_count)"
   set pass 0
} else {
   puts "DRC PASS (No violations)"
}
# Final result
if { $pass } {
   puts "\n*** DESIGN CHECK : PASS ***"
} else {
   puts "\n*** DESIGN CHECK : FAIL ***"
}
```

Output :

```
Timing PASS
(WNS = 0.012)
Area PASS
(Area = 850320)
DRC PASS
(No violations)

*** DESIGN
CHECK : PASS ***

-------------

(or)

Timing FAIL
(WNS = -0.045)
Area PASS
(Area = 850320)
DRC FAIL
(Violations = 12)

*** DESIGN
CHECK : FAIL ***
```

Key Points

- [ ] Use control flow constructs (if, loops) for real EDA checks.
- [ ] Common checks include timing, area, congestion and DRC.
- [ ] PASS/FAIL conditions help in automated regression flows.
- [ ] These scripts are useful in ICC2, PrimeTime, Calibre, etc.
- [ ] Make scripts more reliable, reusable and efficient.

5.1 Procedures

- A procedure (proc) is a block of TCL code that performs a specific task and can be reused.
- Helps in modular programming, code reusability and better organization.
- Widely used in EDA scripts for automation, reporting and flow control.
- A procedure can take arguments and can optionally return a value using return.

1. proc syntax

- The proc command is used to create a procedure.
- Specify the procedure name, argument list (in braces) and the procedure body (in braces).

Syntax :

```
proc proc_name {arg1 arg2 ... } {
    statements
}
```

Example 1 : A simple procedure with no arguments

```
proc greet {} {
    puts "Hello! Welcome to TCL"
}

# Calling the procedure
greet
```

Output :

```
Hello!
Welcome to TCL
```

2. Procedure with arguments

- Arguments are specified inside braces { } after the procedure name.
- The arguments act as local variables within the procedure.

Example 2 : Procedure with a single argument

```
proc greet_user {name} {
    puts "Hello $name!"
}

# Calling the procedure
greet_user "VLSI Engineer"
```

Output :

```
Hello VLSI
Engineer!
```

3. Multiple arguments

- A procedure can take multiple arguments.
- Arguments are space separated inside the braces.
- Use them inside the procedure like normal variables.

Example 3 : Procedure with multiple arguments

```
proc add {a b} {
    set sum [expr {$a + $b}]
    puts "Sum of $a and $b = $sum"
}
# Calling the procedure
add 10 20
```

Output :

```
Sum of 10 and
20 = 30
```

4. Return values

- Use the return command to return a value from a procedure.
- If no return is specified, the procedure returns an empty string.
- The returned value can be used in a variable or another command.

Example 4 : Procedure with return value

```
proc square {num} {
    set result [expr {$num * $num}]
    return $result
}

# Calling the procedure and store the return value
set val [square 5]
puts "Square = $val"
```

Output :

```
Square = 25
```

5. Using return for early exit

- return can also be used to exit from a procedure early.
- Useful for error handling and conditional checks.

Example 5 : Procedure with condition and return

```
proc check_positive {num} {
    if { $num < 0 } {
        puts "Error: Number is negative"
        return -1
    }
    return 1
}
# Calling the procedure
set status [check_positive -5]
puts "Status = $status"
```

Output :

```
Error: Number
is negative
Status = -1
```

6. Using procedures in expressions

- The return value can be used directly inside expressions or other commands.
- Helps in building complex scripts using modular functions.

Example 6 : Using procedure inside an expression

```
proc get_area {width height} {
    return [expr {$width * $height}]
}
set w 10
set h 5
set area [get_area $w $h]
puts "Area = $area"
```

Output :

```
Area = 50
```

Key Points

- [ ] Use proc to create reusable blocks of code.
- [ ] Specify arguments inside braces { }.
- [ ] Use return to send a value back to the caller.
- [ ] Procedures can have multiple arguments.
- [ ] If no return is specified, an empty string is returned.
- [ ] Widely used in EDA for modular, maintainable and scalable scripts.

5.2 Procedure Arguments

- Procedures can take different types of arguments depending on the use case.
- TCL provides positional, optional, default arguments and variable-length args.
- Argument validation is important for robust and error-free scripts.
- Useful for creating flexible, user-friendly and reusable procedures in EDA environments.

1. Positional arguments

- Arguments are passed in a fixed order (position).
- The number of arguments must match the procedure definition.

Example 1 : Procedure with positional arguments

```
proc add {a b} {
    set sum [expr {$a + $b}]
    puts "Sum of $a and $b = $sum"
}
# Calling the procedure
add 10 20
```

Output :

```
Sum of 10
and 20 = 30
```

Note :
- Arguments a and b are positional.
- They must be passed in the same order.
- If the number of arguments is incorrect, TCL gives an error.

2. Optional arguments

- Use default values by specifying them in the argument list.
- If the argument is not passed, the default value is used.

Example 2 : Procedure with optional arguments

```
proc greet {name {msg "Hello"}} {
    puts "$msg, $name!"
}

# Calling the procedure
greet "VLSI"
greet "Engineer" "Welcome"
```

Output :

```
Hello, VLSI!
Welcome, Engineer!
```

3. Default arguments (using variables)

- Default values can also be set inside the procedure using conditional checks.
- Useful when the default value needs to be computed.

Example 3 : Default argument using conditional logic

```
proc connect {host {port ""}} {
    if { $port == "" } {
        set port 22
    }
    puts "Connecting to $host on port $port"
}
# Calling the procedure
connect "server1"
connect "server2" 1000
```

Output :

```
Connecting to
server1 on port 22

Connecting to
server2 on port 1000
```

4. Using args (variable number of arguments)

- The special argument args collects all extra arguments into a list.
- Useful when the number of inputs is not fixed.
- Can be processed using foreach or llength.

Example 4 : Procedure using args

```
proc print_list {args} {
    puts "Number of arguments : [llength $args]"
    foreach item $args {
        puts "Item : $item"
    }
}

# Calling the procedure
print_list a b c d
print_list 1 2 3
```

Output :

```
Number of arguments : 4
Item : a
Item : b
Item : c
Item : d
   ...
Number of arguments : 3
Item : 1
Item : 2
Item : 3
```

Note :
- The args argument collects all extra inputs into a list.
  It is commonly used in EDA scripts for flexible commands.

5. Argument validation

- Always validate the number and type of arguments.
- Helps to avoid runtime errors and improves script robustness.

Example 5 : Argument count validation

```
proc divide {a b} {
    if { [llength [info args]] != 2 } {
        puts "Error: divide requires exactly 2 arguments"
        return
    }
    if { $b == 0 } {
        puts "Error: Division by zero!"
        return
    }
    set result [expr {$a / $b}]
    puts "Result = $result"
}
# Calling the procedure
divide 10 2
divide 10 0
```

Output :

```
Result = 5

Error: Division
by zero!
```

Note :
- Validate argument count using [info args] or [llength $args].
- Check for valid ranges, non-zero values, or correct types.
- Use meaningful error messages to help the user.

Key Points

- [ ] Procedures can take positional, optional, default and variable arguments.
- [ ] Use args for a variable number of inputs.
- [ ] Always validate arguments to make scripts robust.
- [ ] Default values can be specified in the argument list or inside the body.
- [ ] Useful for building flexible and reusable TCL procedures in EDA tools.

5.3 Return Handling

- The return command is used to send a value back to the caller from a procedure.
- A procedure can return a single value, a list, or a status information (e.g., PASS/FAIL).
- Once return is executed, the procedure exits immediately.
- Useful for passing results, data and error/status information between procedures.

1. Basic return command

- Use return to send a value back to the caller.
- The value can be a string, number, or any TCL data type.

Example 1 : Procedure returning a simple value

```
proc get_chip_name {} {
    return "my_chip"
}

# Calling the procedure
set name [get_chip_name]
puts "Chip name = $name"
```

Output :

```
Chip name =
my_chip
```

2. Returning lists

- A procedure can return a list of values.
- Useful for returning multiple results together.

Example 2 : Procedure returning a list

```
proc get_dimensions {} {
    set width 100
    set height 50
    return [list $width $height]
}

# Calling the procedure
set dims [get_dimensions]
puts "Dimensions = $dims"
puts "Width = [lindex $dims 0]"
puts "Height = [lindex $dims 1]"
```

Output :

```
Dimensions = 100 50
Width = 100
Height = 50
```

3. Returning multiple values (named list)

- Return a list with key-value pairs for clarity.
- Useful when returning related information.

Example 3 : Procedure returning a key-value list

```
proc get_cell_info {cell} {
    set area 123.5
    set ref "DFF_X1"
    return [list area $area ref $ref]
}
# Calling the procedure
set info [get_cell_info U1]
puts "Cell area = [dict get $info area]"
puts "Cell ref  = [dict get $info ref]"
```

Output :

```
Cell area = 123.5
Cell ref  = DFF_X1
```

4. Returning status information

- Procedures can return status such as PASS/FAIL, 0/1, or custom codes.
- Useful for checks (timing, area, DRC, etc.) in EDA flows.

Example 4 : Procedure returning PASS/FAIL status

```
proc check_timing {slack} {
    if { $slack < 0 } {
        return "FAIL"
    } else {
        return "PASS"
    }
}
# Calling the procedure
set result [check_timing -0.05]
puts "Timing check : $result"
```

Output :

```
Timing check :
FAIL
```

Note :
- Status can also be returned as 0/1.
- Use meaningful status strings (e.g., PASS/FAIL/SKIP).
- Useful for flow control in larger scripts.

5. Using return with condition and early exit

- return exits the procedure immediately.
- Useful to stop execution when a condition is met.

Example 5 : Early return on error condition

```
proc get_area {cell} {
    if { $cell == "" } {
        puts "Error: No cell specified"
        return -1
    }
    set area [get_attribute $cell area]
    return $area
}

# Calling the procedure
set a1 [get_area "U1"]
puts "Area = $a1"
set a2 [get_area ""]
puts "Area = $a2"
```

Output :

```
Area = 456.2

Error: No cell
specified
Area = -1
```

Key Points

- ☐ Use return to send a value back from a procedure.
- ☐ A procedure can return a single value, a list, or status information.
- ☐ return exits the procedure immediately.
- ☐ Useful for passing data between procedures and for error handling.
- ☐ Commonly used in EDA scripts for checks and reporting.

5.4 Variable Scope

- Variables in TCL can have different scopes – local or global.
- By default, variables created inside a procedure are local to that procedure.
- Use global to access/modify a global variable inside a procedure.
- Use variable to work with namespace (scoped) variables.
- Understanding scope is important to avoid accidental overwrites and to write modular, maintainable scripts.

1. Local variables

- Variables created inside a procedure are local by default.
- They exist only within that procedure and are not visible outside.

Example 1 : Local variable inside a procedure

```
proc show_message {} {
    set msg "Hello from procedure"
    puts "Inside proc: $msg"
}

# Calling the procedure
show_message

# Trying to access outside (will error)
# puts "Outside: $msg"
```

Output :

```
Inside proc:
Hello from procedure
```

Note :
- $msg is local to the procedure.
- It cannot be accessed outside the procedure.
- TCL gives an error if you try to use it outside.

2. Global variables

- Global variables are accessible from anywhere in the script (inside and outside procedures).
- Use the global command inside a procedure to access/modify global variables.

Example 2 : Using global variables

```
set chip_name "TOP"    ;# Global variable
set count 0

proc increment {} {
    global count chip_name
    incr count
    puts "Chip: $chip_name, Count: $count"
}

increment
increment

puts "Final Count = $count"
```

Output :

```
Chip: TOP, Count: 1
Chip: TOP, Count: 2
Final Count = 2
```

Note :
- Use global to access global variables inside a procedure.
- Changes made inside the procedure affect the global variable.
- Global variables should be used carefully to avoid conflicts.

3. The global command

- The global command links a global variable to be used inside a procedure.
- Multiple variables can be specified.

Example 3 : Using global with multiple variables

```
set a 10
set b 20

proc print_sum {} {
    global a b
    set sum [expr {$a + $b}]
    puts "Sum = $sum"
}
print_sum
```

Output :

```
Sum = 30
```

Note :
- Use global var1 var2 ... to access multiple global variables.
- Without global, TCL will treat them as local variables.

4. The variable command (namespace variables)

- The variable command is used inside a namespace to access namespace variables.
- Useful for creating scoped/global variables within a namespace (commonly used in large EDA scripts).

Example 4 : Using variable in a namespace

```
namespace eval design {
    variable version 1.0
    variable author "VLSI_Team"
}
proc show_info {} {
    variable design::version
    variable design::author
    puts "Version : $version"
    puts "Author  : $author"
}
show_info
```

Output :

```
Version : 1.0
Author  : VLSI_Team
```

Note :
- Use variable to access namespace scoped variables.
- Syntax : variable namespace::var_name
- Helps in modular and well-organized scripts.

5. Procedure scope summary

- Understand what variables are visible inside and outside a procedure.
- Plan variable usage to avoid unintended overwrites.

Variable Scope in a Procedure

Global Variables	Procedure (proc)	Namespace Variables
• Accessible everywhere • Use global inside proc • Persist for entire script	Local Variables • Created inside proc • Visible only within proc • Automatically destroyed	• Scoped within namespace • Use variable command • Useful for modular scripts

Key Points

- ☐ Local variables are local to the procedure.
- ☐ Use global to access global variables inside a procedure.
- ☐ Use variable for namespace (scoped) variables.
- ☐ Understand scope to avoid conflicts and make reusable scripts.
- ☐ Widely used in large EDA flows and automation.

5.5 Reusable Procedures

- Reusable procedures (procs) help in writing modular, clean and maintainable TCL code.
- They can be generic (applicable to many designs) or design-specific (tailored to a particular flow).
- Parameterized procedures make them flexible for different use cases.
- Utility procedures are commonly used in EDA scripts for frequent tasks.
- Reusable code saves time, reduces errors and improves productivity.

1. Generic procedures

- Generic procedures perform common tasks and can be used across different projects.
- They are not tool or design specific.

Example 1 : Generic procedure to print a message

```
proc print_msg {msg} {
    puts "INFO: $msg"
}

# Calling the procedure
print_msg "TCL makes automation easy!"
```

Output :

```
INFO: TCL makes automation easy!
```

2. Parameterized procedures

- Use parameters (arguments) to make the procedure flexible.
- Same procedure can work with different inputs.

Example 2 : Parameterized procedure to add two numbers

```
proc add {a b} {
    set sum [expr {$a + $b}]
    return $sum
}

# Calling the procedure
set result [add 10 20]
puts "Sum = $result"
```

Output :

```
Sum = 30
```

3. Utility procedures

- Utility procedures perform commonly used helper functions in EDA scripts (e.g., file handling, report parsing, string processing, logging).
- They can be reused in multiple flows.

Example 3 : Utility procedure to check file existence

```
proc file_exists {filename} {
    if { [file exists $filename] } {
        puts "File $filename exists"
        return 1
    } else {
        puts "File $filename not found"
        return 0
    }
}
# Calling the procedure
set status [file_exists "design.v"]
```

Output :

```
File design.v exists
```

4. Design-specific procedures

- These procedures are tailored for a specific design or flow.
- They use tool commands (e.g., ICC2, PrimeTime) to perform design tasks.
- Helps in automating repetitive tasks in the implementation flow.

Example 4 : Design-specific procedure to report cell count (ICC2)

```
proc report_cell_count {} {
    set cells [get_cells *]
    set count [sizeof_collection $cells]
    puts "Total cells in design : $count"
    return $count
}

# Calling the procedure
set cnt [report_cell_count]
```

Output :

```
Total cells in design : 125340
```

Note :
- Use design-specific procedures to capture common EDA tasks.
- It improves consistency and reduces manual effort.
- Can be extended for macros, nets, pins, timing reports, etc.

5. Building a reusable procedure library

- Store frequently used procedures in a separate TCL file (e.g., utils.tcl).
- Source the file in your main script.
- Makes your code modular, reusable and easy to maintain.

Example 5 : Using a procedure library

File : utils.tcl

```
# Utility procedures
proc print_banner {msg} {
    puts "================"
    puts "  $msg"
    puts "================"
}

proc get_time {} {
  return [clock format [clock seconds]
  -format "%Y-%m-%d %H:%M:%S"]
}
```

File : main.tcl

```
# Source the utility library
source utils.tcl

print_banner "Design Run Started"
puts "Current Time : [get_time]"
```

Output :

```
==================
Design Run Started
==================
Current Time : 2025-09-04 22:30:15
```

Key Points

- [ ] Reusable procedures improve productivity and code quality.
- [ ] Generic procedures can be used across multiple projects.
- [ ] Parameterized procedures make code flexible.
- [ ] Utility procedures handle common tasks (file, string, logging, etc.).
- [ ] Design-specific procedures automate EDA tool commands.
- [ ] Keep reusable procedures in a separate TCL library file.
- [ ] Use meaningful names and comments for better maintainability.

5.6 Procedure Debugging

- Debugging helps identify and fix issues in procedures.
- Check if arguments are passed correctly, the procedure executes as expected, and the results are correct.
- Useful debugging techniques include print statements, variable checks, return value verification and error messages.
- Common in EDA scripts to ensure correct functionality and reliable automation.

1. Checking procedure arguments

- Verify that the correct number of arguments are passed.
- Print the argument values to ensure they are as expected.
- Use argument validation to catch invalid inputs early.

Example 1 : Debug arguments passed to a procedure

```
proc check_args {name width height} {
    puts "=== Debug: Arguments ==="
    puts "Name   : $name"
    puts "Width  : $width"
    puts "Height : $height"
}

# Calling the procedure
check_args "U1" 100 50
```

Output :

```
=== Debug: Arguments ===
Name   : U1
Width  : 100
Height : 50
```

2. Monitoring procedure execution

- Use puts statements at key points to trace the flow.
- Print messages at start, during execution and at end.
- Helps to identify where the procedure is failing or hanging.

Example 2 : Trace execution flow inside a procedure

```
proc process_cell {cell} {
    puts "Start: process_cell"
    puts "Processing cell : $cell"
    # some processing ...
    puts "End: process_cell"
}

# Calling the procedure
process_cell "U1"
```

Output :

```
Start: process_cell
Processing cell : U1
End: process_cell
```

3. Debugging procedures (using error handling)

- Use catch to handle errors and print meaningful messages.
- Helps to debug and continue script execution without crashing.
- Useful when calling tool commands inside procedures.

Example 3 : Using catch for error handling

```
proc get_cell_area {cell} {
    if { [catch {get_attribute $cell area} result] } {
        puts "Error: Cannot get area for cell $cell"
        puts "Details: $result"
        return -1
    } else {
        puts "Area of $cell : $result"
        return $result
    }
}
# Calling the procedure
set area [get_cell_area "U1"]
```

Output :

```
Area of U1 : 45.6
```

4. Checking procedure results

- Verify the return value of the procedure.
- For list results, print each element or its length.
- Use conditional checks to validate the results.

Example 4 : Check and validate procedure results

```
proc get_pins {cell} {
    set pins [get_pins -of $cell]
    return $pins
}

# Calling the procedure
set pin_list [get_pins "U1"]
puts "Pin list : $pin_list"
puts "Number of pins : [llength $pin_list]"
```

Output :

```
Pin list : U1/A U1/B U1/Z
Number of pins : 3
```

5. Debugging with return status

- Procedures can return status codes (0/1) to indicate success or failure.
- Check the status and take appropriate action.
- Useful in larger scripts for error control.

Example 5 : Procedure returning status

```
proc check_timing {path} {
    # Dummy timing check
    if { $path == "" } {
        puts "Error: Empty path name"
        return 1
    }
    puts "Timing check completed for $path"
    return 0
}

# Calling the procedure
set status [check_timing "U1/A - U1/Z"]
if { $status != 0 } {
    puts "Timing check failed !"
} else {
    puts "Timing check passed !"
}
```

Output :

```
Timing check completed for U1/A - U1/Z

Timing check passed !
```

Key Points

- ☐ Always check the number and values of procedure arguments.
- ☐ Use puts statements to trace procedure execution.
- ☐ Use catch for error handling inside procedures.
- ☐ Verify and validate procedure results (return values or lists).
- ☐ Use meaningful debug messages for easier troubleshooting.
- ☐ Return status information (0/1) for success or failure.
- ☐ Commonly used in EDA scripts to make them robust and reliable.
- ☐ Remove or comment debug prints in final production scripts.

5.7 Source Command

- The source command is used to read and execute commands from another Tcl file.
- It loads the specified Tcl file and runs it in the current interpreter (same session).
- Useful for modular scripts, common setup files and reusable procedure libraries.
- Helps in managing large scripts by splitting into multiple files.

1. Basic source command

- Use source <file> to load and execute a Tcl file.
- The file path can be relative or absolute.

Example 1 : Sourcing a simple Tcl file

```
# main.tcl
puts "Before sourcing..."
source "example.tcl"
puts "After sourcing..."

# example.tcl
puts "This is from example.tcl"
set var 100
```

Output :

```
Before sourcing...
This is from example.tcl
After sourcing...
```

Note :

- The commands in example.tcl are executed in the current session.
- Variables defined in the sourced file are available in the calling script.

2. Loading files with path

- You can use relative or absolute paths to source a file.
- Use [file join] to build portable paths.

Example 2 : Using relative and absolute paths

```
# Relative path
source "./scripts/init.tcl"

# Absolute path
source "/home/user/eda/common.tcl"

# Using file join (recommended)
set script_dir "/home/user/eda"
set file_name "init.tcl"
source [file join $script_dir $file_name]
```

Note :

- Relative paths make scripts portable.
- file join handles path separators automatically.
- Useful when scripts are moved across different machines.

3. Script dependencies

- A main script can source multiple files.
- The sourced files can themselves source other files.
- Maintain correct order to satisfy dependencies.

Example 3 : Sourcing multiple files with dependencies

```
# main.tcl
source "config.tcl"  ;# Load variables
source "utils.tcl"   ;# Load utility procedures
source "design.tcl"  ;# Use the procedures

run_flow
```

File dependency order:

1. config.tcl (variables, settings)
2. utils.tcl (procedures)
3. design.tcl (main flow)

4. Common setup files

- In EDA flows, a common setup file is sourced to set environment variables, tool options, libraries, and design settings.
- This avoids repeating the same commands in every script.

Example 4 : Typical setup.tcl file

```
# setup.tcl
# Tool and design setup
set search_path "/libs /tech /rtl"
set target_library "slow.db"
set link_library "* $target_library"
set design_name "my_chip"
set_app_var timing_enable_multiple_clocks true

puts "Environment setup completed!"
```

Using in main script :

```
# main.tcl
source "setup.tcl"

# Continue with flow
read_verilog top.v
link_design $design_name
```

Note :

- Keep common settings in a single setup file.
- Makes scripts cleaner and easier to maintain.
- Changes in environment need to be done only in one place.

5. Reusable Tcl libraries

- Frequently used procedures can be stored in separate Tcl library files and sourced whenever needed.
- Helps in creating a personal or team utility library.

Example 5 : utils.tcl (reusable procedures)

```
# utils.tcl
proc print_banner {msg} {
    puts "=================="
    puts "  $msg"
    puts "=================="
}

proc get_cell_count {} {
    set cells [get_cells *]
    return [sizeof_collection $cells]
}
```

Using the library :

```
# main.tcl
source "utils.tcl"

print_banner "Design Info"
set count [get_cell_count]
puts "Total cells : $count"
```

Output :

```
==================
    Design Info
==================
Total cells : 125430
```

6. Best practes

- [ ] Keep scripts modular and split into multiple files.
- [ ] Use meaningful file names (e.g., setup.tcl, utils.tcl, flow.tcl).
- [ ] Maintain proper dependency order while sourcing files.
- [ ] Use relative paths or [file join] for portability.
- [ ] Store reusable procedures in a separate library file.
- [ ] Avoid circular dependencies between scripts.
- [ ] Add comments and version information in sourced files.
- [ ] Commonly used in ICC2, PrimeTime and other EDA tools.

Key Takeaways

- [ ] source loads and executes another Tcl file in the current session.
- [ ] Useful for common setup files and reusable procedure libraries.
- [ ] Helps manage large scripts and their dependencies.
- [ ] Makes EDA automation scripts modular, clean and maintainable.
- [ ] Always follow good scripting practices for portability and reusability.

5.8 Modular Tcl Design

- Modular Tcl design means splitting the code into multiple files based on functionality (setup, procedures, stages, config, etc).
- It improves reusability, readability and maintainability.
- Common procedures and utility functions are stored in separate Tcl files and sourced when needed.
- Widely used in EDA flows (ICC2, PrimeTime, etc) to manage complex scripts.

1. Main script

- The main script controls the overall flow.
- It sources other Tcl files (setup, procedures, stage scripts).
- It defines the execution sequence.

Example 1 : main.tcl (top level script)

```
# main.tcl
# Top level script

# Source setup and common files
source "setup.tcl"
source "proc/common_proc.tcl"

# Run flow stages
source "flow/place.tcl"
source "flow/route.tcl"
source "flow/signoff.tcl"
```

Output :

```
Setup completed...
Placement started...
Routing started...
Signoff started...
```

2. Setup script

- The setup script initializes environment, variables, tool settings.
- It is sourced at the beginning of the main script.

Example 2 : setup.tcl

```
# setup.tcl
# Tool and environment setup
set design_name "my_chip"
set tech "7nm"
set search_path "./lib ./rtl"
set_app_var search_path $search_path
set_app_var target_library "slow.db"
set_app_var link_library "* $target_library"

puts "Setup completed for $design_name"
```

Output :

```
Setup completed
for my_chip
```

3. Common procedures

- Common utility procedures are stored in a separate file.
- These can be reused across multiple projects and flows.
- Includes helper functions for reporting, checks, logging, etc.

Example 3 : proc/common_proc.tcl

```
# common_proc.tcl
# Utility procedure
proc print_banner {msg} {
    puts "=================="
    puts "    $msg"
    puts "=================="
}
```

Output :

```
==================
    Design Flow
==================
```

4. Stage-specific scripts

- Each stage of the flow (placement, CTS, routing, signoff, etc) is written in a separate script.
- This keeps the flow organized and easy to maintain.
- You can run individual stages when needed.

Example 4 : flow/place.tcl

```
# flow/place.tcl
# Placement stage
print_banner "Placement"
create_floorplan
place_opt
report_utilization > reports/place.rpt
puts "Placement completed!"
```

Output :

```
================
    Placement
================
Placement completed!
```

Note :
- Similarly create scripts for CTS (cts.tcl), routing (route.tcl), and signoff (signoff.tcl).
- Each script can use common procedures and configuration variables.
- Helps in running partial flows (e.g., only placement).

5. Configuration scripts

- Configuration scripts store project-specific variables, file paths, and settings.
- By changing the config file, the same flow can be used for different designs or corners.
- Useful for multi-corner, multi-mode (MCMM) flows.

Example 5 : config.tcl

```
# config.tcl
# Design configuration
set design_name "my_chip"
set top_module "chip_top"
set rtl_path "./rtl"
set constraint_file "./constraints.sdc"
set tech_node "7nm"
set corners {ss tt ff}
set modes {func scan}
```

Output :

```
Design  : my_chip
Top     : chip_top
Tech    : 7nm
Corners : ss tt ff
Modes   : func scan
```

Project structure (Example)

```
project/
├── main.tcl              # Top level script
├── setup.tcl             # Environment setup
├── config.tcl            # Design configuration
├── proc/
│   └── common_proc.tcl   # Utility procedures
├── flow/
│   ├── place.tcl         # Placement stage
│   ├── cts.tcl           # Clock tree synthesis stage
│   ├── route.tcl         # Routing stage
│   └── signoff.tcl       # Signoff stage
└── reports/              # Output reports
```

Key Points

- [ ] Use a main script to control the overall flow.
- [ ] Keep setup, common procedures, stage scripts and config in separate files.
- [ ] Follow a clear directory structure.
- [ ] Make scripts reusable, modular and easy to maintain.
- [ ] Widely used in ICC2, PrimeTime and other EDA tools.

5.9 Tcl Project Structure

- A well-organized project structure keeps Tcl scripts modular, readable and easy to maintain.
- Each stage of the design flow is written in a separate file.
- Common procedures and configuration files are shared across stages.
- The main run.tcl controls the overall flow by sourcing other scripts in the correct order.

1. Typical directory structure

- All Tcl scripts are kept in a project directory.
- Each file has a specific purpose (setup, flow stages, utilities).
- Additional directories (logs, reports, results) are used for outputs.

Example 1 : Tcl project directory structure

```
tcl_project/
├── run.tcl              # Main script
├── setup.tcl            # Environment setup
├── common.tcl           # Common procedures
├── floorplan.tcl        # Floorplan stage
├── power.tcl            # Power planning stage
├── placement.tcl        # Placement stage
├── cts.tcl              # Clock tree synthesis stage
├── route.tcl            # Routing stage
├── signoff.tcl          # Signoff / final checks
├── logs/                # Log files
├── reports/             # Reports
└── results/             # Output database / saves
```

2. run.tcl (Main script)

- Controls the overall flow and sources other Tcl files.
- Defines the execution order of different stages.
- Can include basic configuration and error handling.

Example 2 : run.tcl

```
# run.tcl - Main script
puts "==== Starting VLSI Flow ===="
source "setup.tcl"          ;# Environment setup
source "common.tcl"         ;# Common procedures
source "floorplan.tcl"      ;# Floorplan stage
source "power.tcl"          ;# Power planning
source "placement.tcl"      ;# Placement
source "cts.tcl"            ;# Clock tree synthesis
source "route.tcl"          ;# Routing
source "signoff.tcl"        ;# Signoff checks
puts "==== VLSI Flow Completed ===="
```

Note :
- run.tcl only controls the flow, actual commands are inside stage-specific scripts.
- The order of sourcing is important.

3. setup.tcl (Environment setup)

- Initializes tool environment, libraries, technology files and design variables.
- Commonly sets search paths, target library, link library, etc.

Example 3 : setup.tcl

```
# setup.tcl - Environment setup
set design_name     "my_chip"
set tech            "7nm"
set search_path     "./lib ./rtl"
set target_library  "slow.db"
set link_library    "* slow.db"
set app_var timing_enable_multiple_clocks true

puts "Environment setup completed for $design_name"
```

4. common.tcl (Common procedures)

- Contains reusable procedures and utility functions.
- Used for reporting, checks, file handling, logging, etc.

Example 4 : common.tcl

```
# common.tcl - Common procedures
proc print_banner {msg} {
    puts "====================="
    puts " $msg"
    puts "====================="
}

proc check_design {} {
    puts "Running basic design checks..."
    # add common check commands here
}
```

5. Stage-specific scripts

- Each stage of the flow is written in a separate file.
- This keeps the code modular and easy to debug/maintain.
- Only stage-specific commands are included in each file.

Script File	Purpose / Key Tasks
floorplan.tcl	Initialize floorplan, create core, IO, macrocells, etc.
power.tcl	Power planning, power grid, straps, voltage areas
placement.tcl	Place standard cells, optimization, placement checks
cts.tcl	Clock tree synthesis, clock optimization, CTS reports.
route.tcl	Global + detailed routing, routing optimization, DRC check.
signoff.tcl	Final checks, signoff DRC/LVS/STA, reports, save design

Key Points

- [ ] Use a clear and consistent file naming convention.
- [ ] Keep scripts modular and stage-wise.
- [ ] Use common.tcl for reusable procedures.
- [ ] run.tcl should control the complete flow.
- [ ] Maintain logs/reports directories for outputs.
- [ ] Makes the flow reusable across projects and technologies.

5.10 Modular Tcl Proctice

- Modular Tcl practice helps in writing clean, reusable and maintainable scripts for real design projects.
- Large scripts are split into smaller, logical files (setup, flow stages, common procedures, etc.).
- Common procedures are shared across multiple projects.
- A well-organized project structure improves productivity, readability and collaboration.

1. Building reusable scripts

- Write generic procedures and utility functions.
- Parameterize scripts so they can be used for different designs.
- Keep common code in separate files (e.g., common.tcl or utils.tcl).
- Use meaningful names and comments.

Example 1 : Reusable utility procedure (in utils.tcl)

```
# utils.tcl
proc print_banner {msg} {
    puts "====================="
    puts "  $msg"
    puts "====================="
}
```

Output :

```
=================
Design Run Started
=================
```

2. Splitting large scripts

- Break the flow into smaller, stage-specific scripts.
- Keep each script focused on a specific task.
- Use the source command to include and run each script.
- Makes debugging and maintenance easier.

Example 2 : Splitting a large flow into multiple scripts

```
# run.tcl (main script)
source "setup.tcl"
source "common.tcl"
source "floorplan.tcl"
source "power.tcl"
source "placement.tcl"
source "cts.tcl"
source "route.tcl"
source "signoff.tcl"
```

Flow execution order



3. Sharing procedures

- Common procedures are stored in a separate file (e.g., common.tcl).
- These can be sourced in multiple projects.
- Helps maintain consistency across teams and designs.
- Include utility functions for reporting, checks, logging, etc.

Example 3 : Shared procedure (in common.tcl)

```
# common.tcl
proc get_time {} {
    return [clock format [clock seconds]
    -format "%Y-%m-%d %H:%M:%S"]
}
```

Output :

```
2025-09-04 22:30:15
```

4. Maintaining Tcl projects

- Use a clear directory structure with separate folders for scripts, logs, reports and results.
- Keep a common setup file for environment and variables.
- Use version control (e.g., Git) for tracking changes.
- Comment your code and follow naming conventions.
- Update and enhance procedures as the project evolves.

Example 4 : Typical project directory structure

```
project/
├── run.tcl          # Main script
├── setup.tcl        # Environment setup
├── common.tcl       # Common procedures
├── utils.tcl        # Utility functions
├── flow/
│   ├── floorplan.tcl  # Floorplan stage
│   ├── power.tcl      # Power planning stage
│   ├── placement.tcl  # Placement stage
│   ├── cts.tcl        # Clock tree synthesis stage
│   ├── route.tcl      # Routing stage
│   └── signoff.tcl    # Signoff stage
├── logs/            # Log files
├── reports/         # Reports
└── results/         # Output databases / saves
```

5. Best practices

- Keep scripts modular, reusable and well-documented.
- Use meaningful file and procedure names.
- Avoid hardcoding paths; use variables and configuration files.
- Maintain a consistent coding style across the project.
- Test individual scripts before running the full flow.

Practice	Description
Modular design	Split scripts by function (setup, flow, utils, etc.)
Reusable procedures	Store common procedures in separate files
Clear naming	Use meaningful names for files and procedures
Configuration files	Use config variables for flexibility
Version control	Track changes using Git or similar tools
Documentation	Add comments and usage instructions
Testing	Run and verify each script independently

Key Takeaways

- [ ] Write reusable and parameterized Tcl scripts.
- [ ] Split large scripts into smaller, stage-specific files.
- [ ] Share common procedures across multiple projects.
- [ ] Maintain a clean project structure with setup, flow, logs and reports.
- [ ] Use best practices for easy maintenance and collaboration.
- [ ] Modular Tcl design improves productivity and code quality.

6.1 File Fundamentals

- File handling is used to read, write and process data from files.
- In EDA flows, Tcl scripts work with reports, logs, constraint files, and configuration files.
- Understanding file paths, names and extensions is the first step before performing file operations.
- Both relative and absolute paths can be used to access files.

1. Files

- A file is a named collection of data stored on disk.
- Tcl can open, read, write, append and close files.
- Files can be text files (e.g., .rpt, .tcl, .log) or other formats.
- In EDA, files typically contain reports, constraints, netlists, etc.

Example 1 : Opening and reading a file

```
# Open a file for reading
set fp [open "report.rpt" r]

# Read entire file content
set data [read $fp]

# Close the file
close $fp
```

Output :

File opened and content read into variable 'data'

2. File paths

- A file path specifies the location of a file in the directory structure.
- Paths can be relative (from current directory) or absolute (full path from root).
- Use forward slash (/) in Tcl (works on all platforms).

Example 2 : File path examples

Path Type	Example	Description
Current directory	./report.rpt	File in current directory
Relative path	scripts/setup.tcl	File in sub-directory
Parent directory	../common.tcl	File in parent directory
Absolute path	/home/user/project/report.rpt	Full path from root

3. File names

- File name identifies the file within a directory.
- It can contain letters, numbers, underscore (_), hyphen (-) etc.
- Use meaningful file names for easy identification.
- Avoid spaces in file names (use underscore instead).

Example 3 : Valid and invalid file names

Valid file names	Invalid file names
✓ report.rpt	× my report.rpt (contains space)
✓ setup.tcl	× design@1.tcl (special character)
✓ my_design.log	× test?.log (special character)
✓ floorplan_01.tcl	× floorplan!.tcl (special character)
✓ timing-report.txt	

4. File extensions

- File extension indicates the type or format of the file.
- It is the part after the last dot (.).
- Common extensions in EDA: .tcl, .rpt, .log, .sdc, .v, .lib, .txt, etc.
- Extensions help in identifying the purpose of the file.

Example 4 : Common file extensions in EDA

Extension	Purpose
.tcl	Tcl script file
.rpt	Report file (timing, area, etc.)
.log	Log file (tool execution logs)
.sdc	Constraint file (SDC)
.v	Verilog netlist file
.lib	Library file (timing/power)
.txt	Text file (general purpose)

5. Relative paths

- Relative path is specified with respect to the current working directory.
- It does not start with / (root).
- Common notations: ./ (current directory) and ../ (parent directory).
- Useful for making scripts portable across different environments.

Example 5 : Using relative paths

```
# Current directory file
open "./report.rpt" r

# File in sub-directory
open "scripts/run.tcl" r

# File in parent directory
open "../common/common.tcl" r
```

Directory view :

```
project/
├── run.tcl
├── scripts/
│   └── run.tcl
└── common/
    └── common.tcl
```

6. Absolute paths

- Absolute path specifies the full path from the root directory (/).
- It always starts with / (on Unix/Linux).
- It will point to the exact file location, regardless of current directory.
- Useful when the file location is fixed or when scripts are run from different directories.

Example 6 : Using absolute paths

```
# Absolute path to a file
open "/home/user/eda/reports/timing.rpt" r

# Another example
open "/data/projects/chip1/setup.tcl" r
```

Note :

- Absolute paths make scripts less portable.
- Use environment variables or relative paths when possible.

Key Takeaways

- ☐ File is a named collection of data.
- ☐ File path specifies the location of a file.
- ☐ File name should be meaningful and avoid special characters.
- ☐ File extension indicates the file type.
- ☐ Relative paths are portable and depend on current directory.
- ☐ Absolute paths specify the full location from root.
- ☐ Understanding file fundamentals is essential for report parsing.

Author : Konda Tharun Vivekananda

"Good file organization leads to smooth automation !"

6.2 Opening & Closing Files

- The open and close commands are used to work with files in Tcl.
- A file is opened using the open command and returns a file handle.
- The file handle is used to read, write or append data to the file.
- Always close the file using the close command to release resources.
- Different modes (read, write, append) allow different types of access.

1. open command

- The open command opens a file and returns a file handle.
- Syntax : open <fileName> <mode>
- If the file does not exist, it will be created in write or append mode.

Example 1 : Opening a file in different modes

```
# Open file in read mode
set fp [open "report.rpt" r]

# Open file in write mode (creates new file / overwrites)
set fp [open "output.txt" w]

# Open file in append mode (adds to existing file)
set fp [open "log.txt" a]
```

2. File modes

- The mode specifies how the file is accessed.
- Common modes are read (r), write (w) and append (a).
- Use the correct mode based on the operation you want to perform.

Example 2 : File open modes

Mode	Description	Behavior
r	Read mode	File must exist, error if not
w	Write mode	Creates new file or overwrites existing file
a	Append mode	Creates file if not exists, appends to end of file

3. File handles

- When a file is opened, Tcl returns a file handle (channel).
- The handle is used in other commands like gets, puts, close.
- You can use any variable name to store the handle (e.g., fp, out, log).
- File handles are like pointers to the opened file.

Example 3 : Using file handles

```
set fp [open "report.rpt" r]
puts "File handle is : $fp"

# use the handle for read/write
gets $fp line
puts "First line: $line"
close $fp
```

Output :

```
File handle is : file3
First line: Design Report
```

4. Reading from a file (Read mode)

- Open the file in read mode (r).
- Use gets to read one line at a time.
- Use eof to check end of file.
- Always close the file after reading.

Example 4 : Reading a file

```
set fp [open "report.rpt" r]
while {![eof $fp]} {
    gets $fp line
    puts $line
}
close $fp
```

Note :

- gets reads one line at a time
- eof returns 1 when end of file is reached
- Always use close to release the file handle.

5. Writing to a file (Write mode)

- Open the file in write mode (w).
- Use puts to write data to the file.
- If the file exists, it will be overwritten.
- Close the file after writing.

Example 5 : Writing to a file

```
set fp [open "output.txt" w]
puts $fp "Design Name : my_chip"
puts $fp "Total Cells : 125430"
puts $fp "WNS : 0.12"
close $fp
```

Output file content :

```
Design Name : my_chip
Total Cells : 125430
WNS         : 0.12
```

6. Appending to a file (Append mode)

- Open the file in append mode (a).
- Use puts to add data at the end of the file.
- If the file does not exist, it will be created.
- Useful for log files and incremental reports.

Example 6 : Appending to a file

```
set fp [open "log.txt" a]
puts $fp "Run completed at [clock format [clock seconds]]"
puts $fp "WNS : 0.12"
close $fp
```

Key Takeaways

- [ ] Use open to open a file and get a file handle.
- [ ] Use correct mode (r / w / a) based on the operation.
- [ ] Use the file handle with commands like gets and puts.
- [ ] Always close the file using close.
- [ ] Read mode is for existing files, write mode overwrites, append mode adds to the end.
- [ ] File handling is essential for parsing reports, logs and automation in EDA flows.

6.3 Reading Files

- Reading files is a common operation in Tcl for processing reports, logs and data.
- You can read the complete file at once or read it line-by-line.
- Commands like read, gets and eof are used to access file content.
- Proper end-of-file handling is important to avoid errors.

1. read command (Read complete file)

- The read command reads the entire file content into a string.
- Useful for small files (e.g., configuration files, short reports).
- Syntax : read <FileHandle>
- After reading, the file should be closed.

Example 1 : Reading complete file using read

```
set fp [open "design.rpt" r]
set content [read $fp]
puts "===== File Content ====="
puts $content
close $fp
```

Sample file (design.rpt) :

```
Design  : my_chip
Cells   : 125430
WNS     : 0.12
TNS     : -12.34
```

Output :

```
===== File Content =====
Design  : my_chip
Cells   : 125430
WNS     : 0.12
TNS     : -12.34
```

2. gets command (Read line-by-line)

- The gets command reads one line at a time from the file.
- Returns the number of characters read, or -1 at end of file.
- Each line is stored in a variable for processing.
- Useful for large files (e.g., log files, timing reports).

Example 2 : Reading file line-by-line using gets

```
set fp [open "design.rpt" r]
puts "===== Line by Line Reading ====="
while {[gets $fp line] >= 0} {
    puts "Line: $line"
}
close $fp
```

Output :

```
===== Line by Line Reading =====
Line: Design  : my_chip
Line: Cells   : 125430
Line: WNS     : 0.12
Line: TNS     : -12.34
```

3. End-of-file handling

- The gets command returns -1 when the end of file is reached.
- You can also use the eof command to check if end of file is reached.
- Always close the file after processing.

Example 3 : Using eof for file reading

```
set fp [open "design.rpt" r]
while {![eof $fp]} {
    gets $fp line
    puts $line
}
close $fp
```

Note :

- eof returns 1 if end of file is reached, otherwise 0.
- Both gets and eof can be used together.
- Always close the file to release the handle.

4. Comparing read vs gets

Aspect	read (complete file)	gets (line-by-line)
What it does	Reads entire file into a single string	Reads one line at a time
Return value	File content (string)	Number of characters read, or -1 at EOF
Memory usage	Higher (loads whole file)	Lower (reads line by line)
Use case	Small files, config files	Large files, reports, logs
EOF handling	Not required	Required (use eof or check return value)
Example	set data [read $fp]	gets $fp line

5. Practical example (Parsing a report)

- Read a timing report line-by-line and extract WNS value.
- Demonstrates real usage in EDA flows.

Example 4 : Extracting WNS from a report

```
set fp [open "timing.rpt" r]
set wns ""
while {[gets $fp line] >= 0} {
    if {[string match "*WNS*" $line]} {
        set tokens [split $line ":"]
        set wns [string trim [lindex $tokens 1]]
    }
}
close $fp
puts "WNS = $wns"
```

Sample timing.rpt :

```
Startpoint : U1/CLK
Endpoint   : U2/D
WNS        : 0.12
TNS        : -12.34
```

Output :

```
WNS = 0.12
```

Key Takeaways

- [ ] Use read to read the complete file content at once.
- [ ] Use gets to read files line-by-line.
- [ ] Use eof or check the return value of gets for end-of-file handling.
- [ ] Choose the method based on file size and use case.
- [ ] Always close the file after reading.
- [ ] File reading is essential for parsing reports, logs and automation in EDA.

6.4 Writing Files

- Writing files is used to store data, logs and reports.
- The puts command is used to write data to a file.
- Well formatted reports with headers and tables are useful for analysis and debugging.
- Tcl file writing is widely used in ICC2, PrimeTime and automation scripts.

1. puts to file

- Use puts with a file handle to write data to a file.
- Open the file in write (w) or append (a) mode.
- You can write single lines or multiple lines using puts.
- Close the file after writing to release resources.

Example 1 : Writing data to a file using puts

```
# Open file in write mode
set fp [open "report.txt" w]
puts $fp "This is a sample report"
puts $fp "Generated using Tcl script"
close $fp
```

Output file (report.txt) :

```
This is a sample report
Generated using Tcl script
```

2. Report generation

- Reports can be generated to capture design information (timing, area, power, etc.).
- Use Tcl commands (report_timing, report_area, etc.) and redirect output to files.
- You can also format custom reports using puts.

Example 2 : Generating a timing report

```
set fp [open "timing_report.txt" w]
puts $fp "====== Timing Report ======"
puts $fp "Generated on : [clock format [clock seconds]]"
puts $fp "============================"
# Run ICC2 command and redirect output
redirect -file $fp {report_timing -max_paths 10}
close $fp
```

3. Headers

- Headers make reports more readable and professional.
- Include title, design name, date/time and column names.
- Use puts to print header lines with separators.

Example 3 : Adding header to a report

```
set fp [open "area_report.txt" w]
puts $fp "================================"
puts $fp "          AREA REPORT"
puts $fp "  Design : my_chip"
puts $fp "  Date   : [clock format [clock seconds]]"
puts $fp "================================"
close $fp
```

4. Tables

- Tables help in presenting data in a structured format.
- Use format command to align columns.
- Define column widths and print each row using puts.
- Useful for reports like timing, area, power, etc.

Example 4 : Writing a table to a file

```
set fp [open "cell_report.txt" w]
# Print header
puts $fp [format "%-20s %-10s %-10s" "Cell" "RefName" "Area"]
puts $fp "------------------------------------------"
# Sample data (in real flow, data comes from tool commands)
set cells {U1 U2 U3 U4}
set refs  {INV_X1 NAND2_X1 DFF_X1 NOR2_X1}
set areas {1.23 2.45 3.10 1.87}
# Write table rows
for {set i 0} {$i < [llength $cells]} {incr i} {
    puts $fp [format "%-20s %-10s %-10.2f" \
        [lindex $cells $i] [lindex $refs $i] [lindex $areas $i]]
}
close $fp
```

Output file (cell_report.txt) :

Cell	RefName	Area
U1	INV_X1	1.23
U2	NAND2_X1	2.45
U3	DFF_X1	3.10
U4	NOR2_X1	1.87

5. Formatted output

- Use the format command to control column width, alignment and precision.
- Supports different formats for strings, integers and floating point.
- Helps in creating neat and readable reports.

Example 5 : Using format for formatted output

```
set fp [open "power_report.txt" w]
puts $fp [format "%-15s %-10s %-10s" "Block" "Power(mW)" "Status"]
puts $fp "-------------------------------------"
puts $fp [format "%-15s %-10.2f %-10s" "TOP"  125.36 "OK"]
puts $fp [format "%-15s %-10.2f %-10s" "CPU"  45.12  "OK"]
puts $fp [format "%-15s %-10.2f %-10s" "MEM"  32.58  "OK"]
close $fp
```

Key Takeaways

- [ ] Use puts to write data to a file.
- [ ] Generate reports by redirecting tool command output.
- [ ] Include headers with title, design name and date/time.
- [ ] Use tables to present data in a structured format.
- [ ] Use format for aligned and readable output.
- [ ] Well formatted reports are essential for analysis and documentation.

6.5 File System Commands

- Tcl provides file system commands to check and manipulate files and directories.
- These commands help in making scripts robust and portable.
- Common operations include checking existence, creating directories, deleting, copying and renaming files.
- Widely used in EDA flows for managing input files, reports, logs, and directory structures.

1. file exists

- Checks whether a file or directory exists.
- Returns 1 if it exists, 0 otherwise.
- Useful before opening a file or performing operations.

Example 1 : Checking file existence

```
# Checks if a file exists
if {[file exists "report.rpt"]} {
    puts "File exists"
} else {
    puts "File does not exist"
}
```

Output :

```
File exists
```

2. file isfile

- Checks whether the given path is a regular file.
- Returns 1 if it is a file, 0 otherwise.
- Useful to differentiate between files and directories.

Example 2 : Using file isFile

```
if {[file isFile "reports/timing.rpt"]} {
    puts "It is a file"
} else {
    puts "Not a file"
}
```

Output :

```
It is a file
```

3. file isdirectory

- Checks whether the given path is a directory.
- Returns 1 if it is a directory, 0 otherwise.
- Useful for directory existence checks in scripts.

Example 3 : Using file isdirectory

```
if {[file isdirectory "logs"]} {
    puts "logs is a directory"
} else {
    puts "logs is not a directory"
}
```

Output :

```
logs is a directory
```

4. file mkdir

- Creates a new directory.
- If the directory already exists, it will not give an error.
- You can create nested directories (if supported by tool).

Example 4 : Creating a directory

```
# Create a directory for reports
file mkdir "reports"
puts "Directory 'reports' created"
```

Note :

- If directory already exists, no error.
- Useful to organize output files and logs.

5. file delete

- Deletes a file or directory.
- Use -force option to delete non-empty directories.
- Be careful while deleting files (irreversible operation).

Example 5 : Deleting files and directories

```
# Delete a file
file delete "temp.rpt"

# Delete a non-empty directory
file delete -force "old_runs"
```

Note :

- Use -force to remove directories with content.
- Check existence before deleting to avoid errors.

6. file copy

- Copies a file from source to destination.
- Destination can be a file or a directory.
- Useful for backing up reports, logs or configuration files.

Example 6 : Copying a file

```
# Copy a file to another location
file copy "timing.rpt" "backup/timing.rpt"
puts "File copied successfully"
```

Output :

```
File copied successfully
```

7. file rename

- Renames a file or directory.
- Useful for versioning or moving files to a new name.
- Works across directories (move and rename).

Example 7 : Renaming a file

```
# Rename a file
file rename "old_report.rpt" "new_report.rpt"
puts "File renamed successfully"
```

Output :

```
File renamed successfully
```

Key Takeaways

- [ ] file exists checks if a file or directory is present.
- [ ] file isFile and file isdirectory help identify the type.
- [ ] file mkdir creates directories (can be nested).
- [ ] file delete removes files/directories (use -force carefully).
- [ ] file copy copies files to a new location.
- [ ] file rename changes the name or moves a file.
- [ ] Always check existence and handle errors in scripts.
- [ ] File system commands are essential for robust automation in EDA flows.

6.6 File Path Manipulation

- Tcl provides built-in commands to manipulate file paths.
- These commands help in constructing, extracting and modifying paths.
- Useful for writing portable scripts that work across different environments.
- Common commands include file join, file dirname, file tail and file extension.

1. file join

- Combines multiple path components into a single path.
- Automatically uses the correct path separator (/ or \).
- Useful for constructing file paths in a portable way.

Example 1 : Using file join

```
set dir  [file join "home" "user" "project"]
set path [file join $dir "reports" "timing.rpt"]
puts $path
```

Output :

```
home/user/project/reports/timing.rpt
```

Note :

- Works with any number of arguments.
- Uses the correct separator for the platform.

2. file dirname

- Returns the directory part of a given path.
- Useful to extract the parent directory from a file path.

Example 2 : Using file dirname

```
set path "/home/user/project/reports/timing.rpt"
set dir [file dirname $path]
puts "Directory : $dir"
```

Output :

```
Directory : /home/user/project/reports
```

Note :

- Only the directory portion is returned.
- The file name is removed.

3. file tail

- Returns the last component of a path (file or directory name).
- Useful to extract the file name from a full path.

Example 3 : Using file tail

```
set path "/home/user/project/reports/timing.rpt"
set file [file tail $path]
puts "File name : $file"
```

Output :

```
File name : timing.rpt
```

Note :

- Returns only the last component of the path.
- Works for both files and directories.

4. file extension

- Returns the file extension from a path.
- The extension includes the dot (.).
- Useful to identify file type (e.g., .rpt, .tcl, .log).

Example 4 : Using file extension

```
set file "/home/user/project/reports/timing.rpt"
set ext  [file extension $file]
puts "Extension : $ext"
```

Output :

```
Extension : .rpt
```

Note :

- Includes the dot (.).
- If no extension exists, returns an empty string.

5. Path construction example

- Combine multiple commands to construct and manipulate paths.
- Useful in real scripts for creating output directories and files.
- Helps in making scripts flexible and reusable.

Example 5 : Complete example

```
# Define base directory
set base "/home/user/project"

# Create sub-directory path
set rpt_dir [file join $base "reports" "timing"]

# Get parent directory
set parent  [file dirname $rpt_dir]

# Get file name and extension
set file    [file tail "timing.rpt"]
set ext     [file extension "timing.rpt"]

# Print results
puts "Report directory : $rpt_dir"
puts "Parent directory : $parent"
puts "File name        : $file"
puts "Extension        : $ext"
```

Output :

```
Report directory : /home/user/project/reports/timing
Parent directory : /home/user/project/reports
File name        : timing.rpt
Extension        : .rpt
```

Directory structure :

```
project/
├── reports/
│   └── timing/
│       ├── timing.rpt
│       └── area.rpt
├── logs/
│   └── run.log
└── scripts/
    ├── run.tcl
    └── setup.tcl
```

Path components :



Key Takeaways

- [ ] Use file join to construct paths in a portable way.
- [ ] Use file dirname to get the parent directory.
- [ ] Use file tail to extract the file or directory name.
- [ ] Use file extension to get the file type.
- [ ] These commands help in writing clean, flexible and reusable scripts.
- [ ] Path manipulation is widely used in ICC2, PrimeTime and EDA automation.

6.7 Regular Expressions

- Regular expressions (regex) provide a powerful way to match patterns in strings.
- In Tcl, the regexp command is used for pattern matching.
- Useful for parsing reports, log files and extracting useful information.
- Helps in matching characters, numbers, names and complex patterns.
- Supports capturing groups for extracting specific parts of a match.

1. regexp command

- The regexp command searches a string for a pattern.
- Returns 1 if a match is found, 0 otherwise.
- Can be used with capturing groups to extract matched text.
- Syntax : regexp ?pattern? string ?matchVar? ?subMatchVar ...?

Example 1 : Basic pattern matching

```
set str "Total area = 125430"
if {[regexp {area} $str]} {
    puts "Found the word area"
} else {
    puts "Not found"
}
```

Output :

```
Found the word area
```

2. Pattern matching

- A pattern can be a literal string or a special regex expression.
- Common symbols : . ^ $ * + ? [ ] ( ) |
- Useful for finding specific keywords or formats in a string.

Example 2 : Using pattern to find a number

```
set str "WNS = 0.12  TNS = -12.34"
if {[regexp {WNS\s*=\s*[-+]?[0-9.]+} $str]} {
    puts "WNS value found"
}
```

Output :

```
WNS value found
```

3. Character matching

- Use character classes to match specific types of characters.
- [a-z] lowercase letters
- [A-Z] uppercase letters
- [0-9] digits
- [a-zA-Z0-9_] alphanumeric and underscore

Example 3 : Matching alphanumeric names

```
set str "U1 U2 NAND2_X1 FF_2"
regexp {[A-Za-z0-9_]+} $str match
puts "First match : $match"
```

Output :

```
First match : U1
```

4. Numeric matching

- Useful for extracting integers, floating point numbers, positive or negative values.
- Common pattern : [-+]?[0-9]+(\.[0-9]+)?

Example 4 : Extracting numbers

```
set str "Delay = -12.34 ns"
regexp {[-+]?[0-9]+(\.[0-9]+)?} $str num
puts "Number : $num"
```

Output :

```
Number : -12.34
```

5. Name matching

- Useful for extracting design object names (cells, nets, pins).
- Typically names contain letters, digits and underscores.
- Pattern : [A-Za-z_][A-Za-z0-9_]*

Example 5 : Matching cell names

```
set line "Cell: NAND2_X1 Pin: A Net: data_1"
regexp {Cell:\s*([A-Za-z0-9_]+)} $line -> cell
puts "Cell name : $cell"
```

Output :

```
Cell name : NAND2_X1
```

6. Capturing groups

- Use parentheses ( ) to capture parts of the matched string.
- Captured values can be stored in variables.
- Useful for extracting multiple fields from a line.

Example 6 : Extracting multiple fields

```
set line "U1  NAND2_X1  0.12  OK"
regexp {([A-Za-z0-9_]+)\s+([A-Za-z0-9_]+)\s+
        ([-+]?[0-9.]+)\s+([A-Za-z]+)} $line -> inst cell delay status
puts "Instance : $inst"
puts "Cell     : $cell"
puts "Delay    : $delay"
puts "Status   : $status"
```

Output :

```
Instance : U1
Cell     : NAND2_X1
Delay    : 0.12
Status   : OK
```

7. Useful regex patterns

Pattern	Meaning	Example Match
.	Any single character	a, 1, _
[abc]	Any one of a, b or c	a
[0-9]	Any digit	5
[A-Za-z]	Any letter	Z
[A-Za-z0-9_]	Alphanumeric or underscore	net_1
^	Start of string	^U (U at beginning)
$	End of string	OK$ (ends with OK)
*	Zero or more occurrences	a* (a, aa, aaa)
+	One or more occurrences	a+ (a, aa, aaa)
?	Zero or one occurrence	colou?r (color/colour)
(...)	Capturing group	(U\d+)
|	OR operation	(WNS|TNS)

Key Takeaways

- ☐ Use regexp for powerful pattern matching in Tcl.
- ☐ Learn character classes and common regex symbols.
- ☐ Use numeric and name patterns for extracting data from reports.
- ☐ Use capturing groups to get specific fields.
- ☐ Regular expressions are essential for parsing timing, area and log files in ICC2, PrimeTime and other EDA tools.

6.8 Regex Operators

- Regular expressions (regex) use special operators to define search patterns.
- In Tcl, the regexp command supports a rich set of operators.
- These operators help in matching characters, strings, repetitions and groups.
- Useful for parsing tool reports, log files and extracting information.

1. Dot (.) - Any single character

- . matches any single character (except newline).
- Useful when the exact character is unknown.

Example 1 : Using . operator

```
set str "net1 net2 net3"
if {[regexp {net.} $str]} {
    puts "Match found"
}
```

Output :

Match found
(matches net1, net2, net3)

2. Star (*) - Zero or more occurrences

- * matches zero or more occurrences of the preceding character or pattern.
- Useful for matching variable length strings.

Example 2 : Using * operator

```
set str "aaa a ac b"
if {[regexp {a*} $str]} {
    puts "Match found"
}
```

Output :

Match found
(matches a, aa, aaa)

3. Plus (+) - One or more occurrences

- + matches one or more occurrences of the preceding character or pattern.
- Useful when at least one occurrence is required.

Example 3 : Using + operator

```
set str "a aa aaa b"
if {[regexp {a+} $str]} {
    puts "Match found"
}
```

Output :

Match found
(matches a, aa, aaa but not empty)

4. Question mark (?) - Zero or one occurrence

- ? matches zero or one occurrence of the preceding character or pattern.
- Useful for optional characters.

Example 4 : Using ? operator

```
set str "color colour"
if {[regexp {colou?r} $str]} {
    puts "Match found"
}
```

Output :

Match found
(matches color and colour)

5. Caret (^) - Start of string

- ^ matches the beginning of a string.
- Useful to ensure the pattern appears at the start.

Example 5 : Using ^ operator

```
set str "Start End"
if {[regexp {^Start} $str]} {
    puts "Match at start"
}
```

Output :

Match at start

6. Dollar ($) - End of string

- $ matches the end of a string.
- Useful to ensure the pattern appears at the end.

Example 6 : Using $ operator

```
set str "data_end"
if {[regexp {end$} $str]} {
    puts "Match at end"
}
```

Output :

Match at end

7. Square brackets ([ ]) - Character set

- [ ] matches any one character from the set.
- Can also specify a range (e.g., [0-9], [a-z]).

Example 7 : Using [ ] operator

```
set str "net1 netA net_
regexp {net[0-9]} $str match
puts "Matched : $match"
```

Output :

Matched : net1

Note :

- Ranges can be used:
  [a-z] → lowercase
  [A-Z] → uppercase
  [0-9] → digits

8. Parentheses (( )) - Capturing group

- ( ) is used to group patterns and capture the matched text.
- Captured values can be accessed using variables.

Example 8 : Using ( ) operator

```
set str "Cell: NAND2_X1"
if {[regexp {Cell: (\S+)} $str -> cell]} {
    puts "Cell name : $cell"
}
```

Output :

Cell name : NAND2_X1

9. Curly braces ({ }) - Repetition

- { } specifies the exact or range number of occurrences.
- Syntax : {n} (exact n), {n,} (at least n), {n,m} (between n and m).

Example 9 : Using { } operator

```
set str "a aa aaa aaaa"
regexp {a{2,3}} $str match
puts "Matched : $match"
```

Output :

Matched : aaa

Note :

- a{2} → exactly 2 a's
- a{2,} → 2 or more a's
- a{2,4} → between 2 and 4

Summary Table : Regex Operators

Operator	Meaning	Example	Matches	Operator	Meaning	Example	Matches
.	Any single character	a.t	act, a1t	$	End of string	end$	data_end
*	Zero or more	a*	, a, aa, aaa	[ ]	Character set	[0-9]	0,1,2,...,9
+	One or more	a+	a, aa, aaa	( )	Capturing group	(\w+)	NAND2_X1
?	Zero or one	colou?r	color, colour	{ }	Repetition	a{2,4}	aa, aaa, aaaa
^	Start of string	^net	net1 (at start)				

Author : Konda Tharun Vivekananda

"Small patterns, big insights !"

6.9 Regular Expression Substitution

- The regsub command is used to search and replace text using regular expressions.
- It replaces the first match by default, or all matches using the -all option.
- Useful for report cleanup, name transformation and formatting data.
- Commonly used in ICC2, PrimeTime and other EDA tools to process report files.

1. regsub syntax

- Syntax : regsub ?-all?pattern string replacement ?varName?
- If varName is provided, result is stored in the variable.
- If varName is not provided, the modified string is returned.
- Use -all to replace all occurrences.

Example 1 : Basic regsub usage

```
set str "abc123def123"
set new_str [regsub 123 $str XYZ]
puts "Original : $str"
puts "Modified : $new_str"
```

Output :

```
Original : abc123def123
Modified : abcXYZdef123
```

2. Replace all occurrences (-all)

- By default, only the first match is replaced.
- Use -all option to replace all matches in the string.

Example 2 : Using -all option

```
set str "net1 net2 net3 net4"
set new_str [regsub -all {net[0-9]+} $str NET]
puts $new_str
```

Output :

```
NET NET NET NET
```

3. Search and replace (Report cleanup)

- Useful to remove unwanted text, special characters or extra spaces.
- Helps in cleaning EDA tool reports.

Example 3 : Removing extra spaces

```
set str "  cell  NAND2_X1  area  1.23  "
set new_str [regsub -all {\s+} $str " "]
puts "Cleaned : $new_str"
```

Output :

```
Cleaned :
cell NAND2_X1 area 1.23
```

4. Name transformation

- Useful for modifying design object names (cells, nets, pins).
- Can be used to add prefixes, remove suffixes or change formats.

Example 4 : Adding prefix to cell names

```
set cell "U1"
set new_name [regsub {^} $cell "top_"]
puts "New name : $new_name"
```

Output :

```
New name : top_U1
```

Example 5 : Removing suffix

```
set name "net_data[3]"
set new_name [regsub {\[.*\]} $name ""]
puts "Modified : $new_name"
```

Output :

```
Modified : net_data
```

5. Using a variable to store result

- If a variable name is provided, the substituted string is stored in that variable.
- Useful when multiple substitutions are needed.

Example 6 : Storing result in a variable

```
set str "PATH=/usr/bin:/bin"
regsub {/usr/bin} $str "/user/tools" result
puts "Result : $result"
```

Output :

```
Result :
PATH=/user/tools:/bin
```

6. Practical examples in EDA

- Remove timing units from reports.
- Convert hierarchical names.
- Clean unwanted characters.
- Extract and modify object names.

Use Case	Input String	regsub Command	Output
Remove units	12.34ns	regsub {ns} $str ""	12.34
Remove commas	1,234,567	regsub -all {,} $str ""	1234567
Convert / to _	top/u1/net	regsub -all {/} $str "_"	top_u1_net
Add prefix	U1	regsub {^} $str "top_"	top_U1
Remove [ ]	net[3]	regsub {\[.*\]} $str ""	net
Replace multiple	a:b:c	regsub -all {:} $str "_"	a_b_c

7. Advanced example (Report parsing)

- Extract a specific field and clean it using regsub.
- Remove special characters and extra spaces.

Example 7 : Cleaning a report line

```
set line "Cell: NAND2_X1  (area = 1.23)  ;  delay = 0.45ns"
# Remove parentheses and semicolon
set clean [regsub -all {[();]} $line ""]
# Replace multiple spaces with single space
set clean [regsub -all {\s+} $clean " "]
puts "Clean line : $clean"
```

Output :

```
Clean line :
Cell: NAND2_X1 area = 1.23 delay = 0.45ns
```

Key Takeaways

- ☐ Use regsub to replace text using regular expressions.
- ☐ By default, only the first match is replaced; use -all for all matches.
- ☐ Useful for report cleanup, name transformation and data formatting.
- ☐ Can be used to remove, add or modify text patterns.
- ☐ Widely used in ICC2, PrimeTime and other EDA automation flows.
- ☐ Helps in creating clean, consistent and tool-friendly outputs.

6.10 Report Parsing

- Report parsing means reading tool generated reports and extracting useful information using Tcl (file handling + regex).
- Helps in automation, data analysis and customized reporting.
- Common reports in VLSI flows: Timing, Area, Power, Congestion and DRC.
- Steps: open file → read line by line → use regexp → extract data → store/process → generate summary.

1. Timing report parsing

- Extracts timing information like slack, startpoint, endpoint.
- Helps to identify critical paths and worst violations.
- Useful in PrimeTime, Tempus or ICC2 timing reports.

Example 1 : Extract worst negative slack (WNS)

```
set fp [open "timing.rpt" r]
set wns 0
while {[gets $fp line] >= 0} {
  if {[regexp {slack\s+(-?\d+\.\d+)} $line -> val]} {
    if {$val < $wns} { set wns $val }
  }
}
close $fp
puts "Worst Negative Slack : $wns"
```

Output :

```
Worst Negative
Slack : -0.123
```

2. Area report parsing

- Extracts total cell area, cell count or block area.
- Useful to track design size and compare across runs.
- Common in synthesis, PnR and signoff reports.

Example 2 : Extract total cell area

```
set fp [open "area.rpt" r]
set total_area 0
while {[gets $fp line] >= 0} {
  if {[regexp {Total cell area\s+(:|=)\s+(\d|".)+} $line -> val]} {
    set total_area $val
  }
}
close $fp
puts "Total Cell Area : $total_area"
```

Output :

```
Total Cell Area :
123456.78
```

3. Power report parsing

- Extracts total/dynamic/leakage power numbers.
- Helps in power analysis and optimization.
- Useful in PrimeTime or power analysis tools.

Example 3 : Extract total power

```
set fp [open "power.rpt" r]
set total_power 0
while {[gets $fp line] >= 0} {
  if {[regexp {Total Power\s+(:|=)\s+(\d+\.\d+)} $line -> val]} {
    set total_power $val
  }
}
close $fp
puts "Total Power : $total_power"
```

Output :

```
Total Power :
0.567 W
```

4. Congestion report parsing

- Extracts congestion metrics like overflow, usage or hot spots.
- Useful for checking routing congestion after global/detailed routing.
- Helps in identifying congested layers or regions.

Example 4 : Extract congestion information

```
set fp [open "congestion.rpt" r]
while {[gets $fp line] >= 0} {
  if {[regexp {Layer\s+(\w+)\s+.*overflow\s*[:=]\s+(\d+\.?\d*)}
      $line -> layer val]} {
    puts "Layer $layer Overflow : $val"
  }
}
close $fp
```

Output :

```
Layer M1 Overflow : 2.5
Layer M2 Overflow : 0.0
Layer M3 Overflow : 1.8
```

5. DRC report parsing

- Extracts total number of DRC errors and categorizes by rule.
- Helps in tracking signoff quality.
- Useful in ICC2, Pegasus or other DRC tools.

Example 5 : Extract DRC violation counts

```
set fp [open "drc.rpt" r]
set total_errors 0
while {[gets $fp line] >= 0} {
  if {[regexp {Total Errors\s*[:=]\s+(\d+)} $line -> val]} {
    set total_errors $val
  } elseif {[regexp {(\S+)\s+Violations\s*[:=]\s+(\d+)} $line -> rule cnt]} {
    puts "$rule : $cnt"
  }
}
close $fp
puts "Total DRC Errors : $total_errors"
```

Output :

```
M1.SPACING     : 25
VIA.ENCLOSURE  : 10
METAL.WIDTH    : 5
Total DRC Errors : 40
```

Summary Table

Report Type	Key Information Extracted	Example Pattern
Timing	Slack, startpoint, endpoint	slack\s+(-?\d+\.\d+)
Area	Total cell area, cell count	Total cell area\s+\d+\.\d+
Power	Total, dynamic, leakage power	Total Power\s+\d+\.\d+
Congestion	Layer overflow, usage	Layer\s+\w+.*overflow\s+\d+\.\d+
DRC	Total errors, rule counts	(\S+)\s+Violations\s+\d+

Key Takeaways

- ☐ Use file handling and regex to parse EDA tool reports.
- ☐ Extract only the required information (slack, area, power, congestion, DRC).
- ☐ Store results in variables, arrays or files for further analysis.
- ☐ Customize scripts as per report format (different tools may have different syntax).
- ☐ Report parsing saves time and helps in automated signoff flows.
- ☐ Widely used in ICC2, PrimeTime and other EDA tools.

6.11 Timing Data Extraction

- Timing data extraction means parsing STA timing reports to get key information like WNS, TNS, slack, startpoint, endpoint, path group and violation count.
- Useful for automating signoff checks, tracking progress and generating custom reports.
- Commonly used in PrimeTime, Tempus and ICC2.
- Approach : open report → read line by line → use regexp → extract required fields → store in variables / arrays.

1. Extracting WNS (Worst Negative Slack)

- WNS is the most critical (worst) slack value in the design.
- Generally reported in the summary section of timing report.

Example 1 : Extract WNS from report

```
set fp [open "timing.rpt" r]
set wns ""
while {[gets $fp line] >= 0} {
  if {[regexp {WNS\s*:\s*([-0-9.]+)} $line -> val]} {
     set wns $val
     break
  }
}
close $fp
puts "WNS : $wns"
```

Output :

```
WNS : -0.123
```

2. Extracting TNS (Total Negative Slack)

- TNS is the sum of all negative slacks in the design.
- Indicates the total timing violation.

Example 2 : Extract TNS from report

```
set fp [open "timing.rpt" r]
set tns ""
while {[gets $fp line] >= 0} {
  if {[regexp {TNS\s*:\s*([-0-9.]+)} $line -> val]} {
     set tns $val
     break
  }
}
close $fp
puts "TNS : $tns"
```

Output :

```
TNS : -12.345
```

3. Extracting Path Details (Slack, Startpoint, Endpoint)

- Parse each path line to get slack, startpoint and endpoint.
- Useful for generating violation lists.

Example 3 : Extract path information

```
set fp [open "timing.rpt" r]
while {[gets $fp line] >= 0} {
  if {[regexp {^\s*([-0-9.]+)\s+(\S+)\s+(\S+)} $line -> slack sp ep]} {
     puts "Slack: $slack  Startpoint: $sp  Endpoint: $ep"
  }
}
close $fp
```

Sample Output :

```
Slack: -0.123  Startpoint: U1/CLK  Endpoint: U5/D
Slack: -0.256  Startpoint: U2/Q    Endpoint: U9/D
```

4. Extracting Path Group Information

- Path groups categorize paths (e.g., reg2reg, in2reg, reg2out).
- Helps in analyzing which path group has worst violations.

Example 4 : Extract worst slack per path group

```
set fp [open "timing.rpt" r]
set current_group ""
array set group_wns {}
while {[gets $fp line] >= 0} {
  if {[regexp {Path Group:\s+(\S+)} $line -> grp]} {
     set current_group $grp
  } elseif {[regexp {WNS\s*:\s*([-0-9.]+)} $line -> val]} {
     set group_wns($current_group) $val
  }
}
close $fp
puts "WNS per Path Group:"
foreach grp [array names group_wns] {
   puts "$grp : $group_wns($grp)"
}
```

Output :

```
WNS per Path Group:
reg2reg : -0.123
in2reg  : -0.045
reg2out : -0.210
in2out  : 0.000
```

5. Extraction of Violation Count

- Counts the number of violating paths (negative slack).
- Useful for tracking signoff progress.

Example 5 : Count number of violating paths

```
set fp [open "timing.rpt" r]
set vcount 0
while {[gets $fp line] >= 0} {
  if {[regexp {^\s*([-0-9.]+)\s+} $line -> slack]} {
     if {$slack < 0} {
        incr vcount
     }
  }
}
close $fp
puts "Total Violations : $vcount"
```

Output :

```
Total Violations : 257
```

Sample Timing Report (Partial)

```
Path Group: reg2reg
 Slack       Startpoint                    Endpoint
 ------------------------------------------------------
 -0.123      U1/CLK                        U5/D
 -0.256      U2/Q                          U9/D
 -0.045      U3/Q                          U7/D

Path Group: in2reg
 Slack       Startpoint                    Endpoint
 ------------------------------------------------------
 -0.089      IN1                           U4/D
 -0.012      IN2                           U6/D

 Summary
 WNS  : -0.123
 TNS  : -12.345
 Violating Paths : 257
```

Key Takeaways

- ☐ Use file handling and regex to extract timing information from reports.
- ☐ Extract key metrics : WNS, TNS, slack, startpoint, endpoint.
- ☐ Parse path group information to identify critical path groups.
- ☐ Count number of violating paths for signoff tracking.
- ☐ Useful in ICC2, PrimeTime, Tempus and other EDA tools.
- ☐ Helps in generating custom reports and dashboards.

6.12 Physical Data Extraction

- Physical data extraction means parsing synthesis/place/route reports to get design size, area, utilization, density, cell/macro count and other physical metrics.
- Helps in design analysis, tracking growth and meeting physical constraints.
- Useful for automation, dashboards and signoff checks.
- Approach : open report file → read line by line → use regexp to extract required fields → store and generate summary.

1. Extracting Cell Count

- Cell count includes total number of standard cells in the design.
- Useful to track design size and compare across runs.

Example 1 : Extract cell count from report

```
set fp [open "area.rpt" r]
set cell_count 0
while {[gets $fp line] >= 0} {
  if {[regexp {^Number of cells\s*:\s*([0-9]+)} $line → val]} {
    set cell_count $val
  }
}
close $fp
puts "Total Cell Count : $cell_count"
```

Output :

```
Total Cell Count :
125678
```

2. Extracting Area

- Area is the total cell area (or core area) reported by the tool.
- Useful for comparing design size and ensuring it meets targets.

Example 2 : Extract total area from report

```
set fp [open "area.rpt" r]
set total_area 0
while {[gets $fp line] >= 0} {
  if {[regexp {^Total cell area\s*:\s*([0-9.]+)} $line → val]} {
    set total_area $val
  }
}
close $fp
puts "Total Cell Area : $total_area"
```

Output :

```
Total Cell Area :
123456.78
```

3. Extracting Utilization

- Utilization = (Total cell area / Core area) x 100.
- Helps in checking how much of the core area is used.

Example 3 : Calculate utilization

```
set fp [open "area.rpt" r]
set cell_area 0
set core_area 0
while {[gets $fp line] >= 0} {
  if {[regexp {^Total cell area\s*:\s*([0-9.]+)} $line → val]} {
    set cell_area $val
  } elseif {[regexp {^Core area\s*:\s*([0-9.]+)} $line → val]} {
    set core_area $val
  }
}
close $fp
set util [expr {($cell_area/$core_area)*100}]
puts "Utilization : $util %"
```

Output :

```
Utilization :
72.5 %
```

4. Extracting Density

- Density indicates the area usage per unit core area.
- Helps in analyzing placement congestion and routability.

Example 4 : Extract density from report

```
set fp [open "place.rpt" r]
set density 0
while {[gets $fp line] >= 0} {
  if {[regexp {^Placement density\s*:\s*([0-9.]+)} $line → val]} {
    set density $val
  }
}
close $fp
puts "Placement Density : $density"
```

Output :

```
Placement Density :
0.68
```

5. Extracting Macro Count

- Macros are large blocks like memories, hard macros, IPs.
- Useful to track number of macros in the design.

Example 5 : Extract macro count

```
set fp [open "design.rpt" r]
set macro_count 0
while {[gets $fp line] >= 0} {
  if {[regexp {^Number of macros\s*:\s*([0-9]+)} $line → val]} {
    set macro_count $val
  }
}
close $fp
puts "Macro Count : $macro_count"
```

Output :

```
Macro Count : 12
```

6. Extracting Macro Area

- Macro area is the total area occupied by all macros.
- Helps in capacity planning and floorplan analysis.

Example 6 : Extract macro area

```
set fp [open "design.rpt" r]
set macro_area 0
while {[gets $fp line] >= 0} {
  if {[regexp {^Total macro area\s*:\s*([0-9.]+)} $line → val]} {
    set macro_area $val
  }
}
close $fp
puts "Total Macro Area : $macro_area"
```

Output :

```
Total Macro Area :
45678.90
```

Sample Extracted Report Summary

Metric	Value	Description
Cell Count	125678	Total number of standard cells
Total Cell Area	123456.78	Total area of standard cells (µm²)
Core Area	170000.00	Core area (µm²)
Utilization	72.5 %	(Total cell area / Core area) × 100
Density	0.68	Placement density
Macro Count	12	Number of macros (memories, IPs, etc.)
Macro Area	45678.90	Total area occupied by macros (µm²)

Key Takeaways

- ☐ Use file handling and regex to extract physical data from reports
- ☐ Extract key metrics : cell count, area, utilization, density, macro count, macro area
- ☐ Useful for design analysis, capacity planning and signoff checks.
- ☐ Can be extended to extract other information like core dimensions, aspect ratio, etc.
- ☐ Widely used in ICC2, PrimeTime, Innovus, Tempus and other EDA tools.

6.13 Routing Data Extraction

- Routing data extraction means parsing routing related reports to get important metrics like DRC count, antenna violations, via count, wirelength and congestion.
- Helps in checking routing quality, finding violations and tracking design progress.
- Useful for signoff checks and tapeout readiness.
- Approach : open report file → read line by line → use regexp to extract required fields → store and generate summary.

1. Extracting DRC Count

- DRC count is the total number of design rule violations.
- Useful to check routing clean pass criteria.

Example 1 : Extract DRC count from report

```
set fp [open "drc.rpt" r]
set drc_count 0
while {[gets $fp line] >= 0} {
  if {[regexp {Total Number of DRC Errors\s*:\s*(\d+)} $line -> val]} {
    set drc_count $val
  }
}
close $fp
puts "DRC Count : $drc_count"
```

Output :

```
DRC Count :
257
```

2. Extracting Antenna Violations

- Antenna violations occur due to charge buildup during etch.
- Useful to check reliability and signoff readiness.

Example 2 : Extract antenna violation count

```
set fp [open "antenna.rpt" r]
set ant_count 0
while {[gets $fp line] >= 0} {
  if {[regexp {Antenna Violation\s*:\s*(\d+)} $line -> val]} {
    set ant_count $val
  }
}
close $fp
puts "Antenna Violations : $ant_count"
```

Output :

```
Antenna Violations :
42
```

3. Extracting Via Count

- Via count is the total number of vias used in the design.
- Helpful to understand routing complexity and reliability.

Example 3 : Extract via count from report

```
set fp [open "route.rpt" r]
set via_count 0
while {[gets $fp line] >= 0} {
  if {[regexp {Total Vias\s*:\s*(\d+)} $line -> val]} {
    set via_count $val
  }
}
close $fp
puts "Via Count : $via_count"
```

Output :

```
Via Count :
876543
```

4. Extracting Wirelength

- Wirelength is the total routed wire length in the design.
- Useful for congestion analysis, timing and power estimation.

Example 4 : Extract total wirelength

```
set fp [open "route.rpt" r]
set wirelength 0
while {[gets $fp line] >= 0} {
  if {[regexp {Total Wirelength\s*:\s*([0-9.]+)} $line -> val]} {
    set wirelength $val
  }
}
close $fp
puts "Total Wirelength : $wirelength um"
```

Output :

```
Total Wirelength :
12345678.9 um
```

5. Extracting Congestion Information

- Congestion shows how crowded the routing resources are.
- Useful to identify hotspots and take floorplan/routing actions.
- Reports usually provide congestion as a percentage or in terms of overflow.

Example 5 : Extract congestion from report

```
set fp [open "congestion.rpt" r]
set max_cong 0
while {[gets $fp line] >= 0} {
  if {[regexp {Maximum Congestion\s*:\s*([0-9.]+)} $line -> val]} {
    set max_cong $val
  }
}
close $fp
puts "Maximum Congestion : $max_cong %"
```

Output :

```
Maximum
Congestion :
78.5 %
```

6. Sample Routing Report (Partial)

```
ICC2 Routing Report (Partial)
-----------------------------------------------------
Total Number of DRC Errors        : 257
Antenna Violation                 :  42
Total Vias                        : 876543
Total Wirelength (um)             : 12345678.9
Maximum Congestion (%)            : 78.5
-----------------------------------------------------
```

Summary Table

Metric	Value	Description
DRC Count	257	Total number of routing DRC violations
Antenna Violations	42	Number of antenna violations
Via Count	876543	Total number of vias in the design
Wirelength (um)	12345678.9	Total routed wirelength
Congestion (%)	78.5	Maximum routing congestion

Key Takeaways

- [ ] Use file handling and regex to extract routing data from tool reports.
- [ ] Extract metrics like DRC count, antenna violations, via count, wirelength and congestion.
- [ ] Routing data helps in design analysis, timing/power estimation and signoff.
- [ ] Can be extended to extract layer-wise information and hotspot details.
- [ ] Widely used in ICC2, PrimeTime, Pegasus and other EDA tools.

6.14 Automated Report Generation

- Automated report generation means creating well-formatted reports using Tcl scripts from tool outputs or parsed data.
- Helps in generating summary, detailed, PASS/FAIL, stage and QoR reports.
- Useful for tracking design progress, signoff metrics and sharing results with teams.
- Commonly used in ICC2, PrimeTime and other EDA tools.
- Typically involves file handling, regex parsing, data extraction and formatted output.

1. Summary Reports

- Summary reports provide a high-level view of key metrics.
- Useful for quick status check and dashboard generation.

Example 1 : Generate summary report

```
set fp [open "summary.rpt" w]
puts $fp "============ Design Summary ============"
puts $fp "Cell Count    : $cell_count"
puts $fp "Total Area    : $total_area"
puts $fp "WNS           : $wns"
puts $fp "TNS           : $tns"
puts $fp "DRC Errors    : $drc_count"
close $fp
```

Sample Output : summary.rpt

```
============ Design Summary ============
Cell Count    : 125678
Total Area    : 123456.78
WNS           : -0.123
TNS           : -12.345
DRC Errors    : 0
```

2. Detailed Reports

- Detailed reports provide in-depth information with full lists.
- Useful for debug, analysis and signoff checks.

Example 2 : Generate detailed timing report

```
set fp [open "detailed_timing.rpt" w]
puts $fp "Path Group  Slack     Startpoint     Endpoint"
puts $fp "-------------------------------------------------------"
foreach path $path_list {
    puts $fp "[get_attr $path group]  \
              [get_attr $path slack]  \
              [get_attr $path startpoint]  \
              [get_attr $path endpoint]"
}
close $fp
```

Sample Output : detailed_timing.rpt

```
Path Group    Slack     Startpoint      Endpoint
-------------------------------------------------------
reg2reg       -0.123    U1/CLK          U5/D
reg2reg       -0.256    U2/Q            U9/D
in2out        -0.045    U3/Q            U7/D
...
```

3. PASS/FAIL Reports

- PASS/FAIL reports check if design meets defined criteria.
- Useful for regression runs, signoff checks and automated flows.

Example 3 : Generate PASS/FAIL report

```
set fp [open "check_status.rpt" w]
puts $fp "======= Design Check Status ======="
if { $wns < 0 } {
    puts $fp "Timing    : FAIL (WNS = $wns)"
} else {
    puts $fp "Timing    : PASS"
}
if { $drc_count > 0 } {
    puts $fp "DRC       : FAIL (Errors = $drc_count)"
} else {
    puts $fp "DRC       : PASS"
}
close $fp
```

Sample Output : check_status.rpt

```
======= Design Check Status =======
Timing      : PASS
DRC         : PASS
Antenna     : PASS
Congestion  : FAIL (Max = 78.5%)
```

4. Stage Reports

- Stage reports capture metrics at different stages (e.g., placement, CTS, routing, signoff).
- Useful for tracking progress and improvement across stages.

Example 4 : Generate stage report

```
set fp [open "stage_report.rpt" w]
puts $fp "Stage   WNS      TNS      Area      DRC"
puts $fp "-------------------------------------------"
foreach stage {init place cts route signoff} {
    set wns  [get_stage_metric $stage wns]
    set tns  [get_stage_metric $stage tns]
    set area [get_stage_metric $stage area]
    set drc  [get_stage_metric $stage drc]
    puts $fp "$stage   $wns   $tns   $area   $drc"
}
close $fp
```

Sample Output : stage_report.rpt

Stage	WNS	TNS	Area	DRC
init	-1.234	-56.789	100000.0	125
place	-0.456	-23.456	110000.0	45
cts	-0.234	-10.123	120000.0	12
route	-0.123	-2.345	123456.7	3
signoff	-0.010	-0.100	123456.7	0

5. QoR Reports

- QoR (Quality of Results) reports combine multiple metrics (timing, area, power, congestion, etc.).
- Useful for comparing runs and choosing the best results.
- Often used in design exploration and optimization.

Key Takeaways

- [ ] Use Tcl scripting to automate summary, detailed, PASS/FAIL, stage and QoR reports.
- [ ] Helps in tracking design progress and meeting signoff criteria.
- [ ] Save time, ensure consistent format and avoid manual effort.
- [ ] Can be customized as per design requirements and tool outputs.
- [ ] Widely used in ICC2, PrimeTime and other EDA tools.

7.1 Error Handling in Tcl

- Error handling helps to detect, manage and recover from errors in a Tcl script.
- Useful to prevent unexpected script termination.
- Tcl provides commands like catch, error, return and try to handle errors gracefully.
- Errors can be propagated to calling procedures for higher level handling.

1. Using catch

- catch executes a command and catches any error.
- It returns 0 if command is successful, 1 if an error occurs.
- The error message is stored in a variable.

Example 1 : Using catch to handle an error

```
set ret [catch {set x [expr {10/0}]} errMsg]
if {$ret} {
    puts "Error occurred: $errMsg"
} else {
    puts "Result : $x"
}
```

Output :

```
Error occurred:
divide by zero
```

2. Using error

- error is used to explicitly generate an error with a message.
- Useful for custom error checking and validation.

Example 2 : Using error for custom validation

```
proc check_file {file} {
    if {![file exists $file]} {
        error "File not found : $file"
    }
    puts "File exists : $file"
}

catch {check_file "input.rpt"} msg
puts "Status : $msg"
```

Output :

```
Status :
File not found :
input.rpt
```

3. Using return

- return is used to exit from a procedure and optionally return a value or an error code.
- Can be used inside procedures for controlled exit.

Example 3 : Using return for early exit

```
proc valid_cell {cell} {
    if {$cell eq ""} {
        return -code error "Cell name is empty"
    }
    puts "Valid cell : $cell"
    return 0
}

catch {valid_cell ""} msg
puts "Result : $msg"
```

Output :

```
Result :
Cell name is empty
```

4. Using try (Tcl 8.6+)

- try provides a structured way to handle errors (similar to exception handling).
- Supports multiple clauses: on error, on ok, finally.
- Makes the script more readable and maintainable.

Example 4 : Using try-catch-finally

```
try {
    set fp [open "non_existent.rpt" r]
    puts "File opened successfully"
    close $fp
} on error {errMsg opts} {
    puts "Error : $errMsg"
} finally {
    puts "Cleanup: executed always"
}
```

Output :

```
Error : couldn't open
"non_existent.rpt":
no such file or directory
Cleanup: executed always
```

5. Error Propagation

- Errors can be propagated to the calling procedure using return -code error or by not catching the error.
- The caller can decide how to handle the error.
- Useful for building modular and robust scripts.

Example 5 : Error propagation between procedures

```
proc read_report {file} {
    if {![file exists $file]} {
        return -code error "Report $file not found"
    }
    puts "Reading report: $file"
}

proc main {} {
    if {[catch {read_report "timing.rpt"} msg]} {
        puts "ERROR in main : $msg"
        return -code error $msg
    }
    puts "Report read successfully"
}

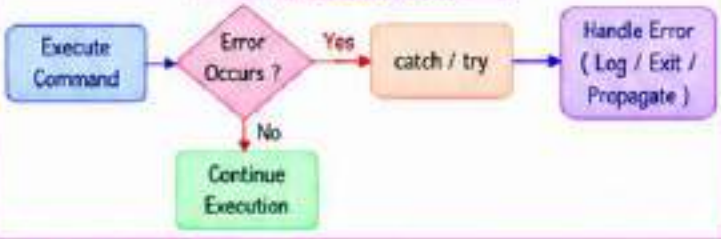
catch {main} result
puts "Final Status : $result"
```

Output :

```
ERROR in main :
Report timing.rpt
not found

Final Status :
Report timing.rpt
not found
```

Error Handling Flow



Key Takeaways

- [ ] Use catch to handle errors and get error messages.
- [ ] Use error to raise custom errors.
- [ ] Use return to exit from procedures with status or value.
- [ ] Use try (Tcl 8.6+) for structured error handling.
- [ ] Understand error propagation for building robust scripts.
- [ ] Essential for reliable automation in ICC2, PrimeTime and other EDA tools.

7.2 Tcl Errors

- Tcl errors occur when the interpreter cannot execute a command due to syntax, runtime or usage issues.
- Understanding common errors helps in quick debugging.
- Error messages provide useful information like the type of error, command and line number.
- Common in ICC2, PrimeTime and other EDA tools while writing custom scripts.
- Use error handling and careful coding to avoid these errors.

1. Syntax Errors

- Occurs when the Tcl command is not written with correct syntax.
- Usually due to missing braces, quotes or wrong command format.

Example 1 : Missing closing brace

```
set a 10
if {$a > 5 {
    puts "Value is greater than 5"
}
```

Output :

```
missing "}"
while executing
"if {$a > 5 {"
```

2. Runtime Errors

- Occurs during script execution due to invalid operation.
- Examples include divide by zero, accessing invalid index, etc.

Example 2 : Divide by zero

```
set a 10
set b 0
set c [expr $a / $b]
puts "Result = $c"
```

Output :

```
divide by zero
while executing
"expr $a / $b"
```

3. Invalid Commands

- Occurs when a command is not recognized by the Tcl interpreter.
- Can be due to a typo or using a tool specific command in wrong context.

Example 3 : Typo in command name

```
puts "Hello"
putss "World"    ;# wrong command
```

Output :

```
invalid command name
"putss"
```

4. Wrong Arguments

- Occurs when a valid command is used with incorrect number or type of arguments.
- Check the usage and required arguments.

Example 4 : Wrong number of arguments

```
expr 10 +
```

Output :

```
wrong # args: should be
"expr arg ?arg ...?"
```

5. Missing Variables

- Occurs when a variable is used before it is defined.
- Tcl treats undefined variables as an error (unlike some other languages).

Example 5 : Using undefined variable

```
puts "Value = $myvar"
```

Output :

```
can't read "myvar":
no such variable
```

Fix : Define the variable before use

```
set myvar 100
puts "Value = $myvar"
```

6. Invalid Files

- Occurs when trying to open or source a file that does not exist or is not accessible.
- Common while reading reports or sourcing Tcl scripts.

Example 6 : Opening a non-existent file

```
set fp [open "non_existing.rpt" r]
```

Output :

```
couldn't open
"non_existing.rpt":
no such file or directory
```

Fix : Check file existence before opening

```
if {[file exists "non_existing.rpt"]} {
    set fp [open "non_existing.rpt" r]
} else {
    puts "File not found!"
}
```

7. Summary of Common Tcl Errors

Error Type	Cause	Example	Error Message
Syntax Error	Incorrect syntax (missing brace/quote)	if {$a > 5 {	missing "}"
Runtime Error	Invalid operation during execution	expr 10 / 0	divide by zero
Invalid Command	Command not recognized	putss "Hello"	invalid command name "putss"
Wrong Arguments	Incorrect number/ type of arguments	expr 10 +	wrong # args
Missing Variable	Variable not defined	puts $myvar	can't read "myvar"
Invalid File	File does not exist or not accessible	open "test.rpt" r	couldn't open "test.rpt"

Key Takeaways

- ☐ Understand different types of Tcl errors and their causes.
- ☐ Read and analyze error messages carefully.
- ☐ Use catch, error and proper checks to handle errors gracefully.
- ☐ Validate inputs, files and variables before use.
- ☐ Helps in writing robust and reliable scripts for ICC2, PrimeTime and other EDA tools.

7.3 Debugging Commands

- Debugging helps in finding and fixing issues in Tcl scripts.
- Tcl provides several built-in commands to inspect variables, commands, procedures and arrays.
- These commands are very useful in interactive shells and in script development.
- Commonly used in ICC2, PrimeTime and other EDA tools for troubleshooting.
- Using these commands along with meaningful print messages makes debugging faster and easier.

1. puts

- puts is used to print messages or variable values to the console.
- Useful for debugging and tracking script flow.

Example 1 : Print messages using puts

```
set design_name "my_block"
set count 10
puts "Design : $design_name"
puts "Count  : $count"
```

Output :

```
Design : my_block
Count  : 10
```

2. info vars

- info vars lists all defined variables in the current scope.
- Helps in checking what variables are available.

Example 2 : List all variables

```
set a 100
set b "test"
set c 3.14
puts "Defined variables:"
puts [info vars]
```

Output :

```
Defined variables:
a
b
c
```

3. info commands

- info commands lists all available Tcl commands.
- Can also be used with a pattern to search for specific commands.

Example 3 : Search for commands

```
puts "All commands starting with get:"
puts [info commands get*]

puts "Check if report_timing exists:"
puts [info commands report_timing]
```

Output :

```
All commands starting with get:
get_cells get_nets get_pins ...
Check if report_timing exists:
report_timing
```

Summary of Debugging Commands

Command	Purpose	Example	Typical Output
puts	Print messages / values	puts $var	Prints value
info vars	List variables	info vars	List of variables
info commands	List Tcl commands	info commands get*	Matching commands
info procs	List procedures	info procs *util*	Matching procs
info exists	Check variable exists	info exists var	1 (true) / 0 (false)
parray	Print array contents	parray arr	key-value pairs

4. info procs

- info procs lists all user-defined procedures.
- Useful to check what procedures are available in the current script or session.
- Can also use pattern matching.

Example 4 : List user defined procedures

```
proc my_proc {} {
    puts "Inside my_proc"
}
proc util_proc {arg} {
    puts "Arg = $arg"
}

puts "All procs:"
puts [info procs]

puts "Procs with pattern '*util*':"
puts [info procs *util*]
```

Output :

```
All procs:
my_proc util_proc

Procs with pattern '*util*':
util_proc
```

5. info exists

- info exists checks if a variable exists or not.
- Very useful to avoid runtime errors due to undefined variables.

Example 5 : Check variable existence

```
if {[info exists mode]} {
    puts "mode is defined: $mode"
} else {
    puts "mode is NOT defined"
}
```

Output :

```
mode is NOT defined
```

6. parray

- parray is used to print the contents of an array variable.
- Very helpful when working with associative arrays.

Example 6 : Print array contents

```
array set cfg {
    top    my_block
    lib    saed.lib
    corner ss
}
parray cfg
```

Output :

```
cfg(top)    = my_block
cfg(lib)    = saed.lib
cfg(corner) = ss
```

Key Takeaways

- ☐ Use puts to print values and debug messages.
- ☐ Use info vars to see defined variables.
- ☐ Use info commands to explore available commands.
- ☐ Use info procs to list user-defined procedures.
- ☐ Use info exists to check if a variable is defined.
- ☐ Use parray to print array contents.
- ☐ These commands are essential for debugging and interactive exploration in ICC2, PrimeTime and other EDA tools.
- ☐ Combine with proper error handling for robust scripts.

7.4 Debugging Techniques

- Debugging techniques help in identifying and fixing issues in Tcl scripts quickly and efficiently.
- Use print statements, inspection commands and step-by-step checks to understand the script behavior.
- Apply different debugging methods for variables, commands, procedures, files and loops.
- Useful in real EDA tools like ICC2, PrimeTime and other flows.
- Good debugging improves script reliability and saves time.

1. Variable Debugging

- Check the value, type and existence of variables.
- Use puts, info exists and parray for debugging.

Example 1 : Variable debugging example

```
set design "top"
set count 5
puts "design = $design"
puts "count = $count"
if {[info exists mode]} {
    puts "mode = $mode"
} else {
    puts "mode is not defined"
}
parray env
parray user_vars
```

Output :

```
design = top
count = 5
mode is not defined
env(DESIGN_MODE) =
  func
user_vars(count) = 5
user_vars(design) = top
```

2. Command Debugging

- Verify if a command exists and use correct syntax.
- Use info commands and catch to debug command execution.

Example 2 : Command debugging example

```
set cmd "get_cells"
if {[llength [info commands $cmd]]} {
    puts "Command '$cmd' is available"
    set result [catch {$cmd -hier *} msg]
    if {$result} {
        puts "Error executing $cmd : $msg"
    } else {
        puts "Command executed successfully"
    }
} else {
    puts "Command '$cmd' not found"
}
```

Output :

```
Command 'get_cells'
is available
Command executed
successfully
```

3. Procedure Debugging

- Print inputs, intermediate values and return values.
- Use puts and catch to debug procedures.

Example 3 : Procedure debugging example

```
proc compute_area {cell_list} {
    puts "[Info] Entering compute_area"
    puts "[Info] Input list : $cell_list"
    set area 0
    foreach c $cell_list {
        set a [get_attribute $c area]
        puts "  Cell: $c  Area: $a"
        set area [expr $area + $a]
    }
    puts "[Info] Total Area = $area"
    return $area
}
set cells {U1 U2 U3}
set total [compute_area $cells]
puts "Returned Area = $total"
```

Output :

```
[Info] Entering
compute_area
[Info] Input list : U1 U2 U3
  Cell: U1  Area: 12.5
  Cell: U2  Area: 8.0
  Cell: U3  Area: 10.2
[Info] Total Area = 30.7
Returned Area = 30.7
```

4. File Debugging

- Check file existence, path and contents.
- Use file commands and puts to debug file operations.
- Useful for parsing reports or reading input files.

Example 4 : File debugging example

```
set file "report_timing.rpt"
if {![file exists $file]} {
    puts "Error: File $file not found"
} else {
    set fp [open $file r]
    puts "File opened successfully"
    set line_num 1
    while {[gets $fp line] >= 0} {
        if {$line_num <= 5} {
            puts "Line $line_num : $line"
        }
        incr line_num
    }
    close $fp
}
```

Output :

```
File opened
successfully
Line 1 :
  # Timing Report
Line 2 :
  Design: top
Line 3 :
  Date : 2025-09-04
Line 4 :
  ----------
Line 5 :
  Path Group: clk
```

5. Loop Debugging

- Print loop variables, iteration count and conditions.
- Helps in identifying infinite loops or incorrect loop behavior.

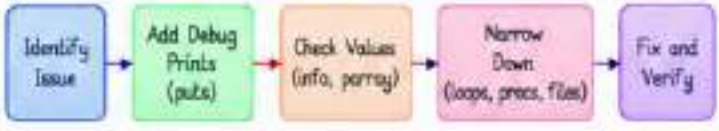
Example 5 : Loop debugging example

```
set count 0
foreach cell [get_cells *] {
    puts "Iteration $count : Cell = $cell"
    # do some operation
    incr count
    if {$count >= 5} {
        puts "Stopping after 5 iterations (debug)"
        break
    }
}
```

Output :

```
Iteration 0 : Cell = U1
Iteration 1 : Cell = U2
Iteration 2 : Cell = U3
Iteration 3 : Cell = U4
Iteration 4 : Cell = U5
Stopping after 5
iterations (debug)
```

Debugging Flow



Key Takeaways

- [ ] Use puts to print values and debug messages.
- [ ] Use info vars, info commands, info procs and info exists to inspect.
- [ ] Debug variables, commands, procedures, files and loops step by step.
- [ ] Use catch to handle errors while debugging.
- [ ] Limit loop iterations during debug to avoid long runtimes.
- [ ] File checks are important to avoid missing file errors.
- [ ] Good debugging practices lead to clean, reliable and maintainable scripts.

7.5 Arrays in Tcl

- Arrays in Tcl are associative arrays (key-value pairs).
- Each element is accessed using a name (string) as the index.
- Useful for storing related data like configuration, cell counts, timing data etc.
- Commonly used in ICC2, PrimeTime and other EDA scripts.
- Tcl provides several built-in commands to work with arrays.

1. Array Creation

- An array is created automatically when a value is assigned using an array index.
- No explicit declaration is required.

Example 1 : Creating an array

```
set design(name)    "my_chip"
set design(process) "7nm"
set design(voltage) "0.8V"
set design(freq)    "1GHz"
```

Output :

```
Array 'design' created
with 4 elements
```

2. array set

- array set is used to create an array and initialize multiple elements at once.
- Values are provided as key value pairs inside a list.

Example 2 : Using array set

```
array set chip {
    name    my_chip
    process 7nm
    voltage 0.8V
    freq    1GHz
    mode    func
}
```

Output :

```
Array 'chip' created
with elements :
name    = my_chip
process = 7nm
voltage = 0.8V
freq    = 1GHz
mode    = func
```

3. array get

- array get returns all key-value pairs of an array.
- Useful for printing or processing the entire array.

Example 3 : Using array get

```
set all_data [array get chip]
puts $all_data
```

Output :

```
name my_chip process 7nm
voltage 0.8V freq 1GHz
mode func
```

Note : Output format

- array get returns a flat list (key value key value ...).
- You can use foreach to iterate through the list.

```
foreach {key value} [array get chip] {
    puts "$key = $value"
}
```

Output :

```
name = my_chip
process = 7nm
voltage = 0.8V
freq = 1GHz
mode = func
```

4. array names

- array names returns a list of all keys (indices) in the array.
- Useful to iterate over array elements.

Example 4 : Using array names

```
set keys [array names chip]
puts "Array keys: $keys"
```

Output :

```
Array keys:
name process voltage
freq mode
```

Looping example :

```
foreach key [array names chip] {
    puts "Key = $key Value = $chip($key)"
}
```

Output :

```
Key = name      Value = my_chip
Key = process   Value = 7nm
Key = voltage   Value = 0.8V
Key = freq      Value = 1GHz
Key = mode      Value = func
```

5. array exists

- array exists checks if an array exists.
- Useful to avoid errors when accessing an undefined array.

Example 5 : Using array exists

```
if {[array exists chip]} {
    puts "Array chip exists"
} else {
    puts "Array chip does not exist"
}
```

Output :

```
Array chip exists
```

Check for specific element :

```
if {[info exists chip(name)]} {
    puts "Element chip(name) exists"
} else {
    puts "Element chip(name) not found"
}
```

Output :

```
Element chip(name)
exists
```

6. array size

- array size returns the number of elements in the array.
- Useful for quick checks and loops.

Example 6 : Using array size

```
set count [array size chip]
puts "Number of elements = $count"
```

Output :

```
Number of elements = 5
```

Common Array Commands Summary

Command	Purpose	Example	Output
Array creation	Create array by assignment	set a(x) 10	Creates array 'a'
array set	Create and initialize	array set a {x 10 y 20}	Array with multiple elements
array get	Return all elements	array get a	key value list
array names	List all keys	array names a	x y
array exists	Check if array exists	array exists a	1 (true) / 0 (false)
array size	Number of elements	array size a	Count of elements

Key Takeaways

- Arrays are associative key-value pairs in Tcl.
- Use array set for easy initialization.
- Use array get and array names to read array contents.
- Always check array exists before accessing.
- array size helps in quick validation.
- Arrays are very useful in EDA scripts for configuration and data storage.

7.6 Array Applications

- Arrays are very useful in real EDA flows to store and process different types of data.
- They provide an easy way to map keys to values and access them quickly.
- Common applications include cell data, scenario data, timing data, report data and QoR data.
- Arrays help in parsing tool reports and generating custom summaries.

1. Cell-to-Value Mapping

- Map cell names to attributes like area, type, drive strength etc.
- Useful for storing and analyzing cell data from reports.

Example 1 : Cell to area mapping

```
# Create array and store cell to area mapping
array set cell_area {
    U1    12.5
    U2    8.0
    U3    15.2
}

# Access value
puts "Area of U2 = $cell_area(U2)"
```

Output :

```
Area of U2 = 8.0
```

2. Scenario-to-Value Mapping

- Map analysis scenarios to values like WNS, TNS or status.
- Useful in multi-corner multi-mode (MCMM) analysis.

Example 2 : Scenario to WNS mapping

```
array set scen_wns {
    FUNC_SS     -0.12
    FUNC_TT     0.05
    FUNC_FF     -0.08
    TEST_TT     0.00
}
puts "WNS in FUNC_TT = $scen_wns(FUNC_TT)"
```

Output :

```
WNS in FUNC_TT
= 0.05
```

3. Timing Data using Arrays

- Store timing information like slack, startpoint, endpoint etc.
- Useful when parsing timing reports.

Example 3 : Store timing data in array

```
array set timing_slack {
    U1/U2     -0.15
    U3/U4     0.02
    U5/U6     -0.08
}
foreach path [array names timing_slack] {
  puts "Path: $path  Slack: $timing_slack($path)"
}
```

Output :

```
Path: U1/U2
Slack: -0.15
Path: U3/U4
Slack: 0.02
Path: U5/U6
Slack: -0.08
```

4. Report Data using Arrays

- Store information extracted from tool reports (e.g., DRC, violations).
- Helps in generating custom summary reports.

Example 4 : Store DRC data from report

```
array set drc_count {
    Metal_Spacing    102
    Via_Enclosure    45
    Antenna          12
    Min_Area         5
}
puts "Antenna violations = $drc_count(Antenna)"
```

Output :

```
Antenna
violations = 12
```

5. QoR Data using Arrays

- Store quality of results (QoR) metrics like area, power, WNS, TNS.
- Useful for tracking and comparing results across runs.

Example 5 : Store QoR metrics

```
array set qor {
    area        1250000
    power       0.85
    wns         -0.03
    tns         -150.5
    util        72.4
}
puts "Design Area  : $qor(area)"
puts "Design Power : $qor(power)"
puts "Design WNS   : $qor(wns)"
```

Output :

```
Design Area  : 1250000
Design Power : 0.85
Design WNS   : -0.03
```

Common Array Applications in EDA

Application	Key	Value	Example Use Case
Cell-to-Value Mapping	Cell name	Area / Type / Drive	Store cell attributes from reports
Scenario-to-Value Mapping	Scenario name	WNS / TNS / Status	MCMM result tracking
Timing Data	Startpoint/Endpoint	Slack / Delay	Parsed timing report data
Report Data	Violation type	Count	DRC / Antenna violations
QoR Data	Metric name	Value	Area, Power, WNS, TNS, Utilization

Key Takeaways

- [ ] Arrays are key-value pairs and very useful in EDA scripting.
- [ ] Use arrays to store and access cell, scenario, timing, report and QoR data.
- [ ] array set helps to initialize arrays.
- [ ] Use array names, array get, array exists and array size for operations.
- [ ] Arrays simplify parsing, data handling and report generation.
- [ ] Widely used in ICC2, PrimeTime and other EDA tools.
- [ ] Help in creating robust and scalable automation scripts.

7.7 Dictionaries in Tcl

- Dictionaries (dict) store key-value pairs, similar to associative arrays but more powerful.
- Keys can be strings (and also numbers), and values can be any Tcl object.
- Dictionaries are widely used in EDA scripts to store structured data like configurations, reports and analysis results.
- Useful for complex data handling, nested information and passing data between procedures.

1. dict set

- dict set is used to create a dictionary and add or update key-value pairs.
- If key already exists, the value is updated.

Example 1 : Creating and populating a dictionary

```
set design_info {}
dict set design_info name "my_chip"
dict set design_info process "7nm"
dict set design_info voltage "0.8V"
dict set design_info frequency "1GHz"
```

Output :

```
Dictionary created
with 4 keys
```

2. dict get

- dict get is used to retrieve the value for a given key.
- Throws an error if the key does not exist.

Example 2 : Accessing values using dict get

```
set name [dict get $design_info name]
set process [dict get $design_info process]
puts "Design : $name"
puts "Process : $process"
```

Output :

```
Design  : my_chip
Process : 7nm
```

3. dict exists

- dict exists checks if a given key exists in the dictionary.
- Useful to avoid runtime errors.

Example 3 : Checking key existence

```
if {[dict exists $design_info voltage]} {
    puts "Voltage is defined"
} else {
    puts "Voltage is NOT defined"
}
```

Output :

```
Voltage is defined
```

4. dict keys

- dict keys returns a list of all keys in the dictionary.
- Useful for iteration or debugging.

Example 4 : Getting all keys

```
set keys [dict keys $design_info]
puts "All keys: $keys"
```

Output :

```
All keys:
name process voltage
frequency
```

5. dict values

- dict values returns a list of all values in the dictionary.
- Useful when only values are needed.

Example 5 : Getting all values

```
set vals [dict values $design_info]
puts "All values: $vals"
```

Output :

```
All values:
my_chip 7nm 0.8V
1GHz
```

6. dict for

- dict for is used to iterate over all key-value pairs in a dictionary.
- Very useful for processing and printing dictionary contents.

Example 6 : Iterating over dictionary

```
dict for {key value} $design_info {
    puts "$key : $value"
}
```

Output :

```
name      : my_chip
process   : 7nm
voltage   : 0.8V
frequency : 1GHz
```

Additional Example : Nested Dictionary

```
set tech_info {}
dict set tech_info node 7nm
dict set tech_info limits [dict create \
        max_voltage 0.9 \
        max_temp   125 \
        min_voltage 0.7 ]
puts "Max Voltage = [dict get [dict get $tech_info limits] max_voltage]"
```

Output :

```
Max Voltage = 0.9
```

Common Dictionary Commands Summary

Command	Purpose	Example	Typical Output
dict set	Add / update key-value	dict set d key val	Dictionary updated
dict get	Get value for a key	dict get d key	Returns value
dict exists	Check if key exists	dict exists d key	1 (true) / 0 (false)
dict keys	List all keys	dict keys d	List of keys
dict values	List all values	dict values d	List of values
dict for	Iterate over key-value pairs	dict for {k v} d { ... }	Loops through all pairs

Key Takeaways

- [ ] Dictionaries store key-value pairs in Tcl.
- [ ] Use dict set to create and update entries.
- [ ] Use dict get to access values.
- [ ] Use dict exists to check if a key is present.
- [ ] Use dict keys and dict values to list keys or values.
- [ ] Use dict for to iterate over all key-value pairs.
- [ ] Dictionaries are widely used in EDA scripts for reports, configurations and QoR data.
- [ ] Help in organizing complex data in a clean and efficient way.

7.8 Dictionary Applications

- Dictionaries provide a flexible way to store and manage structured data in Tcl.
- They are widely used in EDA scripts for configurations, metadata, reports and QoR tracking.
- Help in organizing complex information with meaningful keys and values.
- Easy to access, update and iterate through data.
- Make scripts more readable, scalable and maintainable.

1. Structured Data

- Store related data like cell attributes, pin information or design hierarchy.
- Useful for handling multi-field information.

Example 1 : Store cell information as structured data

```
# Dictionary to store cell information
dict set cell_info U1 {
    name    "U1"
    type    "DFF"
    area    12.5
    drive   "X1"
}
# Access fields
puts "Cell name : [dict get $cell_info U1 name]"
puts "Cell type : [dict get $cell_info U1 type]"
puts "Cell area : [dict get $cell_info U1 area]"
```

Output :

```
Cell name : U1
Cell type : DFF
Cell area : 12.5
```

2. Configuration Data

- Store tool or flow configurations like paths, constraints, modes and switches.
- Useful for maintaining settings in a structured way.

Example 2 : Store flow configuration

```
dict set config tool        "ICC2"
dict set config tech        "7nm"
dict set config mode        "timing"
dict set config max_threads 8
dict set config output_dir  "/proj/run1/reports"

puts "Tool    : [dict get $config tool]"
puts "Tech    : [dict get $config tech]"
puts "Mode    : [dict get $config mode]"
puts "Threads : [dict get $config max_threads]"
puts "Output  : [dict get $config output_dir]"
```

Output :

```
Tool    : ICC2
Tech    : 7nm
Mode    : timing
Threads : 8
Output  : /proj/run1/reports
```

Common Dictionary Applications in EDA

Application	Key	Value (Example)	Use Case
Structured Data	Cell name	{type DFF area 12.5}	Store cell/pin info
Configuration Data	tool	ICC2	Tool/flow settings
QoR Information	WNS	-0.032	Timing results
	area	1250000	Area results
Design Metadata	version	1.0	Design tracking
	author	VLSI_Team	Documentation

3. QoR Information

- Store and analyze QoR metrics like WNS, TNS, area, power and congestion.
- Useful for tracking results across runs or scenarios.

Example 3 : Store QoR metrics in dictionary

```
dict set qor WNS        -0.032
dict set qor TNS        -12.5
dict set qor area       1250000
dict set qor power      0.85
dict set qor congestion 12.5

puts "WNS        : [dict get $qor WNS]"
puts "TNS        : [dict get $qor TNS]"
puts "Area       : [dict get $qor area]"
puts "Power      : [dict get $qor power]"
puts "Congestion : [dict get $qor congestion]"
```

Output :

```
WNS        : -0.032
TNS        : -12.5
Area       : 1250000
Power      : 0.85
Congestion : 12.5
```

Note : Dictionary can store both numeric and string values.

4. Design Metadata

- Store design-level information like block name, version, author, date and description.
- Useful for documentation, tracking and database generation.

Example 4 : Store design metadata

```
dict set design_meta name        "my_chip"
dict set design_meta version     "1.0"
dict set design_meta date        "2025-09-04"
dict set design_meta author      "VLSI_Team"
dict set design_meta description "Top level SoC design"

puts "Design  : [dict get $design_meta name]"
puts "Version : [dict get $design_meta version]"
puts "Date    : [dict get $design_meta date]"
puts "Author  : [dict get $design_meta author]"
puts "Desc    : [dict get $design_meta description]"
```

Output :

```
Design  : my_chip
Version : 1.0
Date    : 2025-09-04
Author  : VLSI_Team
Desc    : Top level
          SoC design
```

Key Takeaways

- [ ] Dictionaries are powerful for storing structured data in Tcl.
- [ ] Use dict set to create and update key-value pairs.
- [ ] Use dict get to access values.
- [ ] Dictionary applications include structured data, configuration data, QoR information and design metadata.
- [ ] Helps in organizing complex data for reports and automation.
- [ ] Widely used in ICC2, PrimeTime and other EDA tools.
- [ ] Makes scripts more flexible, scalable and maintainable.
- [ ] Essential for real-world VLSI design flows.

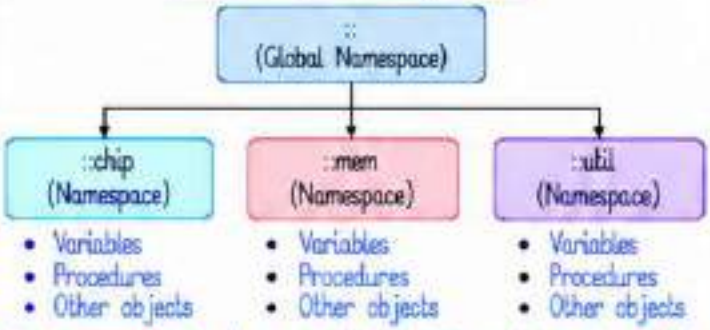
7.9 Namespaces in Tcl

- Namespaces provide a way to group variables, procedures and other objects.
- They create a separate scope and avoid naming conflicts.
- Each namespace has its own variables and procedures.
- Very useful in large scripts, libraries and EDA flows.
- Helps in code organization, reusability and maintainability.

1. Namespace Concept

- A namespace is a container that holds Tcl objects (variables, procedures, etc).
- Each namespace has its own scope.
- Objects inside a namespace are accessed using the namespace prefix (e.g., ns::var).
- Helps to avoid name clashes between different parts of a script.

Namespace Concept



Each namespace has its own set of objects and does not conflict with others.

2. Creating Namespaces

- Use the namespace eval command to create a namespace.
- You can also create multiple namespaces in a hierarchical way.

Example 1 : Creating namespaces

```
# Create simple namespaces
namespace eval chip {}
namespace eval mem {}
namespace eval util {}

# Create a hierarchical namespace
namespace eval chip::block {}
namespace eval chip::block::sub {}
```

Output :

```
Namespaces created:
::chip
::mem
::util
::chip::block
::chip::block::sub
```

3. Namespace Variables

- Variables inside a namespace are local to that namespace.
- Access them using the namespace prefix.

Example 2 : Using namespace variables

```
namespace eval chip {
    set width 100
    set height 200
}
puts "Width  : $chip::width"
puts "Height : $chip::height"
```

Output :

```
Width  : 100
Height : 200
```

4. Namespace Procedures

- Procedures can also be defined inside a namespace.
- They are accessed using the namespace prefix.
- Helps to keep related functionality together.

Example 3 : Defining and calling namespace procedures

```
namespace eval util {
    proc add {a b} {
        return [expr {$a + $b}]
    }
    proc sub {a b} {
        return [expr {$a - $b}]
    }
}
puts "5 + 3 = [util::add 5 3]"
puts "5 - 3 = [util::sub 5 3]"
```

Output :

```
5 + 3 = 8
5 - 3 = 2
```

5. Naming Conflicts

- Namespaces help to avoid naming conflicts between variables or procedures with the same name.
- You can have the same variable or procedure name in different namespaces without any conflict.

Example 4 : Avoiding naming conflicts

```
set value 10          # Global variable
namespace eval chip {
    set value 100     # chip::value
}
namespace eval mem {
    set value 200     # mem::value
}
puts "Global value : $value"
puts "Chip value   : $chip::value"
puts "Mem value    : $mem::value"
```

Output :

```
Global value : 10
Chip value   : 100
Mem value    : 200
```

Common Namespace Commands Summary

Command	Purpose	Example	Typical Output / Usage
namespace eval	Create a namespace and evaluate commands	namespace eval chip {}	Creates namespace ::chip
namespace exists	Check if namespace exists	namespace exists chip	1 (true) / 0 (false)
namespace delete	Delete a namespace	namespace delete chip	Namespace removed
namespace children	List sub-namespaces	namespace children chip	List of child namespaces
namespace vars	List variables in namespace	namespace vars chip	List of variables
namespace procs	List procedures in namespace	namespace procs chip	List of procedures
namespace inscope	Execute command in a namespace	namespace inscope chip { ... }	Runs command in ::chip scope

Key Takeaways

- ☐ Namespaces group related variables, procedures and other objects.
- ☐ Use namespace eval to create namespaces.
- ☐ Access namespace variables and procedures using the :: prefix.
- ☐ Namespaces help to avoid naming conflicts in large scripts.
- ☐ Useful for modular, scalable and maintainable Tcl code.
- ☐ Widely used in ICC2, PrimeTime and other EDA automation flows.

7.10 Tcl Introspection

- Tcl provides powerful introspection commands to examine the current interpreter state.
- These commands help to query existing commands, procedures, arguments, body and variables.
- Useful for debugging, code exploration and dynamic scripting.
- Widely used in EDA tools to understand built-in and user-defined functionality.

1. info commands

- info commands returns a list of all available Tcl commands.
- You can also filter using a pattern.
- Useful to check if a command exists or to discover commands provided by the tool.

Example 1 : Using info commands

```
# List all commands
puts [info commands]

# Search for commands with pattern
puts [info commands *db*]
```

Output :

```
... (list of all commands)
dbGet  dbSet  dbSave
dbOpen dbClose ...
```

2. info procs

- info procs returns a list of all defined procedures.
- You can use a pattern to filter the list.
- Useful to see user-defined procs and tool procs.

Example 2 : Using info procs

```
# List all procedures
puts [info procs]

# Search for procs with pattern
puts [info procs my_*]
```

Output :

```
my_proc1   my_proc2
my_check   my_report
```

3. info args

- info args returns the list of arguments for a procedure.
- Helps to understand how a procedure should be called.

Example 3 : Using info args

```
proc add {a b {c 0}} {
    return [expr {$a + $b + $c}]
}
puts [info args add]
```

Output :

```
a  b  c
```

Note :
- Optional arguments also appear in the list.
- You can use this to check the interface of existing procs.
- Helpful when working with tool built-in procedures.

4. info body

- info body returns the complete body (implementation) of a procedure.
- Useful to see how a procedure is implemented.
- Helpful for debugging and learning existing code.

Example 4 : Using info body

```
proc add {a b} {
    set sum [expr {$a + $b}]
    return $sum
}
puts [info body add]
```

Output :

```
set sum [expr {$a + $b}]
return $sum
```

Note : info body works only for Tcl procedures (not for built-in commands).

5. info vars

- info vars returns a list of all currently defined variables.
- You can use a pattern to filter the list.
- Useful to debug and check variable names in the current scope.

Example 5 : Using info vars

```
set chip "my_chip"
set voltage 0.8
set freq 1GHz
puts [info vars]
puts [info vars v*]
```

Output :

```
chip voltage freq ...
voltage
```

6. Combined Example

Using multiple introspection commands together.

Example 6 : Introspection in a script

```
proc report_design {name type} {
    puts "Reporting $name of type $type"
}
puts "Available commands :"
puts [info commands *report*]
puts "Procedure arguments :"
puts [info args report_design]
puts "Procedure body :"
puts [info body report_design]
puts "Current variables :"
puts [info vars]
```

Output :

```
Available commands :
report_design
Procedure arguments :
name type
Procedure body :
puts "Reporting $name
of type $type"
Current variables :
chip voltage freq ...
```

Key Takeaways

- [ ] Use info commands to list and search available commands.
- [ ] Use info procs to list defined procedures.
- [ ] Use info args to get procedure arguments.
- [ ] Use info body to see procedure implementation.
- [ ] Use info vars to list current variables.
- [ ] Tcl introspection helps in debugging, exploration and automation.
- [ ] Widely used in EDA scripts to understand tool commands and user code.

7.11 Advanced Evaluation

- Advanced evaluation techniques allow dynamic construction and execution of commands in Tcl.
- The eval command and command expansion provide flexibility to build commands at runtime.
- Useful in EDA scripting for tool automation, loops, and parameterized flows.
- Must be used carefully to avoid errors and security risks.
- Helps in creating generic and reusable scripts.

1. eval

- eval executes a Tcl command after performing another level of substitution.
- It treats its arguments as a single command.
- Useful when a command is stored in a variable or built dynamically.

Example 1 : Using eval

```
set cmd "puts Hello Tcl"
# execute using eval
eval $cmd

# eval with multiple arguments
eval puts "Sum =" [expr 10 + 20]
```

Output :

```
Hello Tcl
Sum = 30
```

Note :
- eval joins all arguments and executes as one command.
- Without eval, the command would not be executed.
- Check the constructed command before using eval.

2. Dynamic Command Construction

- Commands can be built dynamically using variables, loops or conditions.
- Useful for creating flexible and reusable scripts.
- Common in EDA flows where tool commands are built based on design names, scenarios, etc.

Example 2 : Building command dynamically

```
set design "chip1"
set report_type "timing"
set cmd "report_$(report_type) -design $design"
puts "Command : $cmd"
eval $cmd
```

Output :

```
Command :
report_timing
-design chip1
# (tool executes
the command)
```

Tip :
- Use variables and string substitution to build commands.
- Display the command (using puts) before executing.
- Helps in debugging and avoiding mistakes.

3. Command Expansion

- Tcl performs command substitution using square brackets [].
- The command inside [] is executed first and its result is used in the outer command.
- Useful for embedding commands within other commands.

Example 3 : Using command expansion

```
set design "top"
set cell_count [llength [get_cells]]
puts "Design : $design"
puts "Total cells : $cell_count"
```

Output :

```
Design : top
Total cells : 5236
```

```
set date [exec date]
puts "Current date : $date"
```

Output :

```
Current date :
Thu Sep 4 22:00:00 2025
```

Note :
- Command substitution using [] is commonly used in Tcl.
- The inner command is executed first, then substituted.
- Helps in getting dynamic information at runtime.

4. Safe Command Construction

- When building commands dynamically, ensure that the input is valid and does not cause errors or security issues.
- Use proper quoting, input validation, and whitelisting of allowed commands/options.
- Avoid executing untrusted user input directly with eval.

Example 4 : Safe command construction

```
# Safe way using list
set design "chip1"
set rpt_cmd [list report_timing -design $design]
puts "Command : $rpt_cmd"
eval $rpt_cmd
```

Output :

```
Command :
report_timing
-design chip1
# (tool executes)
```

```
# Validate input before executing
set valid_reports {timing power area}
set rpt "timing"
if {[lsearch $valid_reports $rpt] != -1} {
    set cmd "report_$(rpt)"
    eval $cmd
} else {
    puts "Error: Invalid report type $rpt"
}
```

Output :

```
report_timing
# (tool executes)
```

Caution :
- Do not pass untrusted input directly to eval.
- Use list to build commands safely (prevents unwanted substitution).
- Validate user inputs and allow only expected commands/options.

Key Takeaways

- [ ] eval executes a command after substitution.
- [ ] Dynamic command construction helps in flexible and reusable scripts.
- [ ] Command expansion using [] executes inner commands first.
- [ ] Use safe command construction techniques (list, input validation).
- [ ] Widely used in EDA tools for automation and complex flows.

8.1 ICC2 Tcl Environment

- ICC2 provides a Tcl-based command interface to interact with the tool.
- Tcl is the underlying scripting language and ICC2 adds its own set of commands for physical implementation.
- You can run commands interactively or through scripts.
- ICC2 Tcl environment is used for design setup, implementation, analysis and reporting.
- Understanding the Tcl environment is the first step to efficient automation in ICC2.

1. ICC2 Command Interpreter

- ICC2 uses a Tcl interpreter with additional ICC2 commands.
- You can enter commands interactively at the ICC2 shell prompt or run them from a Tcl script.
- The prompt usually appears as: icc2>
- You can use standard Tcl commands as well as ICC2 specific commands.
- Supports command history, auto-completion and help.

Example 1 : ICC2 interactive shell

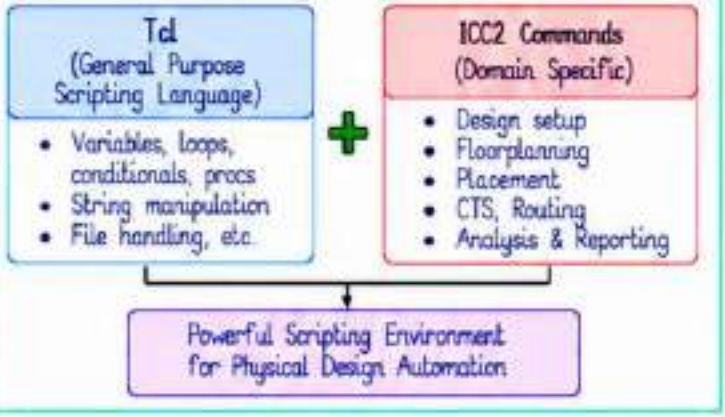
```
$ icc2
Loading ICC2 ...
icc2> version
ICC2 v2023.12
icc2> get_designs
top
icc2>
```

Note :
The icc2> prompt indicates the tool is ready to accept Tcl/ICC2 commands.

2. Tcl + ICC2 Relationship

- ICC2 is built on top of the Tcl language.
- Tcl provides the language features (variables, loops, conditionals, procedures, etc.).
- ICC2 adds its own domain-specific commands for physical design (e.g., create_floorplan, place_opt, route_opt).
- You can freely mix standard Tcl commands and ICC2 commands in the same script.
- This allows you to build powerful scripts for complex flows.

Tcl + ICC2 Relationship



3. ICC2 Command Syntax

- ICC2 commands follow Tcl syntax.
- General format:

```
command_name  [options]  arguments
```

- Options usually start with - (dash).
- You can use help to get command usage.

Example 2 : Basic command syntax

```
create_floorplan  -core_utilization 0.7 \
                  -core_aspect_ratio 1.0
```

Output :

```
Creating floorplan ...
Done
```

4. ICC2 Options

- Options are used to specify the behavior of a command.
- Most options use a dash (-) followed by the option name.
- Options can take values (numbers, strings, lists).
- You can use help <command> to see all available options.
- Some options are mandatory, others are optional.

Example 3 : Using options

```
read_verilog  -top top -format sverilog \
              -file {rtl/top.sv}
```

Output :

```
Reading Verilog file
rtl/top.sv ...
Done
```

Common ICC2 Options (Examples)

Option	Purpose	Example
-top	Specify top module	-top top
-format	Input file format	-format sverilog
-file	Input file name	-file {rtl/top.sv}
-output	Output file name	-output {reports/rpt.rpt}
-verbose	Enable detailed messages	-verbose

5. ICC2 Variables

- You can use Tcl variables to store values (strings, numbers, lists, etc.)
- Variables are useful for parameterizing scripts.
- You can also access ICC2 tool variables (e.g., design information).

Example 4 : Using variables

```
set top_block "top"
set util 0.7
create_floorplan -core_utilization $util
puts "Top block : $top_block"
```

Output :

```
Top block : top
```

6. Script Execution

- You can run Tcl scripts using the source command.
- Scripts can contain multiple ICC2 and Tcl commands.
- Useful for automating complete implementation flows.

Example 5 : Running a script

```
source setup_design.tcl
```

Output :

```
Sourcing file setup_design.tcl ...
Done
```

Key Takeaways

- ☐ ICC2 provides a Tcl-based command environment.
- ☐ Tcl and ICC2 commands can be used together.
- ☐ ICC2 commands follow Tcl syntax with options.
- ☐ Variables and scripts help in automation and flexible flows.
- ☐ Use source to run Tcl scripts.
- ☐ Essential knowledge for efficient ICC2 implementation.

8.2 ICC2 Command Structure

- ICC2 commands follow a well-defined structure.
- A command consists of a command name, options, arguments and switches.
- Commands can take design objects and their attributes as input.
- The command returns a result which can be used in subsequent commands.
- Understanding the command structure helps in writing correct and efficient scripts.

1. Command

- A command is the main keyword that performs a specific task.
- ICC2 has built-in commands for design setup, implementation, analysis and reporting.
- Commands are case-sensitive and follow Tcl syntax.

Example 1 : Basic ICC2 command

```
create_floorplan
compile_pg
report_timing
```

Note : These are built-in ICC2 commands, not user-defined Tcl procedures.

2. Options

- Options are used to modify the behavior of a command.
- Options usually start with a -(dash) followed by the option name.
- Some options take values, while others are just switches.

Example 2 : Using options

```
create_floorplan -core_utilization 0.7 \
                 -core_aspect_ratio 1.0
```

Output : Floorplan created with specified options.

3. Arguments

- Arguments provide input values to a command.
- Arguments can be numbers, strings, lists or design objects.
- The order of arguments usually matters.

Example 3 : Command with arguments

```
set_dont_touch_network VDD
create_clock -name clk -period 5 [get_ports clk]
```

4. Object Arguments

- Object arguments are design objects such as cells, nets, ports, clocks, etc.
- They are usually obtained using get_* commands.

Example 4 : Using object arguments

```
set_attribute [get_cells U1] dont_touch true
report_net -connections [get_nets data_net]
```

5. Attribute Arguments

- Attribute arguments are used to get or set properties of design objects.
- Attributes define the characteristics of an object (e.g., area, timing, physical properties).
- Attributes are accessed using get_attribute and set_attribute.

Example 5 : Using attribute arguments

```
set_attribute [get_cells U1] ref_name NAND2_X1
set util [get_attribute [get_cells U1] area]
```

6. Switches

- Switches are options that do not take a value.
- They are used to enable or disable a specific behavior.
- Common switches: -verbose, -quiet, -force, -hier, etc.

Example 6 : Using switches

```
report_timing -verbose
remove_cell U1 -force
```

7. Command Results

- Commands return results such as numbers, strings, lists or objects.
- The result can be stored in a variable and used in other commands.
- Many report commands return a list of objects.

Example 7 : Using command results

```
set cells [get_cells *]
puts "Total cells: [llength $cells]"
set area [get_attribute $cells area]
puts "Area: $area"
```

Output :

```
Total cells : 5236
Area : 125489.3
```

ICC2 Command Structure Summary

Element	Description	Example
Command	Main keyword to perform a task	create_floorplan
Options	Modify command behavior	-core_utilization 0.7
Arguments	Input values (numbers, strings, lists)	VDD, 5, "clk"
Object arguments	Design objects (cells, nets, ports, etc.)	[get_cells U1]
Attribute arguments	Object properties	ref_name, area
Switches	Options without values	-verbose, -force
Command results	Return value from command	List of objects, numbers, or strings

Key Takeaways

- ☐ ICC2 commands have a clear structure: command, options and arguments.
- ☐ Options modify behavior; switches are options without values.
- ☐ Arguments can be values, objects or attributes.
- ☐ Use get_* commands to obtain design objects.
- ☐ Command results can be stored and used in subsequent commands.
- ☐ Understanding command structure helps in writing efficient and error-free ICC2 scripts.

8.3 ICC2 Design Objects

- ICC2 provides a rich hierarchy of design objects to represent the physical design.
- Design objects model the design database (blocks, cells, nets, etc.).
- Each object has attributes and can be queried/modified using Tcl.
- Understanding design objects is important for writing effective scripts for implementation, analysis and reporting.
- You can use get_* commands to access these objects.

1. Design

- Top level object representing the entire chip.
- Contains one or more blocks (top block is the current design).
- Holds global information like technology, libraries, constraints, etc.

Example 1 : Getting design object

```
set des [get_designs]
puts "Current design : $des"
```

Output :

```
Current design : chip_top
```

2. Block

- A block is a hierarchical container within the design.
- Top block represents the chip, and it can have sub-blocks.
- Each block contains cells, nets, ports, etc.

Example 2 : Getting block objects

```
set top [get_blocks]
puts "Top block : $top"
```

Output :

```
Top block : chip_top
```

3. Cell

- Cells are instances of library cells placed in the design.
- They can be standard cells, macros or other instances.
- Cells have pins, placement information and other attributes.

Example 3 : Getting cell objects

```
set cells [get_cells *]
puts "Total cells : [sizeof_collection $cells]"
```

Output :

```
Total cells : 5256
```

4. Pin

- Pins are connection points on a cell, port or net.
- Each cell has input, output and inout pins.
- Pins have attributes like direction, net, timing info, etc.

Example 4 : Getting pin objects

```
set pins [get_pins U1/*]
puts "Pins of U1 : $pins"
```

5. Port

- Ports are the interface points of a block (top or sub-block).
- They connect internal nets to the external environment.
- Ports have direction (in, out, inout) and timing constraints.

Example 5 : Getting port objects

```
set ports [get_ports *]
puts "Total ports : [sizeof_collection $ports]"
```

Output :

```
Total ports : 256
```

6. Net

- Nets connect pins, ports and cells in the design.
- Nets can be signal nets, power nets or ground nets.
- Nets have attributes like wire length, capacitance, fanout, etc.

Example 6 : Getting net objects

```
set nets [get_nets *]
puts "Total nets : [sizeof_collection $nets]"
```

Output :

```
Total nets : 10432
```

7. Clock

- Clocks are timing objects used for sequential elements.
- Defined using create_clock and propagated through the design.
- Clocks have attributes like period, waveform, source, etc.

Example 7 : Getting clock objects

```
set clks [get_clocks *]
puts "Clocks : $clks"
```

Output :

```
Clocks : clk
         test_clk
```

8. Library Cell

- Library cells are master definitions from the technology library.
- They define the logical and physical implementation (size, power, etc.).
- Instantiated in the design as cells.

Example 8 : Getting library cells

```
set libcells [get_lib_cells *]
puts "Library cells : [sizeof_collection $libcells]"
```

Output :

```
Library cells : 8765
```

9. Library Pin

- Library pins are pins of library cells (master definition).
- They define logical direction, function, and electrical characteristics.

Example 9 : Getting library pins

```
set lpins [get_lib_pins NAND2_X1/*]
puts "Lib pins : $lpins"
```

10. Layer

- Layers are used for routing and manufacturing (metal, via, etc.).
- Each layer has properties like type, direction, pitch, width, etc.

Example 10 : Getting layer objects

```
set layers [get_layers *]
puts "Routing layers : $layers"
```

Output :

```
Routing layers :
M1 M2 M3 ...
```

11. Via

- Vias connect two routing layers.
- They have attributes like cut size, resistance, etc.

Example 11 : Getting via objects

```
set vias [get_vias *]
puts "Vias : [sizeof_collection $vias]"
```

Output :

```
Vias : 250
```

12. Shape

- Shapes represent the physical geometry (wires, vias, blockages, etc.).
- Used for physical analysis, DRC and custom queries.

Example 12 : Getting shape objects

```
set shapes [get_shapes *]
puts "Total shapes : [sizeof_collection $shapes]"
```

Output :

```
Total shapes : 15872
```

13. Site

- Sites define the placement grid for standard cells.
- They control where cells can be placed in the design.

Example 13 : Getting site objects

```
set sites [get_sites *]
puts "Sites : $sites"
```

Output :

```
Sites : core_site
```

- get_* commands are widely used in ICC2
- They return design objects from the current design
- Useful for querying and filtering objects
- Support patterns, filters and hierarchical search
- Essential for automation and analysis scripts

- Use -hierarchical to search in sub-blocks
- Use -filter to apply conditions
- Combine with other commands for automation
- Speeds up design exploration and debug
- One of the most used features in ICC2 Tcl

8.4 ICC2 get_* Commands

- The get_* commands return design objects from the current design.
- They can be used with patterns, filters and hierarchical options.
- Returned objects can be used in other ICC2 commands.
- Very useful for querying design data and writing automation scripts.

1. get_cells

- Returns cells (instances) in the design.
- Can filter by name, ref_name, is_hierarchical, etc.

Example 1 : Using get_cells

```
get_cells *
get_cells U1
get_cells -hierarchical *ALU*
get_cells -filter {is_hierarchical == true}
```

Output :

```
U1 U2 U3 ...
ALU/U1 ...
ALU U_CPU ...
```

2. get_pins

- Returns pins of cells, ports or nets.
- Useful to access pin properties and connectivity.

Example 2 : Using get_pins

```
get_pins U1/*
get_pins -of_objects [get_cells U1]
get_pins -filter {direction == in}
```

Output :

```
U1/A U1/Z ...
U1/A U1/B ...
U1/A U2/CK ...
```

3. get_ports

- Returns top-level ports of the design.
- Ports can be input, output or inout.

Example 3 : Using get_ports

```
get_ports *
get_ports clk
get_ports -filter {direction == in}
```

Output :

```
clk rst data ...
clk
clk rst ...
```

4. get_nets

- Returns nets in the design.
- Can be used with filters (e.g., wire load, fanout, etc.).

Example 4 : Using get_nets

```
get_nets *
get_nets -hierarchical *clk*
get_nets -filter {is_clock == true}
```

Output :

```
n1 n2 n3 ...
top/clk ...
clk core_clk ...
```

5. get_clocks

- Returns clock objects defined in the design.
- Useful for timing analysis and clock queries.

Example 5 : Using get_clocks

```
get_clocks *
get_clocks clk
get_clocks -filter {is_generated == true}
```

Output :

```
clk test_clk ...
clk
gen_clk ...
```

Key Takeaways

- get_* commands return design objects from the current design.
- Use patterns and -filter to get specific objects.
- Returned objects can be used in other ICC2 commands.
- These commands are essential for design exploration, debug and automation.
- Know the commonly used get_* commands and their applications.

6. get_lib_cells

- Returns library cells from the technology libraries.
- Useful to query cell properties and perform mappings.

Example 6 : Using get_lib_cells

```
get_lib_cells *
get_lib_cells NAND2_X1
get_lib_cells -filter {is_sequential == true}
```

Output :

```
NAND2_X1 ...
NAND2_X1
DFF_X1 ...
```

7. get_lib_pins

- Returns pins of library cells.
- Useful to query pin direction, function, timing, etc.

Example 7 : Using get_lib_pins

```
get_lib_pins NAND2_X1/*
get_lib_pins -of_objects [get_lib_cells NAND2_X1]
get_lib_pins -filter {direction == out}
```

Output :

```
NAND2_X1/A ...
NAND2_X1/A ...
NAND2_X1/Z ...
```

8. get_layers

- Returns routing layers from the technology file.
- Useful for routing, layer constraints and analysis.

Example 8 : Using get_layers

```
get_layers *
get_layers M1
get_layers -filter {type == metal}
```

Output :

```
M1 M2 M3 ...
M1
M1 M2 M3 ...
```

9. get_vias

- Returns via definitions from the technology.
- Useful to check via properties like resistance, size, etc.

Example 9 : Using get_vias

```
get_vias *
get_vias VIA12
get_vias -filter {cut_class == "single"}
```

Output :

```
VIA12 VIA23 ...
VIA12
VIA12 VIA34 ...
```

10. get_shapes

- Returns physical shapes (wires, vias, blockages, etc.).
- Useful for DRC, routing and physical analysis.

Example 10 : Using get_shapes

```
get_shapes *
get_shapes -of_objects [get_nets clk]
get_shapes -filter {layer_name == M2}
```

Output :

```
shape1 shape2 ...
shape100 shape101 ...
shape10 shape11 ...
```

11. get_sites

- Returns placement sites from the technology.
- Useful to check site names, size and symmetry.

Example 11 : Using get_sites

```
get_sites *
get_sites core_site
get_sites -filter {is_core == true}
```

Output :

```
core_site 10_site ...
core_site
core_site ...
```

Common get_* Commands Summary

Command	Returns	Command	Returns
get_cells	Design cells (instances)	get_lib_pins	Pins of library cells
get_pins	Pins of cells/ports/nets	get_layers	Routing layers
get_ports	Top-level ports	get_vias	Via definitions
get_nets	Nets in the design	get_shapes	Physical shapes
get_clocks	Clock objects	get_sites	Placement sites
get_lib_cells	Library cells		

Author : Konda Tharun Vivekananda

"Query objects today, build better designs tomorrow !"

* Wildcards simplify object selection in ICC2
* Useful for large hierarchical designs
* Save time and make scripts more flexible
* Combine with get_* commands and filters
* Essential for efficient design exploration

* Powerful patterns for object selection
* Works with hierarchical and full names
* Useful in analysis, debugging and automation
* Reduces manual effort in large designs
* A must-know feature for productive scripting

8.5 ICC2 Wildcards

- Wildcards allow pattern-based matching to select design objects.
- They are used with get_* commands in ICC2.
- Help in querying multiple objects without specifying exact names.
- Support simple and hierarchical matching.
- Very useful for large designs with complex hierarchies.

1. * (Asterisk)

- * matches zero or more characters.
- It can be used at the beginning, middle or end of a name.
- Commonly used to get a group of objects with similar names.

Example 1 : Using * wildcard

```
get_cells *
get_cells U1/*
get_nets clk*
get_ports *rst*
get_pins */D*
```

Explanation :

All cells in design
All cells under U1
Nets starting with clk
Ports containing rst
Pins ending with /D

2. ? (Question Mark)

- ? matches exactly one character.
- Useful when only one character is unknown.
- Helps to match a specific pattern.

Example 2 : Using ? wildcard

```
get_cells U?1
get_nets n?5
get_ports clk?
get_pins */Q?
```

Explanation :

Matches U01, U11, U21 ...
Matches n15, n25 ...
Matches clk1, clk2 ...
Matches /Q0, /Q1 ...

3. Pattern Matching

- Use * and ? together to create flexible patterns.
- Pattern matching helps to select multiple related objects.
- You can also combine with filters for more refined selection.

Example 3 : Pattern matching examples

```
get_cells U*_REG*
get_nets *data*
get_ports ?clk
get_pins *U1*/D*
```

Explanation :

Cells starting with U and containing REG
Nets containing the word data
Ports like aclk, bclk, etc.
All D pins under U1 hierarchy

Note :

- Wildcard matching is case-sensitive in ICC2.
- Use quotes if pattern contains special characters.
- You can combine wildcards with filters (-filter option).

4. Hierarchical Matching

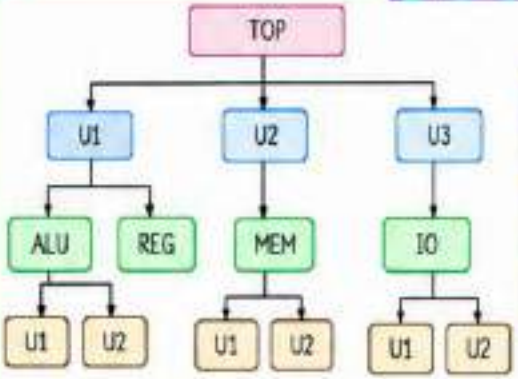
- Wildcards can be used to search across hierarchy.
- Use / to navigate hierarchy levels.
- Helps to select objects in specific or multiple hierarchical levels.
- Useful for block-level or top-level queries.

Example 4 : Using wildcards with hierarchy

```
get_cells TOP/U1/*
get_cells */ALU/*
get_nets */data/*
get_pins TOP/*/U2/D*
```

Explanation :

All cells under TOP/U1
All cells under any ALU block
All nets under any data block
D pins of U2 under TOP



Example :
get_cells */ALU/*
will match:
TOP/U1/ALU/U1
TOP/U1/ALU/U2

5. Full-name Matching

- ICC2 objects have full hierarchical names (e.g., TOP/U1/ALU/U1).
- You can use wildcards with full names for exact hierarchy matching.
- Useful when the same instance name exists in multiple blocks.
- Helps to avoid ambiguity in large designs.

Example 5 : Using full-name matching

```
get_cells TOP/U1/ALU/*
get_pins TOP/U1/ALU/U2/*
get_nets TOP/*/MEM/*
get_ports TOP/*/IO/*
```

Explanation :

All cells under this block
All pins of U2 in ALU
All nets under any MEM block
All ports under any IO block

Note :

- Full-name matching is more specific and avoids multiple matches.
- Use wildcards to handle repeated hierarchy.
- Combine with -hierarchical option if required.

Key Takeaways

- [ ] * matches zero or more characters, ? matches one character.
- [ ] Use wildcards with get_* commands to select multiple objects.
- [ ] Wildcards work with hierarchical and full names.
- [ ] Pattern matching helps in flexible and efficient querying.
- [ ] Essential for handling large hierarchical designs in ICC2.

Author : Konda Tharun Vivekananda

"Small patterns, big possibilities !"

8.6 Hierarchical Object Access

- ICC2 supports hierarchical access to all design objects.
- You can access objects at any level using their full hierarchical names.
- Hierarchical access is essential for large SoC designs with multiple blocks.
- You can use get_* commands with hierarchical paths.
- Helps in debug, analysis, constraints and targeted modifications.

1. Top-level Objects

- Top-level objects are directly under the current design.
- They can be accessed without hierarchy prefix.
- Examples: top-level cells, nets, ports, clocks, etc.

Example 1 : Accessing top-level objects

```
get_cells U1
get_ports clk
get_nets data
get_clocks core_clk
```

Output :

```
U1
clk
data
core_clk
```

2. Hierarchical Instances

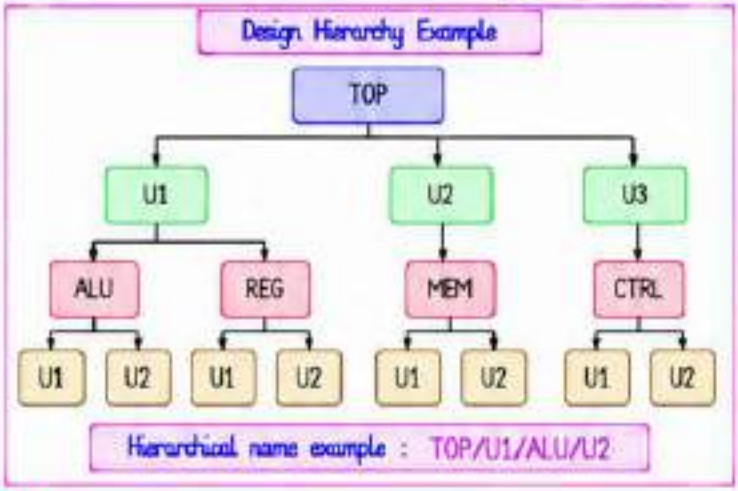
- Instances can be hierarchical (contain sub-instances).
- Use hierarchical names to access instances at any level.
- Hierarchy is separated by "/" (slash).

Example 2 : Accessing hierarchical instances

```
get_cells U1
get_cells U1/ALU
get_cells U1/ALU/U2
get_cells TOP/SUB1/U3
```

Output :

```
U1
ALU
U2
U3
```



3. Hierarchical Pins

- Pins of hierarchical instances can be accessed using full names.
- You can access instance pins or hierarchical port connections.

Example 3 : Accessing hierarchical pins

```
get_pins U1/ALU/U2/A
get_pins U1/ALU/*
get_pins TOP/U3/CTRL/clk
```

Output :

```
A
A B Y ...
clk
```

4. Hierarchical Nets

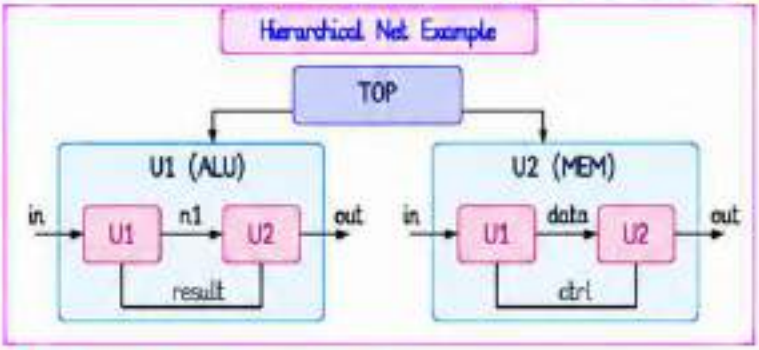
- Nets can also be accessed hierarchically.
- Useful for tracing connectivity across blocks.
- Helps in analysis, debugging and applying constraints.

Example 4 : Accessing hierarchical nets

```
get_nets U1/ALU/n1
get_nets U1/REG/*
get_nets TOP/U2/MEM/data
```

Output :

```
n1
n1 n2 n3 ...
data
```



5. Full Object Names

- Full object name specifies the complete hierarchical path.
- Format: <inst1>/<inst2>/.../<object>
- Full names are unique and avoid ambiguity.
- Useful when the same instance or net name exists in multiple blocks.

Example 5 : Using full object names

```
get_cells TOP/U1/ALU/U2
get_pins TOP/U1/ALU/U2/Y
get_nets TOP/U1/ALU/n1
get_ports TOP/clk
```

Output :

```
U2
Y
n1
clk
```

Note :

- Use full names when there are name conflicts.
- You can combine full names with wildcards (e.g., TOP/U1/*).
- Full names are case-sensitive.
- Use get_attribute to get properties of hierarchical objects.

More Examples

Command	Purpose
get_cells */U2	Get all instances named U2 in design
get_pins U1/*/A	Get pin A in all sub-instances under U1
get_nets */data	Get all nets named data in hierarchy
get_ports */clk	Get all ports named clk
get_cells -hier U2	Get all U2 instances in hierarchy

Key Takeaways

- ☐ Hierarchical access allows querying objects at any level.
- ☐ Use "/" to specify hierarchical paths.
- ☐ Can access hierarchical instances, pins, nets, ports, etc.
- ☐ Full object names avoid ambiguity.
- ☐ Essential for debug, analysis and automation in large designs.

8.7 ICC2 Collections

- ICC2 collections are special Tcl objects used to store and manipulate design objects (cells, nets, pins, etc.).
- They are more efficient than Tcl lists for large designs.
- Collections are widely used with get_* commands in ICC2.
- Understanding collections is essential for writing efficient and scalable ICC2 scripts.

1. What is a Collection ?

- A collection is a handle to a set of design objects in ICC2.
- It stores references to objects (not their names as strings).
- Collections are created as a result of get_* commands.
- They allow efficient operations on large number of objects.

Example 1 : Creating a collection

```
set c1 [get_cells *]
puts $c1
```

Note :
$c1 is a collection handle (pointing to cells), not a list of cell names.

2. Tcl List vs ICC2 Collection

- A Tcl list is a simple list of strings.
- An ICC2 collection is a special object handle optimized for design data.
- Collections are memory efficient and support advanced operations (filtering, attributes, etc.).

Tcl List vs ICC2 Collection

Feature	Tcl List	ICC2 Collection
Content	List of strings (names)	Handle to design objects
Example	{U1 U2 U3}	[get_cells *]
Memory usage	Higher (stores strings)	Lower (stores references)
Performance	Slower for large data	Faster and optimized
Supported operations	Basic list operations	Collection-specific commands
Use case	General Tcl scripting	Design object manipulation in ICC2

3. Collection Handles

- A collection is referenced by a handle (e.g., c1, c2).
- The handle can be stored in a variable.
- The handle represents a set of objects in memory.
- You can pass the handle to ICC2 commands directly.

Example 2 : Collection handle usage

```
set all_cells [get_cells *]
set reg_cells [get_cells -hier -filter {is_sequential == true}]
puts $reg_cells
```

Note :
- all_cells and reg_cells are collection handles.
- They can be used directly in other ICC2 commands.

4. Collection Size

- use sizeof_collection to get the number of objects in a collection.
- Useful to check if a collection is empty or has objects.

Example 3 : Finding collection size

```
set c1 [get_nets *]
set count [sizeof_collection $c1]
puts "Total nets : $count"
```

Output :
Total nets : 10432

Note :
- If collection is empty, size will be 0.
- Useful in conditional checks.

5. Collection Manipulation

- ICC2 provides several commands to manipulate collections.
- You can filter, combine, subtract and get attributes of collections.
- Common commands: filter_collection, add_to_collection, remove_from_collection, intersect_collections, sizeof_collection, get_attribute.

5.1 Filtering a collection

Example 4 : Using filter_collection

```
set regs [get_cells *]
set reg_dff [filter_collection $regs {ref_name == DFF*}]
puts "DFF count : [sizeof_collection $reg_dff]"
```

Output :
DFF count : 5236

5.2 Combining collections

Example 5 : Using add_to_collection

```
set c1 [get_cells U1/*]
set c2 [get_cells U2/*]
set all [add_to_collection $c1 $c2]
puts "Total cells : [sizeof_collection $all]"
```

Output :
Total cells : 8765

5.3 Removing objects from a collection

Example 6 : Using remove_from_collection

```
set all [get_cells *]
set dont_touch [get_cells -filter {dont_touch == true}]
set useful [remove_from_collection $all $dont_touch]
puts "Useful cells : [sizeof_collection $useful]"
```

Output :
Useful cells : 7201

5.4 Collection intersection

Example 7 : Using intersect_collections

```
set c1 [get_nets -of_objects [get_cells U1/*]]
set c2 [get_nets -of_objects [get_cells U2/*]]
set common [intersect_collections $c1 $c2]
puts "Common nets : [sizeof_collection $common]"
```

Output :
Common nets : 152

Key Takeaways

- ☐ ICC2 collections are efficient handles to design objects.
- ☐ They are different from Tcl lists.
- ☐ Use sizeof_collection to get the number of objects.
- ☐ Collections can be manipulated using filter, add, remove, intersect.
- ☐ Essential for handling large designs in ICC2 scripting.

8.8 Collection Commands

- ICC2 provides collection commands to operate on design object collections (cells, nets, pins, etc.).
- Collections are more efficient than Tcl lists for large designs.
- These commands help in querying, manipulating and processing design data.
- They are widely used with get_* commands in ICC2.
- Understanding these commands is essential for writing powerful and scalable scripts.

1. sizeof_collection

- Returns the number of objects in a collection.
- Useful to check how many objects are selected.
- If the collection is empty, it returns 0.

Example 1 : Using sizeof_collection

```
set cells [get_cells *]
set count [sizeof_collection $cells]
puts "Total cells : $count"
```

Output :

```
Total cells : 8765
```

2. get_object_name

- Returns the full or short name of objects in a collection.
- Useful for printing or logging object names.
- Can be used with loops to iterate over collection objects.

Example 2 : Using get_object_name

```
set ports [get_ports *]
foreach p $ports {
    puts [get_object_name $p]
}
```

Output :

```
clk
rst
data_in
data_out
...
```

3. index_collection

- Returns the object at a specific index from a collection.
- Index starts from 0.
- Useful to access individual objects.

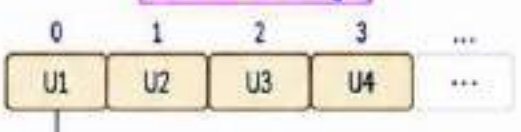
Example 3 : Using index_collection

```
set cells [get_cells *]
set first_cell [index_collection $cells 0]
puts [get_object_name $first_cell]
```

Output :

```
U1
```

Collection Indexing



4. add_to_collection

- Adds one or more objects to an existing collection.
- Useful to build a collection incrementally.
- Returns a new collection with the added objects.

Example 4 : Using add_to_collection

```
set c1 [get_cells U1]
set c2 [get_cells U2]
set all_cells [add_to_collection $c1 $c2]
puts "Total cells : [sizeof_collection $all_cells]"
```

Output :

```
Total cells : 2
```



5. remove_from_collection

- Removes objects from a collection.
- Useful to exclude unwanted objects.
- Returns a new collection with the specified objects removed.

Example 5 : Using remove_from_collection

```
set all_cells [get_cells *]
set reg_cells [get_cells -hier -filter {is_sequential == true}]
set non_regs [remove_from_collection $all_cells $reg_cells]
puts "Combinational cells : [sizeof_collection $non_regs]"
```

Output :

```
Combinational cells : 7201
```

6. sort_collection

- Sorts the objects in a collection based on a given attribute.
- Useful for ordered processing and consistent results.
- Common options: -dictionary, -ascending, -descending.

Example 6 : Using sort_collection

```
set cells [get_cells *]
set sorted_cells [sort_collection $cells -dictionary]
foreach c $sorted_cells {
    puts [get_object_name $c]
}
```

Output :

```
U1
U10
U100
U101
...
```

7. filter_collection

- Filters a collection based on a given condition.
- Widely used to select objects with specific attributes.
- Supports logical expressions and attribute-based filtering.

Example 7 : Using filter_collection

```
set cells [get_cells *]
set big_cells [filter_collection $cells {area > 1000}]
puts "Big cells : [sizeof_collection $big_cells]"
```

Output :

```
Big cells : 256
```

Key Takeaways

- Collection commands operate on ICC2 collections, not Tcl lists.
- They help in querying, manipulating and filtering design objects.
- Use sizeof_collection to check size, get_object_name to get names.
- Use add_to_collection, remove_from_collection, sort_collection and filter_collection for powerful data handling.
- Essential for efficient and scalable ICC2 scripting.

8.9 Collection Iteration

- ICC2 provides `foreach_in_collection` to iterate over collection objects
- Iteration is more efficient than converting to Tcl lists
- Used to access and process each object in a collection
- Collection variables can be used to store and pass collections
- Nested iteration is useful for hierarchical designs
- Essential for writing flexible and scalable ICC2 scripts

1. foreach_in_collection

- Iterates over each object in a collection
- Syntax is similar to Tcl `foreach` but works with ICC2 collections
- The loop variable represents a single design object

Syntax — foreach_in_collection syntax

```
foreach_in_collection <var> <collection> {
    # commands
}
```

Example 1 : Iterate over cells

```
set cells [get_cells *]
foreach_in_collection c $cells {
    puts [get_object_name $c]
}
```

Output :

```
U1
U2
U3
...
```

2. Collection Variables

- Collections can be stored in Tcl variables
- These variables hold handles to design objects
- Can be passed between procedures and used in loops

Example 2 : Using collection variables

```
set my_cells [get_cells -hier *]
puts "Total cells : [sizeof_collection $my_cells]"
foreach_in_collection c $my_cells {
    puts [get_object_name $c]
}
```

Output :

```
Total cells : 5234
U1
U2
U3
...
```

3. Object Names

- Use `get_object_name` to get the full or short name
- Helps in logging, reporting and debugging
- Can also get attributes of each object during iteration

Example 3 : Printing object names and attributes

```
set nets [get_nets *]
foreach_in_collection n $nets {
    set name [get_object_name $n]
    set drv  [get_attribute $n driver]
    puts "$name -> driver : [get_object_name $drv]"
}
```

Output :

```
net1 -> driver : U1/A
net2 -> driver : U2/Z
net3 -> driver : U3/Y
...
```

4. Collection Processing

- You can perform any ICC2 command inside the loop
- Useful for analysis, reporting and modifications
- Common operations: get attributes, set attributes, filter, etc.

Example 4 : Processing collection objects

```
set ports [get_ports *]
foreach_in_collection p $ports {
    set dir  [get_attribute $p direction]
    set type [get_attribute $p port_type]
    puts "[get_object_name $p] : $dir , $type"
}
```

Output :

```
clk : in , clock
rst : in , signal
data_in : in , signal
data_out : out , signal
...
```

Note :
- All ICC2 commands can be used inside the loop
- Avoid heavy operations inside the loop for performance
- Use filters to reduce the collection size if possible

5. Nested Collection Processing

- You can use nested `foreach_in_collection` loops
- Useful for hierarchical designs (e.g., blocks → cells → pins)
- Helps in in-depth analysis and data extraction

Example 5 : Nested iteration (blocks → cells → pins)

```
set blks [get_blocks *]
foreach_in_collection b $blks {
    puts "Block : [get_object_name $b]"
    set cells [get_cells -of_objects $b]
    foreach_in_collection c $cells {
        puts "  Cell : [get_object_name $c]"
        set pins [get_pins -of_objects $c]
        foreach_in_collection p $pins {
            puts "    Pin : [get_object_name $p]"
        }
    }
}
```

Output :

```
Block : TOP
  Cell : U1
    Pin : U1/A
    Pin : U1/Z
  Cell : U2
    Pin : U2/A
    Pin : U2/Z
Block : U3
  Cell : U3_1
    Pin : U3_1/A
    Pin : U3_1/Z
  ...
```

Best Practices :
- Use meaningful variable names for readability
- Apply filters to limit the collection size
- Avoid unnecessary nested loops
- Use `get_object_name` for clear logging
- Combine with `filter_collection` for efficient processing

Key Takeaways

- [ ] `foreach_in_collection` is used to iterate over ICC2 collections
- [ ] Collection variables store handles to design objects
- [ ] Use `get_object_name` to access object names
- [ ] Collections can be processed for analysis, reporting and modifications
- [ ] Nested collection processing is useful for hierarchical designs
- [ ] Essential for writing efficient and scalable ICC2 scripts

- Filters help to select specific design objects
- Essential for analysis and automation
- Can filter based on attributes, patterns or logic
- Reduces collection size and improves performance
- Widely used in real-world ICC2 scripts
- Use filters to quickly find the required objects
- Combine multiple conditions using logical operators
- Very useful for timing, physical and DRC data
- Saves time in debugging and analysis
- A must-know feature for productive scripting

8.10 ICC2 Filters

- Filters are used to select a subset of objects from a collection based on specified conditions.
- You can use -filter option with get_* commands or use filter_collection command.
- Filtering can be done using attributes, patterns and logical expressions.
- Filters make scripts more efficient and easy to read.
- Very useful for large designs with millions of objects.

1. -filter Option

- -filter is an option available with most get_* commands in ICC2.
- It selects objects that satisfy the specified condition.
- The condition uses attribute names and values.

Example 1 : Using -filter with get_cells

```
get_cells -filter {is_sequential == true}
get_cells -filter {ref_name =~ "DFF*"}
get_cells -filter {area > 1000}
```

Output :

```
Sequential cells
All DFF cells
Cells with area > 1000
```

2. filter_collection Command

- filter_collection takes an input collection and returns a filtered collection.
- It can be used when you already have a collection.
- More flexible for complex filtering.

Example 2 : Using filter_collection

```
set c1 [get_cells *]
set c2 [filter_collection $c1 \
        {is_combinational == true}]
puts "Combinational cells : [sizeof_collection $c2]"
```

Output :

```
Combinational cells : 5231
```

3. Attribute-based Filtering

- Filters can use object attributes like ref_name, area, is_sequential, num_pins, etc.
- Use ==, !=, >, <, >=, <= for comparison.

Example 3 : Attribute-based filters

```
get_cells -filter {ref_name == "INVX1"}
get_cells -filter {area > 200}
get_nets  -filter {num_pins >= 3}
get_ports -filter {direction == in}
```

Output :

```
All INVX1 cells
Cells with area > 200
Nets with 3 or more pins
Input ports
```

Common Attributes for Filtering

Object Type	Common Attributes
Cell	ref_name, is_sequential, is_combinational, area
Net	num_pins, is_clock, is_reset, is_power
Pin	direction, is_clock, is_data, full_name
Port	direction, is_clock, is_reset
Clock	name, period, is_generated
Shape	layer, area, bbox
Via	via_definition, layer
Instance/Block	name, full_name, is_hard_macro

4. Pattern Filtering

- You can use wildcards (*, ?) and pattern matching operators (=~) in filters.
- Useful to select objects based on name or reference pattern.

Example 4 : Using pattern filters

```
get_cells -filter {ref_name =~ "DFF*"}
get_nets  -filter {name =~ "clk*"}
get_ports -filter {name =~ "rst?"}
```

Output :

```
All DFF cells
Nets starting with clk
Ports like rst0, rst1 ...
```

Pattern Matching Operators

Operator	Meaning
*	Matches zero or more characters
?	Matches exactly one character
=~	Pattern matching (supports * and ?)
!~	Not matching pattern

5. Logical Filtering

- You can combine multiple conditions using logical operators.
- Supported operators: && (AND), || (OR), ! (NOT)
- Use parentheses for complex expressions.

Example 5 : Using logical filters

```
get_cells -filter {is_sequential == true && \
                   ref_name =~ "DFF*"}
get_nets  -filter {is_clock == true ||
                   is_reset == true}
get_pins  -filter {direction == out &&
                   !is_clock}
```

Output :

```
Sequential DFF cells
Clock or reset nets
Output pins (not clock)
```

Logical Operators

Operator	Meaning
&&	Logical AND (both conditions true)
||	Logical OR (any condition true)
!	Logical NOT (condition false)
( )	Parentheses for grouping

6. Practical Examples

- Filters are widely used in reporting, analysis and modifications.
- They help to target only the required objects.

Example 6 : Real-world filter examples

```
get_cells -filter {is_sequential == true}
get_nets  -filter {num_pins > 10}
get_pins  -filter {is_clock == true && direction == in}
get_cells -filter {ref_name =~ "AOI*" && area < 500}
```

Output :

```
Sequential cells
High fanout nets
Clock input pins
Small AOI cells
```

Key Takeaways

- Use -filter or filter_collection to select specific objects
- Filters can be based on attributes, patterns or logical expressions
- Common operators: ==, !=, >, <, >=, <=, =~, &&, ||, !
- Filtering improves efficiency and reduces collection size
- Essential for analysis, debugging and automation in ICC2

Author : Konda Tharun Vivekananda

"Filter smartly, work efficiently !"

8.11 ICC2 Attributes

- Attributes are properties associated with design objects (cells, nets, pins, etc.) in ICC2.
- Use get_attribute to read attribute values.
- Use set_attribute to modify attribute values.
- Attributes provide useful information for analysis, constraints and design modifications.
- Different objects have different attributes.

1. get_attribute

- get_attribute returns the value of an attribute for an object.
- You can query single or multiple objects.
- Useful for reading object properties.

Example 1 : Reading attribute value

```
get_attribute [get_cells U1] ref_name
get_attribute [get_nets n1] net_type
get_attribute [get_pins U1/A] direction
```

Output :

```
INVX1
signal
in
```

Note :
- Attribute names are case-sensitive.
- If attribute is not valid, ICC2 returns an error.
- You can use get_attribute on collections.

2. set_attribute

- set_attribute is used to modify the value of an attribute.
- Useful for changing design properties, constraints, etc.
- Can be applied to single or multiple objects.

Example 2 : Modifying attribute value

```
set_attribute [get_cells U1] dont_touch true
set_attribute [get_nets n1] net_type clock
set_attribute [get_pins U1/A] is_clock true
```

Output :

```
N/A
N/A
N/A
```

Note :
- Not all attributes are writable.
- Check ICC2 documentation for supported attributes.
- Use with caution to avoid unintended changes.

3. Reading Object Properties

- Object properties like name, type, location, size, etc. can be read using get_attribute.
- Useful for reporting, debugging and analysis.

Example 3 : Reading common properties

```
get_attribute [get_cells U1] {ref_name is_hierarchical}
get_attribute [get_nets n1] {net_type total_cap}
get_attribute [get_pins U1/A] {direction full_name}
```

Output :

```
INVX1 false
signal 0.0123
in TOP/U1/A
```

Tip :
- You can query multiple attributes using a list.
- Use get_object_name to print clean object names.

Key Takeaways

- [ ] Use get_attribute to read object properties.
- [ ] Use set_attribute to modify properties.
- [ ] Different objects (cells, nets, pins) have different attributes.
- [ ] Attributes are essential for analysis, debugging and optimization.
- [ ] Always verify changes after modifying attributes.

4. Modifying Object Properties

- You can modify certain attributes to control design behavior.
- Common use cases: setting dont_touch, changing net type, updating placement properties, etc.

Example 4 : Setting useful attributes

```
set_attribute [get_cells U1] dont_touch true
set_attribute [get_cells U1] is_fixed true
set_attribute [get_nets n1] net_type clock
set_attribute [get_pins U1/A] is_clock true
```

Output :

```
N/A
N/A
N/A
N/A
```

Note :
- Use set_attribute only on valid and writable attributes.
- Always verify the change using get_attribute.
- Some attributes may require design updates (e.g. save).

5. Cell Attributes

- Cells have attributes related to their type, size, placement, status, etc.
- Common for optimization, placement and analysis.

Common Cell Attributes

Attribute	Description
ref_name	Reference library cell name (e.g. INVX1)
is_hierarchical	Indicates if cell is a hierarchical block
dont_touch	Prevents optimization or modification
is_fixed	Cell is fixed in placement
location	Cell placement location (x y)
size	Cell size (width height)
area	Cell area
is_physical_only	Physical only cell (no logical function)

6. Net Attributes

- Nets have attributes related to type, load, routing, etc.
- Useful for timing, routing and analysis.

Common Net Attributes

Attribute	Description
net_type	Type of net (signal, clock, power, ground)
total_cap	Total capacitance of the net
total_res	Total resistance of the net
is_clock	Indicates if net is a clock
is_const	Indicates if net is a constant net
fanout	Number of loads on the net
wire_length	Total routed wire length
is_critical	Indicates if net is on a critical path

7. Pin Attributes

- Pins have attributes related to direction, timing, load, etc.
- Useful for connectivity, timing and analysis.

Common Pin Attributes

Attribute	Description
direction	Pin direction (in, out, inout)
full_name	Hierarchical pin name (e.g. TOP/U1/A)
is_clock	Indicates if pin is a clock pin
is_data	Indicates if pin is a data pin
net	Connected net name
capacitance	Pin load capacitance
is_timing	Indicates if pin is used in timing analysis
is_physical_only	Physical only pin

8.12 Important ICC2 Attributes

- ICC2 objects (cells, nets, pins, etc.) have various attributes that describe their properties.
- These attributes can be queried using get_attribute command.
- Important attributes help in design analysis, debugging, filtering and automation.
- Different object types have different attributes.
- Some commonly used attributes are name, full_name, ref_name, is_hierarchical, is_sequential, is_combinational, is_clock_cell and is_physical_only.

1. name

- Returns the simple name of the object (without hierarchy).
- Useful for display and basic identification.

Example 1 : Getting object name

```
set c [get_cells U1]
puts [get_attribute $c name]
```

Output :
```
U1
```

2. full_name

- Returns the hierarchical name of the object.
- Includes the full hierarchy path from the top.
- Useful to uniquely identify an object in hierarchical designs.

Example 2 : Getting full name

```
set c [get_cells U1/A]
puts [get_attribute $c full_name]
```

Output :
```
TOP/U1/A
```

3. ref_name

- Returns the reference library cell name (for cells).
- Indicates which library cell the instance is using.
- Useful to check cell type, size, etc.

Example 3 : Getting reference name

```
set c [get_cells U1]
puts [get_attribute $c ref_name]
```

Output :
```
INVX1
```

4. is_hierarchical

- Indicates whether the cell is a hierarchical (block) or leaf cell.
- Returns true for block instances and false for leaf cells.

Example 4 : Checking hierarchical cell

```
set c [get_cells U1]
puts [get_attribute $c is_hierarchical]
```

Output :
```
true
```



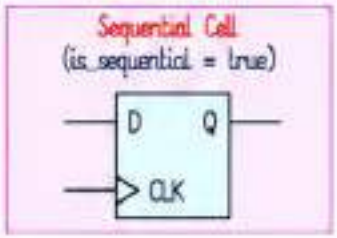
5. is_sequential

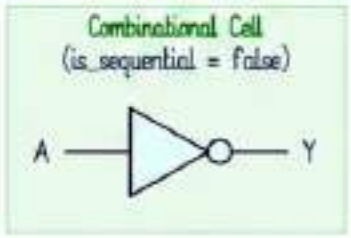
- Indicates whether the cell is a sequential element.
- Returns true for flip-flops, latches, registers, etc.
- Returns false for combinational cells.

Example 5 : Checking sequential cell

```
set c [get_cells U1]
puts [get_attribute $c is_sequential]
```

Output :
```
true
```





6. is_combinational

- Indicates whether the cell is a combinational element.
- Returns true for logic gates, muxes, adders, etc.
- Returns false for sequential cells.

Example 6 : Checking combinational cell

```
set c [get_cells U2]
puts [get_attribute $c is_combinational]
```

Output :
```
true
```

7. is_clock_cell

- Indicates whether the cell is a clock cell (e.g., buffer, inverter) used in clock network.
- Returns true for clock gating/clock buffer/inverter cells.
- Useful to identify clock related cells in the design.

Example 7 : Checking clock cell

```
set c [get_cells CLK_IBUF]
puts [get_attribute $c is_clock_cell]
```

Output :
```
true
```



8. is_physical_only

- Indicates whether the cell is a physical-only cell.
- Returns true for filler cells, end caps, tap cells, decaps, etc.
- Returns false for functional cells.

Example 8 : Checking physical-only cell

```
set c [get_cells FILL1]
puts [get_attribute $c is_physical_only]
```

Output :
```
true
```

Key Takeaways

- Use get_attribute to query important object properties.
- Know commonly used attributes for cells, nets and pins.
- Attributes help in filtering, analysis, debugging and automation.
- Combine attributes with -filter or filter_collection.
- Understand the difference between hierarchical, sequential, combinational, clock and physical-only cells.

- Attributes give useful information about design objects
- Help in querying, filtering and automation
- Essential for analysis, debugging and design modifications
- Different objects (cells, nets, pins, etc.) have different attributes
- Learn commonly used attributes and their meaning

TCL for Complete ICC2 Physical Design

9. Important ICC2 Attributes

- Use get_attribute and set_attribute
- Attributes can be used in filters and reports
- Helps in large block and hierarchical designs
- Very useful in scripting and automation
- A must-know topic for ICC2 users

9.1 Important ICC2 Attributes

- ICC2 design objects (cells, nets, pins, etc.) have many attributes that describe their properties.
- Attributes can be queried using get_attribute and modified using set_attribute (where applicable).
- These attributes help in design analysis, debugging, filtering and automation.
- Some of the most commonly used attributes are: name, full_name, ref_name, is_hierarchical, is_sequential, is_combinational, is_clock_cell and is_physical_only

1. name

- Returns the simple name of the object (without hierarchy).
- Useful for basic identification and display.

Example 1 : Getting object name

```
set c [get_cells U1]
puts [get_attribute $c name]
```

Output :
U1

2. full_name

- Returns the full hierarchical name of the object.
- Includes the entire hierarchy path from the top.
- Useful for uniquely identifying an object in hierarchical designs.

Example 2 : Getting full hierarchical name

```
set c [get_cells U1/A]
puts [get_attribute $c full_name]
```

Output :
TOP/U1/A

3. ref_name

- Returns the reference library cell name (for cells).
- Indicates which library cell the instance is using.
- Useful to check cell type, size, drive strength, etc.

Example 3 : Getting reference name

```
set c [get_cells U1]
puts [get_attribute $c ref_name]
```

Output :
INVX1

Note : ref_name is mainly applicable to cells. For nets and ports, other attributes are used (e.g., net_type, direction).

4. is_hierarchical

- Indicates whether the cell is a hierarchical (block) or leaf cell.
- Returns true for block instances and false for leaf cells.

Example 4 : Checking hierarchical cell

```
set c [get_cells U1]
puts [get_attribute $c is_hierarchical]
```

Output :
true



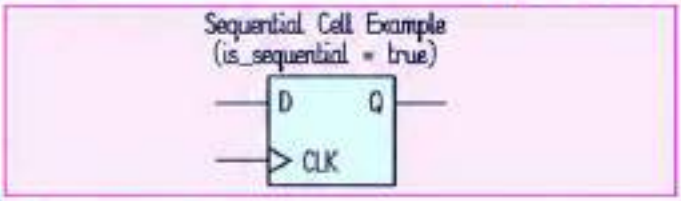
5. is_sequential

- Indicates whether the cell is a sequential element (flip-flop, latch, register, etc.).
- Returns true for sequential cells and false for others.
- Useful for separating sequential and combinational logic.

Example 5 : Checking sequential cell

```
set c [get_cells UFF1]
puts [get_attribute $c is_sequential]
```

Output :
true



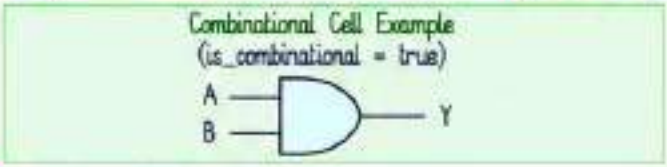
6. is_combinational

- Indicates whether the cell is a combinational element (gate, mux, adder, etc.).
- Returns true for combinational cells and false for sequential cells.
- Useful for logic analysis and filtering.

Example 6 : Checking combinational cell

```
set c [get_cells U2]
puts [get_attribute $c is_combinational]
```

Output :
true



7. is_clock_cell

- Indicates whether the cell is a clock cell (e.g., buffer, inverter, clock gate, mux used in clock network).
- Returns true for clock cells and false for others.
- Useful to identify clock related cells in the design.

Example 7 : Checking clock cell

```
set c [get_cells CLKBUF1]
puts [get_attribute $c is_clock_cell]
```

Output :
true

8. is_physical_only

- Indicates whether the cell is a physical-only cell (filler, tap, end cap, decap, etc.).
- Returns true for physical-only cells and false for functional cells.
- Useful for placement, filler cell handling and signoff checks.

Example 8 : Checking physical-only cell

```
set c [get_cells FILL1]
puts [get_attribute $c is_physical_only]
```

Output :
true

Summary of Important ICC2 Attributes

Attribute	Applicable To	Description
name	All objects	Simple name (no hierarchy)
full_name	All objects	Full hierarchical name
ref_name	Cells	Reference library cell name
is_hierarchical	Cells	True if hierarchical (block) cell
is_sequential	Cells	True if sequential cell
is_combinational	Cells	True if combinational cell
is_clock_cell	Cells	True if clock cell
is_physical_only	Cells	True if physical-only cell (filler, tap, etc.)

Key Takeaways

- Use get_attribute to query object properties.
- Important attributes help in filtering, analysis and automation.
- Different attributes are available for different object types.
- Use these attributes with filters for efficient queries.
- Essential for debugging, design analysis and complete ICC2 physical design flow.

Author : Konda Tharun Vivekananda

"Know your design, query with attributes, build better chips!"

10.1 Technology & Library Setup

- Technology and library setup defines the design environment in ICC2.
- Technology file provides process information, design rules, layers, and manufacturing constraints.
- Reference libraries provide standard cells, macros, and I/O cells.
- NDM (Native Design Model) libraries are the recommended format for ICC2.
- Search paths are used to locate technology and library files.
- Always perform library checks to ensure consistency and completeness.

1. Technology Files

- Technology file (.tf) contains process technology information.
- Includes layer definitions, design rules, via definitions, etc.
- Provided by the foundry (e.g., .tf or .tch format).
- Mandatory for ICC2 to understand the fabrication process.

Example 1 : Setting technology file

```
set_technology [list /path/to/tech/tech.tf]
```

2. Reference Libraries

- Reference libraries contain pre-compiled cell information used for design implementation.
- Include standard cells, I/O cells, and macros.
- Can be in Milkyway (.db) or NDM (.ndm) format.
- Specified using set_reference_libs command.

Example 2 : Setting reference libraries

```
set_reference_libs [list \
  /path/to/lib/tsmc_std.ndm \
  /path/to/lib/tsmc_io.ndm]
```

Common Reference Libraries

Library Type	Purpose	Example
Standard Cell	Logic implementation	stdcell.ndm
I/O Library	Pad and ESD cells	io.ndm
Memory Compiler	SRAM/ROM/Custom memories	mem.ndm
Analog/PHY	Analog, PLL, SerDes	phy.ndm
Macro	Hard macros / IP blocks	macro.ndm

3. NDM Libraries

- NDM (Native Design Model) is the recommended library format for ICC2.
- Provides better performance and lower memory usage.
- Contains all necessary views (logical, physical, timing, etc.).
- Use NDM instead of older Milkyway format whenever possible.

Example 3 : Using NDM libraries

```
set_reference_libs [list /libs/stdcell.ndm]
set link_library [list * /libs/stdcell.ndm]
```

Note :
- NDM libraries are created using library compiler.
- Ensure you have the correct NDM version for your ICC2 release.
- Check that all required views are available in the NDM.

4. Search Paths

- Search paths tell ICC2 where to look for technology files, libraries and other design files.
- Use set_search_path to specify directories.
- Can include multiple paths (separated by space).
- Helps avoid using full paths in commands.

Example 4 : Setting search path

```
set_search_path [list \
  ./tech \
  ./libs \
  ./rtl \
  ./scripts]
```

Example Directory Structure

```
project/
├── tech/
│   └── tech.tf
├── libs/
│   ├── stdcell.ndm
│   └── io.ndm
├── rtl/
│   └── design.v
└── scripts/
    └── setup.tcl
```

5. Library Checks

- Check that libraries are properly loaded and accessible.
- Verify required views (timing, physical, etc.) are available.
- Use check_library and report commands to validate.
- Helps catch issues early in the flow.

Example 5 : Checking libraries

```
check_library
report_lib
report_lib -verbose
```

Sample Output (report_lib)

```
Library  : stdcell
Type     : NDM
Cells    : 1024
Timing   : Available
Physical : Available
Status   : OK
```

6. Reference-Library Checks

- Verify that all required reference libraries are correctly loaded.
- Check for missing cells, views, or inconsistencies.
- Use report_reference_libs and check_design -lib.
- Ensure that the libraries match the technology and PDK version.

Example 6 : Checking reference libraries

```
report_reference_libs
check_design -lib
report_design -lib
```

Sample Output

```
Reference Libraries:
  stdcell.ndm   (OK)
  io.ndm        (OK)
  mem.ndm       (OK)
No issues found.
```

Key Checks

Check	Purpose
check_library	Validates library integrity
report_lib	Shows library details
report_reference_libs	Lists all loaded reference libraries
check_design -lib	Checks for missing cells or views
report_missing_lib_cells	Reports any missing cells
report_design -lib	Overall library consistency report

Key Takeaways

- [ ] Set the correct technology file provided by the foundry.
- [ ] Use NDM reference libraries for better performance.
- [ ] Configure search paths to locate all required files.
- [ ] Always perform library checks before starting the design flow.
- [ ] Verify that all required views are available in the reference libraries.
- [ ] Reference-library checks help avoid late-stage errors.

Author : Konda Tharun Vivekananda

"Setup right, design fast !"

11.1 MMMC & Timing Setup Overview

- MMMC (Multi-Mode Multi-Corner) defines the different operating modes and process-voltage-temperature (PVT) corners for timing analysis.
- In ICC2, we create modes, corners and combine them into scenarios.
- MMMC setup is required for accurate timing analysis, signoff and optimization.
- Each scenario represents a unique mode-corner combination with specific constraint and library settings.
- Use the following commands to create, manage and verify timing setup in ICC2.

1. create_mode

- Creates a user-defined mode (functional or test).
- A mode typically corresponds to a specific set of constraints (SDC).
- Multiple modes can be defined for different operating conditions.

Example 1 : Creating modes

```
create_mode -name FUNC -sdc_files {func.sdc}
create_mode -name TEST -sdc_files {test.sdc}
```

2. create_corner

- Creates a PVT corner using technology and library information.
- Each corner represents a specific combination of process, voltage and temperature.
- Specify the appropriate lib files for the corner.

Example 2 : Creating corners

```
create_corner -name TT -lib_files {tt.lib} -rc_corner rc_tt
create_corner -name SS -lib_files {ss.lib} -rc_corner rc_ss
create_corner -name FF -lib_files {ff.lib} -rc_corner rc_ff
```

Common Corner Types

Corner	Process	Voltage	Temperature	Usage
TT	Typical	Nominal	25°C	Functional
SS	Slow	Low		Worst case (max delay)
FF	Fast	High		Best case (min delay)
SF	Slow	High		Setup corners
FS	Fast	Low		Hold corners

3. create_scenario

- Combines a mode and a corner to create a scenario.
- A scenario inherits the constraints from the mode and the libraries from the corner.
- Multiple scenarios are created to cover all mode-corner combinations.

Example 3 : Creating scenarios

```
create_scenario -name FUNC_TT -mode FUNC -corner TT
create_scenario -name FUNC_SS -mode FUNC -corner SS
create_scenario -name TEST_FF -mode TEST -corner FF
```

4. set_scenario_status

- Enables or disables a scenario for analysis.
- Useful to exclude unnecessary scenarios during a particular run.

Example 4 : Enabling / Disabling scenarios

```
set_scenario_status FUNC_TT -status true
set_scenario_status TEST_FF -status false
```

Status Values	
true	Enable scenario
false	Disable scenario

5. report_scenarios

- Reports all defined scenarios and their status.
- Shows the associated mode, corner and constraint files.

Example 5 : Reporting scenarios

```
report_scenarios
```

Sample Output :

Scenario	Mode	Corner	Status
FUNC_TT	FUNC	TT	true
FUNC_SS	FUNC	SS	true
TEST_FF	TEST	FF	false

6. report_corners

- Reports all defined corners and their library/RC information.
- Useful to verify that the correct libraries are loaded.

Example 6 : Reporting corners

```
report_corners
```

Sample Output :

Corner	Lib Files	RC Corner	Status
TT	tt.lib	rc_tt	OK
SS	ss.lib	rc_ss	OK
FF	ff.lib	rc_ff	OK

7. check_timing

- Checks the timing setup for the defined scenarios.
- Verifies that modes, corners, libraries and constraints are consistent and ready for analysis.
- Reports any issues before running full timing analysis.

Example 7 : Checking timing setup

```
check_timing
```

Sample Output :

```
Checking timing setup ...
Info    : All scenarios are valid.
Info    : No missing libraries.
Info    : No constraint issues found.
Info    : Timing setup is ready.
```

Key Takeaways

- [ ] Use create_mode to define functional and test modes.
- [ ] Use create_corner to specify PVT corners with correct libraries.
- [ ] Use create_scenario to combine mode and corner.
- [ ] Use set_scenario_status to enable/disable scenarios.
- [ ] Use report_scenarios and report_corners to verify setup.
- [ ] Always run check_timing to validate the MMMC setup.
- [ ] A correct MMMC setup is the foundation for accurate timing analysis and signoff.

12.1 Floorplan Automation Overview

- Floorplan defines the die, core, IO, macros and keepout regions
- ICC2 provides TCL commands to automate floorplan creation
- initialize_floorplan is a high-level command that sets die/core area, aspect ratio, utilization, etc.
- We can also explicitly set die area and core area.
- Placement blockages, routing blockages and keepouts help control placement and routing.
- A good floorplan reduces congestion and improves timing closure.

1. initialize_floorplan

- High-level command to automatically create die and core area
- Takes aspect ratio and target utilization as inputs
- Creates a square/rectangular core area with default margins

Example 1 : Basic floorplan initialization

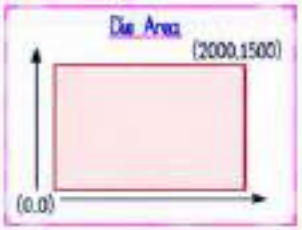
```
initialize_floorplan -aspect_ratio 1.0 \
                     -core_utilization 0.7 \
                     -padding 50
```

2. Die area

- Die area is the total chip area (including IO pads).
- Defines the chip boundary
- Can be set manually or by initialize_floorplan.

Example 2 : Setting die area explicitly

```
set die_ll {0 0}
set die_ur {2000 1500}
create_die_area $die_ll $die_ur
```



3. Core area

- Core area is the region where standard cells are placed.
- It is inside the die area, leaving space for IO and margins.
- Can be set automatically or manually

Example 3 : Setting core area explicitly

```
set core_ll {100 100}
set core_ur {1900 1400}
create_core_area $core_ll $core_ur
```



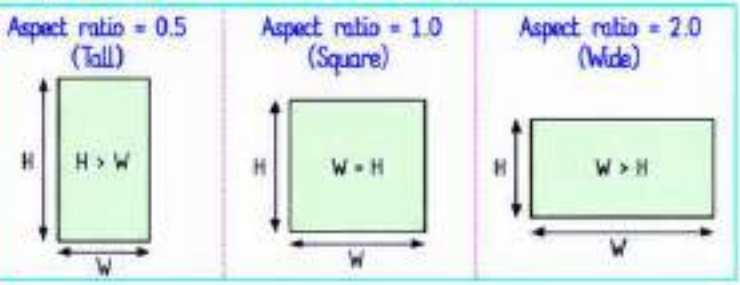
Key Takeaways

- [ ] Use initialize_floorplan for quick floorplan setup.
- [ ] Define die and core area based on design requirements.
- [ ] Choose appropriate aspect ratio and target utilization.
- [ ] Use placement blockages, routing blockages and keepouts to guide P&R.
- [ ] A well-planned floorplan is critical for successful implementation.

4. Aspect ratio

- Aspect ratio = Width / Height of the core area.
- Common values: 0.5 (tall), 1.0 (square), 2.0 (wide).
- Choose based on design shape, IO distribution and floorplan constraints.

Example 4 : Different aspect ratios



5. Utilization

- Core utilization = (Total cell area) / (Core area).
- Typical range: 0.6 - 0.75 (60% - 75%).
- Higher utilization → smaller area but more congestion.
- Lower utilization → better routability but larger area.

Example 5 : Setting target utilization

```
initialize_floorplan -aspect_ratio 1.0 -core_utilization 0.7
```

Note : You can change utilization later using:

```
set_fp_placement_strategy -core_utilization 0.65
```

6. Placement blockages

- Placement blockages prevent cells from being placed in a region.
- Useful for macros, analog blocks or reserved areas.
- Can be hard or soft blockages.

Example 6 : Creating placement blockage

```
create_placement_blockage -name PB1 \
                          -rect {500 500 800 800} \
                          -type hard
```



7. Routing blockages

- Routing blockages prevent routing in a region.
- Useful to reserve area for future use or to protect sensitive blocks.
- Can be applied on specific metal layers.

Example 7 : Creating routing blockage

```
create_routing_blockage -name RB1 \
                        -rect {1000 400 1300 700} \
                        -layers {M1 M2 M3}
```

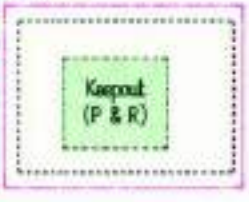


8. Keepouts

- Keepouts are general regions to restrict placement, routing or both.
- Useful for IP keepouts, antenna fixes, or future expansion.
- Can specify type: placement, routing or both.

Example 8 : Creating keepout region

```
create_keepout -name KO1 \
               -rect {300 300 600 600} \
               -type both
```



- Macros are pre-designed blocks (IP / hard macros)
- Macro planning is critical for PPA and routability
- Use TCL to automate identification, placement and checks
- Maintain proper halos, channels and blockages
- Well planned macro floorplan reduces iterations

TCL for Complete ICC2 Physical Design

13. Macro Planning Automation

- Ensure enough space for routing and power
- Use groups for logical macro placement
- Run placement checks to validate setup
- Avoid congestion around macros
- Essential for SoC designs with multiple IPs
- A must-know topic for ICC2 users

13.1 Macro Planning Automation Overview

- Macros (IP blocks) such as SRAM, CPU, DSP, PLL, IO, etc., are pre-designed and delivered as hard macros.
- Macro planning decides where to place macros and how to create space around them for routing, power and other cells.
- ICC2 provides TCL commands to automate macro identification, placement, grouping, halos, channels and blockages.
- Proper macro planning improves timing, routability and overall design quality.
- Macro placement checks help validate the floorplan setup.

1. Macro identification

- Identify all macro cells in the design.
- Macros are usually hard macros (is_hierarchical = true).
- Filter based on attributes like ref_name, is_physical_only, etc.

Example 1 : Getting list of macros

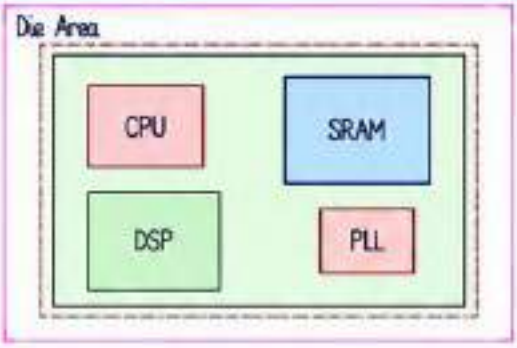
```
set macros [get_cells -hier -filter {is_hierarchical == true}]
puts "Number of macros: [sizeof_collection $macros]"
get_object_name $macros
```

2. Macro placement

- Place macros at desired locations (absolute or relative).
- Use create_placement or set_location commands.
- Can automate placement from a file or predefined coordinates.

Example 2 : Placing a macro at a location

```
set mem [get_cells U_SRAM]
create_placement -coordinate {500 800} -cell $mem
# or
set_location {500 800} $mem
```



3. Macro grouping

- Group related macros (e.g., memory macros) for easy handling.
- Useful for block-level floorplanning and constraints.
- Can apply common settings (halos, blockages) to the group.

Example 3 : Creating a macro group

```
set mems [get_cells -hier -filter {ref_name == "*SRAM*"}]
group_objects -name MEM_GRP -objects $mems
set_attribute is_guide_only false [get_groups MEM_GRP]
```

Key Takeaways

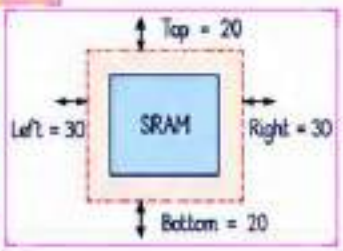
- Identify all macros and understand their type and size
- Automate macro placement, halos, channels and blockages using TCL
- Use groups to manage multiple macros efficiently
- Run placement checks to validate the macro floorplan
- Proper macro planning is key for routability, timing and overall PPA

4. Macro halos

- Halo creates keepout space around macros for standard cells.
- Helps avoid congestion and improves routability.
- Can specify different halo sizes for each side.

Example 4 : Setting macro halo

```
set mem [get_cells U_SRAM]
set_attribute halo_top 20    $mem
set_attribute halo_bottom 20 $mem
set_attribute halo_left  30  $mem
set_attribute halo_right 30  $mem
```



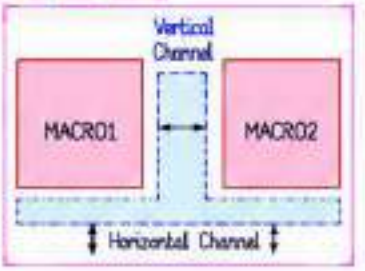
5. Macro channels

- Channels are spaces between macros for routing (signals, power).
- Vertical and horizontal channels help reduce congestion.
- Can be created using placement blockages or create_keepout.

Example 5 : Creating channel using keepout

```
create_keepout -name CH1 \
    -type placement \
    -coordinate {800 0 820 1500}

create_keepout -name CH2 \
    -type placement \
    -coordinate {0 700 2000 720}
```



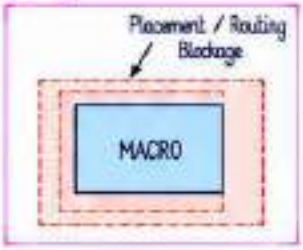
6. Macro blockages

- Blockages prevent standard cell or routing placement in a region.
- Useful to reserve area around macros or for future expansion.
- Can create placement, routing or both types of blockages.

Example 6 : Creating macro blockage

```
create_placement_blockage -name PB1 \
    -coordinate {450 750 550 900} \

create_routing_blockage -name RB1 \
    -coordinate {450 750 550 900} \
    -layers {M1 M2 M3 M4}
```



7. Macro placement checks

- Run checks to verify macro placement, halos, blockages and spacing.
- Ensure no overlap and sufficient routing channels.
- Helps catch issues early in the design flow.

Example 7 : Running macro placement checks

```
check_placement
check_blockages
report_placement -macro
report_macros
report_areas
check_design -checks {macro_placement}
```

Sample Check Output

Check	Status
Macro Overlap	OK
Halo Spacing	OK
Blockage Violations	0
Channel Width	OK
Macro Placement	OK

Author : Konda Tharun Vivekananda

"Place macros smartly, route freely !"

14.1 I/O & Pin Planning Overview

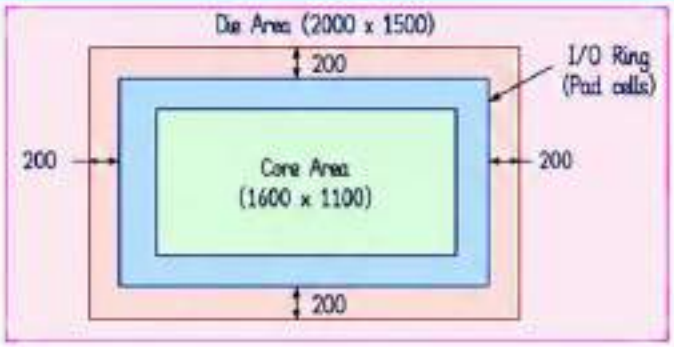
- I/O and pin planning defines the external interface of the chip
- Includes I/O ring, I/O guides, pin guides, port placement and pin constraints.
- Proper pin planning ensures good routability, signal integrity and manufacturability.
- ICC2 provides TCL commands to automate and control pin planning.
- Pin-placement checks help validate the final setup.

1. I/O ring

- I/O ring is the peripheral region between core and die area.
- Contains I/O cells, pads and bump cells (in advanced nodes).
- Defined by die area and core area, with margins.
- Provides space for I/O placement and routing.

Example 1 : Setting up I/O ring

```
initialize_floorplan -die_area {0 0 2000 1500} \
                     -core_area {200 200 1800 1300} \
                     -io_ring
```



2. I/O guides

- I/O guides define the regions on each side of the chip where pins can be placed.
- Helps in evenly distributing pins and avoiding congestion.
- Guides can be defined for top, bottom, left and right.

Example 2 : Creating I/O guides

```
create_io_guide -side top    -start 200 -end 1800 -layer M4
create_io_guide -side bottom -start 200 -end 1800 -layer M4
create_io_guide -side left   -start 200 -end 1300 -layer M4
create_io_guide -side right  -start 200 -end 1300 -layer M4
```



Key Takeaways

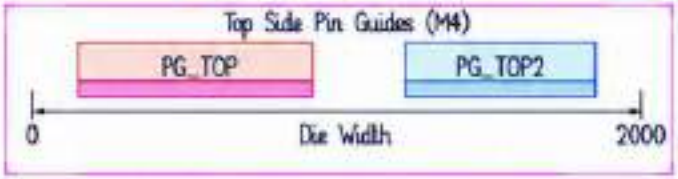
- Use I/O guides and pin guides for structured pin planning.
- Keep critical signals away from congested regions.
- Apply pin constraints for clocks, resets and high-speed signals.
- Always run pin-placement checks to validate the setup.
- A well-planned I/O interface improves routability and signoff quality.

3. Pin guides

- Pin guides specify detailed locations and layers for pins.
- More granular than I/O guides.
- Useful for critical signals, power/ground or specific interfaces.
- Can define multiple guides per side.

Example 3 : Creating pin guides

```
create_pin_guide -name PG_TOP -side top \
                 -start 200 -end 800 -layer M4 -width 2
create_pin_guide -name PG_BOT -side bottom \
                 -start 1000 -end 1800 -layer M4 -width 2
```



4. Port placement

- Ports are logical interface points in the design (input/output/inout).
- Ports need to be placed on the top level boundary.
- Can be placed automatically or manually using guides.

Example 4 : Placing ports

```
create_port IO_IN  -direction in
create_port IO_OUT -direction out
set_port_location IO_IN  -coordinate {100 1500}
set_port_location IO_OUT -coordinate {1900 300}
```

5. Pin placement

- Pin placement assigns the physical location to ports/pins.
- Can be done based on I/O guides, pin guides or manual coordinates.
- Supports automatic pin assignment using place_pins.

Example 5 : Placing pins using automatic mode

```
place_pins -ports [IO_*] -io_guide
```

6. Pin constraints

- Pin constraints control location, layer, spacing and assignment rules.
- Useful for critical pins (clocks, resets, high-speed signals).
- Ensures compliance with design and package rules.

Example 6 : Setting pin constraints

```
set_pin_constraint -ports {CLK} -side top -layer M4
set_pin_constraint -ports {RST} -side left -layer M5
set_pin_spacing -min_distance 10 -ports {CLK RST}
```

7. Pin-placement checks

- Run checks to verify pin placement, spacing and rule compliance.
- Detects overlaps, violations and unplaced ports.
- Helps ensure a clean interface for signoff.

Example 7 : Running pin placement checks

```
check_pin_placement
report_pin_placement
report_io_guide
report_pin_guide
```

Sample Check Output

Check	Value	Status
Unplaced Ports	0	OK
Pin Overlap	0	OK
Spacing Violations	0	OK
Guide Violations	0	OK

Author : Konda Tharun Vivekananda

"Right pins, brighter connections, better chips !"

TCL for Complete ICC2 Physical Design

15. Multi-Voltage / UPF Automation

- Multi-voltage design is essential for low power
- UPF (Unified Power Format) defines power intent
- ICC2 supports UPF for automated implementation
- Power domains, supply sets and voltage areas control power
- Level shifters and isolation cells ensure safe operation
- Use load_upf to load the UPF file
- Define and verify power intent in ICC2
- Automatically insert level shifters/isolation
- Check power domain connections and supply nets
- Ensure correct power-state management
- A must-know topic for ICC2 users

15.1 Multi-Voltage / UPF Automation Overview

- UPF (Unified Power Format) describes the power intent of a design.
- It defines power domains, supply nets, supply sets, voltage areas and low-power cells (level shifters, isolation, etc.).
- ICC2 uses UPF to automatically implement multi-voltage design.
- Helps in power-aware placement, routing and signoff.
- Correct UPF setup ensures proper voltage connectivity and functional correctness.

1. load_upf

- Loads the UPF file into ICC2.
- Parses power intent and creates power domains, supply sets, etc.
- Should be loaded early in the flow (after libraries).
- Use check_upf to verify the UPF.

Example 1 : Loading UPF file

```
load_upf ./design.upf
check_upf
report_upf
```

2. Power domains

- Power domain is a collection of logic that operates at a specific voltage supply.
- Each block/module can be assigned to a power domain.
- Domains define isolation, level shifting and power control.

Example 2 : Creating and reporting power domains

```
create_power_domain PD_A
create_power_domain PD_B
report_power_domains
```



3. Supply nets

- Supply nets carry the power (VDD) and ground (VSS) for domains.
- They are connected to the design power pins.
- Can be specified in UPF or manually in ICC2.
- ICC2 uses supply nets for power-aware checks and placement.

Example 3 : Defining supply nets

```
create_supply_net VDD_A
create_supply_net VSS_A
connect_supply_net VDD_A -ports {VDD}
connect_supply_net VSS_A -ports {VSS}
```

Key Takeaways

- [ ] Use load_upf to load and verify the UPF file.
- [ ] Define power domains, supply nets and supply sets correctly.
- [ ] Use voltage areas for physical separation of domains.
- [ ] Ensure level shifters and isolation cells are inserted.
- [ ] Verify power states and run UPF checks for a robust multi-voltage design.

4. Supply sets

- Supply set is a group of related supply nets (power and ground).
- Each power domain is associated with a supply set.
- Helps in defining voltage for a domain.

Example 4 : Creating supply sets

```
create_supply_set SS_A \
    -function {power ground} \
    -power VDD_A \
    -ground VSS_A

set_domain_supply_set PD_A -supply_set SS_A
```

Supply Set Example

Supply Set	Power Net	Ground Net
SS_A	VDD_A	VSS_A
SS_B	VDD_B	VSS_B
SS_C	VDD_C	VSS_C

5. Voltage areas

- Voltage area specifies a physical region with a specific power domain.
- Helps in physical separation of different voltage domains.
- Used during floorplanning and placement.

Example 5 : Creating voltage areas

```
create_voltage_area VA_B \
    -power_domain PD_B \
    -box {1000 500 2000 1500}
```

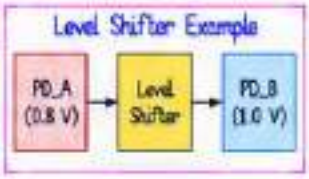


6. Level shifters

- Level shifters are inserted between different voltage domains.
- Convert signals from lower voltage to higher voltage (and vice versa).
- ICC2 can automatically insert level shifters based on UPF.
- Use reports to check level shifter insertion.

Example 6 : Reporting level shifters

```
report_level_shifters
report_cells -filter {is_level_shifter == true}
```

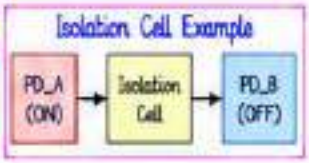


7. Isolation cells

- Isolation cells prevent floating outputs when a domain is powered down.
- Inserted at domain boundaries based on UPF rules.
- Can be automatically inserted by ICC2.

Example 7 : Reporting isolation cells

```
report_isolation_cells
report_cells -filter {is_isolation_cell == true}
```



8. Power-state management

- Defines different power states (ON, OFF, retention, etc.).
- Specifies control signals, power switches and state transitions.
- Used for multi-mode low power verification and implementation.

Example 8 : Reporting power states

```
report_power_states
report_power_state_table
```

Power State Example

State	PD_A	PD_B	PD_C
S0 (Active)	ON	ON	ON
S1 (Low Power)	ON	OFF	ON
S2 (Deep Sleep)	OFF	OFF	ON

Author : Konda Tharun Vivekananda

"Power aware design today, efficient chips tomorrow !"

TCL for Complete ICC2 Physical Design

16. Power Planning Automation

- Proper power planning ensures reliable power delivery
- Use ICC2 Tcl commands to automate PG creation
- Define PG rings, straps, mesh and standard-cell rails
- Ensure sufficient PG vias and connectivity
- Run PG verification to check for opens, shorts and coverage
- A good power plan improves IR drop, EM and overall chip reliability
- Plan PG early in the flow
- Use technology recommended rules
- Ensure proper PG connectivity to all power domains
- Check PG with built-in ICC2 verification commands
- Fix violations before placement and routing
- A must-know topic for ICC2 users

16.1 Power Planning Automation Overview

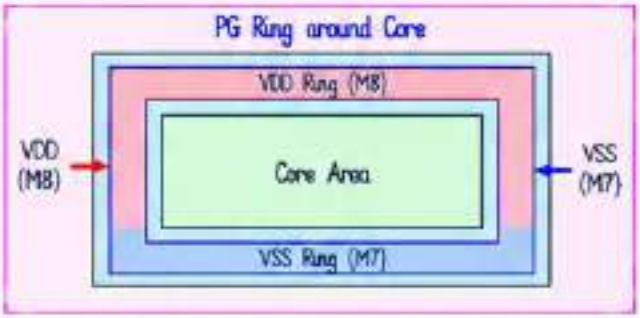
- Power planning creates the power distribution network (PDN) for the design.
- Includes PG rings, straps, mesh, standard-cell rails and PG vias.
- Ensures low IR drop, good reliability and uniform power delivery.
- ICC2 provides Tcl commands to automate and customize PG creation.
- PG connectivity and verification are critical before placement.
- A well-planned power network is essential for timing, signal integrity and signoff.

1. PG rings

- PG rings surround the core area and provide a low resistance path.
- Typically implemented on top metal layers (e.g. M7/M8).
- Connects to pads, bumps or top-level power.
- Encloses the core and can be configured for multiple nets.

Example 1 — Creating PG ring

```
create_pg_ring \
    -nets {VDD VSS} \
    -layers {top M8 bottom M7 left M8 right M7} \
    -width 10 -spacing 5 -offset 10
```

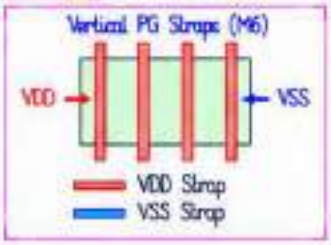


2. PG straps

- PG straps are wide wires that distribute power across the core.
- Usually run in vertical and horizontal directions.
- Implemented on multiple metal layers (e.g. M4, M5, M6).
- Helps in connecting rings to standard-cell rails and macro blocks.

Example 2 — Creating PG straps

```
create_pg_straps \
    -nets {VDD VSS} \
    -layers {M6 M5} \
    -direction vertical \
    -width 4 -spacing 4 -start_offset 10
```

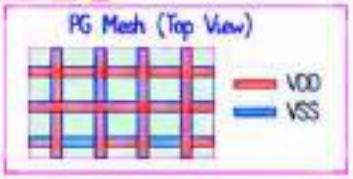


3. PG mesh

- PG mesh is a grid of straps in multiple layers (horizontal + vertical).
- Provides better power distribution and lower IR drop.
- Used in high-performance and advanced nodes.

Example 3 — Creating PG mesh

```
create_pg_mesh \
    -nets {VDD VSS} \
    -layers {M4 M5} \
    -pitch {10 10} -width {2 2}
```

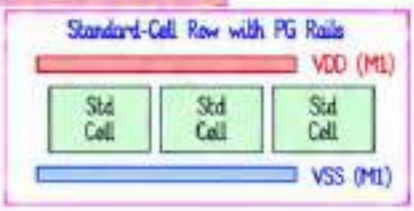


4. Standard-cell rails

- Standard-cell rails are power rails in the standard-cell rows.
- Automatically created during row creation or explicitly using PG commands.
- Connects the standard cells to the global PG network.
- Ensures proper connection to straps/mesh.

Example 4 — Creating standard-cell rails

```
create_pg_std_cell_rail \
    -nets {VDD VSS} \
    -layer M1 \
    -direction horizontal
```

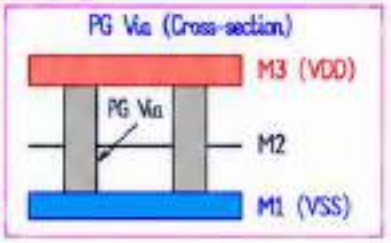


5. PG vias

- PG vias connect PG nets between different metal layers.
- Ensure low resistance and reliable power connectivity.
- Can be created automatically or explicitly.
- Important for connecting rings, straps, rails and mesh.

Example 5 — Creating PG vias

```
create_pg_vias \
    -nets {VDD VSS} \
    -from_layers {M1 M2} \
    -to_layers   {M2 M3} \
    -via_rule default
```



6. PG connectivity

- Ensure all power components (rings, straps, rails, macros) are connected.
- Check for floating PG nets or disconnected regions.
- Use built-in ICC2 commands to verify connectivity.
- Fix missing connections by adjusting straps, vias or blockages.

Example 6 — Checking PG connectivity

```
check_pg_connectivity
report_pg_connectivity
report_power_net -verbose
```

Sample Report Output

Power Net	Status	No. of Objects
VDD	OK	1256
VSS	OK	1248

7. PG verification

- Run PG verification to check for opens, shorts, unconnected rails.
- Verify PG coverage, via count, spacing and design rule compliance.
- Use ICC2 built-in checks and reports.
- Resolve all violations before placement and routing.

Example 7 — PG verification commands

```
check_pg_drc
check_pg_connectivity
report_pg_drc -verbose
report_power_grid
report_power_net -voltage
```

Sample DRC Report

Check	Violations	Status
PG Open	0	OK
PG Short	0	OK
Via Spacing	0	OK
PG Coverage	0	OK

Key Takeaways

- [ ] Use ICC2 Tcl commands to automate complete power planning.
- [ ] Create PG rings, straps, mesh and standard-cell rails.
- [ ] Ensure sufficient PG vias and proper connectivity.
- [ ] Run PG verification and fix all violations.
- [ ] A robust power plan is critical for a successful physical design.

Author : Konda Tharun Vivekananda

"Strong power, stronger designs !"

TCL for Complete ICC2 Physical Design

17. Physical-Only Cell Automation

- Physical-only cells are used for manufacturing requirements
- They do not affect logic functionality
- ICC2 provides Tcl commands to automatically insert and manage tap, endcap, boundary, filler and decap cells
- Use physical-only cells to ensure DRC/LVS clean design
- Run checks to verify correct insertion and spacing
- Ensure well taps for latch-up prevention
- Insert endcaps at row boundaries
- Use filler cells to fill placement gaps
- Use decap cells to improve IR drop and stability
- Run physical-only cell checks after insertion
- A must-know topic for ICC2 users

17.1 Physical-Only Cell Automation Overview

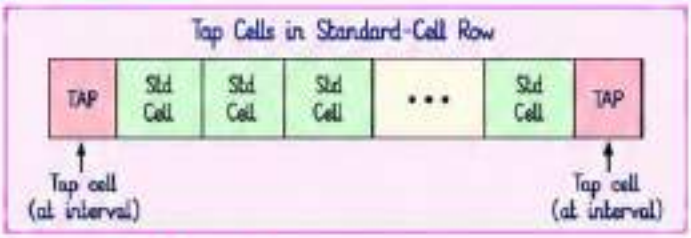
- Physical-only cells are non-functional cells used to meet manufacturing, reliability and physical design requirements
- Includes tap cells, endcap cells, boundary cells, filler cells and decap cells
- ICC2 provides Tcl commands to automate insertion, removal and verification of physical-only cells
- Helps in DRC/LVS clean design, prevents latch-up, improves IR drop and ensures proper row termination
- Run checks to verify correct insertion, spacing and coverage

1. Tap cells

- Tap cells connect the well (N-well / P-well) to supply (VDD/VSS)
- Inserted periodically in standard-cell rows
- Prevent latch-up and ensure substrate/well biasing

Example 1 : Inserting tap cells

```
add_tap_cells \
    -cell TAPCELL_X1 \
    -distance 20 \
    -in_rows \
    -prefix TAP
```



2. Endcap cells

- Endcap cells are inserted at the start and end of each row
- Provide well continuity and edge termination
- Different endcap cells for left and right ends

Example 2 : Inserting endcap cells

```
add_endcaps \
    -left_endcap_cell ENDCAP_L_X1 \
    -right_endcap_cell ENDCAP_R_X1 \
    -in_rows
```

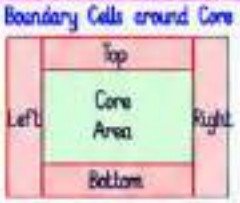


3. Boundary cells

- Boundary cells are placed around the core boundary
- Used to provide well and metal density at the edges
- Helps in DRC compliance and avoids edge effects

Example 3 : Inserting boundary cells

```
add_boundary_cells \
    -top_cell BOUND_TOP_X1\
    -bottom_cell BOUND_BOTTOM_X1\
    -left_cell BOUND_LEFT_X1\
    -right_cell BOUND_RIGHT_X1
```

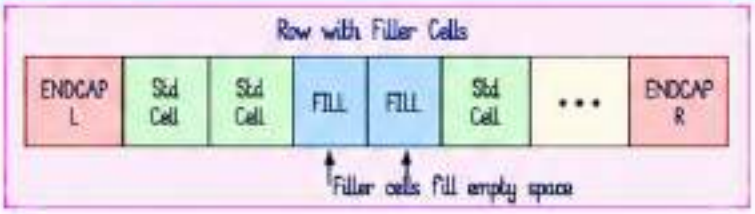


4. Filler cells

- Filler cells are inserted to fill the empty spaces in rows
- Ensure continuous well and avoid implant density issues
- Do not have any logic function
- Different sizes (1x, 2x, 4x, etc.) are used to fill gaps efficiently

Example 4 : Inserting filler cells

```
add_filler_cells \
    -cell {FILL1_X1 FILL2_X2 FILL4_X4} \
    -in_rows \
    -prefix FILL
```

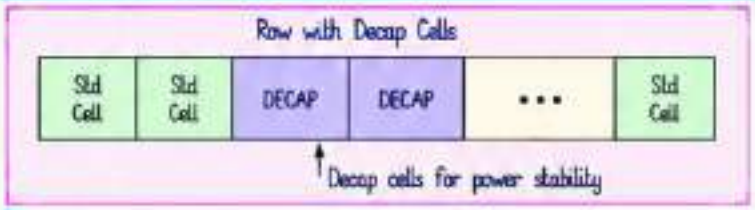


5. Decap cells

- Decap cells (decoupling capacitors) are used for power stability
- Provide additional capacitance between VDD and VSS
- Help in reducing IR drop and power noise
- Inserted in empty spaces or in dedicated decap rows/areas

Example 5 : Inserting decap cells

```
add_decap_cells \
    -cell DECAP_X4 \
    -max_utilization 0.05
    -in_rows \
    -prefix DECAP
```



6. Physical-only cell checks

- Run checks to verify proper insertion of tap, endcap, boundary, filler and decap cells
- Check for missing cells, incorrect spacing or DRC violations
- Use built-in ICC2 reports for verification

Example 6 : Physical-only cell verification

```
check_physical_only_cells
report_physical_only_cells
report_tap_cells
report_filler_cells
report_decap_cells
report_endcap_cells
report_boundary_cells
```

Sample Report Output

Check	Count	Status
Tap Cells	1250	OK
Endcap Cells	320	OK
Boundary Cells	180	OK
Filler Cells	4520	OK
Decap Cells	860	OK
DRC Violations	0	OK

Key Takeaways

- Use appropriate physical-only cells for manufacturing and reliability
- Automate insertion using ICC2 Tcl commands
- Ensure proper spacing and coverage
- Run physical-only cell checks to verify the setup
- A clean physical design requires correct use of tap, endcap, boundary, filler and decap cells

Author : Konda Tharun Vivekananda

"Well placed cells build reliable chips !"

- Good placement is the foundation for timing, power and routability
- Use ICC2 automation to achieve high-quality placement
- Apply constraints and density control for better results
- Iterate with place_opt and refinement for convergence
- Always check placement after each stage
- A must-know topic for ICC2 users

TCL for Complete ICC2 Physical Design

18. Placement Automation

- Set proper constraints (location, fences, groups)
- Control placement density to avoid congestion
- Use legalize and refine for clean placement
- Monitor QoR using built-in reports
- Fix placement issues early in the flow
- Better placement leads to easier routing and signoff

18.1 Placement Automation Overview

- Placement decides the physical locations of standard cells.
- ICC2 provides powerful automation commands for placement.
- Includes global placement, legalization and optimization.
- Supports various constraints and density controls.
- Good placement improves timing, power, routability and overall design quality.

1. place_design

- Performs initial placement (global placement).
- Places standard cells based on constraints, timing and density.
- Generates a legal or near-legal placement.
- Can be run in different modes (e.g., -timing_driven, -congestion_driven).

Example 1 : Running initial placement

```
place_design \
    -timing_driven \
    -congestion_driven \
    -effort high
```

2. place_opt

- Performs placement optimization after initial placement.
- Improves timing, reduces congestion and optimizes wirelength.
- Can be run in incremental mode.
- Commonly used before and after legalization.

Example 2 : Running place_opt

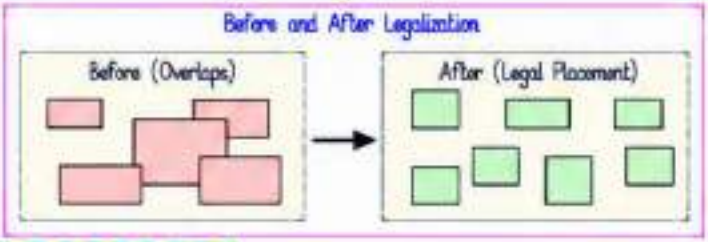
```
place_opt \
    -timing_driven \
    -congestion_driven \
    -power_driven \
    -effort high
```

3. legalize_placement

- Moves cells to legal sites without changing the placement significantly.
- Fixes overlaps and site violations.
- Ensures design is fully legal for routing.

Example 3 : Legalizing placement

```
legalize_placement \
    -incremental \
    -effort high
```



Key Takeaways

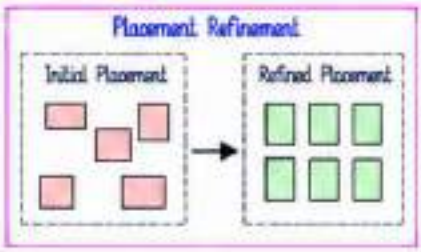
- [ ] Use place_design for initial placement.
- [ ] Run place_opt to improve QoR.
- [ ] Always legalize placement before detailed steps.
- [ ] Apply placement constraints and density control.
- [ ] Check placement regularly using reports.
- [ ] Good placement reduces congestion and improves timing, power and routability.

4. refine_placement

- Performs incremental improvement to the placement.
- Reduces wirelength, congestion and improves timing.
- Can be run in specific regions or for selected cells.
- Useful after legalization or after applying constraints.

Example 4 : Refining placement

```
refine_placement \
    -effort high \
    -congestion_driven \
    -incremental
```



5. refine_opt

- Runs optimization during placement refinement.
- Targets timing, congestion and power improvement.
- Useful for final placement tuning before CTS.
- Can be run with different effort levels.

Example 5 : Running refine_opt

```
refine_opt \
    -timing_driven \
    -congestion_driven \
    -power_driven \
    -effort high
```

6. Placement constraints

- Control where cells can or cannot be placed.
- Use location constraints, placement blockages, fences and groups.
- Helps to meet design intent and avoid congestion in critical areas.

Example 6 : Setting placement constraints

```
create_placement_blockage -name BLK1 \
    -coordinate {100 200 300 400}

set_placement_constraint -cells U1 -location {500 600}

create_placement_region -name REG1 \
    -coordinate {1000 1000 2000 2000}

set_placement_fence -cells {U2 U3 U4} -region REG1
```

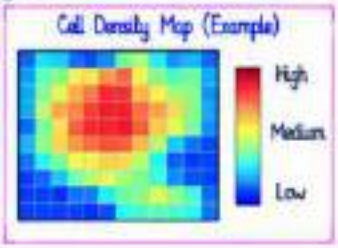


7. Density control

- Controls the cell density in the design or in specific regions.
- Helps to reduce congestion and improve routability.
- Can set global or local density targets.

Example 7 : Setting placement density

```
set_placement_density -global 0.7
set_placement_density -region REG1 0.6
report_placement_density
```



Author : Konda Tharun Vivekananda

"Better placement today, smoother routing tomorrow !"

- Analyze placement quality before routing
- Use ICC2 reports to check density, congestion and timing
- Identify high-fanout nets and critical paths early
- Fix placement legality issues to avoid route failures
- Automate analysis using TCL for quick debugging
- A must know topic for ICC2 physical design automation

TCL for Complete ICC2 Physical Design

19. Placement Analysis Automation

- Run analysis after placement and place_opt
- Use built-in ICC2 reports and visualizations
- Check hotspots and take corrective actions
- Ensure placement is legal and meets utilization
- Correlate placement with timing and congestion
- Helps achieve better QoR and faster convergence

19.1 Placement Analysis Automation Overview

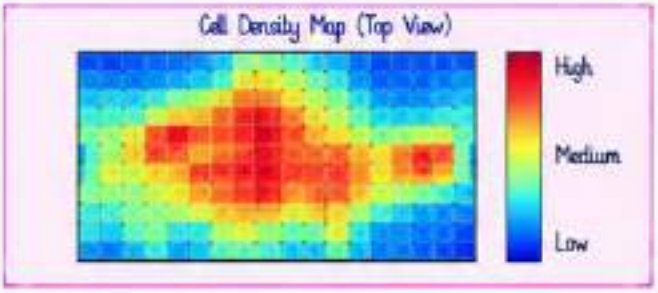
- Placement analysis checks the quality of the placed design.
- Helps identify issues like high cell density, congestion, timing violations, high-fanout nets and placement legality problems.
- ICC2 provides various reports and visualizations to analyze placement and guide optimization.
- Early analysis helps in fixing problems before routing, improving QoR, runtime and overall design convergence.

1. Cell density

- Cell density is the ratio of cell area to the available area in a region (e.g., core or placement block).
- High density can cause congestion and routing issues.
- ICC2 provides reports and density maps to analyze and control density.

Example 1 : Report cell density

```
report_placement_density
report_density -by_region
report_utilization
set_placement_density -global 0.7
set_placement_density -region REG1 0.6
```



2. Utilization

- Utilization indicates how much of the core area is used by cells.
- Proper utilization improves area efficiency without hurting routability.
- ICC2 reports overall and per-region utilization.

Example 2 : Report design utilization

```
report_utilization
report_utilization -by_hierarchy
report_utilization -by_region
get_attribute [current_design] core_utilization
```

Sample Utilization Report

Region	Cell Area (um²)	Total Area (um²)	Utilization (%)
Core	450000	600000	75.0
REG1	120000	200000	60.0
REG2	180000	200000	90.0
REG3	150000	200000	75.0

Key Takeaways

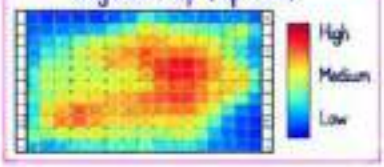
- Run placement analysis after placement / place_opt.
- Check cell density and utilization to avoid congestion.
- Use congestion and timing reports to identify problem areas.
- Monitor high-fanout nets and critical paths.
- Ensure placement legality before routing.
- Automate analysis using TCL for faster debugging and better QoR.

3. Congestion

- Congestion occurs when too many cells/nets compete for routing resources in a region.
- High congestion leads to routing detours, timing violations and increased runtime.
- ICC2 provides congestion reports and global/detail congestion maps.

Example 3 : Check congestion

```
report_congestion
report_congestion -by_region
report_congestion -verbose
create_congestion_map
gui_show congestion
```



4. Timing

- Placement analysis checks for setup/hold violations after placement.
- Helps identify timing-critical areas and guides further optimization.
- ICC2 provides timing reports and path-based analysis.

Example 4 : Run timing analysis

```
report_timing -max_paths 10
report_timing -slack_lesser_than 0
report_timing -path_type full_clock
report_timing -from [get_ports clk] -to [all_registers]
```

Sample Timing Report (Post-Placement)

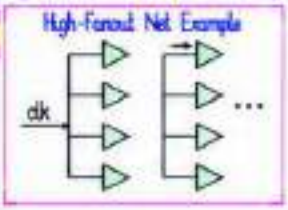
Path	Slack (ns)	Startpoint	Endpoint	Path Type
1	-0.125	U1/FF1	U2/FF2	Setup
2	-0.082	U3/FF5	U4/FF6	Setup
3	0.015	U5/FF8	U6/FF9	Hold

5. High-fanout nets

- High-fanout nets drive a large number of loads and can cause congestion, high delay and signal integrity issues.
- ICC2 can report high-fanout nets and help guide replication or buffering.

Example 5 : Report high-fanout nets

```
report_high_fanout_nets -threshold 100
report_net -fanout -greater_than 100
```



6. Critical paths

- Critical paths are the longest (setup) or shortest (hold) paths in the design.
- Identifying critical paths helps focus optimization efforts.
- ICC2 provides path reporting and visualization.

Example 6 : Report critical paths

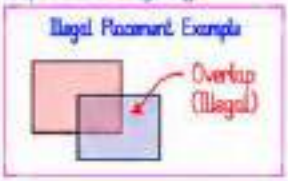
```
report_timing -max_paths 10 -nworst 10
report_timing -slack_lesser_than 0.05
```

7. Placement legality

- Placement legality ensures no cell overlaps, all cells are within rows and site/grid rules are met.
- Illegal placement can cause routing failures and DRC violations.
- ICC2 provides commands to check and fix placement legality.

Example 7 : Check placement legality

```
check_placement
report_placement -illegal
legalize_placement
```



Author : Konda Tharun Vivekananda

"Analyze today, optimize tomorrow !"

- Clock Tree Synthesis (CTS) distributes clocks to all sequential elements with low skew
- ICC2 provides powerful automation for CTS using Tcl
- Proper clock definition and constraints are essential
- CTS impacts timing, power, area and signal integrity
- Analyze clock tree quality using reports after CTS

TCL for Complete ICC2 Physical Design

20. CTS Automation

- Use ICC2 CTS commands for clock tree synthesis
- Define and identify clocks correctly
- Control CTS optimization with useful options
- Analyze clock skew, latency and transition
- Check CTS QoR using reports
- Fix violations and improve clock tree quality
- A must-know topic for ICC2 users

20.1 CTS Automation Overview

- Clock Tree Synthesis (CTS) builds a clock distribution network from clock source to all sequential elements
- ICC2 provides Tcl commands to automate and control CTS
- Proper clock identification, constraints and CTS options ensure low skew, balanced latency and acceptable transition
- Analyze CTS results using built-in reports and fix any issues
- A good CTS improves timing, power, reliability and overall design performance

1. Clock identification

- Clocks must be defined in the design using create_clock
- Specify period, waveform, and clock source / port
- Identify all functional clocks (core, IO, scan, test, etc.)
- Set clock properties like uncertainty, latency and transition
- Use get_clocks and report_clocks to verify

Example 1 : Creating and identifying clocks

```
create_clock -name CLK -period 2.5 -waveform {0 1.25} [get_ports clk]
set_clock_uncertainty 0.1 [get_clocks CLK]
report_clocks
get_clocks
```

Sample Clock Report

Clock Name	Period (ns)	Waveform (ns)	Source	Type
CLK	2.5	{0 1.25}	clk	Root
CLK2	5.0	{0 2.5}	clk2	Root
SCAN_CLK	10.0	{0 5.0}	scan_clk	Root

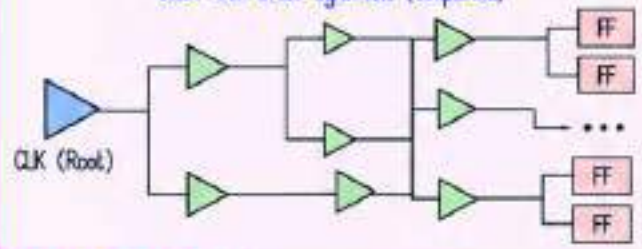
2. synthesize_clock_tree

- Builds an initial clock tree based on already defined clocks
- Inserts clock buffers / inverters automatically
- Tries to meet target skew, latency and transition
- Can be run in different modes with various options

Example 2 : Running initial clock tree synthesis

```
synthesize_clock_tree \
    -clock_list {CLK} \
    -max_skew 0.1 \
    -max_cap 0.05 \
    -target_skew 0.05
```

Clock Tree after Synthesis (Simplified)



Key Takeaways

- Define and identify all clocks with correct constraints
- Use synthesize_clock_tree and clock_opt commands for CTS
- Analyze clock skew, latency and transition after CTS
- Fix any violations and iterate with optimization
- A well-balanced clock tree is critical for timing and reliability

3. clock_opt

- Performs clock tree optimization (post-synthesis)
- Improves skew, latency, transition and overall clock QoR
- Can be run incrementally or in full mode
- Useful options to control optimization effort
- Commonly used after synthesize_clock_tree

Example 3 : Running clock optimization

```
clock_opt \
    -clock_list {CLK} \
    -optimize_for skew \
    -max_transition 0.1 \
    -max_capacitance 0.05 \
    -num_iterations 3
```

4. clock_opt_cts

- Integrated command to run CTS and optimization in one step
- Automatically performs clock tree synthesis followed by optimization
- Simplifies the flow for common use cases
- Useful for quick CTS implementation

Example 4 : One-step CTS using clock_opt_cts

```
clock_opt_cts \
    -clock_list {CLK} \
    -optimize_for skew \
    -max_transition 0.1 \
    -max_capacitance 0.05
```

5. Clock-tree construction

- Inserts clock buffers, inverters and clock gating cells
- Builds a balanced tree structure to minimize skew
- Uses useful skew and useful latency techniques
- Honors placement blockages, congestion and physical constraints
- Supports multi-clock and hierarchical designs

Clock Tree Construction Flow



6. Clock skew and latency

- Clock skew is the difference in arrival time between clock sinks
- Clock latency is the total delay from clock source to sink
- Low skew and balanced latency are critical for timing
- Use reports to analyze and check skew / latency

Example 5 : Reporting clock skew and latency

```
report_clock_tree -skew
report_clock_latency
report_clock_timing
```

Sample Clock Skew Report

Clock	Min Lat (ns)	Max Lat (ns)	Skew (ns)
CLK	0.45	0.52	0.07
CLK2	1.10	1.18	0.08

7. Clock transition

- Clock transition (slew) affects timing, power and signal integrity
- CTS tries to meet target transition at all clock sinks
- Use reports to check clock transition
- Fix any violations by running clock_opt with proper constraints

Example 6 : Reporting clock transition

```
report_clock_tree -transition
report_clocks -attributes {transition}
```

Sample Clock Transition Report

Clock	Min Tran (ns)	Max Tran (ns)
CLK	0.03	0.08
CLK2	0.04	0.09

Author : Konda Tharun Venkanandla

"Balanced clocks, better timing, brighter chips !"

- Analyze CTS quality after clock tree synthesis
- Use ICC2 built-in reports to check clock QoR and timing
- Verify skew, latency, transition and violations
- Fix issues and re-run CTS if required
- A clean clock tree is critical for timing signoff
- A must-know topic for ICC2 users

21. CTS Analysis Automation

- Use report_clock_* commands for analysis
- Check global and per-clock QoR numbers
- Analyze skew, latency and transition
- Identify and fix clock violations
- Use reports and GUI for detailed debug
- Helps achieve balanced and reliable clock tree

21.1 CTS Analysis Automation Overview

- CTS analysis checks the quality, timing and integrity of the generated clock tree.
- ICC2 provides dedicated reports to analyze clock QoR.
- Key metrics include skew, latency, transition and violations.
- Helps identify issues and guide further optimization.
- A good CTS analysis ensures reliable timing and better design convergence.

1. report_clock_qor

- Provides overall clock tree quality of results (QoR).
- Shows metrics like number of sinks, skew, latency, transition, and violations.
- Can be run for all clocks or a specific clock.
- Useful to quickly assess CTS quality.

Example 1 : Reporting clock QoR

```
report_clock_qor
report_clock_qor -clock CLK
```

Sample report_clock_qor Output

Clock	#Sinks	Skew (ns)	Latency (ns)	Transition (ns)	Violations
CLK	1250	0.045	1.23	0.08	0
CLK2	980	0.062	0.95	0.11	2
SCAN_CLK	420	0.010	0.60	0.07	0

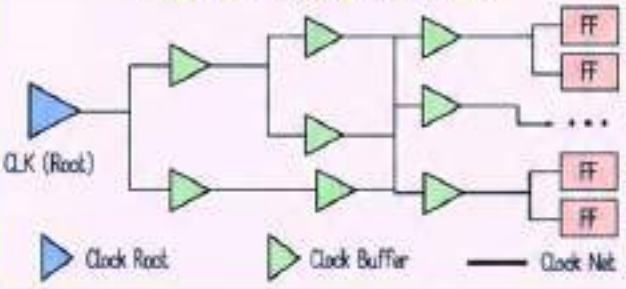
2. report_clock_tree

- Provides detailed clock tree structure information.
- Shows clock buffers, inverters, routing, and insertion delays.
- Helps to analyze clock tree topology and balance.
- Useful for debug and understanding the clock tree.

Example 2 : Reporting clock tree structure

```
report_clock_tree
report_clock_tree -clock CLK
report_clock_tree -levels 5
report_clock_tree -attributes {cell ref_name}
```

Sample Clock Tree Report (Simplified)



3. report_clock_timing

- Provides timing information for clock paths.
- Shows insertion delay, arrival times and required times.
- Helps to identify clock timing violations.
- Useful for detailed analysis of clock timing after CTS.

Example 3 : Reporting clock timing

```
report_clock_timing
report_clock_timing -clock CLK
report_clock_timing -from [get_ports CLK] -to [all_registers]
```

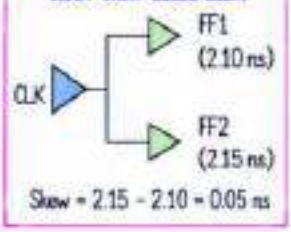
4. Skew analysis

- Clock skew is the difference in arrival time between clock sinks.
- High skew can cause setup/hold violations.
- ICC2 provides skew reports and histograms.
- Skew should be within target limits for reliable operation.

Example 4 : Reporting clock skew

```
report_clock_qor -skew
report_clocks -attributes {skew}
```

Clock Skew Illustration



Skew = 2.15 - 2.10 = 0.05 ns

5. Latency analysis

- Clock latency is the total delay from clock source to sink.
- Includes source latency and network latency.
- ICC2 reports latency for each clock.
- Helps to ensure balanced clock distribution.

Example 5 : Reporting clock latency

```
report_clocks -attributes {latency}
report_clock_timing -clock CLK
```

Sample Clock Latency Report

Clock	Source Latency (ns)	Network Latency (ns)	Total Latency (ns)
CLK	0.25	1.00	1.25
CLK2	0.30	0.80	1.10

6. Clock violations

- Clock violations include setup, hold, transition and skew violations.
- It can occur due to high skew, large transition or improper constraints.
- ICC2 provides dedicated reports to identify violations.
- Fix violations by adjusting CTS options, constraints or running incremental clock optimization.

Example 6 : Reporting clock violations

```
report_clock_qor -violators
report_timing -path_type full_clock
report_constraint -all_violators
```

Sample Clock Violations Report

Clock	Violation Type	Count	Worst (ns)
CLK	Setup	0	-
CLK	Hold	1	0.012
CLK2	Transition	3	0.18
CLK2	Skew	2	0.09

Key Takeaways

- [ ] Use report_clock_qor to check overall CTS quality.
- [ ] Use report_clock_tree to analyze clock structure.
- [ ] Use report_clock_timing for detailed clock timing.
- [ ] Monitor skew, latency and transition after CTS.
- [ ] Check and fix clock violations for reliable design.
- [ ] A well analyzed clock tree leads to better timing closure.
- [ ] Use both reports and GUI for deeper analysis and debug.

Author : Konda Tharun Vrekumanda

"Good clocks make great designs !"

TCL for Complete ICC2 Physical Design

22. Routing Automation

- Routing is a major step in physical design
- ICC2 provides powerful routing automation using Tcl
- Includes global, track and detailed routing
- Use routing options and constraints for better PPA
- Run route_opt to improve timing, congestion and DRC
- Always check routing results and fix violations
- Use proper routing strategy and layer rules
- Monitor congestion and guide routing if needed
- Use route_opt for timing and DRC improvement
- Verify with built-in reports and GUI
- A clean routing ensures good performance
- A must-know topic for ICC2 users

22.1 Routing Automation Overview

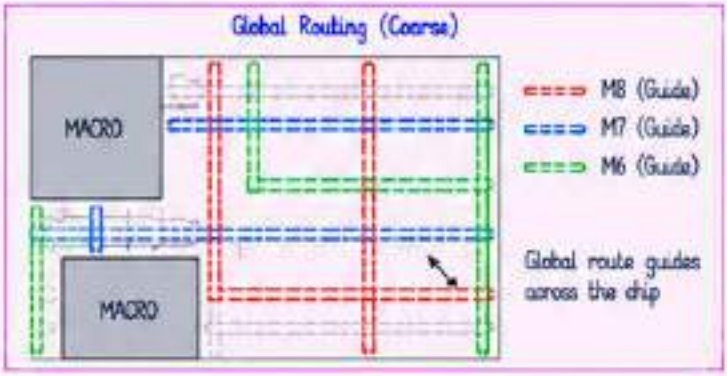
- Routing connects all the placed cells using metal layers based on design rules and constraints.
- ICC2 provides automated routing commands: route_global, route_track, route_detail and route_opt.
- Routing is done in stages: Global Routing → Track Routing → Detail Routing.
- Proper routing improves timing, reduces congestion and ensures DRC clean design.
- Routing automation saves time and provides better QoR.

1. route_global

- Performs global routing to determine route guides and overall routing topology.
- Considers congestion, layer usage and routing constraints.
- Generates global route guides for detailed routing.
- Helps in early congestion analysis and fixing hot spots.

Example 1 : Running global routing

```
route_global \
    -congestion_driven true \
    -use_ndr true \
    -timing_driven true \
    -output_dir ./route_global
```

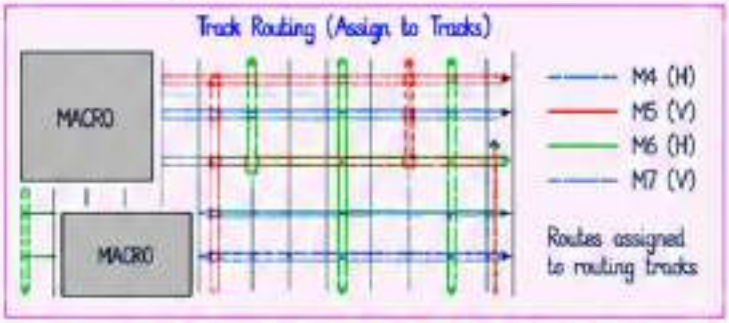


2. route_track

- Assigns global routes to specific routing tracks.
- Converts global route guides into detailed track routes.
- Resolves track assignments, layer directions and preferred routes.
- Helps to prepare for detailed routing.

Example 2 : Running track routing

```
route_track \
    -timing_driven true \
    -congestion_driven true \
    -use_track_guide true
```

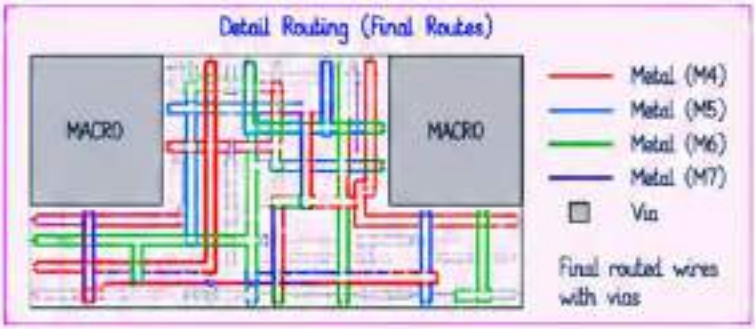


3. route_detail

- Performs detailed routing to create actual wires, vias and final connections.
- Resolves design rule checks, spacing, and via constraints.
- Uses track routes as input and generates final routed design.
- Can be run incrementally for ECOs.

Example 3 : Running detail routing

```
route_detail \
    -timing_driven true \
    -drc_driven true \
    -honor_ndr true \
    -incremental false
```



4. route_opt

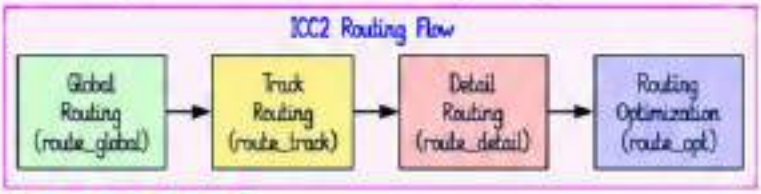
- Optimizes the routed design for timing, congestion and DRC.
- Performs post-routing optimization (similar to ECO routing).
- Can fix setup/hold violations and reduce congestion.
- Useful options to improve overall QoR.

Example 4 : Running route optimization

```
route_opt \
    -timing_driven true \
    -drc_driven true \
    -congestion_driven true
    -max_iterations 5
```

5. Routing Flow Summary

- Routing is done in multiple stages: global, track and detail.
- Each stage refines the routing and resolves more details.
- route_opt is used for final improvement and fixing violations.
- Always analyze routing results using reports and GUI.
- Adjust constraints, NDRs and routing strategies for better results.



Key Takeaways

- [ ] Use route_global to generate global route guides.
- [ ] Use route_track to assign tracks.
- [ ] Use route_detail to create final routed wires.
- [ ] Run route_opt to improve timing, congestion and fix violations.
- [ ] Check congestion, DRC and timing after routing.
- [ ] Use NDRs, routing constraints and layer strategies for better QoR.
- [ ] A clean and optimized routing ensures reliable and high-performance design.

Author : Konda Tharun Vivekananda

"Well routed designs power a better tomorrow !"

TCL for Complete ICC2 Physical Design

23. Routing Analysis Automation

- Analyze routed design to ensure DRC clean and reliable
- Use ICC2 reports for antenna, congestion, crosstalk, etc.
- Identify and fix routing issues early
- Correlate routing with timing and signal integrity
- Automate analysis using Tcl for faster debug and closure
- A must-know topic for ICC2 signoff
- Use reports and GUI to analyze and visualize
- Check global and detailed routing quality
- Monitor wirelength and congestion
- Fix antenna and crosstalk violations
- Ensure timing is met after routing
- Iterate with route_opt and constraints
- Better analysis leads to clean, reliable and robust design

23.1 Routing Analysis Overview

- Routing analysis verifies the quality, correctness and reliability of the routed design.
- ICC2 provides various reports to check DRC, antenna, via issues, wirelength, congestion, crosstalk and routing timing.
- These reports help identify violations, optimize routing and achieve signoff readiness.
- A good routing analysis ensures manufacturability, signal integrity and timing closure.

1. DRC analysis

- Checks for design rule violations (spacing, width, enclosure, etc.).
- Ensures routed design meets technology rules.
- Run DRC after global and detail routing.
- Fix violations using route_opt or ECO routing.

Example 1 : Running DRC report

```
report_drc \
    -type all \
    -verbose \
    -outfile drc.rpt
check_drc
```

Sample DRC Report (Summary)

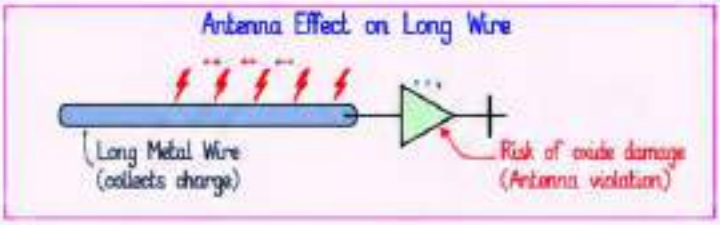
DRC Type	Count	Example
Spacing	12	M4 spacing < 0.1um
Width	5	M3 width < 0.08um
Enclosure	3	Via enclosure violation

2. Antenna analysis

- Checks for antenna violations during fabrication.
- Long interconnect can collect charge and damage gate oxide.
- Use antenna reports to identify violations and insert diodes/ run route_opt fix.

Example 2 : Reporting antenna violations

```
report_antenna \
    -thresholds \
    -verbose \
    -outfile antenna.rpt
```



3. Via analysis

- Checks for via related issues (redundancy, cut count, spacing).
- Ensures proper via usage for reliability and current handling.
- Helps identify missing or insufficient vias.

Example 3 : Reporting via information

```
report_via \
    -type all \
    -verbose \
    -outfile via.rpt
```

4. Wirelength analysis

- Reports total wirelength of the routed design.
- Helps in power, timing and congestion analysis.
- Can be broken down by nets, layers or blocks.
- Useful to compare different routing strategies.

Example 4 : Reporting wirelength

```
report_wire_length \
    -net_type all \
    -by_layer \
    -outfile wirelength.rpt
```

Sample Wirelength Report (by Layer)

Layer	Wirelength (um)
M1	1,250,000
M2	980,000
M3	1,650,000
M4	2,100,000
Total	5,980,000

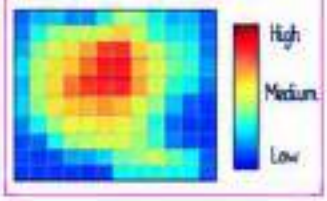
5. Congestion analysis

- Checks routing congestion in global and detailed routing.
- Helps identify over-utilized regions and routing blockages.
- Use congestion maps and reports to guide optimization.
- High congestion can lead to DRC, timing and SI issues.

Example 5 : Reporting congestion

```
report_congestion \
    -type global \
    -by_region \
    -outfile congestion.rpt

gui_show_congestion
```

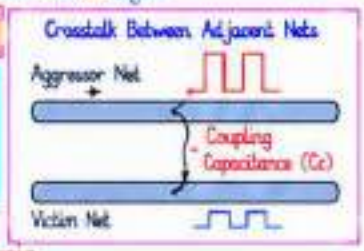


6. Crosstalk analysis

- Analyzes coupling between adjacent nets.
- Reports noise, delta delay and SI violations.
- Helps identify sensitive nets and optimize spacing/shielding.
- Important for high-speed and low-noise designs.

Example 6 : Reporting crosstalk

```
report_xtalk \
    -significant \
    -threshold 0.1 \
    -outfile xtalk.rpt
```



7. Routing timing analysis

- Checks timing after routing with extracted parasitics.
- Reports critical paths, path delays and setup/hold violations.
- Helps correlate routing with overall timing closure.
- Use post-route timing reports for signoff verification.

Example 7 : Reporting routing timing

```
report_timing \
    -significant_digits 4 \
    -path_type full_clock \
    -max_paths 10 \
    -sort_by group
```

Sample Post-Route Timing Report

Path	Slack (ns)	Delay (ns)	Type
1	-0.032	1.203	Setup
2	-0.010	0.987	Setup
3	0.015	1.450	Hold

Key Takeaways

- Run DRC, antenna, via, congestion and crosstalk analysis after routing.
- Use wirelength and timing reports to evaluate design quality.
- Fix violations using route_opt, constraints and ECO routing.
- Analyze reports and GUI for detailed debugging.
- A clean routing analysis ensures reliable, manufacturable and high-performance design.

Author : Konda Tharun Vekananda

"Clean routes, reliable chips, brighter tomorrows !"

- Post-route optimization improves timing, signal integrity and physical quality of the routed design
- ICC2 provides route_opt and related commands
- Addresses timing, crosstalk, DRC, antenna and via issues
- Uses advanced algorithms with incremental optimization
- Iterate analysis and optimization for best QoR
- Use reports to identify violations before optimize
- Apply proper constraints and optimization strategies
- Run incremental optimization for faster convergence
- Verify improvement using timing, SI and DRC reports
- A clean post-route design is critical for signoff
- Automate the flow using Tcl for efficiency
- A must-know topic for ICC2 physical design users

24. Post-Route Optimization

24.1 Post-Route Optimization Overview

- Post-route optimization (PRO) fixes violations after detailed routing
- Targets timing, crosstalk, DRC, antenna and via issues
- Uses route_opt and related commands in ICC2
- Can run in incremental or full-chip mode
- Improves overall QoR and ensures signoff readiness
- Requires proper constraints, analysis and iterative optimization

1. Tuning optimization

- Fixes setup and hold timing violations after routing
- Optimizes by sizing, buffer insertion, upsizing/downsizing
- Uses route_opt with timing-driven options
- Can target specific paths or run globally
- Helps achieve signoff timing closure

Example 1 Run post-route timing optimization

```
route_opt \
    -timing_driven true \
    -max_iterations 5 \
    -incremental true \
    -report_detail true
```



2. Crosstalk optimization

- Fixes noise-induced delay and SI violations
- Uses shielding, spacing, re-routing or layer change
- Optimizes aggressor/victim nets
- Helps improve signal integrity and reduce setup/hold violations
- Run with SI-aware options in route_opt

Example 2 Run crosstalk optimization

```
route_opt \
    -si_driven true \
    -crosstalk_reduction true \
    -max_iterations 5
```



3. DRC fixing

- Fixes routing design rule violations (spacing, width, enclosure, etc.)
- Uses route_opt with -drc option
- Can run incremental DRC fixing to preserve timing
- Helps achieve clean design for signoff

Example 3 Run DRC fixing

```
route_opt \
    -drc true \
    -drc_effort high \
    -max_iterations 10
```

Common DRC Issues

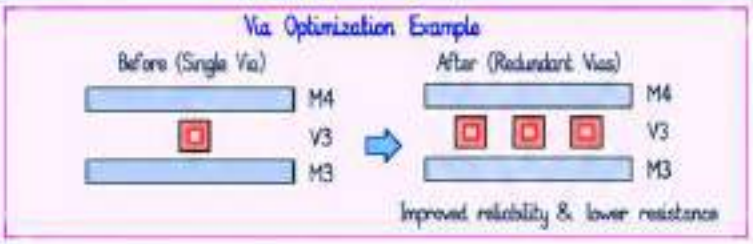
- Min spacing
- Min width
- Enclosure violation
- Cut spacing
- Parallel run length

4. Via optimization

- Optimizes via count, spacing and redundancy
- Fixes via DRC violations
- Ensures reliable connections and better current handling
- Can add redundant vias on critical nets
- Run during route_opt or as a separate step

Example 4 Run via optimization

```
route_opt \
    -via_optimization true \
    -via_effort high \
    -max_iterations 5
```

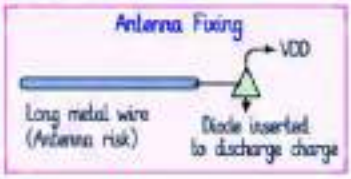


5. Antenna fixing

- Fixes antenna violations due to long metal wires during fabrication
- Inserts diode or antenna cell automatically
- Can also re-route or layer change to reduce antenna effect
- Uses route_opt with antenna options
- Ensures reliability and yield in manufacturing

Example 5 Run antenna fixing

```
route_opt \
    -antenna_fix true \
    -antenna_diode_insertion true \
    -max_iterations 5
```



6. Route optimization (Overall)

- Performs overall post-route optimization
- Fixes remaining violations (timing, SI, DRC, antenna, etc.)
- Supports incremental optimization for quick convergence
- Uses multiple cost functions for best trade-off
- Final step before signoff verification

Example 6 Run overall route optimization

```
route_opt \
    -timing_driven true \
    -si_driven true \
    -drc true \
    -antenna_fix true \
    -max_iterations 10 \
    -incremental true \
    -report_detail true
```



Key Takeaways

- Use route_opt with appropriate options for each type of violation
- Run timing, SI, DRC, antenna and via optimization iteratively
- Analyze reports before and after optimization to measure improvement
- Use incremental mode to save runtime
- Ensure clean design for signoff with no violations
- Automate post-route optimization using Tcl scripts

Author : Konda Tharun Vrekumanda

"Fix today, signoff tomorrow !"

- ECO (Engineering Change Order) fixes late design changes
- ICC2 provides powerful ECO capabilities using Tcl
- Supports timing, SI, DRC and functional ECOs
- Minimizes design disruption and turnaround time
- Can be done before or after signoff
- A must-know topic for real chip development

TCL for Complete ICC2 Physical Design

25. ECO Automation

- Use ECO commands with proper scope and constraints
- Verify changes after every ECO step
- Maintain design integrity (timing, DRC, SI, etc.)
- Use incremental mode for better runtime
- Run ECO placement and routing for new cells/nets
- Check final QoR and signoff readiness
- A practical and important topic for ICC2 users

25.1 ECO Automation Overview

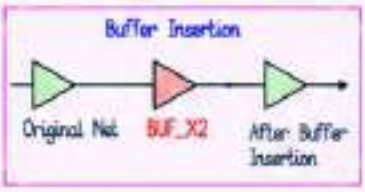
- ECO is used to fix timing, DRC, SI or functional issues with minimal design changes.
- ICC2 provides dedicated ECO commands for cell insertion, resizing, net modification, placement and routing.
- ECO flow is incremental and preserves existing design as much as possible.
- Requires re-verification after ECO to ensure no new violations.
- Common in post-route fixing, late-stage changes and metal eco (Meco).

1. ECO cell insertion

- Inserts new cells (buffers, inverters, etc.) to fix timing or SI issues.
- Can insert cells on existing nets automatically.
- ICC2 provides insert_cell or eco_insert_cell commands.
- Useful for adding buffers, level shifters, or tie cells.

Example 1 : Inserting a buffer cell

```
insert_cell BUF_X2 \
    -net [get_nets net1] \
    -location {100.5 200.3} \
    -name ECO_BUF1
```



2. Cell sizing

- Changes the size of existing cells to improve timing.
- Can be done using size_cell or eco_change_cell_size.
- Useful for fixing setup/hold violations.
- Can be applied to multiple cells using collections.

Example 2 : Resizing a cell

```
size_cell \
    -cell U1 \
    -to BUFX4
```

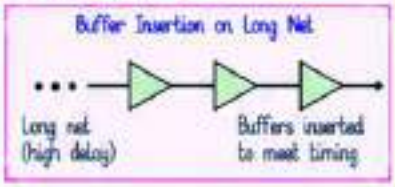


3. Buffer insertion

- Adds buffers to reduce net delay or fix hold violations.
- Can be inserted automatically using eco_add_buffer.
- Useful for high-fanout nets and long interconnects.
- ICC2 can choose optimal location and size.

Example 3 : Adding buffer on a net

```
eco_add_buffer \
    -net [get_nets net2] \
    -buffer_cell BUFX2 \
    -name ECO_BUF2
```



4. Net modification

- Modifies existing nets (reconnect, split, or add new connections).
- Can be used to change net topology or fix connectivity issues.
- Useful in ECO for metal fixes or logical changes.

Example 4 : Modifying a net

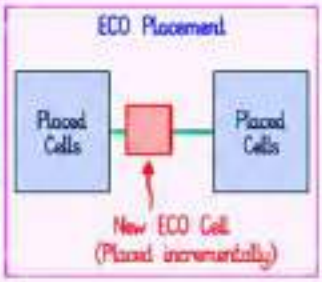
```
disconnect_net -net net3 -pins {U1/Z2}
connect_net    -net net3 -pins {U2/A}
```

5. ECO placement

- Places newly inserted or modified cells.
- Use place_eco or incremental placement.
- Ensures minimum disturbance to existing design.
- Can be constrained to specific regions.

Example 5 : ECO placement

```
place_eco \
    -cells {ECO_BUF1 ECO_INV1} \
    -incremental true \
    -preserve_routing true
```

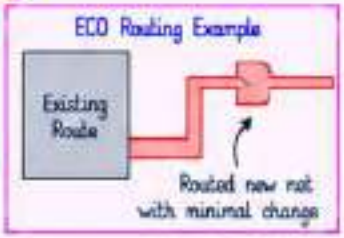


6. ECO routing

- Routes the newly inserted/modified cells and nets.
- Use route_eco for incremental routing.
- Fixes DRC and ensures connectivity.
- Supports different routing options (nanoRoute, track, detail).

Example 6 : ECO routing

```
route_eco \
    -cells {ECO_BUF1 ECO_INV1} \
    -incremental true \
    -fix_drc true
```



7. ECO verification

- Verify timing, DRC, SI and connectivity after ECO.
- Use ICC2 reports to check that fixes are successful.
- Ensure no new violations are introduced.
- Run incremental signoff checks if required.

Example 7 : ECO verification commands

```
report_timing -from U1 -to U2
check_drc
report_crosstalk
report_antenna
report_design_status
```

ECO Verification Checklist

Check Item	Command
Timing	report_timing
DRC	check_drc
Crosstalk	report_crosstalk
Antenna	report_antenna
Connectivity	report_design_status
QoR Summary	report_qor

Key Takeaways

- ☐ ECO fixes timing, SI, DRC or functional issues with minimal changes.
- ☐ Use insert_cell, size_cell, eco_add_buffer for cell modifications.
- ☐ Modify nets using disconnect_net and connect_net.
- ☐ Run place_eco and route_eco for incremental implementation.
- ☐ Always verify after ECO using timing, DRC, SI and antenna reports.
- ☐ Maintain design integrity and avoid introducing new violations.
- ☐ ECO is a critical skill for real-world chip development.

Author : Konda Tharun Vrekumanda

"Small changes. Big impact !"

- Physical verification ensures the design is manufacturable
- ICC2 provides built-in signoff checks using Tcl commands
- Helps catch DRC, LVS, timing, routing and connectivity issues
- Should be run at multiple stages (pre-route, post-route, signoff)
- Use reports and GUI to analyse and fix violations
- A clean design is critical for tapeout success

TCL for Complete ICC2 Physical Design

26. Physical Verification Automation

- Use check_* commands for automated verification
- Run all checks after major design stages
- Analyse reports, fix violations and re-run
- Perform PG, antenna and connectivity checks
- Ensure timing is met along with physical correctness
- A must-know topic for ICC2 users

26.1 Physical Verification Automation Overview

- Physical verification checks the design for rule violations, timing issues, connectivity and manufacturability.
- ICC2 provides Tcl commands to run DRC, LVS, timing, routing and other checks.
- These checks ensure the design is clean before signoff.
- Can be run in batch mode and integrated into flows.
- Helps reduce iterations and achieves reliable tapeout.

1. check_design

- Performs general design checks (connectivity, unplaced cells, PG connectivity, etc.).
- Identifies common issues early in the flow.
- Can be run with different options for specific checks.
- Useful after placement, CTS and routing.

Example 1 Running check_design

```
check_design \
    -unplaced \
    -misc \
    -pg \
    -verbose
```

Sample check_design Report

Check Item	Status
Unplaced cells	0
PG connectivity	0
Floating nets	2
Multiple drivers	0
Misc. issues	0

2. check_timing

- Runs timing checks to verify setup, hold and other constraints.
- Identifies timing violations after placement/CTS/routing.
- Can be run for specific scenarios/corners.
- Useful for quick timing validation before signoff.

Example 2 Running check_timing

```
check_timing \
    -path_type full_clock \
    -max_paths 100 \
    -verbose
```

Sample check_timing Report

Check Type	Total Paths	Violations	Worst (ns)
Setup	1250	12	-0.032
Hold	1250	5	-0.018
Recovery	320	0	0.000
Removal	320	0	0.000

3. check_drc

- Checks for design rule violations (spacing, width, enclosure, etc.).
- Uses internal ICC2 DRC engine or foundry rule deck.
- Generates detailed reports with violation locations.
- Can be run incrementally for faster debug.

Example 3 Running check_drc

```
check_drc \
    -all \
    -verbose \
    -outfile drc.rpt
```

Sample DRC Report

Rule Type	Count	Example
Spacing	15	M1 spacing < 0.1um
Width	8	M2 width < 0.05um
Enclosure	3	Via enclosure
Density	1	Metal density

4. check_routes

- Checks routed nets for connectivity and routing integrity.
- Identifies open, short, unconnected nets, wrong layer usage.
- Helps verify that routing is completed correctly.
- Useful after detailed routing and ECOs.

Example 4 Running check_routes

```
check_routes \
    -all \
    -nets all \
    -verbose \
    -outfile route_check.rpt
```

Sample check_routes Report

Issue Type	Count
Unrouted nets	0
Open nets	2
Shorts	0
Wrong layer	1
Antenna risk	0

5. check_lvs

- Performs layout vs schematic (LVS) checks.
- Verifies connectivity between layout and netlist.
- Ensures no missing or extra devices/nets.
- Usually run at signoff stage with foundry tools (Calibre, etc.).
- ICC2 can invoke LVS flow and report results.

Example 5 Running check_lvs

```
check_lvs \
    -netlist design.v \
    -layout design.gds \
    -report lvs.rpt \
    -hierarchical true
```

Sample LVS Report

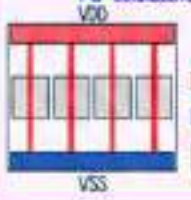
Check Item	Result
Device mismatch	0
Missing devices	0
Extra devices	0
Net mismatch	0
Overall status	PASS

6. PG checks

- Checks power and ground (PG) connectivity and integrity.
- Identifies floating PG nets, missing straps or broken connections.
- Ensures proper power distribution for reliable operation.
- Can be run using check_design or dedicated PG check commands.

Example 6 Running PG checks

```
check_design -pg
check_pg_connectivity \
    -verbose \
    -report pg_check.rpt
```



7. Antenna checks

- Checks for antenna rule violations during manufacturing.
- Identifies nets with high metal area to gate area ratio.
- Can insert diodes or use routing options to fix violations.
- Helps prevent oxide damage during fabrication.

Example 7 Running antenna checks

```
check_antenna \
    -all \
    -report antenna.rpt \
    -verbose
```

Sample Antenna Report

Net Name	Violation	Ratio
net_123	YES	1200
net_456	YES	850
net_789	NO	200

Key Takeaways

- Run physical verification checks at every major stage of the flow.
- Use check_design, check_timing, check_drc, check_routes, check_lvs.
- Verify PG connectivity and antenna violations.
- Analyse reports and fix violations before signoff.
- A clean, verified design ensures reliable and manufacturable silicon.

Author : Konda Tharun Vrekumanda

"Verify well today, tapeout with confidence tomorrow !"

TCL for Complete ICC2 Physical Design

27. Signoff Report Automation

- Signoff reports give final quality checks of the design
- ICC2 provides powerful reporting commands using Tcl
- Generate reports for timing, DRC, LVS, PG, power and QoR
- Use proper options and filters for meaningful reports
- Automate report generation for signoff flow
- A must-know topic for complete ICC2 physical design
- Use reports to ensure clean and consistent sign-off
- Check all violations are fixed before tapeout
- Generate summary and detailed reports
- Use redirection to save reports in files
- Compare QoR and power reports across iterations
- Final signoff reports ensure manufacturability
- A critical topic for ICC2 users

27.1 Signoff Report Automation Overview

- Signoff reporting is the final step to verify the design quality
- ICC2 provides various report commands for timing, DRC, LVS, PG, power and QoR
- Reports help ensure the design meets all specifications
- Can be run after each major stage (post-route, post-ECO, etc.)
- Reports can be customized, filtered and saved to files
- Automating signoff reports saves time and ensures consistency

1. Timing reports

- Provide detailed timing analysis of the design
- Reports setup, hold, critical paths, slack, etc.
- Can be generated for specific clocks, paths or groups
- Useful for final timing signoff

Example 1 : Generate timing report

```
report_timing \
    -max_paths 100 \
    -delay_type max \
    -nworst 10 \
    -out_file timing_signoff.rpt
```

Sample Timing Report (Summary)

Metric	Value
Worst Setup Slack	-0.012 ns
Worst Hold Slack	-0.008 ns
Total Violations	5
Total Endpoints	1250

2. DRC reports

- Checks for physical rule violations (spacing, width, enclosure, etc.)
- Uses internal ICC2 DRC engine
- Reports all violations with location and type
- Should be clean before signoff

Example 2 : Generate DRC report

```
report_drc \
    -all \
    -verbose \
    -out_file drc_signoff.rpt
```

Sample DRC Report

Rule Type	Count	Example
Spacing	0	-
Width	0	-
Enclosure	0	-
Cut Spacing	0	-
Total	0	(CLEAN)

3. LVS reports

- Verifies layout vs schematic (LVS) connectivity
- Ensures no missing or extra devices/nets
- Usually run at signoff stage with Calibre (or equivalent)
- Reports unmatched devices, nets and pins

Example 3 : Generate LVS report

```
report_lvs \
    -netlist design.v \
    -layout design.gds \
    -out_file lvs_signoff.rpt
```

Sample LVS Report

Check Item	Result
Device mismatch	0
Missing devices	0
Extra devices	0
Net mismatch	0
Overall status	PASS

4. PG reports

- Checks power and ground (PG) connectivity and integrity
- Reports floating PG nets, broken straps, and open connections
- Ensures proper power distribution for reliable operation
- Critical before signoff

Example 4 : Generate PG report

```
report_pg \
    -type connectivity \
    -verbose \
    -out_file pg_signoff.rpt
```

Sample PG Report

Check Item	Count
Floating PG nets	0
Broken straps	0
Open connections	0
PG short violations	0
Overall status	PASS

5. QoR reports

- Reports overall design quality metrics
- Includes area, timing, power, congestion, etc.
- Helps track design progress across iterations
- Useful for final signoff comparison

Example 5 : Generate QoR report

```
report_qor \
    -summary \
    -out_file qor_signoff.rpt
```

Sample QoR Reports

Metric	Value
Total Cell Area	1.25 mm²
Total Wire Length	12.8 mm
Worst Setup Slack	-0.012 ns
Worst Hold Slack	-0.008 ns
Total Power	85.4 mW
DRC Violations	0
Congestion (max)	2%

6. Power reports

- Reports total, dynamic and leakage power
- Can be generated for blocks, hierarchies or power domains
- Helps verify power intent and power optimization
- Important for final signoff and tapeout

Example 6 : Generate power report

```
report_power \
    -hierarchy \
    -analysis_effort high \
    -out_file power_signoff.rpt
```

Sample Power Report

Power Type	Value
Total Power	85.4 mW
Dynamic Power	72.1 mW
Leakage Power	13.3 mW

Power Breakdown



7. Final reports

- Collects all signoff reports (timing, DRC, LVS, PG, power, QoR)
- Can be automated using Tcl scripts
- Helps ensure the design is clean and ready for tapeout
- Useful to create a final report summary

Example 7 : Run final signoff reports (Script)

```
# Create reports directory
file mkdir reports/signoff
# Run all reports
report_timing    -out_file reports/signoff/timing.rpt
report_drc       -out_file reports/signoff/drc.rpt
report_lvs       -out_file reports/signoff/lvs.rpt
report_pg        -out_file reports/signoff/pg.rpt
report_power     -out_file reports/signoff/power.rpt
report_qor       -out_file reports/signoff/qor.rpt
```

Key Takeaways

- Use ICC2 report commands to generate signoff reports
- Check timing, DRC, LVS, PG, power and QoR before tapeout
- Automate report generation using Tcl scripts
- Verify that all violations are fixed and reports are clean
- Use filtered and detailed reports for debugging
- Final reports ensure a reliable and manufacturable design

Author : Konda Tharun Vekumanda

"Reports today, a successful tapeout tomorrow !"

TCL for Complete ICC2 Physical Design

28. GDSII / Tapeout Automation

- Final signoff step generates tapeout data for fabrication
- ICC2 provides Tcl commands to write GDSII, DEF, netlist, SDF, parasitics and other signoff data
- Ensure all violations are fixed before data generation
- Use proper options, technology files and naming conventions
- Generated data is used for mask manufacturing
- A critical final step in the ICC2 physical design flow
- Verify design using signoff checks before tapeout
- Generate all required files in a structured directory
- Use write_gds, write_oasis, write_def, write_verilog, write_sdf and write_parasitics commands
- Check file sizes and run basic sanity checks
- Follow foundry guidelines for layer mapping and data format
- A clean and complete tapeout package ensures success

28.1 GDSII / Tapeout Automation Overview

- GDSII (Graphic Data System II) is the industry standard format for mask manufacturing
- ICC2 provides Tcl commands to generate all required tapeout data (GDSII, OASIS, DEF, netlist, SDF, parasitics).
- Final data generation should be done after all signoff checks (timing, DRC, LVS, etc.)
- Use correct options, layer maps and technology files.
- Verify generated files and follow foundry guidelines.
- A well-prepared tapeout package ensures a smooth transition to manufacturing

1. write_gds

- Generates GDSII file for mask manufacturing.
- Includes all physical design information (layers, cells, hierarchy).
- Use proper library name, units and layer map.
- Usually generated in the final signoff stage.

Example 1 — Writing GDSII file

```
write_gds "top_chip.gds" \
    -library_name TOP \
    -units 2000 \
    -map_file layer_map.tf \
    -compress
```

Important Options

- -library_name : top library name
- -units : database units
- -map_file : layer mapping file
- -compress : reduce file size
- -include_physical_only : physical data

2. write_oasis

- Generates OASIS file (efficient, smaller than GDSII).
- Commonly used for advanced nodes due to smaller file size.
- Supports same options as write_gds.

Example 2 — Writing OASIS file

```
write_oasis "top_chip.oas" \
    -library_name TOP \
    -units 2000 \
    -map_file layer_map.tf
```

GDSII vs OASIS

Feature	GDSII	OASIS
File size	Larger	Smaller
Write time	Slower	Faster
Industry use	Mature	Increasing
Supported	Yes	Yes

3. write_def

- Generates DEF file with design physical information.
- Contains placement, routing, pins and blockages.
- Used for flow handoff or for ECO/validation.

Example 3 — Writing DEF file

```
write_def "top_chip.def" \
    -include_components \
    -include_nets \
    -include_pins \
    -include_routing
```

DEF Usage

- Used for downstream tools (P&R, LEC, ECO)
- Contains physical design data
- Can be used for incremental flows
- Verify DEF using third-party DEF viewers

4. write_verilog

- Generates synthesized or placed netlist in Verilog format.
- Can include SI/PG information based on options.
- Used for LVS, formal check or downstream flows.

Example 4 — Writing Verilog netlist

```
write_verilog "top_chip.v" \
    -include_tie_cells \
    -include_physical_nets \
    -hierarchy
```

Common Options

- -include_tie_cells
- -include_power_nets
- -include_physical_nets
- -hierarchy
- -strip_path

5. write_sdf

- Generates SDF file with final timing information.
- Used for gate-level simulation and signoff correlation.
- Includes delays, timing checks and back-annotated information.

Example 5 — Writing SDF file

```
write_sdf "top_chip.sdf" \
    -version 2.1 \
    -context verilog \
    -load_delay net \
    -include_timing_checks
```

SDF Usage

- Used for gate-level simulation
- Includes cell and net delays
- Helps verify timing with extracted parasitics
- Ensure consistency with signoff STA

6. write_parasitics

- Generates extracted parasitics (RC) in SPEF/CPF format.
- Used for signoff STA and SI analysis.
- Includes resistance, capacitance and coupling information.

Example 6 — Writing SPEF file

```
write_parasitics "top_chip.spef" \
    -format spef \
    -compress \
    -include_coupling_capacitance \
    -hierarchy
```

Parasitic File Formats

Format	Description	Usage
SPEF	Standard format	STA, SI
CPF	Cadence format	STA, SI
DSPF	Detailed format	Signoff

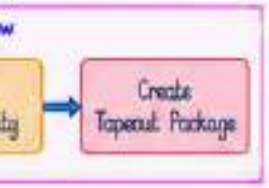
7. Final data generation

- Generate all required files in a structured output directory.
- Run final sanity checks on file sizes and contents.
- Ensure all signoff checks (timing, DRC, LVS, PG, antenna) are clean.
- Create a tapeout package with necessary documentation.

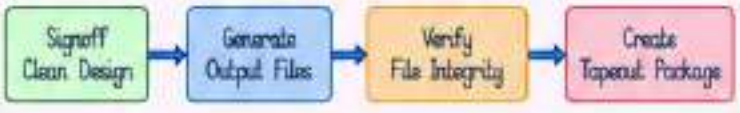
Example 7 — Complete tapeout data generation

```
set out_dir ./tapeout
file mkdir $out_dir
write_gds "$out_dir/top.gds"
write_oasis "$out_dir/top.oas"
write_def "$out_dir/top.def"
write_verilog "$out_dir/top.v"
write_sdf "$out_dir/top.sdf"
write_parasitics "$out_dir/top.spef" -format spef
```

Tapeout Directory Structure



Tapeout Data Generation Flow



Key Takeaways

- Use write_gds, write_oasis, write_def, write_verilog, write_sdf and write_parasitics for final data generation.
- Ensure all signoff checks (timing, DRC, LVS, PG, antenna) are clean.
- Follow foundry guidelines for layer maps and file formats.
- Verify generated files (size, content, hierarchy).
- Maintain a well-organized tapeout directory.
- Final data generation is a critical step for successful manufacturing.

Author : Konda Tharun Vrekaranda

"From design to silicon - it all starts with clean data !"

TCL HERO: ADVANCED ICC2 AUTOMATION

29. Automated QoR Framework

- Automated QoR Framework helps to evaluate overall design quality in ICC2.
- It collects key metrics like area, timing, power, congestion, DRC, wirelength, via count and clock skew.
- Using Tcl automation, reports can be generated, parsed and summarized in a single dashboard.
- Helps in quick comparison across iterations and ensures design is ready for signoff.
- Use reports to extract QoR metrics automatically.
- Monitor area, timing, power, congestion, DRC, etc.
- Run for multiple stages (pre-CTS, post-CTS, post-route).
- Compare results and track improvements.
- Generate summary reports in files (csv / rpt).
- Useful for regression runs and tapeout readiness.
- A must-have automation for advanced ICC2 users.

29.1 Automated QoR Framework Overview

- Collects and reports key QoR metrics from ICC2.
- Provides a unified view of design quality.
- Can be run at different stages (pre-CTS, post-CTS, post-route).
- Helps track improvements across iterations.
- Useful for regression, signoff checks and tapeout readiness.
- Metrics include area, utilization, WNS, TNS, hold, congestion, DRC, power, wirelength, via count and clock skew.
- Output can be saved in a report file (CSV/HTML).

1. Area and Utilization

- Area: Total cell area used in the design.
- Utilization: Percentage of core area occupied.
- Helps to ensure optimal placement and avoid congestion.

Example 1 Get area and utilization

```
report_design_area > area.rpt
set util [get_attribute [get_cells -hier]
    area_utilization]
puts "Utilization = $util"
```

Sample Area Report

Metric	Value
Total Cell Area	1.25 mm²
Core Area	2.00 mm²
Utilization	62.5 %

2. WNS and TNS

- WNS: Worst Negative Slack (setup).
- TNS: Total Negative Slack (setup).
- Indicates overall timing health of the design.

Example 2 Get WNS and TNS

```
report_timing_summary > timing.rpt
set wns [get_attribute [current_design]
    timing_worst_slack]
set tns [get_attribute [current_design]
    timing_total_negative_slack]
puts "WNS = $wns"
puts "TNS = $tns"
```

Sample Timing Summary

Metric	Value
WNS	-0.012 ns
TNS	-1.328 ns
Hold Violations	5
Total Endpoints	1250

3. Hold Violations

- Checks for hold time violations.
- Critical for functional correctness.
- Should be zero before signoff.

Example 3 Check hold violations

```
report_timing -delay_type min > hold.rpt
set hold_viol [get_attribute [current_design]
    timing_hold_violations]
puts "Hold Violations = $hold_viol"
```

Sample Hold Report

Metric	Value
Hold Violations	5
Worst Hold Slack	-0.008 ns

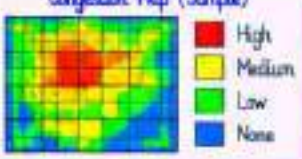
4. Congestion

- Indicates routing congestion in the design.
- High congestion can impact timing and routability.
- Useful to track congested areas and fix early.

Example 4 Check congestion

```
report_congestion > congestion.rpt
set cong [get_design_congestion]
puts "Max Congestion = [get_attribute
    $cong max_congestion]"
```

Congestion Map (Sample)



5. DRC

- Checks for physical design rule violations.
- Should be clean (0 violations) before signoff.
- Can be run incrementally or fully.

Example 5 Run DRC check

```
check_drc > drc.rpt
set drc_viol [get_drc_violations -count]
puts "DRC Violations = $drc_viol"
```

Sample DRC Report

Rule Type	Count
Spacing	0
Width	0
Enclosure	0
Cut Spacing	0
Total DRC	0 (CLEAN)

6. Power

- Reports total, dynamic and leakage power.
- Helps in power optimization and signoff.
- Can be broken down by block or hierarchy.

Example 6 Get power report

```
report_power > power.rpt
set pwr [get_attribute [current_design]
    total_power]
puts "Total Power = $pwr"
```

Sample Power Report

Metric	Value
Total Power	85.4 mW
Dynamic Power	72.1 mW
Leakage Power	13.3 mW

7. Wirelength

- Total routed wirelength in the design.
- Helps measure routing complexity and QoR.
- Should be optimized to reduce RC and power.

Example 7 Get wirelength

```
report_route_length > wirelength.rpt
set wl [get_route_length]
puts "Total Wirelength = $wl"
```

Sample Wirelength Report

Layer	Length (um)
M1	12,450
M2	18,320
M3	25,610
M4	10,220
Total	66,600

8. Via Count

- Total number of vias in the design.
- Important for reliability and manufacturability.
- High via count may indicate routing complexity.

Example 8 Get via count

```
set vias [get_db [get_vias -of_objects
    [get_nets -hier]] .num_objects]
puts "Total Via Count = $vias"
```

Sample Via Report

Via Type	Count
VIA1	125,430
VIA2	98,210
VIA3	45,670
Total	269,310

9. Clock Skew

- Difference in arrival time between clock paths.
- Excessive skew can cause setup/hold violations.
- Should be within target before signoff.

Example 9 Get clock skew

```
report_clock_trees -summary > cts.rpt
set skew [get_attribute [get_clocks *]
    skew]
puts "Clock Skew = $skew"
```

Sample Clock Report

Metric	Value
Clock Skew	12.5 ps
Clock Latency	320 ps
Clock Transition	45 ps

10. Final QoR Summary

- Combine all metrics into a single report.
- Useful for tracking iterations and signoff readiness.
- Can be generated in CSV/HTML format.

Example 10 Generate QoR summary

```
# Run all checks and generate summary
exec python3 generate_qor_report.py \
--o qor_summary.csv
puts "QoR report generated: qor_summary.csv"
```

Sample QoR Summary

Metric	Value
Area (mm²)	1.25
Utilization (%)	62.5
WNS (ns)	-0.012
TNS (ns)	-1.328
Hold Violations	5
Congestion (max)	2%
DRC Violations	0
Total Power (mW)	85.4
Wirelength (um)	66,600
Via Count	269,310
Clock Skew (ps)	12.5

Key Takeaways

- Automated QoR framework provides a complete design health check.
- Use ICC2 report commands to collect all key metrics.
- Monitor area, timing, power, congestion, DRC, wirelength, via count and clock skew.
- Generate a unified summary report for easy comparison.
- Run at multiple stages and track improvements.
- A clean QoR report ensures a successful tapeout.

Author : Konda Tharun Vrekumanda

"Measure. Analyze. Improve. Tapeout with confidence!"

TCL HERO:
ADVANCED ICC2 AUTOMATION
30. Automated Stage Gates

- Stage gates ensure each design stage meets quality criteria before moving to next stage.
- Automated PASS/FAIL checks save time and ensure consistent signoff flow.
- Uses ICC2 reports and user-defined thresholds.
- Prevents late-stage surprises and improves productivity.
- A must-have automation for advanced ICC2 users.

- Define clear metrics and thresholds for each stage.
- Run checks automatically using Tcl scripts.
- Generate PASS/FAIL with detailed reports.
- Stop the flow if any stage fails.
- Ensure GDSII release only when all criteria are met.
- Provides a clean, repeatable and reliable flow.

30.1 Automated Stage Gates Overview

- Stage gates are automated quality checks at each major design stage in ICC2.
- Uses Tcl scripts to collect metrics and compare against predefined thresholds.
- If all criteria meet, stage is marked PASS, else FAIL.
- Helps in early detection of issues and avoids rework.
- Ensures smooth progression from floorplan to GDSII.
- Can be integrated with regression flow for continuous monitoring.

1. Floorplan PASS/FAIL

- Checks core area, utilization, aspect ratio, PG structure.
- Ensures no major DRC violations.
- Validates placement blockages and keepout regions.
- PASS if all metrics meet thresholds.

Example 1 Floorplan stage check (Tcl)

```
set util [get_attribute [get_cells -hier] area_utilization]
set drc_cnt [get_drc_violations -count]
if { $util <= 70 && $drc_cnt == 0 } {
  puts "FLOORPLAN : PASS"
} else {
  puts "FLOORPLAN : FAIL"
}
```

Floorplan Criteria (Example)

Metric	Target	Result	
Utilization (%)	≤ 70	62.5	PASS
Core aspect ratio	0.5–2.0	1.2	PASS
DRC violations	0	0	PASS
PG connectivity	OK	OK	PASS

2. Placement PASS/FAIL

- Checks placement legalization, cell density, HPWL.
- Ensures no overlapping cells and max displacement.
- Validates timing (WNS/TNS) after placement.
- PASS if all metrics are within limits.

Example 2 Placement stage check (Tcl)

```
set overlap [check_legalization -count]
set wns [get_attribute [get_timing_paths
-max_paths 1] slack]
if { $overlap == 0 && $wns >= -0.1 } {
  puts "PLACEMENT : PASS"
} else {
  puts "PLACEMENT : FAIL"
}
```

Placement Criteria (Example)

Metric	Target	Result	
Cell overlap	0	0	PASS
WNS (ns)	≥ -0.1	-0.05	PASS
TNS (ns)	≥ -5	-2.1	PASS
Max cell displacement	≤ 50 µm	32 µm	PASS

3. CTS PASS/FAIL

- Checks clock tree completion and clock QoR.
- Validates clock skew, latency and clock DRCs.
- Ensures no clock-related timing violations.
- PASS if all clock metrics meet thresholds.

Example 3 CTS stage check (Tcl)

```
set skew [get_attribute [get_clocks *] skew]
set drc_cnt [get_drc_violations -type clock -count]
if { $skew <= 50 && $drc_cnt == 0 } {
  puts "CTS : PASS"
} else {
  puts "CTS : FAIL"
}
```

CTS Criteria (Example)

Metric	Target	Result	
Clock skew (ps)	≤ 50	32	PASS
Clock latency (ns)	< 1.0	0.85	PASS
Clock DRCs	0	0	PASS
Clock violations	0	0	PASS

4. Routing PASS/FAIL

- Checks route completion, DRC, congestion and wirelength.
- Ensures all critical nets are routed.
- Validates no unconnected nets or antenna violations.
- PASS if routing meets quality criteria.

Example 4 Routing stage check (Tcl)

```
set drc_cnt [get_drc_violations -count]
set unrouted [get_nets -filter {is_routed == false}]
set congr [get_route_congestion]
if { $drc_cnt == 0 && [sizeof_collection $unrouted] == 0 } {
  puts "ROUTING : PASS"
} else {
  puts "ROUTING : FAIL"
}
```

Routing Criteria (Example)

Metric	Target		Result
DRC violations	0	0	PASS
Unrouted nets	0	0	PASS
Congestion (%)	≤ 10	8	PASS
Antenna violations	0	0	PASS
Total wirelength	< target 66.6mm		PASS

5. Signoff PASS/FAIL

- Checks final timing (WNS/TNS), hold, DRC, LVS, power.
- Ensures design meets signoff quality.
- Validates no open/short and antenna violations.
- PASS if all signoff metrics are clean.

Example 5 Signoff stage check (Tcl)

```
set wns [get_attribute [get_timing_paths -max_paths 1] slack]
set tns [get_attribute [get_timing_paths] total_negative_slack]
set drc_cnt [get_drc_violations -count]
set lvs_status [check_lvs]
if { $wns >= 0 && $tns >= 0 && $drc_cnt == 0 &&
  $lvs_status == "PASS" } {
  puts "SIGNOFF : PASS"
} else {
  puts "SIGNOFF : FAIL"
}
```

Signoff Criteria (Example)

Metric	Target	Result	
WNS (ns)	≥ 0	0.015	PASS
TNS (ns)	≥ 0	0	PASS
Hold violations	0	0	PASS
DRC violations	0	0	PASS
LVS status	PASS	PASS	PASS
Total Power (mW)	≤ target	85.4	PASS

6. GDSII Release Criteria

- All previous stages must be PASS.
- Final checks on database integrity, layer map and data preparation.
- Ensure no debug markers or test cells.
- Verify final outputs (GDSII/DEF/Netlist) are clean.
- PASS if all criteria are met, then release for tapeout.

Example 6 GDSII release check (Tcl)

```
if { $floorplan == "PASS" && $placement == "PASS" &&
  $cts == "PASS" && $routing == "PASS" &&
  $signoff == "PASS" } {
  puts "GDSII RELEASE : PASS"
  write_gds -o top_chip.gds
} else {
  puts "GDSII RELEASE : FAIL"
}
```

GDSII Release Criteria

Check Item	Status	Result
All stage gates	PASS	PASS
No debug cells	0	PASS
Layer map (LDR)	OK	PASS
GDSII file size	> 0	PASS
Final DRC clean	0	PASS
Tapeout ready	YES	PASS

Key Takeaways

- Automated stage gates ensure design quality at every stage.
- Define clear metrics and thresholds for PASS/FAIL.
- Use ICC2 reports and Tcl scripts to collect and evaluate data.
- Stop the flow if any stage fails and fix issues early.
- GDSII release only when all criteria are met.
- A reliable stage gate flow leads to successful tapeout.

Author : Konda Tharun Vrekumanda

"Quality at every stage, Success at tapeout !"

TCL HERO: ADVANCED ICC2 AUTOMATION

31. Automated Decision Making

- Automated decision making helps ICC2 take intelligent actions based on design metrics.
- Uses Tcl scripting to analyze reports and choose the right action automatically.
- Improves QoR, saves runtime and reduces manual effort.
- Combines multiple criteria like timing, congestion, DRC, CTS and power.
- A must-have capability for advanced ICC2 automation.

- Use thresholds based on project/rule of thumb.
- Analyze reports and make conditional decisions.
- Apply the right optimization commands automatically.
- Log decisions for traceability.
- Iterate until convergence is achieved.
- Helps in achieving better QoR with minimal manual effort.
- Essential for tapeout-ready, signoff clean design.

31.1 Automated Decision Making Overview

- Analyzes design metrics (timing, congestion, DRC, power, etc.).
- Compares against predefined thresholds.
- Automatically selects and runs the appropriate optimization commands.
- Iterates until the design meets the target QoR.
- Helps in faster convergence and consistent signoff quality.

1. Timing-based decisions

- Analyze WNS, TNS and critical path information.
- If timing violates the target, run optimization commands.
- Use cell sizing, buffer insertion, placement or routing optimization.
- Iterate until timing meets the threshold.

Example 1 Timing-based decision (Tcl)

```
set wns [get_attribute [get_timing_paths] wns]
if { $wns < -0.05 } {
  puts "WNS = $wns -> Running timing opt"
  optimize_design -timing
} else {
  puts "Timing met: WNS = $wns"
}
```

Timing Decision Thresholds

Metric	Threshold	Action
WNS	< -0.05	timing opt
TNS	< -1.0	timing opt
Critical path slack	< 0	size/buffer
Hold violations	> 0	hold fix

2. Congestion-based decisions

- Analyze routing congestion and hotspot information.
- If congestion is high, apply placement/routing strategies.
- Use cell spreading, re-placement, layer adjustment, or congestion-driven routing.
- Re-analyze until congestion is within limits.

Example 2 Congestion-based decision (Tcl)

```
set cong [get_attribute [get_congestion] max_util]
if { $cong > 0.75 } {
  puts "Congestion = $cong -> Re-placing"
  place_opt -congestion
} else {
  puts "Congestion OK: $cong"
}
```

Congestion Thresholds

Metric	Threshold	Action
Max util	> 75%	re-place
Routing HCs	> 70%	route_opt
Hotspot count	> 0	layer adj
Overflow	> 0	cell spread

3. DRC-based decisions

- Check design rule violations after each stage.
- If DRC violations exist, run auto-fix commands.
- Can use route_opt, legalize_placement or re-route.
- Iterate until DRC is clean.

Example 3 DRC-based decision (Tcl)

```
set drc_count [get_drc_violations -count]
if { $drc_count > 0 } {
  puts "DRC violations = $drc_count -> Fixing"
  route_opt -drc
} else {
  puts "DRC clean"
}
```

DRC Decision Table

Violation Count	Action
0	Clean
1 - 100	route_opt -drc
> 100	re-route / place_opt
Recurring	adjust rules / layers

4. CTS-based decisions

- Analyze clock tree quality after CTS.
- Check clock skew, latency and DRCs on clock nets.
- If metrics are out of range, re-run CTS with different options.
- Ensure clean and balanced clock tree.

Example 4 CTS-based decision (Tcl)

```
set skew [get_attribute [get_clocks *] skew]
if { $skew > 0.10 } {
  puts "Clock skew = $skew -> Re-run CTS"
  clock_opt -rebuild_clock_tree
} else {
  puts "Clock quality OK"
}
```

CTS Thresholds

Metric	Threshold	Action
Clock skew	> 0.10 ns	re-run CTS
Latency	> target	adjust CTS
Clock DRCs	> 0	fix DRC
Unbalanced	> 10%	re-CTS

5. Power-based decisions

- Analyze total and dynamic power.
- If power exceeds the target, apply low-power optimization.
- Use multi-Vt, cell downsizing, clock gating or power-driven placement.
- Ensure power is within budget without hurting timing.

Example 5 Power-based decision (Tcl)

```
set pwr [get_attribute [get_power] total_power]
if { $pwr > 100 } {
  puts "Power = $pwr -> Running low power opt"
  optimize_design -power
} else {
  puts "Power within limit: $pwr"
}
```

Power Decision Thresholds

Metric	Threshold	Action
Total Power	> 100 mW	power opt
Dynamic Power	> 80 mW	clock gating
Leakage Power	> 20 mW	multi-Vt
Power density	> limit	spread cells

6. Automatic optimization selection

- Based on the analysis, select the best optimization strategy.
- Can combine multiple criteria (timing, congestion, DRC, power).
- Use priority order or weighted scoring.
- Automatically run the selected ICC2 commands.

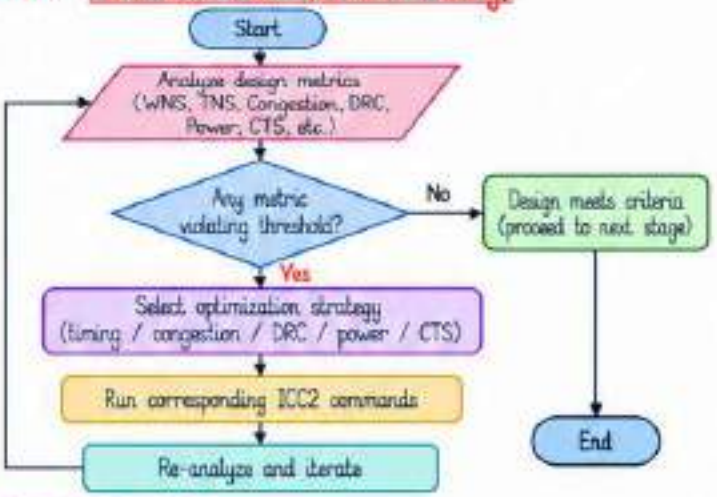
Example 6 Automatic optimization selection (Tcl)

```
if { $wns < -0.05 } {
  set opt_type "timing"
} elseif { $cong > 0.75 } {
  set opt_type "congestion"
} elseif { $drc_count > 0 } {
  set opt_type "drc"
} elseif { $pwr > 100 } {
  set opt_type "power"
} else {
  set opt_type "none"
}
puts "Selected optimization: $opt_type"
```

Optimization Priority (Example)

Priority	Condition	Optimization
1	Timing fail	optimize_design -timing
2	DRC violations	route_opt -drc
3	High congestion	place_opt -congestion
4	Power high	optimize_design -power
5	CTS issues	clock_opt
6	None	No action

7. Automated Flow (Decision Making)



Key Takeaways

- Automated decision making enables intelligent ICC2 flows.
- Use design metrics and thresholds to trigger optimizations.
- Handle timing, congestion, DRC, CTS and power automatically.
- Select the best optimization strategy based on priority or rules.
- Iterate until all criteria are met.
- Saves time, improves QoR and ensures a robust tapeout.

Author : Konda Tharun Vrekumanda

"Smarter decisions today, better silicon tomorrow!"

TCL HERO: ADVANCED ICC2 AUTOMATION

32. Multi-Scenario Automation

- Multi-scenario automation helps verify the design across different modes and corners in ICC2.
- It ensures correct timing, power and physical implementation under all operating conditions.
- Using Tcl, we can automate scenario creation, analysis and report generation.
- Essential for signoff-quality results and tapeout readiness.
- A must-have skill for advanced ICC2 users.

- Define modes, corners and scenarios using Tcl.
- Create and activate scenarios automatically.
- Run setup and hold analysis for all scenarios.
- Generate scenario-specific reports.
- Use loops to handle multiple scenarios efficiently.
- Compare results across scenarios.
- Helps ensure robust multi-corner multi-mode (MCMM) verification.
- Critical for signoff and silicon success.

32.1 Multi-Scenario Automation Overview

- ICC2 supports Multi-Corner Multi-Mode (MCMM) analysis to verify design across different operating conditions.
- Modes represent functional states (e.g., Functional, Test).
- Corners represent PVT conditions (e.g., SS, TT, FF).
- Scenarios are combinations of mode + corner.
- Automated Tcl scripts can create, run and report scenarios.
- Ensures correct setup/hold timing, power and design rule compliance across all conditions.

1. Modes

- Modes define the functional state of the design.
- Examples: functional mode, test mode, low-power mode.
- Each mode can have different constraints (clocks, IOs, etc.).
- Use create_mode and set_mode commands.

Example 1 Define modes (Tcl)

```
create_mode -name FUNC
create_mode -name TEST
create_mode -name LP
set_mode FUNC
```

Sample Mode Definition

Mode Name	Description
FUNC	Normal functional mode
TEST	Scan/test mode
LP	Low power mode

2. Corners

- Corners represent PVT (Process, Voltage, Temperature) conditions.
- Examples: SS (slow), TT (typical), FF (fast).
- Use create_corner to define corners with libraries.
- Each corner has different timing and power characteristics.

Example 2 Define corners (Tcl)

```
create_corner -name SS \
    -process slow \
    -voltage 0.72 \
    -temperature 125
create_corner -name TT \
    -process typical \
    -voltage 0.80 \
    -temperature 25
create_corner -name FF \
    -process fast \
    -voltage 0.88 \
    -temperature -40
```

Sample Corners

Corner	Process	Voltage (V)	Temp (°C)
SS	Slow	0.72	125
TT	Typical	0.80	25
FF	Fast	0.88	-40

3. Scenarios

- A scenario is a combination of a mode and a corner.
- Each scenario represents a unique operating condition.
- Use create_scenario to define scenarios.
- Activate scenarios for MCMM analysis.

Example 3 Create scenarios (Tcl)

```
create_scenario -name FUNC_SS \
    -mode FUNC -corner SS
create_scenario -name FUNC_TT \
    -mode FUNC -corner TT
create_scenario -name TEST_FF \
    -mode TEST -corner FF
create_scenario -name LP_TT \
    -mode LP -corner TT
```

Sample Scenarios

Scenario	Mode	Corner
FUNC_SS	FUNC	SS
FUNC_TT	FUNC	TT
FUNC_FF	FUNC	FF
TEST_TT	TEST	TT
LP_TT	LP	TT

4. Scenario Loops

- Use Tcl loops to run analysis and reports for all scenarios.
- Avoids manual effort and ensures consistency.
- Can run timing, DRC, power and QoR reports automatically.

Example 4 Loop over scenarios (Tcl)

```
set scen_list [get_scenarios]
foreach scen $scen_list {
    current_scenario $scen
    puts "Running analysis for $scen"
    update_timing
}
```

Useful Commands

get_modes
get_corners
get_scenarios
current_scenario
report_scenarios

5. Setup Analysis

- Setup analysis checks for maximum delay violations.
- Run for all scenarios automatically.
- Generate setup timing reports per scenario.
- Ensure WNS and TNS meet targets.

Example 5 Run setup analysis (Tcl)

```
foreach scen [get_scenarios] {
    current_scenario $scen
    update_timing
    report_timing -max_paths 100 \
        -delay_type max \
        -outfile reports/${scen}_setup.rpt
}
```

Sample Setup Report (Summary)

Scenario	WNS (ns)	TNS (ns)	Status
FUNC_SS	-0.012	-1.25	FAIL
FUNC_TT	0.021	0.00	PASS
FUNC_FF	0.045	0.00	PASS
TEST_TT	-0.005	-0.32	FAIL
LP_TT	0.018	0.00	PASS

6. Hold Analysis

- Hold analysis checks for minimum delay violations.
- Run for all scenarios automatically.
- Generate hold timing reports per scenario.
- Ensure no hold violations exist.

Example 6 Run hold analysis (Tcl)

```
foreach scen [get_scenarios] {
    current_scenario $scen
    update_timing
    report_timing -max_paths 100 \
        -delay_type min \
        -outfile reports/${scen}_hold.rpt
}
```

Sample Hold Report (Summary)

Scenario	WNS (ns)	TNS (ns)	Status
FUNC_SS	0.003	0.00	PASS
FUNC_TT	0.002	0.00	PASS
FUNC_FF	-0.001	-0.08	FAIL
TEST_TT	0.004	0.00	PASS
LP_TT	0.001	0.00	PASS

7. Scenario-specific Reports

- Generate timing, power, DRC and QoR reports for each scenario.
- Use consistent naming for easy tracking.
- Reports can be saved in separate directories.
- Helps compare results across modes and corners.

Example 7 Generate reports for all scenarios (Tcl)

```
file mkdir reports
foreach scen [get_scenarios] {
    current_scenario $scen
    file mkdir reports/$scen
    report_timing -summary \
        -outfile reports/$scen/${scen}_timing.rpt
    report_power -summary \
        -outfile reports/$scen/${scen}_power.rpt
    report_qor \
        -outfile reports/$scen/${scen}_qor.rpt
    report_design_rule -summary \
        -outfile reports/$scen/${scen}_drc.rpt
}
```

Report Types per Scenario

Report Type	File Name (example)
Timing (Setup)	FUNC_TT_timing.rpt
Timing (Hold)	FUNC_TT_hold.rpt
Power	FUNC_TT_power.rpt
DRC	FUNC_TT_drc.rpt
QoR	FUNC_TT_qor.rpt

Scenario Flow



Key Takeaways

- Multi-scenario automation ensures robust and reliable design.
- Use Tcl to create modes, corners and scenarios.
- Run setup and hold analysis for all scenarios automatically.
- Generate scenario-specific timing, power, DRC and QoR reports.
- Use loops to save time and reduce manual effort.
- Compare results across scenarios to meet signoff criteria.
- Essential for advanced ICC2 users and successful tapeout.

Author : Konda Tharun Vrekumada

"One design, Many conditions. Zero surprises !"

33. Configuration-Driven Tcl

- Configuration-driven Tcl makes ICC2 flow flexible and reusable.
- All design parameters are defined in external config files.
- Same flow can be used for multiple designs by changing config.
- Improves maintainability, scalability and team collaboration.
- Use Tcl arrays, dictionaries or simple key-value files.
- Keeps technology, library paths and options separate from main flow.
- A must-have methodology for advanced ICC2 automation.

- Separate configuration file from flow script.
- Use meaningful variables and naming conventions.
- Support multiple technology nodes and libraries.
- Easily modify paths, options and design settings.
- Helps run the same flow for different projects.
- Store all reports in organized directories.
- Enables reproducible and portable ICC2 flows.
- Essential for professional and large-scale designs.

33.1 Configuration-Driven Tcl Overview

- Uses external configuration files to define all design parameters.
- Separates flow logic from design-specific settings.
- Makes the flow reusable, portable and easy to maintain.
- Supports different designs, libraries and technology nodes.
- Key parameters: design variables, library variables, tech variables, input files, report/output directories.
- Improves scalability and reduces manual changes.
- Common formats: Tcl, .cfg (key-value), .tcl, .yaml, .json, .txt.

1. Configuration files

- Stores all design settings in a separate file.
- Can be simple Tcl key-value pairs or a structured format.
- Easily editable for different projects.
- Loaded at runtime using source or custom parser.

Example 1 — Sample configuration file (config.tcl)

```
# Design Configuration File
set design_name     my_chip
set top_module      top
set tech_node       7nm
set run_mode        full
set proj_dir        /proj/my_chip
set report_dir      $proj_dir /reports
set output_dir      $proj_dir /results
```

File Format Options

- Tcl script (.tcl)
- Key-value file (.cfg)
- JSON (.json)
- YAML (.yaml)
- Environment variables

2. Design variables

- Define design specific parameters.
- Includes design name, top module, block type, constraints.
- Can be changed per project without modifying main flow.

Example 2 — Design variables in config

```
set design_name     my_soc
set top_module      soc_top
set block_type      block
set target_frequency  1000   ;# MHz
set core_util       0.7
```

Common Design Variables

- design_name
- top_module
- block_type (chip/block)
- target_frequency
- core_utilization
- flow_mode (full/incremental)

3. Library variables

- Define technology libraries used in the flow.
- Include timing, link, and physical libraries.
- Can support multiple library sets for different corners.

Example 3 — Library variables

```
set search_path    /libs
set target_library \
    [list slow.lib typical.lib fast.lib]
set link_library   * $target_library
set synthetic_library  dw_foundation.sldb
```

Library Variable Examples

- search_path
- target_library
- link_library
- synthetic_library
- symbol_library
- tluplus_files

4. Technology variables

- Define technology specific settings.
- Includes tech files, layer map, RC tech, manufacturing node.
- Helps in multi-node support.

Example 4 — Technology variables

```
set tech_node       7nm
set tech_file       /tech/7nm.tf
set layer_map       /tech/7nm_layer.map
set tlup_file       /tech/7nm.tlup
set max_metal_layer   12
```

Tech Variable Examples

- tech_node
- tech_file
- layer_map
- tlup_file
- max_metal_layer
- min_metal_layer

5. Input files

- Specify all input files required for the flow.
- Includes RTL, netlist, constraints, floorplan, etc.
- Can be changed based on project or stage.

Example 5 — Input file variables

```
set rtl_files     [list rtl/top.v rtl/sub.v]
set netlist_file  netlist/design.v
set sdc_file      constraints/top.sdc
set floorplan_def  inputs/floorplan.def
set lef_files     [list stdcell.lef io.lef]
set tf_files      [list tech.tf]
```

Common Input Files

- RTL files
- Gate-level netlist
- SDC constraints
- LEF/DEF files
- Technology files (TF)
- Floorplan files
- UPF/CPF (for low power)

6. Report directories

- Specify directories for different types of reports.
- Keeps all reports organized and easy to find.
- Separate directories for timing, DRC, power, etc.

Example 6 — Report directory variables

```
set report_dir      $proj_dir/reports
set timing_dir      $report_dir/timing
set drc_dir         $report_dir/drc
set power_dir       $report_dir/power
set cong_dir        $report_dir/congestion
set log_dir         $report_dir/logs
```

Report Directory Structure

```
reports/
├── timing/
├── drc/
├── power/
├── congestion/
├── cts/
├── logs/
└── qor/
```

7. Output directories

- Specify directories for all output files.
- Includes databases, netlists, parasitics, GDSII, etc.
- Helps in maintaining a clean and structured flow.

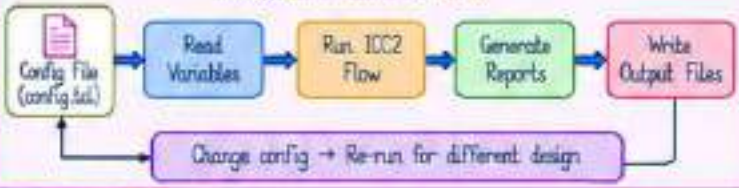
Example 7 — Output directory variables

```
set output_dir      $proj_dir/results
set db_dir          $output_dir/db
set netlist_dir     $output_dir/netlist
set gds_dir         $output_dir/gds
set spef_dir        $output_dir/spef
set sdf_dir         $output_dir/sdf
```

Output Directory Structure

```
results/
├── db/
├── netlist/
├── gds/
├── spef/
├── sdf/
├── reports/
└── final/
```

Configuration-Driven Flow



Key Takeaways

- Use configuration files to separate flow and design parameters.
- Define design, library, technology and file variables clearly.
- Maintain separate report and output directories.
- Makes the flow reusable, scalable and easy to maintain.
- Supports multiple designs, nodes and scenarios.
- Essential for professional ICC2 automation and large projects.

Author : Kanda Tharun Vrekumanda

"Configure once, run anywhere !"

34. Logging Framework

A robust logging framework helps to track the flow execution, debug issues and maintain a clear history.
Provides consistent messaging format (INFO, WARNING, ERROR, etc).
Helps in automated run monitoring and post-analysis.
Useful for large multi-stage ICC2 flows and regressions.
Makes it easy to identify failures and measure runtime.
An essential utility for professional ICC2 automation.

Use a common logging procedure for all scripts.
Include timestamp, stage and message level.
Write logs to both console and log file.
Track all important commands and results.
Helps in debugging and root cause analysis.
Can be extended to log runtime, memory, QoR metrics.
A must-have framework for reliable and scalable ICC2 automation.

34.1 Logging Framework Overview

- A logging framework provides a standardized way to record messages during ICC2 flow execution.
- Supports multiple message levels (INFO, WARNING, ERROR, PASS, FAIL).
- Includes timestamp, stage name and message details.
- Logs can be written to console and to a log file.
- Helps in monitoring, debugging and regression analysis.
- Can capture executed commands and their status.

1. Log Message Levels

- Different message levels are used to classify log messages.
- Helps in quickly identifying the importance and severity.
- Common levels: INFO, WARNING, ERROR, PASS, FAIL.
- Each level can have a different color for easy viewing.

Log Levels and Meaning

Level	Color (Example)	Meaning
INFO	Blue	General information message
WARNING	Orange	Potential issue, non-critical
ERROR	Red	Critical error, flow may stop
PASS	Green	Stage or check passed
FAIL	Magenta/Red	Stage or check failed

2. Timestamp Logging

- Each log message should include a timestamp.
- Helps to track the exact time of events.
- Useful for performance analysis and debugging.
- Format: YYYY-MM-DD HH:MM:SS

Example 1 Get timestamp and print log

```
set ts [clock format [clock seconds]
 -format "%Y-%m-%d %H:%M:%S"]
puts "$ts : INFO : Flow started"
```

Sample Output

```
2025-09-04 21:15:23 :
INFO : Flow started
2025-09-04 21:16:10 :
WARNING : High congestion
2025-09-04 21:20:05 :
ERROR : DRC violations found
```

3. Stage Logging

- Log messages should include the current flow stage.
- Helps to understand which stage is running.
- Examples: FLOORPLAN, PLACEMENT, CTS, ROUTING, SIGNOFF.

Example 2 Stage based logging

```
set stage "PLACEMENT"
set ts [clock format [clock seconds] \
  -format "%Y-%m-%d %H:%M:%S"]
puts "$ts : INFO : [$stage] : Starting placement"
puts "$ts : PASS : [$stage] : Placement completed"
```

Sample Log Messages

```
2025-09-04 22:01:10 : INFO : PLACEMENT : Starting placement
2025-09-04 22:35:42 : PASS : PLACEMENT : Placement completed
```

4. Command Logging

- Log all important ICC2 commands before execution.
- Capture the command status (SUCCESS/FAIL).
- Helps in replaying and debugging failed commands.

Example 3 Command logging

```
proc run_cmd {cmd} {
  set ts [clock format [clock seconds] -format "%Y-%m-%d %H:%M:%S"]
  puts "$ts : CMD  : $cmd"
  if {[catch {eval $cmd} res]} {
    puts "$ts : ERROR: Command failed: $res"
    return -code error $res
  } else {
    puts "$ts : INFO : Command completed"
  }
}
```

5. Logging Procedure

- Create a common logging procedure for all scripts.
- Supports different message levels and automatic formatting.
- Writes to both console and log file.
- Can include stage name and optional message.

Example 4 Logging procedure (Tcl)

```
set log_file "icc2_flow.log"
proc log_msg {level stage message} {
  set ts [clock format [clock seconds]
  -format "%Y-%m-%d %H:%M:%S"]
  set msg "$ts : $level : $stage : $message"
  puts $msg
  set fp [open $::log_file a]
  puts $fp $msg
  close $fp
}
```

Usage Example

```
log_msg INFO "INIT" "Flow started"
log_msg WARNING "FLOORPLAN" "High utilization"
log_msg ERROR "ROUTING" "DRC violations found"
log_msg PASS "SIGNOFF" "Timing clean"
log_msg FAIL "SIGNOFF" "Hold violations found"
```

6. Log File Structure

- Maintain a separate log file for each run. (with date/time or run name).
- Logs can be stored in a dedicated log directory.
- Keep the log file organized and easy to parse.

Example 5 Log file organization

```
set run_name "mychip_r1"
set log_dir "./logs"
file mkdir $log_dir
set log_file "$log_dir/${run_name}.log"
log_msg INFO "INIT" "Log file created at $log_file"
```

Sample Log Directory

```
logs/
├── mychip_r1.log
├── mychip_r2.log
├── mychip_r3.log
├── signoff.log
└── regression.log
```

7. PASS / FAIL Logging

- Use PASS and FAIL messages to indicate stage/check results.
- Helps in quick identification of flow status.
- Can be used in automated regression and reporting.

Example 6 PASS / FAIL Logging (Tcl)

```
if {$drc_count == 0} {
  log_msg PASS "DRC" "No DRC violations"
} else {
  log_msg FAIL "DRC" "$drc_count DRC violations found"
}
```

Sample Output

```
2025-09-04 23:10:15 :
PASS : DRC :
No DRC violations
2025-09-04 23:12:40 :
FAIL : DRC :
52 DRC violations found
```

8. Complete Logging Flow (Example)

- Initialize log file and log directory.
- Log each stage and important commands.
- Capture timestamps and status messages.
- Use consistent format throughout the flow.

Example 7 Complete flow logging (Tcl)

```
set run_name "mychip_r1"
set log_dir "./logs"
file mkdir $log_dir
set log_file "$log_dir/${run_name}.log"

log_msg INFO "INIT" "Flow started"
run_cmd "create_mw_lib mychip -technology ./tf"
log_msg PASS "INIT" "Library created successfully"

log_msg INFO "FLOORPLAN" "Starting floorplan"
run_cmd "create_floorplan"
log_msg PASS "FLOORPLAN" "Floorplan completed"

log_msg INFO "SIGNOFF" "Running final checks"
run_cmd "check_design"
log_msg PASS "SIGNOFF" "All checks clean"

log_msg INFO "FLOW" "Flow completed successfully"
```

Sample Log File (Excerpt)

```
2025-09-04 20:10:01 : INFO : INIT : Flow started
2025-09-04 20:10:05 : CMD  : create_mw_lib mychip
2025-09-04 20:10:08 : INFO : Command completed
2025-09-04 20:25:20 : INFO : FLOORPLAN : Starting floorplan
2025-09-04 20:25:10 : PASS : FLOORPLAN : Completed
2025-09-04 21:05:33 : INFO : SIGNOFF : Running final checks
2025-09-04 21:10:15 : PASS : SIGNOFF : All checks clean
2025-09-04 21:10:16 : INFO : FLOW : Flow completed successfully
```

Key Takeaways

- A logging framework provides clear and consistent messaging.
- Use different log levels (INFO, WARNING, ERROR, PASS, FAIL).
- Include timestamp and stage name in all logs.
- Log important commands and their status.
- Maintain organized log files in a dedicated directory.
- Helps in debugging, regression and flow monitoring.
- Essential for professional and reliable ICC2 automation.

Author : Konda Tharun Vrekumuda

"Good logs today, easier debug tomorrow !"

35. Checkpoint Automation

- Checkpoint automation helps save design state at various stages of ICC2 flow.
- Enables easy recovery, debugging and flow continuation.
- Supports block, library, DEF, netlist and GDSII generation.
- Stage-wise checkpoints ensure design safety.
- Essential for long running flows and tapeout readiness.
- A must-have skill for advanced ICC2 automation.

- Use meaningful names for checkpoints with stage and time.
- Save checkpoints at major milestones.
- Generate all required design outputs automatically.
- Keep multiple recovery checkpoints.
- Verify saved data by re-loading.
- Helps in flow restart, ECO, debug and comparison.
- Ensures a robust and reliable tapeout flow.

35.1 Checkpoint Automation Overview

- Checkpoint automation saves the design state, data and output files at different stages of the flow.
- Includes save_block, save_lib, DEF, netlist and GDSII generation.
- Stage checkpoints allow flow restart from any stage.
- Recovery checkpoints help in case of failure.
- Ensures data integrity, reduces re-run time and improves productivity.
- Can be implemented using Tcl scripts for fully automated flow.

1. save_block (Design Checkpoint)

- save_block saves the current design block state.
- Includes placement, routing, constraints and design data.
- Can be reloaded using open_block or restore.
- Useful for stage and recovery checkpoints.

Example 1 — save_block (Tcl)

```
set block_name "mychip_n1"
save_block -as $block_name

# or save to specific directory
save_block -as $block_name \
  -output_dir /checkpoints/
```

Key Options	
-as <name>	Checkpoint name
-output_dir	Directory to save
-force	Overwrite existing
-compress	Compress checkpoint
-include_physical	Include physical data

2. save_lib (Library Checkpoint)

- save_lib saves the design library with current state.
- Useful when working with custom/tweaked libraries.
- Includes timing, physical and design data.
- Can be re-used in future runs for consistent results.

Example 2 — save_lib (Tcl)

```
set lib_name "mychip_lib"
save_lib -as $lib_name
save_lib -as $lib_name \
  -output_dir ./libs/
  -compress
```

Key Options	
-as <name>	Library name
-output_dir	Directory to save
-compress	Compress library
-force	Overwrite
-include_timing	Include timing data

3. DEF Generation

- Generates DEF file for the current block.
- Useful for interoperability, analysis and signoff.
- Contains placement, routing and physical data.
- Can be generated at different stages (e.g., post-place, post-route).

Example 3 — write_def (Tcl)

```
set def_file "mychip.def"
write_def -output $def_file

# with options
write_def -output $def_file \
  -version 5.8 -compress
```

Key Options	
-output <file>	Output DEF file
-version <v>	DEF version
-compress	Compress file
-include_nets	Include net info
-include_components	Include cells

4. Netlist Generation

- Generates gate-level netlist from the current design.
- Useful for signoff, LVS and tapeout.
- Can include optional information like hierarchy and attributes.
- Can be generated at multiple stages.

Example 4 — write_verilog (Tcl)

```
set netlist "mychip.v"
write_verilog -output $netlist
# with options
write_verilog -output $netlist \
  -hierarchy -include_attributes
```

Key Options	
-output <file>	Netlist file
-hierarchy	Include hierarchy
-include_attributes	Include attributes
-exclude_cells	Exclude specific cells
-format <v/sv>	Verilog/SystemVerilog

5. GDSII Generation

- Generates final GDSII file for tapeout.
- Includes all physical design data.
- Can be generated after signoff checks are clean.
- Usually generated once, after final verification.

Example 5 — write_gds (Tcl)

```
set gds_file "mychip.gds"
write_gds -output $gds_file
# with options
write_gds -output $gds_file \
  -hierarchy -compress -map_file techmap
```

Key Options	
-output <file>	GDS file name
-hierarchy	Include hierarchy
-compress	Compress file
-map_file <file>	Layer mapping file
-lib_cell_view	Include library cells

6. Stage Checkpoints

- Save checkpoints at major flow stages.
- Helps in restarting the flow from a specific stage.
- Common stages: floorplan, placement, CTS, routing, signoff.
- Use consistent naming convention with stage and timestamp.

Example 6 — Stage Checkpoint (Tcl)

```
set stage "place"
set block_name "mychip_${stage}"
set time_stamp [clock format [clock seconds] \
  -format "%Y%m%d_%H%M%S"]
set chk_name "${block_name}_${time_stamp}"
save_block -as $chk_name -output_dir /checkpoints/
```

Common Stage Names:

- floorplan
- placement
- cts
- routing
- post_route
- signoff
- final

7. Recovery Checkpoints

- Save additional checkpoints for recovery.
- Helps in case of tool crash, flow failure or unexpected issues.
- Can be triggered automatically on error.
- Keep multiple recovery points to avoid data loss.

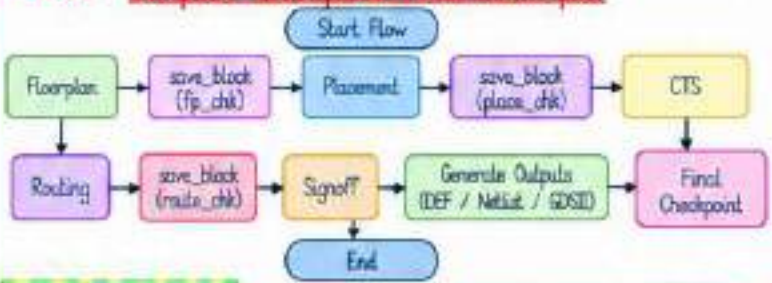
Example 7 — Recovery checkpoint (Tcl)

```
if { [catch { save_block -as "recovery" } err] } {
  puts "ERROR: Checkpoint failed: $err"
  set time_stamp [clock format [clock seconds] \
    -format "%Y%m%d_%H%M%S"]
  save_block -as "recovery_${time_stamp}" \
    -output_dir /recovery/
  puts "Recovery checkpoint saved!"
}
```

Recovery Strategies

- Automatic save on error
- Periodic checkpoints
- Save before major steps
- Keep multiple generations
- Store in separate directory
- Verify checkpoint by reload
- Maintain disk space

8. Complete Checkpoint Flow (Example)



Key Takeaways

- [ ] Checkpoint automation ensures safe and reliable ICC2 flow.
- [ ] Use save_block, save_lib, DEF, netlist and GDSII generation.
- [ ] Save stage checkpoints at major milestones.
- [ ] Maintain recovery checkpoints for error handling.
- [ ] Use consistent naming and timestamp for all checkpoints.
- [ ] Verify saved data by reloading.
- [ ] Essential for professional tapeout-ready flows.

Author : Konda Tharun Vrekumada

"Save your progress, secure your success !"

TCL HERO: ADVANCED ICC2 AUTOMATION

36. Automated Report Management

- Automated report management helps organize, track and analyze all reports generated during ICC2 flow.
- Creates a structured directory, naming and archiving system.
- Generates stage summaries and compares QoR across runs.
- Helps in quick debug, historical tracking and signoff readiness.
- Saves time and improves productivity.
- Essential for professional and tapeout-ready ICC2 automation.

- Maintain consistent directory structure for all rpts.
- Use meaningful names with stage, block and timestamp.
- Archive old reports to save space.
- Generate summary reports for each stage.
- Compare QoR metrics across multiple runs.
- Keep historical data for trend analysis.
- Helps in tracking improvements and regressions.
- A must-have framework for advanced ICC2 users.

36.1 Automated Report Management Overview

- Automated report management handles all reports generated during ICC2 flow in a systematic way.
- Covers directory structure, naming, archiving and comparison.
- Generates stage-wise summaries and tracks QoR over time.
- Helps in debugging, regression analysis and signoff.
- Ensures organized and clean project data.
- Can be implemented using Tcl scripts for full automation.

1. Report directories

- Create separate directories for each stage and report type.
- Keeps reports organized and easy to access.
- Can include sub-directories for timing, DRC, power, etc.
- Use relative or absolute paths.

Example 1 Create report directory structure (Tcl)

```
set rep_dir "./reports"
file mkdir $rep_dir
file mkdir $rep_dir/timing
file mkdir $rep_dir/drc
file mkdir $rep_dir/power
file mkdir $rep_dir/qor
file mkdir $rep_dir/summary
```

Directory Structure (Example)

```
reports/
├── timing/
├── drc/
├── power/
├── qor/
├── summary/
└── archive/
```

2. Report naming

- Use consistent and meaningful naming convention.
- Include block name, stage, type and timestamp.
- Helps in easy identification and sorting.

Example 2 Report naming convention (Tcl)

```
set block_name "mychip"
set stage "place"
set rpt_type "timing"
set time_stamp [clock format [clock seconds] \
        -format "%Y%m%d_%H%M%S"]
set report_name "${block_name}_${stage}_${rpt_type}_${time_stamp}.rpt"
```

Report Naming Format (Example)

Component	Example	Description
Block name	mychip	Design/block name
Stage	place	Flow stage (floorplan/place/CTS/route/signoff)
Report type	timing	Type of report (timing/DRC/power/qor)
Timestamp	20250904_221030	Date and time
Full name	mychip_place_timing_20250904_221030.rpt	Final report name

3. Report archiving

- Archive old reports to save disk space.
- Keep only latest N runs in active directory.
- Move older reports to an archive folder.
- Helps in maintaining a clean environment.

Example 3 Archive old reports (Tcl)

```
set archive_dir "$rep_dir/archive"
file mkdir $archive_dir

# Move reports older than 7 days
exec find $rep_dir -type f -mtime +7 -exec mv {} \
    $archive_dir/ \\;
```

Archive Strategy

- Keep latest N runs
- Move old reports to archive/
- Can also compress (.tar.gz)
- Optional: auto-delete after certain days

4. Stage summaries

- Generate a summary report for each stage.
- Include key QoR metrics (WNS, TNS, DRC, Power, etc.).
- Helps in quick overview of stage results.
- Can be generated automatically after each stage.

Example 4 Generate stage summary (Tcl)

```
set summary_file "$rep_dir/summary/${block_name}_${stage}_summary.rpt"
redirect -file $summary_file {
    report_timing_summary
    report_qor
    report_power_summary
    report_congestion
    report_drc -summary
}
```

Stage Summary Report (Example)

Metric	Value	Status
WNS (ns)	-0.012	PASS
TNS (ns)	-1.25	FAIL
DRC Violations	0	PASS
Total Power (mW)	85.6	PASS
Cell Area (um²)	1.25e6	-
Congestion (%)	72	PASS

5. QoR comparison

- Compare QoR metrics across different runs or stages.
- Helps in tracking improvements and regressions.
- Can be done using custom Tcl scripts.

Example 5 QoR comparison (Tcl)

```
set rpt1 "reports/run1/summary/summary.rpt"
set rpt2 "reports/run2/summary/summary.rpt"
# Extract WNS from reports (example using grep)
set wns1 [exec grep "WNS" $rpt1 | awk '{print $3}']
set wns2 [exec grep "WNS" $rpt2 | awk '{print $3}']
puts "Run1 WNS: $wns1"
puts "Run2 WNS: $wns2"
```

QoR Comparison (Example)

Run	WNS (ns)	TNS (ns)	Power (mW)	DRC	Status
run1	-0.045	-2.10	92.3	12	FAIL
run2	-0.012	-0.85	88.7	0	PASS
run3	-0.018	-1.20	86.5	0	PASS

6. Historical comparison

- Maintain historical data of all runs.
- Track QoR trends over time (WNS, TNS, Power, Area, etc.).
- Helps in regression analysis and long-term tracking.
- Can be exported to CSV/Excel for visualization.

Example 6 Maintain historical database (Tcl)

```
set hist_file "$rep_dir/summary/qor_history.csv"
if { !([file exists $hist_file]) {
    puts $hist_file "Run,Stage,WNS,TNS,Power,DRC,Date"
}
set date [clock format [clock seconds] -format "%Y-%m-%d"]
puts $hist_file "run3,place,-0.012,-0.85,88.7,0,$date"
```

Historical QoR Data (CSV)

Run	Stage	WNS	TNS	Power	DRC	Date
run1	place	-0.045	-2.10	92.3	12	2025-09-01
run2	place	-0.028	-1.50	90.1	5	2025-09-02
run3	place	-0.012	-0.85	88.7	0	2025-09-04

Key Takeaways

- Use structured directories for all reports.
- Follow consistent naming convention with stage and timestamp.
- Archive old reports to maintain clean environment.
- Generate stage summaries with key QoR metrics.
- Compare QoR across runs and maintain historical data.
- Automated report management ensures better tracking, debug and signoff readiness.
- Essential for professional ICC2 automation.

Author : Konda Tharun Vrekumuda

"Well organized reports lead to well designed chips !"

TCL HERO: ADVANCED ICC2 AUTOMATION

37. Multi-Block Automation

- Multi-block automation enables efficient handling of hierarchical designs in ICC2.
- Use a common framework to run multiple blocks automatically.
- Each block can have its own configuration and scripts.
- Helps in large SoC projects with block-level and top-level flows.
- Improves productivity, consistency and reusability.
- Essential for real-world multi-block SoC designs.

- Supports multiple blocks with minimal manual effort.
- Use block-specific configuration files.
- Run block-level and hierarchical flows.
- Enable reuse of scripts and common procedures.
- Generate individual and consolidated reports.
- Maintain consistent flow across all blocks.
- A scalable approach for complex SoC designs.

37.1 Multi-Block Automation Overview

- Multi-block automation handles multiple design blocks efficiently.
- Supports hierarchical design flow (block → sub-block → top).
- Uses configuration files for each block.
- Block-specific scripts can be run using a common framework.
- Ensures consistency, scalability and easy maintenance.
- Useful for SoC and large hierarchical designs.

1. Block configuration

- Each block has its own configuration file (Tcl/.cfg/.yaml).
- Contains block name, paths, libraries, constraints, modes, etc.
- Allows flexibility to run different blocks with different settings.
- Can be simple key-value pairs or structured format.
- Helps in reusability and easy modification.

Example 1 Block configuration file (block1.cfg)

```
set block_name      cpu
set design_dir      /proj/blocks/cpu
set lib_dir         /proj/libs
set tech_file       /proj/tech/tech.tf
set constraint_file /proj/blocks/cpu/constraints.sdc
set mode_list       {FUNC TEST}
set corner_list     {SS TT FF}
set output_dir      /proj/blocks/cpu/results
```

Common Parameters

- block_name
- design_dir
- lib_dir
- tech_file
- constraint_file
- mode_list
- corner_list
- output_dir
- reports_dir
- run_options

2. Multiple blocks

- Framework should support running multiple blocks in one go.
- Blocks can be independent or hierarchical.
- Use a list of blocks from a master configuration file.
- Run blocks sequentially or in parallel (if resources available).
- Generate individual block reports and a consolidated summary.

Example 2 Block list file (blocks.tcl)

```
set block_list {
    cpu
    gpu
    dsp
    peripherals
    top
}
```

Example Block Details

Block	Type	Description
cpu	Leaf Block	CPU core block
gpu	Leaf Block	GPU core block
dsp	Leaf Block	DSP accelerator
peripherals	Leaf Block	IO and interface
top	Top Block	SoC top integrating all blocks

3. Block-specific scripts

- Each block can have its own script or reuse common scripts.
- Block-specific script can handle unique constraints or flow steps.
- Common procedures can be sourced from a shared library.
- Helps to handle special requirements for certain blocks.

Example 3 Block-specific script (run_block.tcl)

```
proc run_block {block} {
    source ./config/${block}.cfg
    puts "[clock format [clock seconds]] : Running block $block"
    source ./scripts/common_flow.tcl
    source ./scripts/${block}_flow.tcl    ;# block specific steps
    puts "[clock format [clock seconds]] : Completed block $block"
}
```

Common vs Block-Specific Scripts

Script Type	Purpose	Example
Common script	Generic flow (setup, place, route, report)	common_flow.tcl
Block-specific script	Block unique steps / constraints	cpu_flow.tcl
Utility script	Helper functions (logging, checks, etc.)	utils.tcl

4. Reusable framework

- Use modular Tcl procedures and functions.
- Common framework can be reused across projects.
- Supports easy addition/removal of blocks.
- Centralized logging, error handling and report generation.
- Makes the flow scalable and maintainable.

Example 4 Reusable framework structure

```
icc2_flow/
  config/
    blocks.tcl
    cpu.cfg
    gpu.cfg
    dsp.cfg
    top.cfg
  scripts/
    common_flow.tcl
    cpu_flow.tcl
    gpu_flow.tcl
    dsp_flow.tcl
    top_flow.tcl
    utils.tcl
  run/
    run_all_blocks.tcl
  reports/
```

Key Features

- Modular structure
- Configuration driven
- Reusable procedures
- Supports multiple blocks
- Easy to maintain
- Scalable for SoC designs
- Consistent flow and reporting

5. Hierarchical design automation

- Supports hierarchical flow: block-level and top level integration.
- Run block-level flows first, then integrate at top level.
- Generates block abstracts (netlist/DEF) for top-level use.
- Handles dependencies between blocks.
- Useful for large SoC designs with multiple IPs and hierarchical levels.

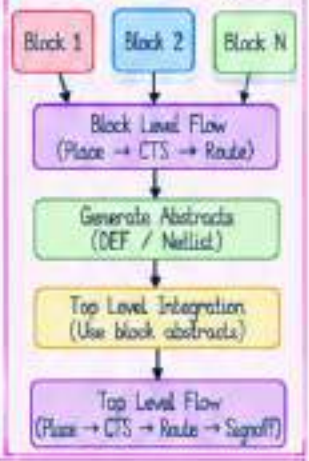
Example 5 Hierarchical flow (Tcl)

```
# Run block level flows
foreach blk $block_list {
   run_block $blk
}

# Generate abstracts for top
foreach blk $block_list {
   save_block -as ${blk}_abstract \
     -output_dir /abstracts/${blk}
}

# Run top level integration
run_block top
```

Hierarchical Flow Diagram



Generated Outputs

Stage	Output Files	Used For
Block Level	Netlist, DEF, Reports	Top level integration
Abstract	{blk}_abstract.def/netlist	Import to top
Top Level	Final DEF, GDSII, Reports	Tapeout

Key Takeaways

- Use configuration files for each block.
- Support multiple blocks with a common framework.
- Enable block-specific scripts for unique requirements.
- Reuse scripts and procedures for scalability.
- Automate hierarchical design flow (block → top).
- Generate individual and consolidated reports.
- Improves productivity and ensures consistent flow.
- Essential for complex SoC and multi-block designs.

Author : Konda Tharun Vrekumada

"Build blocks, automate flows, create bigger possibilities !"

38. Tcl Performance Optimization

- Efficient Tcl scripting is critical for large ICC2 designs.
- Optimize collections, queries and loops to reduce runtime.
- Avoid unnecessary operations and repeated database access.
- Use proper filtering and caching techniques.
- Write scalable and maintainable scripts for production use.
- A well-optimized script saves hours of runtime and resources.
- Write efficient, scalable and optimized Tcl scripts.
- Use collections wisely and avoid excessive queries.
- Cache results and reuse them wherever possible.
- Minimize loops and nested iterations.
- Use efficient filtering instead of multiple queries.
- Essential for large designs with millions of objects.
- Good scripting practice improves runtime and productivity.
- A must-know skill for advanced ICC2 automation.

38.1 Tcl Performance Optimization Overview

- Optimized Tcl scripts run faster and use fewer resources.
- Key factors: collections, queries, loops, filtering and caching.
- Reduces runtime for large designs (millions of instances/nets).
- Improves productivity in daily design automation flow.
- Helps in building scalable and reusable frameworks.
- Essential for production-level ICC2 flows.

1. Efficient collections

- Use collections instead of lists wherever possible.
- Collections are native to ICC2 and are faster.
- Avoid converting collections to lists unnecessarily.
- Use get_*/filter_collection instead of foreach on lists.
- Work with small, targeted collections.

Example 1 Using collections (efficient)

```
# Get all sequential cells
set seq_cells [get_cells -hier -filter
  "is_sequential==true"]

# Get their names only when required
set seq_names [get_object_name $seq_cells]
```

Avoid (slower way)

```
set all_cells [get_cells -hier]
set seq_cells {}
foreach c $all_cells {
    if {[get_attribute $c
        is_sequential]} {
        lappend seq_cells $c
    }
}
```

2. Collection caching

- Query results can be cached in variables for reuse.
- Avoid repeated database queries for the same objects.
- Cache frequently used collections (cells, nets, pins).
- Improves runtime significantly in large designs.
- Use global variables or namespace variables for common data.

Example 2 Caching collections

```
# Cache frequently used collections
set all_regs [get_cells -hier -filter
  "is_sequential==true"]
set all_clks [get_clocks *]
# Reuse cached data
set reg_count [sizeof_collection $all_regs]
puts "Total registers: $reg_count"
```

Benefits

- Reduces repeated queries
- Faster script execution
- Useful in multiple procs
- Keeps code clean
- Essential for large designs

3. Avoiding unnecessary loops

- Minimize the use of foreach loops on large collections.
- Use built-in collection commands and filters.
- Avoid nested loops where possible.
- Operate on collections directly instead of element-wise iteration.
- Use vectorized operations (e.g., set_attribute on collection).

Example 3 Avoid loop (use direct commands)

```
# Slower way
foreach c $all_regs {
    set_attribute $c dont_touch true
}
# Faster way
set_attribute $all_regs dont_touch true
```

Loop Optimization Tips

- Avoid element-wise loops
- Use collection commands
- Minimize nested loops
- Use filters instead of if conditions
- Perform bulk operations

4. Efficient filtering

- Use -filter option with get_* commands.
- Apply multiple filter conditions in a single query.
- Filter on attributes instead of post-processing in loops.
- Use logical operators (&&, ||) for complex filters.
- Reduces the amount of data processed and improves runtime.

Example 4 Efficient filtering

```
# Get high fanout nets
set hf_nets [get_nets -hier -filter "fanout_count > 100"]
# Get non-clock, non-power cells
set cells [get_cells -hier -filter "is_clock==false && \
                                    is_power==false"]
```

Common Filter Attributes

- is_sequential
- is_combinational
- is_clock / is_power
- ref_name
- fanout_count
- cell_type
- is_hierarchical

5. Reducing repeated queries

- Avoid querying the same data multiple times.
- Store query results in variables and reuse.
- Combine multiple operations in a single query.
- Use report commands only when needed.
- Use -collection and -filter to get targeted results.

Example 5 Reduce repeated queries

```
# Slower - multiple queries
set regs [get_cells -hier -filter "is_sequential"]
set reg_cnt [sizeof_collection [get_cells -hier -filter
  "is_sequential"]]
set reg_names [get_object_name [get_cells -hier -filter
  "is_sequential"]]

# Faster - single query and reuse
set regs [get_cells -hier -filter "is_sequential"]
set reg_cnt [sizeof_collection $regs]
set reg_names [get_object_name $regs]
```

Best Practices

- Query once, reuse many times
- Combine filters
- Avoid redundant reports
- Use collections efficiently
- Store results in variables
- Reduces runtime and memory

6. Large-design scripting

- Use scalable scripting techniques for million-instance designs.
- Process data in smaller chunks if needed.
- Use hierarchical queries and filters.
- Avoid generating large lists or writing huge logs.
- Use modular procedures and configuration files.

Example 6 Handling large designs

```
# Process design in blocks (hierarchical)
set blocks [get_blocks -hier]
foreach blk $blocks {
  set blk_cells [get_cells -of_objects $blk -filter "is_sequential"]
  puts "Block [get_object_name $blk] : [sizeof_collection
      $blk_cells] regs"
}
# Use filters to limit data
set critical_paths [get_timing_paths -max_paths 100 \
                                     -filter "slack < 0"]
```

Large Design Tips

- Use hierarchical queries
- Process in smaller chunks
- Limit report size
- Avoid huge log files
- Use efficient filters
- Use checkpoint-based flow
- Keep scripts modular
- Test on smaller design first

Performance Comparison (Example)

Method	Runtime (s)	Memory (MB)	Notes
List + foreach loop	120	850	Slow, high memory
Repeated queries	95	600	Redundant DB access
No filtering	80	520	Processes extra data
Optimized collections	15	120	Fast and efficient
Caching + filtering	10	100	Best performance

Large-Design Scripting Flow



Key Takeaways

- [ ] Use collections instead of lists for better performance.
- [ ] Cache query results and reuse them.
- [ ] Avoid unnecessary and nested loops.
- [ ] Use efficient filtering with -filter option.
- [ ] Reduce repeated database queries.
- [ ] Follow scalable scripting practices for large designs.
- [ ] Optimized Tcl scripts improve runtime, productivity and maintainability.
- [ ] Essential for professional ICC2 automation and signoff flows.

Author : Konda Tharun Vrekumanda

"Optimize your scripts, accelerate your design !"

39. Professional Tcl Coding Standards

- Follow consistent Tcl coding standards for clean, readable and maintainable ICC2 scripts.
- Use meaningful names, modular design and proper comments.
- Helps in team collaboration and long-term maintenance.
- Reduces debugging time and improves reusability.
- Essential for professional and production-level automation.

- Write clean code, so future you (or others) can understand it.
- Follow a consistent style across all scripts.
- Handle errors gracefully and provide useful messages.
- Use modular and reusable procedures.
- Document the purpose, usage and examples.
- Good coding standards make automation reliable, scalable and professional.
- A must for industry-level ICC2 Tcl development.

39.1 Professional Tcl Coding Standards Overview

- Coding standards ensure clean, consistent and maintainable scripts.
- Helps in team collaboration and long-term support.
- Covers naming, comments, formatting, error handling, modular design, reusability and documentation.
- Improves readability, reduces bugs and makes scripts portable.
- Essential for professional ICC2 automation.

1. Naming conventions

- Use meaningful and descriptive names.
- Follow a consistent naming style (e.g., lower_case_with_underscore).
- Use clear prefixes for variables, procedures and constants.
- Avoid single letter names (except loop indices).
- Use block/stage specific names for clarity.

Example 1 — Good vs Bad Naming

Bad Practice	Good Practice
set a 10	set max_utilization 10
set x1 [get_cells U1]	set core_cells [get_cells U1]
proc run {} { ... }	proc run_place {} { ... }

Naming Prefixes (Example)

- var_ : variable
- proc_ : procedure
- const_ : constant
- blk_ : block related
- stage_ : flow stage
- rpt_ : report related

2. Comments

- Use comments to explain the purpose, logic and assumptions.
- Include author, date and version information.
- Use section headers for better organization.
- Comment complex logic and non-obvious commands.

Example 2 — Commenting Style

```
# ------------------------------------------------
# Procedure   : proc_run_placement
# Description : Runs placement for the given block
# Author      : Your Name
# Date        : 2025-09-04
# ------------------------------------------------
proc proc_run_placement {blk_name} {
    # Get block object
    set blk [get_blocks $blk_name]
    # Run placement
    place_opt -block $blk
}
```

Comment Types

- File header comments
- Procedure comments
- Inline comments
- Section separators
- TODO / FIXME notes

3. Formatting

- Use consistent indentation (e.g., 4 spaces).
- Keep line length reasonable (<= 80-100 chars).
- Use blank lines to separate logical blocks.
- Align related options for readability.
- Follow a consistent style across all scripts.

Example 3 — Good Formatting Practice

```
set cells [get_cells -hier -filter "is_sequential"]
foreach cell $cells {
    set ref_name [get_attribute $cell ref_name]
    if { $ref_name eq "DFFX1" } {
        set dont_touch true
    } else {
        set dont_touch false
    }
    set_attribute dont_touch $dont_touch $cell
}
```

Formatting Tips

- Use consistent indent
- Align options
- Use blank lines
- Keep lines readable
- Avoid very long lines
- Use braces for blocks

4. Error handling

- Use error handling to catch and report issues gracefully.
- Provide meaningful error messages.
- Use catch, if-else and return codes.
- Log errors to a file and stop execution if needed.
- Helps in robust and reliable automation.

Example 4 — Error Handling in Tcl

```
proc safe_execute {cmd} {
    if { [catch {eval $cmd} errMsg] } {
        puts "ERROR: Command failed : $cmd"
        puts "       Error message : $errMsg"
        return -code error $errMsg
    } else {
        puts "INFO: Command executed successfully : $cmd"
    }
}
```

Common Techniques

- Use catch for error trapping
- Check return codes
- Print meaningful messages
- Log errors to file
- Exit gracefully on fatal errors
- Provide recovery options

5. Modular design

- Break scripts into smaller, logical procedures.
- Use separate procedures for each stage or task.
- Pass arguments instead of using global variables.
- Keeps code organized, readable and easy to debug.
- Enables reuse across different projects.

Example 5 — Modular Procedure Example

```
proc run_cts {blk_name args} {
    puts "INFO: Starting CTS for $blk_name"
    # CTS specific commands
    clock_opt -block $blk_name {*}$args
    puts "INFO: CTS completed for $blk_name"
}
# Usage
run_cts "mychip" -verbose
```

Modular Design Benefits

- Improves readability
- Easier debugging
- Reusable across projects
- Supports team development
- Simplifies maintenance
- Allows independent testing

6. Reusable procedures

- Create generic, reusable procedures for common tasks.
- Use arguments and default values.
- Store in a common utility file (e.g., utils.tcl).
- Can be reused across multiple designs and flows.
- Reduces effort and improves consistency.

Example 6 — Reusable Procedure (Generic)

```
proc get_cell_list {pattern {hier 1}} {
    if { $hier } {
        return [get_cells -hier -filter "name =~ $pattern"]
    } else {
        return [get_cells -filter "name =~ $pattern"]
    }
}
# Usage
set u_list [get_cell_list "U*"]
```

Best Practices

- Use meaningful names
- Use default arguments
- Keep procedures generic
- Store in a common file
- Add usage examples
- Ensure error handling

7. Documentation

- Document the purpose, inputs, outputs and examples.
- Use a standard header format for each procedure.
- Maintain a README for the overall script.
- Helps in knowledge transfer and long-term maintenance.
- Essential for team and production environments.

Example 7 — Procedure Documentation Template

```
# ------------------------------------------------
# Procedure   : get_utilization
# Description : Returns core utilization of the block
# Arguments   : blk_name - block name
# Returns     : utilization (float)
# Example     : set u [get_utilization "mychip"]
# ------------------------------------------------
proc get_utilization {blk_name} {
    set blk [get_blocks $blk_name]
    set util [get_attribute $blk core_utilization]
    return $util
}
```

Documentation Checklist

- Procedure purpose
- Input arguments
- Output/return values
- Usage examples
- Assumptions/constraints
- Version history
- Author information

Author : Konda Tharun Vrekumada

"Good code today, reliable automation tomorrow !"

TCL HERO
ADVANCED ICC2 AUTOMATION
40. Complete ICC2 Run Manager

- A complete ICC2 run manager automates the entire flow from setup to GDSII using Tcl.
- Provides a single, configurable and robust script.
- Ensures consistent, repeatable and error-free implementation.
- Supports MMMC, multiple blocks and signoff flow.
- Saves time, reduces manual effort and improves productivity.
- Essential for production-level ICC2 tapeout.

- Modular stage-based flow with logging and checkpoints.
- Supports error handling and recovery.
- Configurable for different designs and technologies.
- Generates reports at each stage.
- Can be extended for multi-block and hierarchical flows.
- A must-have framework for professional ICC2 users.

1. Setup

- Set design, technology and library information.
- Define directory structure (inputs, outputs, reports).
- Set environment variables and tool options.
- Parse configuration file (Tcl/JSON/YAML).
- Create log file and initialize run.

Example 1 — Setup (run.tcl)

```
source ./config.tcl
set DESIGN $::env(DESIGN_NAME)
set TOP    $::env(TOP)
set tech_file ./inputs/tech.tf
set log_file ./logs/run.log
redirect -tee -file $log_file {
puts "INFO: ICC2 Run started at [clock format
    [clock seconds]]"
}
```

Directory Structure

```
run/
├── config/
├── scripts/
├── inputs/
├── logs/
├── reports/
├── checkpoints/
└── outputs/
```

2. Initialization

- Read technology and library files.
- Read design netlist and constraints.
- Set search paths and run tool initialization commands.
- Check design and environment readiness.

Example 2 — Initialization (init.tcl)

```
read_tech_file $tech_file
read_libs $lib_list
read_verilog $netlist
read_sdc $sdc_file
initialize_floorplan
```

Key Checks

- Files exist
- Libraries loaded
- Netlist read correctly
- Constraints loaded
- Tool initialized

3. MMMC (Multi-Mode Multi-Corner)

- Create modes, corners and scenarios.
- Read mode/corner specific constraints.
- Setup analysis views for setup and hold.
- Verify scenarios are correct.

Example 3 — MMMC setup

```
create_mode func -sdc func.sdc
create_mode test -sdc test.sdc
create_corner ss -lib ss.lib
create_corner ff -lib ff.lib
create_scenario func_ss -mode func -corner ss
create_scenario func_ff -mode func -corner ff
```

Typical Scenarios

- func_ss (setup)
- func_ff (hold)
- test_ss
- test_ff
- low_power
- high_performance

4. Floorplan

- Define core, die area and aspect ratio.
- Create floorplan, IO placement and blockages.
- Insert tap cells, well taps and endcaps.
- Save checkpoint and generate reports.

Example 4 — Floorplan (floorplan.tcl)

```
create_floorplan -core_area {0 0 1000 800}
create_io_ring
create_tap_cells
create_placement_blockage
save_block -as fp -dir ./checkpoints
```

Floorplan Outputs

- DEF (optional)
- Floorplan report
- Placement blockage
- IO report
- Checkpoint (fp)

5. Power planning

- Setup power grid (rings, stripes, mesh).
- Create power/ground nets and special routes.
- Connect PG pins and verify.
- Run PG DRC checks.

Example 5 — Power planning (power.tcl)

```
create_pg_net -net VDD -voltage 0.8
create_pg_net -net VSS -voltage 0.0
create_pg_stripe -nets {VDD VSS}
connect_pg_net
check_pg_drc
save_block -as power -dir ./checkpoints
```

Common PG Structures

- Power rings
- Power stripes
- Power mesh
- Via stitching
- PG connectivity check
- IR drop check

6. Placement

- Set placement mode and options.
- Run global placement and legalization.
- Fix placement issues and optimize.
- Save checkpoint and generate reports.

Example 6 — Placement (place.tcl)

```
set_place_mode -effort high
place_opt
legalize_placement
check_placement
save_block -as place -dir ./checkpoints
```

7. CTS (Clock Tree Synthesis)

- Define clocks and CTS options.
- Run clock tree synthesis.
- Verify clock network (skew, latency, transition).
- Save checkpoint and generate reports.

Example 7 — CTS (cts.tcl)

```
create_clock_tree_spec
set_cts_options -target_skew 0.05
clock_opt
check_clock_tree
save_block -as cts -dir ./checkpoints
```

CTS Reports

- Clock tree report
- Skew and latency
- Transition report
- Clock DRC check
- Checkpoint (cts)

8. Routing

- Set routing options and track assignments.
- Run global and detailed routing.
- Fix DRC and antenna violations.
- Save checkpoint and generate reports.

Example 8 — Routing (route.tcl)

```
set_route_mode -effort high
route_opt
check_route
check_antenna
save_block -as route -dir ./checkpoints
```

Routing Reports

- Routing report
- DRC report
- Antenna report
- Congestion report
- Checkpoint (route)

9. Post-route optimization

- Run timing, SI and power optimizations.
- Fix setup/hold violations.
- Improve QoR (WNS, TNS, power, area).
- Save checkpoint and generate reports.

Example 9 — Post-route opt (opt.tcl)

```
opt_design -post_route
opt_timing
opt_power
opt_si
save_block -as opt -dir ./checkpoints
```

Optimization Targets

- Fix setup violations
- Fix hold violations
- Reduce power/area
- Improve SI (crosstalk)
- Meet signoff QoR

10. ECO

- Apply late stage fixes (timing, DRC, ECO).
- Support iterative ECO flow.
- Maintain change logs.
- Save checkpoint after ECO.

Example 10 — ECO (eco.tcl)

```
read_eco_changes eco.eco
apply_eco
check_design
save_block -as eco -dir ./checkpoints
}
```

ECO Use Cases

- Fix timing violations
- Fix DRC violations
- Netlist changes
- Late functional fixes
- Pre-signoff cleanup

11. Signoff

- Run final STA, DRC, LVS (if available).
- Check QoR across all scenarios.
- Generate final reports and summaries.
- Ensure tapeout readiness.

Example 11 — Signoff (signoff.tcl)

```
report_timing -all_scenarios
report_qor
check_drc
report_power
report_area
save_block -as signoff -dir ./checkpoints
```

Signoff Checklist

- WNS/TNS met
- DRC clean
- Power within budget
- Area within target
- All scenarios clean
- Final checkpoint

12. GDSII

- Stream out final GDSII for tapeout.
- Encrypt if required.
- Verify GDSII and run final checks.
- Archive outputs and reports.

Example 12 — GDSII (gds.tcl)

```
set_stream_out_options -format gds2
stream_out ./outputs/${TOP}.gds
verify_gds -input ./outputs/${TOP}.gds
save_block -as final -dir ./checkpoints
```

Final Outputs

- GDSII file
- Stream log
- Final netlist
- DEF (optional)
- All reports archived

Complete ICC2 Run Flow



Key Takeaways

- [ ] A complete run manager automates the entire ICC2 flow.
- [ ] Use modular stage-based scripts.
- [ ] Maintain consistent configuration and directory structure.
- [ ] Generate reports and checkpoints at every stage.
- [ ] Support MMMC, multiple blocks and hierarchical designs.
- [ ] Include error handling, logging and recovery.
- [ ] Ensure signoff QoR before GDSII generation.
- [ ] Essential for professional and production-level ICC2 automation.

Author : Kenda Tharun Vrekumenda

"Automate today, tapeout tomorrow !"

- Automate the complete ASIC flow from RTL to GDSII using Tcl.
- Integrates synthesis handoff with full ICC2 implementation.
- Ensures consistent, repeatable and signoff-ready flow.
- Generates final tapeout data and release package.
- Helps in reducing manual effort and improves productivity.
- Essential for industrial SoC and block-level designs.

TCL HERO
ADVANCED ICC2 AUTOMATION
41. Complete RTL-to-GDSII Automation

- Single automated flow from RTL to GDSII.
- Configurable, modular and scalable.
- Includes synthesis handoff, implementation, signoff.
- Generates final tapeout release package.
- Can be used for block, IP or full SoC.
- A complete, production-ready ICC2 automation flow.

1. Synthesis handoff

- Receive synthesized netlist, SDC, and constraints from synthesis.
- Check design consistency and completeness.
- Verify library, technology and timing information.
- Prepare input files for ICC2 initialization.
- Can also run synthesis as part of the flow (optional).

Example 1 — Synthesis Handoff (Tcl)

```
set DESIGN_NAME mychip
set netlist ./inputs/${DESIGN_NAME}_syn.v
set sdc  /inputs/${DESIGN_NAME}_syn.sdc
set tech_file ./tech/tech.tf
set lib_files ./libs/*.db

# Check files exist
foreach f [list $netlist $sdc $tech_file] {
    if {![file exists $f]} {
        puts "ERROR: Missing file $f"
        exit 1
    }
}
```

Handoff Files

- Synthesized netlist
- SDC constraints
- Technology file (.tf)
- Library files (.db)
- UPF/CPF (optional)
- Design spec
- Synthesis reports

2. ICC2 initialization

- Set up technology, libraries and design environment.
- Read netlist and constraints.
- Create working directories.
- Initialize design and check consistency.

Example 2 — Initialization (Tcl)

```
create_mw_lib $DESIGN_NAME -technology $tech_file \
    -mw_reference_library $lib_files
open_mw_lib $DESIGN_NAME
read_verilog $netlist
read_sdc $sdc
link_design
check_design
```

Key Settings

- Technology file
- Library paths
- Search paths
- RTL/Netlist
- Constraints (SDC)
- UPF/CPF (optional)
- Working directories

3. Floorplan

- Define core, die size and aspect ratio.
- Set site, rows and tracks.
- Create floorplan, IO placement and blockages.
- Insert tap cells, well taps and endcaps.

Example 3 — Floorplan (Tcl)

```
create_floorplan -core_area {0 0 1000 800} \
    -aspect_ratio 1.0
create_io_ring
create_tap_cells
create_placement_blockage
save_block -as fp -dir ./checkpoints
```

Floorplan Outputs

- DEF (optional)
- Floorplan report
- Placement blockage
- Tap cell report
- Checkpoint (fp)

4. Power

- Read UPF/CPF for power intent.
- Create power domains, power rings, stripes and grids.
- Connect power to standard cells.
- Run PG DRC and verify connectivity.

Example 4 — Power Planning (Tcl)

```
read_upf ./inputs/${DESIGN_NAME}.upf
create_power_rings
create_power_stripes
connect_pg_net
check_pg_drc
save_block -as power -dir ./checkpoints
```

Power Outputs

- Power grid report
- PG DRC report
- UPF/CPF summary
- Connectivity report
- Checkpoint (power)

5. Placement

- Set placement mode and options.
- Run global placement and legalization.
- Fix placement violations (overlap, density, etc.).
- Save checkpoint and generate reports.

Example 5 — Placement (Tcl)

```
set_placement_mode -effort high
place_opt
legalize_placement
check_placement
report_placement -summary
save_block -as place -dir ./checkpoints
```

Placement Outputs

- Placement report
- Congestion report
- Cell density report
- Placement DRC
- Checkpoint (place)

6. CTS (Clock Tree Synthesis)

- Define clocks and CTS options.
- Run clock tree synthesis.
- Verify clock network (skew, latency, transitions).
- Fix clock violations and optimize.

Example 6 — CTS (Tcl)

```
create_clock_tree_spec
set_cts_options -target_skew 0.05
clock_opt -only_cts
check_clock_tree
report_clock_tree -summary
save_block -as cts -dir ./checkpoints
```

CTS Reports

- Clock tree report
- Skew and latency
- Transition report
- Clock DRC report
- Clock power report
- Checkpoint (cts)

7. Routing

- Set routing mode and layer assignments.
- Run global and detailed routing.
- Fix DRC and antenna violations.
- Save checkpoint and generate reports.

Example 7 — Routing (Tcl)

```
set_route_mode -effort high
route_opt
check_route
check_antenna
report_route -summary
save_block -as route -dir ./checkpoints
```

Routing Reports

- Routing report
- DRC report
- Antenna report
- Congestion report
- Layer utilization
- Checkpoint (route)

8. Signoff

- Run final STA, DRC, LVS (if available).
- Verify timing, power, SI and physical checks.
- Generate signoff reports and summaries.
- Ensure design is tapeout ready.

Example 8 — Signoff (Tcl)

```
report_timing -all_scenarios -max_paths 100
report_power -analysis_effort high
report_si
check_drc
report_drc
save_block -as signoff -dir ./checkpoints
```

Signoff Checklist

- Timing (setup/hold)
- Power (dynamic/leakage)
- Signal integrity (SI)
- DRC (clean)
- LVS (if available)
- All signoff reports clean

9. Tapeout data

- Generate final GDSII, netlist and collateral.
- Include timing, power and physical constraint files.
- Run final checks (stream-out, antenna, metal fill, etc.).
- Create a release package for tapeout.

Example 9 — Tapeout Data (Tcl)

```
set_stream_out_options -format gds2
stream_out ./outputs/${DESIGN_NAME}.gds
write_verilog -hierarchy -output ./outputs/${DESIGN_NAME}.v
write_def ./outputs/${DESIGN_NAME}.def
write_sdc ./outputs/${DESIGN_NAME}.sdc
save_block -as final -dir ./checkpoints
```

Tapeout Outputs

- GDSII file
- Final netlist (Verilog)
- DEF (optional)
- SDC (Final)
- Antenna/Fill data
- Release package (tar.gz)

10. Final release

- Archive all inputs, scripts, checkpoints and reports.
- Create a structured release directory.
- Verify completeness and checksum.
- Hand over to foundry / production team.

Example 10 — Final Release (Tcl)

```
file mkdir ./release
exec tar -czf ${DESIGN_NAME}_release.tar.gz \
    ./inputs ./scripts ./checkpoints ./outputs ./reports
puts "INFO: Release package created!"
```

Release Contents

- All input files
- Tcl scripts
- Checkpoints
- Reports
- GDSII and netlist
- README / documentation

Complete RTL-to-GDSII Flow



Key Takeaways

- Complete, end-to-end RTL-to-GDSII automation using Tcl.
- Modular and configurable flow with checkpoints at each stage.
- Generates signoff-clean GDSII and release package.
- Reduces manual effort and ensures consistent, repeatable results.
- Essential for professional ASIC/SoC tapeout.

Author : Konda Tharun Vrekumarda

"From RTL to reality – Automate. Implement. Tapeout !"

TCL HERO

ADVANCED ICC2 AUTOMATION

42. Advanced Automation Projects

- Build advanced analysis tools using Tcl for ICC2.
- Analyze cell, macro, timing, congestion, placement, CTS and routing QoR automatically.
- Generate useful reports and visualizations.
- Helps in faster debugging, better QoR and signoff readiness.
- Essential for production-level automation and tapeout.
- Real-world projects to apply Tcl skills.
- Can be extended and customized for any design.
- Generates comprehensive QoR reports and dashboards.
- Helps in data-driven decision making.
- Improves productivity and reduces manual effort.
- A must-have skill for advanced ICC2 users.

1. Cell analyzer

- Analyzes standard cells in the design.
- Reports cell types, area, count, utilization, and critical cells.
- Identifies high-fanout, large cells and potential issues.
- Helps in optimization and area reduction.

Example 1 Cell analyzer (cell_analyzer.tcl)

```
set all_cells [get_cells -hier *]
set total_cells [sizeof_collection $all_cells]
set total_area 0
foreach c $all_cells {
    set a [get_attribute $c area]
    set total_area [expr $total_area + $a]
}
puts "Total cells: $total_cells"
puts "Total cell area: $total_area"
```

Sample Report

- Total cells : 250450
- Total area : 1.25e+08
- Sequential : 120450
- Combinational : 130000
- Largest cell : AOI222_X8
- High fanout (>100) : 320

2. Macro analyzer

- Analyzes hard macros and IP blocks.
- Reports macro area, location, orientation and spacing.
- Checks for overlaps and spacing violations.
- Useful for floorplan and integration checks.

Example 2 Macro analyzer (macro_analyzer.tcl)

```
set macros [get_cells -hier -filter "is_hard_macro==true"]
foreach m $macros {
    set name [get_attribute $m name]
    set bbox [get_attribute $m bbox]
    set area [get_attribute $m area]
    puts "$name  $bbox  $area"
}
```

Macro Report (Sample)

Macro	Area (µm²)	Location (x y)
CPU	2.5e+06	(100 200)
SRAM0	1.8e+06	(500 200)
SRAM1	1.8e+06	(500 800)
DSP	3.2e+06	(1200 400)

3. Timing analyzer

- Analyzes design timing and finds critical paths.
- Reports WNS, TNS, endpoint slacks and critical paths.
- Identifies top violating paths and cells.
- Helps in timing closure and optimization.

Example 3 Timing analyzer (timing_analyzer.tcl)

```
set wns [get_attribute [get_timing_paths -max_paths 1] \
        slack]
set tns [report_timing -return_string \
        | grep "Total Negative Slack"]

puts "WNS: $wns"
puts "TNS: $tns"

report_timing -max_paths 10 > timing_top10.rpt
```

Timing Summary (Sample)

- WNS : -0.025 ns
- TNS : -12.45 ns
- Total paths : 2.5M
- Violating paths : 1250
- Top endpoint : U123/Q
- Clock : clk_main

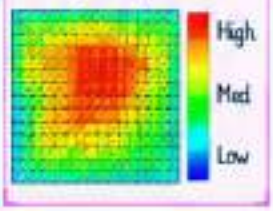
4. Congestion analyzer

- Analyzes routing congestion in the design.
- Reports congested areas, layers and hotspots.
- Helps in placement and routing optimization.
- Can generate congestion map and reports.

Example 4 Congestion analyzer (congestion_analyzer.tcl)

```
set cong_map [report_congestion -return_string]
puts $cong_map
report_congestion -verbose > congestion.rpt
create_congestion_map \
    -output congestion.png
```

Congestion Map (Sample)



5. Placement QoR analyzer

- Analyzes placement quality and metrics.
- Reports HPWL, cell density, displacement and utilization.
- Identifies potential placement issues.
- Helps in improving placement strategy.

Example 5 Placement QoR analyzer (place_qor.tcl)

```
set hpwl [report_qor -return_string | \
    grep "HPWL"]
set density [report_placement -density]
puts "HPWL: $hpwl"
puts "Density: $density"
report_placement -summary > place_qor.rpt
```

Placement QoR (Sample)

- HPWL : 1.25e+09
- Total cell area : 1.25e+08
- Core utilization : 72%
- Average density : 0.68
- Max density : 0.92
- Displacement : 12.5 µm

6. CTS QoR analyzer

- Analyzes clock tree quality after CTS.
- Reports clock skew, latency, insertion delay and clock violations.
- Identifies clock tree issues and unbalanced clocks.
- Helps in improving CTS settings and clock quality.

Example 6 CTS QoR analyzer (cts_qor.tcl)

```
report_clock_tree -summary > cts_qor.rpt
set skew [get_attribute [get_clocks *] skew]
set latency [get_attribute [get_clocks *] latency]
puts "Clock Skew    : $skew"
puts "Clock Latency : $latency"
```

CTS QoR (Sample)

- Clock skew : 0.032 ns
- Insertion delay : 0.410 ns
- CTS cells : 15420
- Buffer count : 12000
- Inverter count : 3420
- Clock violations : 0

7. Routing QoR analyzer

- Analyzes post-route design quality.
- Reports DRC, antenna, wire length, vias and routing efficiency.
- Identifies routing hotspots and violations.
- Helps in routing optimization and signoff readiness.

Example 7 Routing QoR analyzer (route_qor.tcl)

```
report_route -summary > route_qor.rpt
report_drc -summary > drc.rpt
report_antenna -summary > antenna.rpt
set wire_len [report_route -wire_length]
puts "Total wire length: $wire_len"
```

Routing QoR (Sample)

- Total wire length : 3.25e+08 µm
- Total vias : 12.5M
- DRC violations : 0
- Antenna violations : 0
- Routing efficiency : 92%
- Congested areas : 2

8. Signoff dashboard

- Comprehensive dashboard with all signoff metrics.
- Collects timing, power, DRC, LVS and QoR data.
- Generates a single summary report and status.
- Helps in tapeout readiness and final signoff.

Example 8 Signoff dashboard (signoff_dashboard.tcl)

```
set rpt_dir ./signoff_reports
file mkdir $rpt_dir
report_qor > $rpt_dir/qor.rpt
report_timing > $rpt_dir/timing.rpt
report_power > $rpt_dir/power.rpt
report_drc > $rpt_dir/drc.rpt
report_signoff_summary > $rpt_dir/signoff.rpt
puts "Signoff dashboard generated in $rpt_dir"
```

Signoff Dashboard (Sample)

Check	Status	Metric
Timing	PASS	WNS = 0.015 ns
Power	PASS	456 mW
DRC	PASS	0
LVS	PASS	Match
Antenna	PASS	0
QoR	PASS	All targets met

Key Takeaways

- Build advanced analyzers using Tcl for complete design analysis.
- Generate meaningful reports for each stage (cell, macro, timing, congestion, etc.).
- Use automation to improve QoR and reduce manual effort.
- Create a unified signoff dashboard for tapeout readiness.
- Extend and customize these projects for real-world designs.
- Essential for professional ICC2 automation and production flows.

Author : Kanda Tharun Mekananda

"Analyze today, automate tomorrow !"

TCL HERO
ADVANCED ICC2 AUTOMATION
43. Complete Automated ICC2 Flow

- Build a complete reusable ICC2 Tcl framework.
- Automate PD flow from setup to signoff.
- Generate stage reports and enable QA checks.
- Implement PASS/FAIL gates for each stage.
- Automate checkpoints and QoR analysis.
- Generate final GDSII release package.
- A production-ready flow for real-world projects.

- A modular, reusable and fully automated flow.
- Ensures consistent, repeatable and error-free runs.
- Captures reports, checkpoints and QoR metrics.
- Supports PASS/FAIL criteria and early error detection.
- Generates final GDSII package for tapeout.
- Can be adapted for block, SoC or hierarchical designs.
- Essential for professional ICC2 users.

1. Reusable ICC2 Tcl Framework

- Create a modular and reusable framework.
- Use separate scripts for each stage.
- Common utilities for logging, reporting, error handling,
- Configurable through a config file (Tcl/JSON).
- Easy to extend for new designs and blocks.

Example 1 Framework Structure

```
icc2_flow/
├── config/          # design config files
├── scripts/         # stage scripts
│   ├── 01_setup.tcl
│   ├── 02_floorplan.tcl
│   ├── 03_power.tcl
│   ├── 04_placement.tcl
│   ├── 05_cts.tcl
│   ├── 06_routing.tcl
│   ├── 07_opt.tcl
│   └── 08_signoff.tcl
├── utils/           # common procs (log, report, checks)
├── scripts/         # stage reports
├── checkpoints/     # stage checkpoints
└── results/         # final GDSII package
```

Key Features
- Modular scripts
- Configuration driven
- Common utility procs
- Logging & error handling
- Reusable across projects
- Supports block/SoC flow

2. Automate the complete PD flow

- Run the entire ICC2 flow using Tcl.
- Stages: setup → init → floorplan → power → placement → CTS → routing → post-route optimization → signoff.
- Each stage executed via dedicated script.
- Automatic handoff between stages.

Example 2 Main Run Script (run.tcl)

```
source ./config/design.tcl
source ./utils/common_procs.tcl

set stages {setup init floorplan power \
        placement cts routing opt signoff}

foreach stage $stages {
  log_info "===== Running $stage stage ====="
  source ./scripts/${stage}.tcl
  if {[check_stage_result $stage] == 0} {
     log_error "Stage $stage failed!"
     exit 1
  }
}
log_info "PD flow completed successfully!"
```

Flow Stages
1. Setup
2. Initialization
3. Floorplan
4. Power planning
5. Placement
6. CTS
7. Routing
8. Post route optimization
9. Signoff

→ Final GDSII

3. Generate stage reports

- Generate detailed reports at each stage.
- Store reports in organized directories.
- Use consistent naming convention.
- Include key QoR metrics in stage summary.

Example 3 Report Generation

```
set rpt_dir "reports/${stage}"
file mkdir $rpt_dir
report_qor      > $rpt_dir/qor_${stage}.rpt
report_timing   > $rpt_dir/timing_${stage}.rpt
report_congestion > $rpt_dir/congestion_${stage}.rpt
report_power    > $rpt_dir/power_${stage}.rpt
report_route    > $rpt_dir/route_${stage}.rpt
report_utilization > $rpt_dir/util_${stage}.rpt
```

Stage Report Contents
- QoR summary
- Timing report
- Power report
- Congestion report
- Utilization report
- DRC/LVS (if applicable)
- Custom analysis reports

4. Implement PASS/FAIL gates

- Define QoR criteria for each stage.
- Check metrics after report generation.
- Stop the flow if criteria are not met.
- Provide clear PASS/FAIL messages in log.
- Helps in early issue detection.

Example 4 PASS/FAIL Check (Timing)

```
set wns [get_attribute [current_design] wns]
set tns [get_attribute [current_design] tns]

if { $wns < -0.05 || $tns < -10.0 } {
   log_error "Timing check FAILED."
   log_error "  WNS = $wns, TNS = $tns"
   return 0
} else {
   log_info "Timing check PASSED"
   return 1
}
```

Typical Checks
- WNS / TNS
- Max leakage / Total power
- DRC violations (0)
- Congestion < threshold
- Utilization within limit
- Route overflow (0)
- LVS clean (for signoff)
- Custom design checks

5. Automate checkpoints

- Save design checkpoints after each stage.
- Use save_block, save_lib and export files as needed.
- Maintain recovery checkpoints for debugging.
- Helps to restart flow from any stage.

Example 5 Checkpoint Automation

```
set chk_dir "checkpoints/${stage}"
file mkdir $chk_dir
save_block -as ${stage} -in $chk_dir
save_lib   -as ${stage} -in $chk_dir
write_def  $chk_dir/${stage}.def
write_verilog $chk_dir/${stage}.v
```

Checkpoint Types
- Stage checkpoint
- Recovery checkpoint
- Block (.enc)
- Library (.lib)
- DEF / Netlist
- Debug checkpoints

6. Automate QoR analysis

- Collect and analyze key QoR metrics across stages.
- Track timing, power, area, congestion and DRC.
- Generate QoR summary and compare with previous runs.
- Highlight improvements or regressions.

Example 6 QoR Analysis (Summary)

```
set fp_area [get_attribute [current_design] area]
set util    [get_attribute [current_design] utilization]
set power   [get_attribute [current_design] total_power]
set wns     [get_attribute [current_design] wns]

puts $qor_fp "${stage}, $fp_area, $util, $power, $wns"
```

QoR Metrics
- Area / Utilization
- Timing (WNS, TNS)
- Power (Total, Leakage)
- Congestion
- DRC violations
- Route overflow
- Stage-to-stage comparison
- Trend analysis (historical)

7. Generate final GDSII release package

- Run final signoff checks (STA, DRC, LVS).
- Generate GDSII, final netlist and collateral.
- Create a release package with all required files.
- Archive reports, scripts and configuration.
- Deliver a production-ready tapeout package.

Example 7 Final Release (GDSII)

```
set rel_dir "results/release"
file mkdir $rel_dir

write_gds -output $rel_dir/${DESIGN_NAME}.gds
write_verilog -hierarchy -output $rel_dir/${DESIGN_NAME}.v
write_def $rel_dir/${DESIGN_NAME}.def

exec tar -czf $rel_dir/release_package.tar.gz $rel_dir/*

log_info "GDSII release package generated successfully!"
```

Release Package Contents
- GDSII file
- Final netlist (Verilog)
- DEF file
- SDC file (final)
- Signoff reports (STA/DRC/LVS)
- Scripts & config
- README / documentation
- Release archive (tar.gz)

Author : Konda Tharun Vrekumada

"Automate the flow, accelerate the silicon !"

Automate | Script | Simplify | Innovate

```
#!/usr/bin/tclsh
# TCL Script
set msg "Hello TCL!"
puts $msg

# Keep Learning :)
```

Tcl

SIMPLE
POWERFUL
FLEXIBLE
EVERYWHERE

Variables

```
set a 10
set b 20
set sum [expr $a + $b]
puts "Sum = $sum"
```

AUTOMATION
EFFICIENCY
PRODUCTIVITY

THANK YOU

set
puts
expr
if
foreach
proc

list
lappend
string
file
incr
info

"Small Scripts Big Impact"



Thank you !



LEARN • SCRIPT • AUTOMATE • GROW

